

FIT1058 — Notes  
Course Note(CN) + Rosen Discrete Mathematic (Rosen)

Alston

25 August 2026

## Contents

|          |                                                                                     |           |
|----------|-------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Sets</b>                                                                         | <b>6</b>  |
| 1.1      | Sets and elements . . . . .                                                         | 6         |
| 1.2      | Specifying sets . . . . .                                                           | 6         |
| 1.3      | Cardinality . . . . .                                                               | 7         |
| 1.4      | Sets of numbers . . . . .                                                           | 7         |
| 1.5      | Sets of strings . . . . .                                                           | 7         |
| 1.6      | Subsets and supersets . . . . .                                                     | 7         |
| 1.7      | Minimal, minimum, maximal, maximum . . . . .                                        | 8         |
| 1.8      | Counting all subsets . . . . .                                                      | 8         |
| 1.9      | The power set of a set . . . . .                                                    | 8         |
| 1.10     | Counting subsets by size using binomial coefficients . . . . .                      | 9         |
| 1.11     | Complement and set difference . . . . .                                             | 9         |
| 1.12     | Union and intersection . . . . .                                                    | 10        |
| 1.13     | Symmetric difference . . . . .                                                      | 12        |
| 1.14     | Cartesian product . . . . .                                                         | 12        |
| 1.15     | Partitions . . . . .                                                                | 13        |
| 1.16     | Cardinality of infinite sets (Rosen §2.5 – extension, beyond CN syllabus) . . . . . | 13        |
| <b>2</b> | <b>Functions</b>                                                                    | <b>15</b> |
| 2.1      | Definitions . . . . .                                                               | 15        |
| 2.1.1    | Domain . . . . .                                                                    | 15        |
| 2.1.2    | Codomain & co. . . . .                                                              | 15        |
| 2.1.3    | Rule . . . . .                                                                      | 16        |
| 2.2      | Functions in computer science . . . . .                                             | 16        |
| 2.2.1    | Functions from analysis . . . . .                                                   | 16        |
| 2.2.2    | Functions in mathematics and programming . . . . .                                  | 16        |
| 2.3      | Notation . . . . .                                                                  | 16        |
| 2.4      | Some special functions . . . . .                                                    | 17        |
| 2.5      | Functions with multiple arguments and values . . . . .                              | 17        |
| 2.6      | Restrictions . . . . .                                                              | 18        |
| 2.7      | Injections, surjections, bijections . . . . .                                       | 18        |
| 2.8      | Inverse functions . . . . .                                                         | 19        |
| 2.9      | Composition . . . . .                                                               | 20        |
| 2.10     | Cryptosystems . . . . .                                                             | 21        |
| 2.11     | Loose composition . . . . .                                                         | 22        |

|          |                                                                   |           |
|----------|-------------------------------------------------------------------|-----------|
| 2.12     | Counting functions . . . . .                                      | 22        |
| 2.13     | Binary relations . . . . .                                        | 23        |
| 2.14     | Properties of binary relations . . . . .                          | 24        |
| 2.15     | Combining binary relations . . . . .                              | 25        |
| 2.16     | Equivalence relations . . . . .                                   | 27        |
| 2.17     | Relations . . . . .                                               | 29        |
| 2.18     | Counting relations . . . . .                                      | 30        |
| 2.19     | Matrices (Rosen §2.6 – extension, beyond CN syllabus) . . . . .   | 30        |
| <b>3</b> | <b>Proofs</b>                                                     | <b>33</b> |
| 3.1      | Theorems and proofs . . . . .                                     | 33        |
| 3.2      | Logical deduction . . . . .                                       | 33        |
| 3.3      | Proofs of existential and universal statements . . . . .          | 34        |
| 3.4      | Finding proofs . . . . .                                          | 35        |
| 3.5      | Types of proofs . . . . .                                         | 36        |
| 3.6      | Proof by symbolic manipulation . . . . .                          | 37        |
| 3.7      | Proof by construction . . . . .                                   | 37        |
| 3.8      | Proof by cases . . . . .                                          | 38        |
| 3.9      | Proof by contradiction . . . . .                                  | 39        |
| 3.10     | Proof by mathematical induction . . . . .                         | 40        |
| 3.11     | Induction: more examples . . . . .                                | 41        |
| 3.12     | Induction: extended example . . . . .                             | 48        |
| 3.13     | Mathematical induction and statistical induction . . . . .        | 49        |
| 3.14     | Programs and proofs . . . . .                                     | 49        |
| <b>4</b> | <b>Propositional Logic</b>                                        | <b>51</b> |
| 4.1      | Truth values . . . . .                                            | 51        |
| 4.2      | Boolean variables . . . . .                                       | 51        |
| 4.3      | Propositions . . . . .                                            | 51        |
| 4.4      | Logical operations . . . . .                                      | 51        |
| 4.5      | Negation . . . . .                                                | 51        |
| 4.6      | Conjunction . . . . .                                             | 52        |
| 4.7      | Disjunction . . . . .                                             | 52        |
| 4.8      | De Morgan's Laws . . . . .                                        | 53        |
| 4.9      | Implication . . . . .                                             | 54        |
| 4.10     | Equivalence . . . . .                                             | 54        |
| 4.11     | Exclusive-or . . . . .                                            | 54        |
| 4.12     | Tautologies and logical equivalence . . . . .                     | 55        |
| 4.13     | History . . . . .                                                 | 56        |
| 4.14     | Distributive Laws . . . . .                                       | 56        |
| 4.15     | Laws of Boolean algebra . . . . .                                 | 56        |
| 4.16     | Disjunctive Normal Form . . . . .                                 | 57        |
| 4.17     | Conjunctive Normal Form . . . . .                                 | 57        |
| 4.18     | Satisfiability . . . . .                                          | 58        |
| 4.19     | Representing logical statements . . . . .                         | 60        |
| 4.20     | Statements about how many variables are true . . . . .            | 61        |
| 4.21     | Universal sets of operations . . . . .                            | 61        |
| 4.22     | Boolean algebra (Rosen Ch. 12 – new section, not in CN) . . . . . | 62        |

|          |                                                                           |           |
|----------|---------------------------------------------------------------------------|-----------|
| 4.22.1   | Boolean functions . . . . .                                               | 62        |
| 4.22.2   | Representing Boolean functions . . . . .                                  | 63        |
| 4.22.3   | Logic gates . . . . .                                                     | 63        |
| 4.22.4   | Minimization of circuits . . . . .                                        | 64        |
| <b>5</b> | <b>Predicate Logic</b>                                                    | <b>65</b> |
| 5.1      | Relations, predicates, and truth-valued functions . . . . .               | 65        |
| 5.2      | Variables and constants . . . . .                                         | 65        |
| 5.3      | Predicates and variables . . . . .                                        | 65        |
| 5.4      | Arguments of predicates . . . . .                                         | 66        |
| 5.5      | Building logical expressions with predicates . . . . .                    | 67        |
| 5.6      | Existential quantifier . . . . .                                          | 67        |
| 5.7      | Restricting existentially quantified variables . . . . .                  | 68        |
| 5.8      | Universal quantifier . . . . .                                            | 68        |
| 5.9      | Restricting universally quantified variables . . . . .                    | 68        |
| 5.10     | Multiple quantifiers . . . . .                                            | 69        |
| 5.11     | Predicate logic expressions . . . . .                                     | 69        |
| 5.12     | Doing logic with quantifiers . . . . .                                    | 70        |
| 5.13     | Duality between quantifiers . . . . .                                     | 70        |
| 5.14     | Some examples . . . . .                                                   | 71        |
| 5.15     | Summary of rules for logic with quantifiers . . . . .                     | 73        |
| 5.16     | Rules of inference (Rosen §1.6 – extension, beyond CN syllabus) . . . . . | 73        |
| <b>6</b> | <b>Sequences &amp; Series</b>                                             | <b>75</b> |
| 6.1      | Definitions and notation . . . . .                                        | 75        |
| 6.2      | Recursive definitions of sequences . . . . .                              | 75        |
| 6.3      | Arithmetic sequences . . . . .                                            | 76        |
| 6.4      | Geometric sequences . . . . .                                             | 76        |
| 6.5      | Harmonic sequences . . . . .                                              | 76        |
| 6.6      | From recursive definitions to formulas . . . . .                          | 76        |
| 6.7      | The Fibonacci sequence . . . . .                                          | 82        |
| 6.7.1    | Upper bounds . . . . .                                                    | 82        |
| 6.7.2    | Lower bounds . . . . .                                                    | 82        |
| 6.7.3    | Asymptotic behaviour . . . . .                                            | 82        |
| 6.7.4    | An exact formula . . . . .                                                | 83        |
| 6.8      | Limits of infinite sequences . . . . .                                    | 84        |
| 6.9      | Big-O notation . . . . .                                                  | 86        |
| 6.10     | Sums and summation . . . . .                                              | 88        |
| 6.11     | Finite series . . . . .                                                   | 90        |
| 6.12     | Finite arithmetic series . . . . .                                        | 90        |
| 6.13     | Finite geometric series . . . . .                                         | 91        |
| 6.14     | Infinite geometric series . . . . .                                       | 91        |
| 6.15     | Harmonic numbers . . . . .                                                | 91        |
| <b>7</b> | <b>Number Theory</b>                                                      | <b>93</b> |
| 7.1      | Multiples and divisors . . . . .                                          | 93        |
| 7.2      | Prime numbers . . . . .                                                   | 94        |
| 7.3      | Remainders and the mod operation . . . . .                                | 95        |

|           |                                                                             |            |
|-----------|-----------------------------------------------------------------------------|------------|
| 7.4       | Parity . . . . .                                                            | 96         |
| 7.5       | The greatest common divisor . . . . .                                       | 96         |
| 7.6       | The Euclidean algorithm . . . . .                                           | 97         |
| 7.7       | The gcd and integer linear combinations . . . . .                           | 98         |
| 7.8       | The extended Euclidean algorithm . . . . .                                  | 98         |
| 7.9       | Coprimality . . . . .                                                       | 99         |
| 7.10      | Modular arithmetic . . . . .                                                | 100        |
| 7.11      | Modular inverses . . . . .                                                  | 102        |
| 7.12      | The Euler totient function . . . . .                                        | 103        |
| 7.13      | Fast exponentiation . . . . .                                               | 104        |
| 7.14      | Modular exponentiation . . . . .                                            | 104        |
| 7.15      | Primitive roots . . . . .                                                   | 105        |
| 7.16      | One-way functions . . . . .                                                 | 106        |
| 7.17      | Modular exponentiation with fixed base . . . . .                            | 107        |
| 7.18      | Diffie–Hellman key agreement . . . . .                                      | 107        |
| <b>8</b>  | <b>Counting &amp; Combinatoric</b>                                          | <b>109</b> |
| 8.1       | Counting by addition . . . . .                                              | 109        |
| 8.2       | Counting by multiplication . . . . .                                        | 109        |
| 8.3       | Inclusion-exclusion . . . . .                                               | 110        |
| 8.4       | Inclusion-exclusion: derangements . . . . .                                 | 111        |
| 8.5       | Selection . . . . .                                                         | 112        |
| 8.6       | Ordered selection with replacement . . . . .                                | 112        |
| 8.7       | Ordered selection without replacement . . . . .                             | 113        |
| 8.8       | Unordered selection without replacement . . . . .                           | 113        |
| 8.9       | Unordered selection with replacement . . . . .                              | 114        |
| 8.10      | Permutations with repetition, and distributing objects into boxes . . . . . | 114        |
| 8.11      | Generating permutations and combinations . . . . .                          | 115        |
| <b>9</b>  | <b>Discrete Probability I</b>                                               | <b>117</b> |
| 9.1       | The nature of randomness . . . . .                                          | 117        |
| 9.2       | Probability . . . . .                                                       | 117        |
| 9.3       | Choice of sample space . . . . .                                            | 117        |
| 9.4       | Mutually exclusive events . . . . .                                         | 117        |
| 9.5       | Operations on events . . . . .                                              | 118        |
| 9.6       | Inclusion-Exclusion for probabilities . . . . .                             | 118        |
| 9.7       | Independent events . . . . .                                                | 118        |
| 9.8       | Conditional probability . . . . .                                           | 119        |
| 9.9       | Bayes’ Theorem . . . . .                                                    | 119        |
| <b>10</b> | <b>Discrete Probability II</b>                                              | <b>120</b> |
| 10.1      | Random variables . . . . .                                                  | 120        |
| 10.2      | Probability distributions . . . . .                                         | 120        |
| 10.3      | Expectation . . . . .                                                       | 120        |
| 10.4      | Median . . . . .                                                            | 121        |
| 10.5      | Mode . . . . .                                                              | 121        |
| 10.6      | Variance . . . . .                                                          | 121        |
| 10.7      | Uniform distribution . . . . .                                              | 122        |

|           |                                                 |            |
|-----------|-------------------------------------------------|------------|
| 10.8      | Binomial distribution . . . . .                 | 122        |
| 10.9      | Poisson distribution . . . . .                  | 122        |
| 10.10     | Geometric distribution . . . . .                | 123        |
| 10.11     | The coupon collector's problem . . . . .        | 123        |
| <b>11</b> | <b>Graph Theory I</b>                           | <b>125</b> |
| 11.1      | Basic definitions . . . . .                     | 125        |
| 11.2      | Types of graphs . . . . .                       | 125        |
| 11.3      | Graphs and relations . . . . .                  | 126        |
| 11.4      | Representing graphs . . . . .                   | 126        |
| 11.4.1    | Edge list . . . . .                             | 126        |
| 11.4.2    | Adjacency matrix . . . . .                      | 126        |
| 11.4.3    | Adjacency list . . . . .                        | 126        |
| 11.4.4    | Incidence matrix . . . . .                      | 126        |
| 11.5      | Graph isomorphism . . . . .                     | 127        |
| 11.6      | Subgraphs . . . . .                             | 127        |
| 11.7      | Some special graphs . . . . .                   | 127        |
| 11.8      | Degree . . . . .                                | 128        |
| 11.9      | Moving around . . . . .                         | 129        |
| 11.10     | Vertex and edge connectivity . . . . .          | 130        |
| 11.11     | Shortest paths . . . . .                        | 130        |
| 11.12     | Connectivity . . . . .                          | 131        |
| 11.13     | Bipartite graphs . . . . .                      | 131        |
| 11.14     | Euler tours . . . . .                           | 132        |
| <b>12</b> | <b>Graph Theory II</b>                          | <b>134</b> |
| 12.1      | Trees . . . . .                                 | 134        |
| 12.2      | Equivalent characterizations of trees . . . . . | 134        |
| 12.3      | Properties of trees . . . . .                   | 134        |
| 12.4      | Forests . . . . .                               | 137        |
| 12.5      | Spanning tree algorithms . . . . .              | 137        |
| 12.6      | Graph colouring . . . . .                       | 138        |
| 12.7      | Spanning trees . . . . .                        | 139        |
| 12.8      | Planarity . . . . .                             | 140        |
| 12.9      | Games on graphs (omega, advanced) . . . . .     | 142        |
| 12.9.1    | Shannon's Switching Game . . . . .              | 142        |
| 12.9.2    | Slither . . . . .                               | 143        |
|           | <b>Notation summary</b>                         | <b>144</b> |

# 1 Sets

## 1.1 Sets and elements

*Note* (Naive set theory). We work within *naive set theory*: a set is a primitive, undefined notion, and any well-specified collection of objects is assumed to form a set. This suffices for ordinary practice, but unrestricted set formation is inconsistent in general – *Russell’s paradox* considers  $R = \{x : x \notin x\}$ , for which  $R \in R \leftrightarrow R \notin R$ , a contradiction. Axiomatic set theory (e.g. ZFC) resolves this by restricting which collections may form sets.

**Definition 1.1** (Set). A set  $A$  is an unordered collection of distinct objects, called *elements* (or *members*) of  $A$ .

**Definition 1.2** (Membership).

$$a \in A \iff a \text{ is an element of } A, \quad a \notin A \iff \neg(a \in A).$$

**Definition 1.3** (Set equality).

$$A = B \iff \forall x (x \in A \leftrightarrow x \in B).$$

**Definition 1.4** (Empty set).  $\emptyset = \{\}$  is the unique set satisfying  $\forall x (x \notin \emptyset)$ .

**Definition 1.5** (Singleton, unordered pair).  $\{a\}$  has the sole element  $a$ ;  $\{a, b\}$  has elements  $a, b$ , with  $\{a, b\} = \{b, a\}$ .

**Definition 1.6** (Standard number sets).

$$\begin{array}{ll} \mathbb{N} = \{0, 1, 2, 3, \dots\} & \text{(natural numbers)} \\ \mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\} & \text{(integers)} \\ \mathbb{Z}^+ = \{1, 2, 3, \dots\} & \text{(positive integers)} \\ \mathbb{Q} = \{p/q : p \in \mathbb{Z}, q \in \mathbb{Z}, q \neq 0\} & \text{(rationals)} \\ \mathbb{R} & \text{(reals)} \\ \mathbb{C} & \text{(complex numbers)} \end{array}$$

*Note.* Rosen takes  $0 \in \mathbb{N}$ ; confirm this matches CN §1.4 before relying on it elsewhere.

**Definition 1.7** (Universal set). The *universal set*  $U$  is the set containing all objects under discussion in a given context; every set considered is implicitly  $A \subseteq U$ .

**Definition 1.8** (Venn diagram). A *Venn diagram* represents sets as regions enclosed by curves inside a rectangle representing  $U$ ; overlapping regions represent intersections, non-overlapping regions represent disjointness.

## 1.2 Specifying sets

**Definition 1.9** (Roster method).  $A = \{a_1, a_2, \dots, a_n\}$ . Order and repetition do not matter:  $\{1, 2, 3\} = \{3, 2, 1\} = \{1, 1, 2, 3\}$ .

**Definition 1.10** (Ellipsis notation).  $\{1, 2, 3, \dots\}$ , or bounded:  $\{1, 2, \dots, 100\}$ .

**Definition 1.11** (Set-builder notation).  $A = \{x \in U : P(x)\}$ , equivalently  $\{x \mid P(x)\}$ , where  $P$  is a predicate.

**Example 1.1.**  $\{x \in \mathbb{Z} : x^2 < 9\} = \{-2, -1, 0, 1, 2\}$ .

**Definition 1.12** (Interval notation,  $a, b \in \mathbb{R}$ ,  $a < b$ ).

$$[a, b] = \{x \in \mathbb{R} : a \leq x \leq b\}, \quad (a, b) = \{x \in \mathbb{R} : a < x < b\},$$

$$[a, b) = \{x \in \mathbb{R} : a \leq x < b\}, \quad (a, b] = \{x \in \mathbb{R} : a < x \leq b\}.$$



$[a, b)$ : filled dot at  $a$  (included), open circle at  $b$  (excluded)

### 1.3 Cardinality

**Definition 1.13** (Cardinality). For finite  $A$ ,  $|A|$  is the number of distinct elements of  $A$ .  $|\emptyset| = 0$ .

**Definition 1.14** (Finite, infinite set).  $A$  is *finite* if  $A = \emptyset$  or there exists  $n \in \mathbb{Z}^+$  and a bijection  $\{1, 2, \dots, n\} \rightarrow A$ ; then  $|A| = n$ . If no such  $n$  exists,  $A$  is *infinite*.

### 1.4 Sets of numbers

**Definition 1.15** (Signed subsets).

$$\mathbb{Z}^+ = \{1, 2, 3, \dots\}, \quad \mathbb{Z}^- = \{-1, -2, -3, \dots\}, \quad \mathbb{R}^+ = \{x \in \mathbb{R} : x > 0\}, \quad \mathbb{R}^- = \{x \in \mathbb{R} : x < 0\}.$$

*Note.*  $\mathbb{N} \subseteq \mathbb{Z} \subseteq \mathbb{Q} \subseteq \mathbb{R} \subseteq \mathbb{C}$ .

### 1.5 Sets of strings

**Definition 1.16** (Alphabet, string, length). An *alphabet*  $\Sigma$  is a finite nonempty set of symbols. A *string* over  $\Sigma$  is a finite sequence of symbols from  $\Sigma$ .  $|w|$  is the length of  $w$ ;  $\varepsilon$  is the *empty string*,  $|\varepsilon| = 0$ .

**Definition 1.17** ( $\Sigma^*$ ,  $\Sigma^n$ ).

$$\Sigma^n = \{w : w \text{ over } \Sigma, |w| = n\}, \quad \Sigma^* = \bigcup_{n=0}^{\infty} \Sigma^n.$$

### 1.6 Subsets and supersets

**Definition 1.18** (Subset, superset).

$$A \subseteq B \iff \forall x (x \in A \rightarrow x \in B), \quad A \supseteq B \iff B \subseteq A.$$

**Definition 1.19** (Proper subset).

$$A \subsetneq B \iff (A \subseteq B) \wedge (A \neq B).$$

**Theorem 1.1.** For every set  $A$ ,  $\emptyset \subseteq A$ .

*Proof.* By definition  $\emptyset \subseteq A$  iff  $\forall x (x \in \emptyset \rightarrow x \in A)$ . Since  $x \in \emptyset$  is always false, the conditional is vacuously true for every  $x$ , so the universally quantified statement holds. Hence  $\emptyset \subseteq A$ . ■

**Theorem 1.2.**  $\subseteq$  is reflexive ( $A \subseteq A$ ) and transitive ( $A \subseteq B \wedge B \subseteq C \Rightarrow A \subseteq C$ ).

## 1.7 Minimal, minimum, maximal, maximum

**Example 1.2** (Motivating example: cliques). Let  $B$  be a set of people, with some pairs acquainted. A *clique* is a subset  $A \subseteq B$  such that every two people in  $A$  know each other. We want to describe the “biggest” cliques – but “biggest” has two different meanings.

**Definition 1.20** (Maximum, maximal, w.r.t. a family  $\mathcal{F}$  of sets sharing a property). Let  $\mathcal{F}$  be a family of subsets of some set, all sharing a given property (e.g. “is a clique”), ordered by  $\subseteq$ .

- $M \in \mathcal{F}$  is a *maximum* if  $S \subseteq M$  for every  $S \in \mathcal{F}$  (largest possible size).
- $M \in \mathcal{F}$  is *maximal* if  $M$  is not a proper subset of any  $S \in \mathcal{F}$  (cannot be enlarged while keeping the property).

**Theorem 1.3.** *Every maximum is maximal.*

*Proof.* If  $M$  is a maximum, then  $S \subseteq M$  for all  $S \in \mathcal{F}$ , so no  $S \in \mathcal{F}$  can have  $M \subsetneq S$  (that would force  $M \subsetneq S \subseteq M$ , impossible). Hence  $M$  is maximal. ■

*Note.* The converse fails in general: a maximal clique need not be a maximum clique – it just cannot be enlarged, while some other, larger clique may exist elsewhere in  $B$ .

**Definition 1.21** (Minimum, minimal – dual definitions). Dually, for  $\mathcal{F}$  ordered by  $\subseteq$ :

- $m \in \mathcal{F}$  is a *minimum* if  $m \subseteq S$  for every  $S \in \mathcal{F}$ .
- $m \in \mathcal{F}$  is *minimal* if  $m$  is not a proper superset of any  $S \in \mathcal{F}$  (removing anything from  $m$  destroys the property).

Every minimum is minimal; the converse fails in general.

*Note* (Why the distinction matters here but not for  $\mathbb{R}$ ). For real numbers,  $\leq$  is a *total order*: any two reals are comparable, so “cannot be increased” and “is the largest” coincide, and *maximum*/*maximal* are used interchangeably. But  $\subseteq$  is only a *partial order*: two sets  $A, B$  can be *incomparable* ( $A \not\subseteq B$  and  $B \not\subseteq A$ ). This is exactly why a family can have several maximal elements without any single maximum.

## 1.8 Counting all subsets

**Theorem 1.4.** *For finite  $B$  with  $|B| = n$ , the number of subsets of  $B$  is  $2^n$ .*

*Proof.* A subset is determined by choosing, independently for each of the  $n$  elements, whether to include it (2 options per element). By the product rule, the number of independent choice-sequences is  $\underbrace{2 \times 2 \times \cdots \times 2}_n = 2^n$ , and each choice-sequence corresponds to exactly one subset. ■

## 1.9 The power set of a set

**Definition 1.22** (Power set).  $\mathcal{P}(A) = \{S : S \subseteq A\}$ .

**Theorem 1.5.**  $|A| = n \Rightarrow |\mathcal{P}(A)| = 2^n$ .

*Proof.* Immediate from §1.8:  $\mathcal{P}(A)$  is exactly the set of subsets of  $A$ , and there are  $2^n$  of them. ■

*Note.* Holds even for  $A = \emptyset$ :  $n = 0$  gives  $2^0 = 1$ , consistent with  $\emptyset$  having exactly one subset, itself.  $\mathcal{P}(A)$  is also written  $2^A$ .

**Example 1.3.**  $\emptyset$  has exactly one subset, itself, so  $\mathcal{P}(\emptyset) = \{\emptyset\}$ . The set  $\{\emptyset\}$  has exactly two subsets,  $\emptyset$  and  $\{\emptyset\}$  itself, so  $\mathcal{P}(\{\emptyset\}) = \{\emptyset, \{\emptyset\}\}$ .



## 1.10 Counting subsets by size using binomial coefficients

**Definition 1.23** (Binomial coefficient). For  $0 \leq k \leq n$ ,  $\binom{n}{k}$  (“ $n$  choose  $k$ ”) is the number of  $k$ -element subsets of an  $n$ -element set.

**Theorem 1.6** (Sum over  $k$ ).

$$\sum_{k=0}^n \binom{n}{k} = 2^n.$$

*Proof.* The binomial coefficients  $\binom{n}{0}, \binom{n}{1}, \dots, \binom{n}{n}$  partition the subsets of an  $n$ -set by size, so they sum to the total subset count,  $2^n$  (§1.8). ■

**Theorem 1.7** (Symmetry).

$$\binom{n}{k} = \binom{n}{n-k}.$$

*Proof.* Choosing  $k$  elements to include determines which  $n-k$  elements are excluded, and vice versa; this is a bijection between  $k$ -subsets and  $(n-k)$ -subsets. ■

**Theorem 1.8** (Closed form).

$$\binom{n}{k} = \frac{n(n-1)(n-2) \cdots (n-k+1)}{k!} = \frac{n!}{k!(n-k)!}.$$

*Proof.* Count *ordered* choices of  $k$  distinct elements from  $n$ :  $n$  options for the first,  $n-1$  for the second,  $\dots$ ,  $n-k+1$  for the  $k$ -th, giving  $n(n-1) \cdots (n-k+1) = n!/(n-k)!$  ordered sequences. Each  $k$ -subset is counted once for every ordering of its  $k$  elements, i.e.  $k!$  times, so

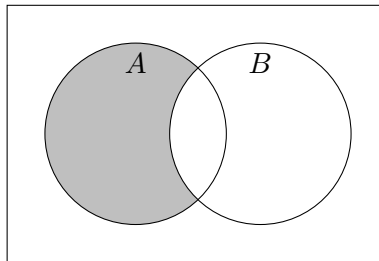
$$\binom{n}{k} = \frac{1}{k!} \cdot \frac{n!}{(n-k)!} = \frac{n!}{k!(n-k)!}.$$

**Example 1.4.**  $\binom{n}{0} = \binom{n}{n} = 1$  (only one way to choose nothing, or everything);  $\binom{n}{1} = \binom{n}{n-1} = n$ .

## 1.11 Complement and set difference

**Definition 1.24** (Set difference, complement w.r.t.  $U$ ).

$$A - B = \{x : x \in A \wedge x \notin B\}, \quad \overline{A} = U - A.$$



$A \setminus B$  (shaded)

**Theorem 1.9** (De Morgan's laws).

$$\overline{A \cup B} = \overline{A} \cap \overline{B}, \quad \overline{A \cap B} = \overline{A} \cup \overline{B}.$$

**Example 1.5.** Prove  $\overline{A \cap B} = \overline{A} \cup \overline{B}$  using set-builder notation and logical equivalences.

$$\begin{aligned} \overline{A \cap B} &= \{x : x \notin A \cap B\} \\ &= \{x : \neg(x \in A \cap B)\} \\ &= \{x : \neg(x \in A \wedge x \in B)\} \\ &= \{x : \neg(x \in A) \vee \neg(x \in B)\} && \text{(De Morgan's law, logic)} \\ &= \{x : x \notin A \vee x \notin B\} \\ &= \{x : x \in \overline{A} \vee x \in \overline{B}\} \\ &= \overline{A} \cup \overline{B}. \end{aligned}$$

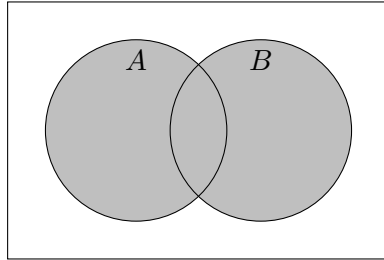
**Example 1.6.** Let  $A, B, C$  be sets. Prove  $\overline{A \cup (B \cap C)} = (\overline{C} \cup \overline{B}) \cap \overline{A}$  by chaining previously established identities.

$$\begin{aligned} \overline{A \cup (B \cap C)} &= \overline{A} \cap \overline{(B \cap C)} && \text{(1st De Morgan's law)} \\ &= \overline{A} \cap (\overline{B} \cup \overline{C}) && \text{(2nd De Morgan's law)} \\ &= (\overline{B} \cup \overline{C}) \cap \overline{A} && \text{(commutativity of } \cap) \\ &= (\overline{C} \cup \overline{B}) \cap \overline{A}. && \text{(commutativity of } \cup) \end{aligned}$$

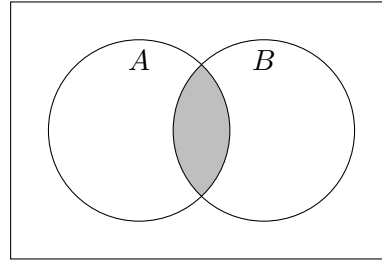
## 1.12 Union and intersection

**Definition 1.25** (Union, intersection).

$$A \cup B = \{x : x \in A \vee x \in B\}, \quad A \cap B = \{x : x \in A \wedge x \in B\}.$$



$A \cup B$  (shaded)



$A \cap B$  (shaded)

**Definition 1.26** (Disjoint).  $A \cap B = \emptyset$ .

**Theorem 1.10** (Inclusion-exclusion).  $|A \cup B| = |A| + |B| - |A \cap B|$ .

**Example 1.7.** Prove  $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$  by the element argument.

( $\subseteq$ ) Let  $x \in A \cap (B \cup C)$ . Then  $x \in A$  and  $(x \in B \text{ or } x \in C)$ . If  $x \in B$  then  $x \in A \cap B$ ; if  $x \in C$  then  $x \in A \cap C$ . Either way  $x \in (A \cap B) \cup (A \cap C)$ .

( $\supseteq$ ) Let  $x \in (A \cap B) \cup (A \cap C)$ . Then  $x \in A \cap B$  or  $x \in A \cap C$ ; either way  $x \in A$  and  $(x \in B \text{ or } x \in C)$ , so  $x \in A \cap (B \cup C)$ .

Both directions hold, so the sets are equal.

*Note* (Methods for proving set identities). (i) *Element argument (double inclusion)*: show  $\text{LHS} \subseteq \text{RHS}$  and  $\text{RHS} \subseteq \text{LHS}$ , as above.

(ii) *Membership table*: tabulate membership (0/1) of an arbitrary  $x$  in each named set for every combination, and compare the LHS/RHS columns.

(iii) *Using established identities*: rewrite one side into the other via a chain of already-proven identities (the Set Identities table), analogous to proving logical equivalences.

**Definition 1.27** (Family of sets). A *family of sets* indexed by  $I$  is a collection  $\{A_i\}_{i \in I}$ , one set  $A_i$  for each  $i \in I$ ;  $I$  is the *index set*.

**Definition 1.28** (Generalized union and intersection, finite).

$$\bigcup_{i=1}^n A_i = A_1 \cup A_2 \cup \cdots \cup A_n = \{x : \exists i \in \{1, \dots, n\}, x \in A_i\},$$

$$\bigcap_{i=1}^n A_i = A_1 \cap A_2 \cap \cdots \cap A_n = \{x : \forall i \in \{1, \dots, n\}, x \in A_i\}.$$

**Definition 1.29** (Generalized union and intersection, arbitrary index set). For a family  $\{A_i\}_{i \in I}$ ,

$$\bigcup_{i \in I} A_i = \{x : \exists i \in I, x \in A_i\}, \quad \bigcap_{i \in I} A_i = \{x : \forall i \in I, x \in A_i\}.$$

When  $I = \mathbb{Z}^+$  one also writes  $\bigcup_{i=1}^{\infty} A_i, \bigcap_{i=1}^{\infty} A_i$ .

**Theorem 1.11** (Set identities). For sets  $A, B, C$  with universal set  $U$ :

$$A \cup \emptyset = A, \quad A \cap U = A \quad (\text{Identity laws})$$

$$A \cup U = U, \quad A \cap \emptyset = \emptyset \quad (\text{Domination laws})$$

$$A \cup A = A, \quad A \cap A = A \quad (\text{Idempotent laws})$$

$$\overline{\overline{A}} = A \quad (\text{Complementation law})$$

$$A \cup B = B \cup A, \quad A \cap B = B \cap A \quad (\text{Commutative laws})$$

$$(A \cup B) \cup C = A \cup (B \cup C), \quad (A \cap B) \cap C = A \cap (B \cap C) \quad (\text{Associative laws})$$

$$A \cup (B \cap C) = (A \cup B) \cap (A \cup C) \quad (\text{Distributive law})$$

$$A \cap (B \cup C) = (A \cap B) \cup (A \cap C) \quad (\text{Distributive law})$$

$$\overline{A \cup B} = \overline{A} \cap \overline{B}, \quad \overline{A \cap B} = \overline{A} \cup \overline{B} \quad (\text{De Morgan's laws})$$

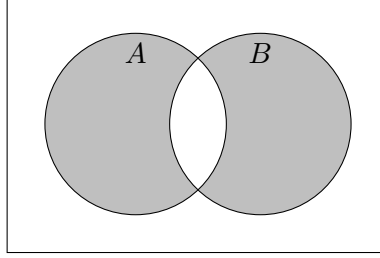
$$A \cup (A \cap B) = A, \quad A \cap (A \cup B) = A \quad (\text{Absorption laws})$$

$$A \cup \overline{A} = U, \quad A \cap \overline{A} = \emptyset \quad (\text{Complement laws})$$

**Definition 1.30** (Bit-string representation). For finite  $U = \{a_1, \dots, a_n\}$  with a fixed ordering,  $A \subseteq U$  corresponds to the bit string  $b_1 b_2 \cdots b_n$  where  $b_i = 1$  iff  $a_i \in A$ . Then  $A \cup B \leftrightarrow$  bitwise OR,  $A \cap B \leftrightarrow$  bitwise AND,  $\overline{A} \leftrightarrow$  bitwise NOT.

### 1.13 Symmetric difference

**Definition 1.31** (Symmetric difference).  $A \triangle B = \{x : x \in A \text{ or } x \in B \text{ but not both}\}$  (exclusive or).



$A \triangle B$  (shaded)

**Theorem 1.12** (Equivalent forms).

$$A \triangle B = (A \setminus B) \cup (B \setminus A) = (A \cap \bar{B}) \cup (\bar{A} \cap B) = (A \cup B) \setminus (A \cap B).$$

*Note.*  $A \triangle A = \emptyset$ , and this is the only case where a symmetric difference is empty.

**Theorem 1.13.**  $A = B \iff A \triangle B = \emptyset$ .

*Proof.*

$$A = B \iff A \subseteq B \text{ and } B \subseteq A \iff A \setminus B = \emptyset \text{ and } B \setminus A = \emptyset \iff (A \setminus B) \cup (B \setminus A) = \emptyset \iff A \triangle B = \emptyset.$$

■

**Theorem 1.14.**  $\overline{A \triangle B} = \bar{A} \triangle \bar{B}$ .

*Proof.*

$$\overline{A \triangle B} = \overline{(A \cap \bar{B}) \cup (\bar{A} \cap B)} = \overline{(A \cap \bar{B})} \cap \overline{(\bar{A} \cap B)} = (\bar{A} \cup B) \cap (A \cup \bar{B}).$$

Expanding and simplifying (using distributivity and  $A \cap \bar{A} = \emptyset$ ) gives  $(A \cap \bar{B}) \cup (\bar{A} \cap B) = A \triangle B$ , so the complement identity holds by the same characterization applied to  $\bar{A}, \bar{B}$ :  $\bar{A} \triangle \bar{B} = (\bar{\bar{A}} \cap B) \cup (\bar{A} \cap \bar{\bar{B}}) = A \triangle B = \overline{A \triangle B}$ . ■

### 1.14 Cartesian product

**Definition 1.32** (Ordered pair).  $(a, b)$  collects  $a, b$  in order;  $(a, b) = (c, d) \iff a = c \wedge b = d$ .

**Definition 1.33** (Cartesian product).  $A \times B = \{(a, b) : a \in A, b \in B\}$ .

**Theorem 1.15.**  $|A \times B| = |A| \cdot |B|$  for finite  $A, B$ .

*Proof.* There are  $|A|$  choices for the first coordinate, and independently  $|B|$  choices for the second, so by the product rule there are  $|A| \cdot |B|$  pairs. ■

**Definition 1.34** ( $n$ -tuple, generalized Cartesian product). For sets  $A_1, \dots, A_n$ :

$$A_1 \times A_2 \times \dots \times A_n = \{(a_1, \dots, a_n) : a_i \in A_i, i = 1, \dots, n\},$$

with equality of  $n$ -tuples componentwise:  $(a_1, \dots, a_n) = (b_1, \dots, b_n) \iff a_i = b_i \forall i$ . If finite,  $|A_1 \times \dots \times A_n| = |A_1| \cdot \dots \cdot |A_n|$ .

**Definition 1.35** (Power notation). For a set  $A$  and  $n \in \mathbb{N}$ ,  $A^n = A \times A \times \cdots \times A$  ( $n$  factors);  $A^0 = \{\emptyset\text{-tuple}\}$  is conventionally treated as a single-element set,  $A^1 = A$ . If  $A$  is finite,  $|A^n| = |A|^n$ .

*Note.* When  $A$  is an alphabet,  $A^n$  is identified with strings of length  $n$  (§1.5): e.g.  $\{0,1\}^3 = \{000, 001, \dots, 111\}$ . Also  $\mathbb{R}^n = \mathbb{R} \times \cdots \times \mathbb{R}$  is the set of coordinates in  $n$ -dimensional space.

**Theorem 1.16.**  $|A_1 \times \cdots \times A_n| = |A_1| \cdot |A_2| \cdots |A_n|$  for finite  $A_i$ .

## 1.15 Partitions

**Definition 1.36** (Partition). A *partition* of  $A$  is a set of pairwise disjoint, nonempty subsets of  $A$  (called *parts*, *blocks*, or *cells*) whose union is  $A$ .

*Note* (Equivalent characterization). A collection of nonempty subsets of  $A$  is a partition of  $A$  iff every element of  $A$  belongs to *exactly one* part.

**Example 1.8.** For  $A = \{a, b, c\}$ :  $\{\{a, c\}, \{b\}\}$  and  $\{\{a\}, \{b\}, \{c\}\}$  are partitions;  $\{\{a, b\}, \{c\}, \emptyset\}$  fails (empty part),  $\{\{a, b\}, \{b, c\}\}$  fails (not disjoint),  $\{\{a\}, \{b\}\}$  fails (union  $\neq A$ , misses  $c$ ).

**Definition 1.37** (Coarsest and finest partitions). For any  $A$ : the *coarsest* partition is  $\{A\}$  (one part, everything together); the *finest* partition is  $\{\{a\} : a \in A\}$  (one singleton part per element). If  $|A| = 1$  these coincide; otherwise they are the two extremes, with every other partition of  $A$  intermediate between them.

*Note* (Connection to equivalence relations (Rosen §9.5, previewed here)). Partitions are closely tied to *equivalence relations* (reflexive, symmetric, transitive relations – CN §2.16, Rosen §9.5): every equivalence relation  $R$  on  $A$  partitions  $A$  into *equivalence classes*  $[a]_R = \{x \in A : x R a\}$ , and conversely every partition of  $A$  arises this way from exactly one equivalence relation (namely,  $x R y \iff x, y$  lie in the same part). We revisit this correspondence formally once binary relations are covered.

## 1.16 Cardinality of infinite sets (Rosen §2.5 – extension, beyond CN syllabus)

**Definition 1.38** (Same cardinality).  $A, B$  have the *same cardinality*,  $|A| = |B|$ , iff there exists a bijection  $A \rightarrow B$ .

**Definition 1.39** (Countable).  $A$  is *countable* if  $A$  is finite, or  $|A| = |\mathbb{N}|$  (a bijection  $\mathbb{N} \rightarrow A$  exists –  $A$  can be listed  $a_0, a_1, a_2, \dots$ ). Otherwise  $A$  is *uncountable*.

**Theorem 1.17.**  $\mathbb{Z}$  is countable.

*Proof.*  $f : \mathbb{N} \rightarrow \mathbb{Z}$ ,  $f(n) = n/2$  if  $n$  even,  $-(n+1)/2$  if  $n$  odd, is a bijection:  $0, 1, 2, 3, 4, \dots \mapsto 0, -1, 1, -2, 2, \dots$ , hitting every integer exactly once. ■

**Theorem 1.18** (Countability of  $\mathbb{Q}$ , Cantor's diagonal enumeration).  $\mathbb{Q}$  is countable.

*Proof sketch.* Arrange positive rationals  $p/q$  in a grid, row  $p$ , column  $q$ . Traverse the grid along successive diagonals ( $p+q = 2, 3, 4, \dots$ ), skipping any  $p/q$  already listed in lowest terms. This lists every positive rational exactly once; interleave with negatives and 0 as in the  $\mathbb{Z}$  proof above. ■

**Theorem 1.19** (Uncountability of  $\mathbb{R}$ , Cantor's diagonal argument – harder).  $\mathbb{R}$  (even just  $(0,1)$ ) is uncountable.

*Proof.* Suppose, for contradiction,  $(0, 1)$  were countable: list all its elements as  $r_1, r_2, r_3, \dots$ , each written as an infinite decimal  $r_i = 0.d_{i1}d_{i2}d_{i3} \dots$ .

Construct  $s = 0.s_1s_2s_3 \dots$  by setting  $s_i = 5$  if  $d_{ii} \neq 5$ , else  $s_i = 6$  (any rule avoiding both  $d_{ii}$  and decimal-expansion ambiguity works).

Then  $s \in (0, 1)$ , but  $s \neq r_i$  for every  $i$  (they differ in digit  $i$ ), so  $s$  is missing from the list – contradicting that the list was exhaustive.

So no such list exists:  $(0, 1)$  is uncountable. ■

*Note (Consequence).* Since  $\mathbb{Q}$  is countable but  $\mathbb{R}$  is not,  $|\mathbb{Q}| \neq |\mathbb{R}|$  – there are strictly more reals than rationals, despite both being infinite. This is the first proof most students see that not all infinities are the same size.

## 2 Functions

### 2.1 Definitions

**Definition 2.1** (Function). A function  $f$  consists of: a set called the *domain*; a set called the *codomain*; and a rule assigning, to each  $x$  in the domain, a unique value  $f(x)$  in the codomain.

$$\begin{array}{ccc} 1 & \xrightarrow{f} & p \\ 2 & \xrightarrow{f} & q \\ 3 & \xrightarrow{f} & r \end{array}$$

A function  $f : \{1, 2, 3\} \rightarrow \{p, q, r\}$ , each argument pointing to its unique value

**Definition 2.2** (Rosen’s phrasing). Let  $A, B$  be nonempty sets. A *function*  $f$  from  $A$  to  $B$  is an assignment of exactly one element of  $B$  to each element of  $A$ . We write  $f(a) = b$  if  $b$  is the element assigned to  $a$ , and  $f : A \rightarrow B$ .

*Note.*  $x$  is the *argument* (parameter, input);  $f(x)$  is the *value* (output).

**Definition 2.3** (Equality of functions).  $f = g$  iff  $\text{dom}(f) = \text{dom}(g)$ , their codomains agree, and  $f(x) = g(x)$  for every  $x \in \text{dom}(f)$ .

**Example 2.1.** Let  $\text{CubeSum4}_{\mathbb{Z}}(w, x, y, z) = w^3 + x^3 + y^3 + z^3$  on  $\text{dom} = \mathbb{Z}^4$ , and  $\text{CubeSum4}_{\mathbb{R}}$  the same rule on  $\text{dom} = \mathbb{R}^4$ . Despite the identical rule, these are *different* functions, since equality of functions requires equal domains.

#### 2.1.1 Domain

**Definition 2.4** (Domain). The *domain* of  $f$ , written  $\text{dom}(f)$ , is exactly the set of valid arguments:  $f(x)$  must be defined for every  $x \in \text{dom}(f)$ , and undefined for every  $x \notin \text{dom}(f)$ .

*Note* (The domain as a promise). Specifying  $\text{dom}(f)$  commits  $f$  to working correctly on *every* member of it – no more, no less. A larger domain is a bigger promise (more work to guarantee); a smaller domain that still covers the intended use case is often preferable.

#### 2.1.2 Codomain & co.

**Definition 2.5** (Codomain). The *codomain* of  $f$  is a set containing every possible value  $f(x)$  for  $x \in \text{dom}(f)$ , but is allowed to be a *loose* superset – it may contain elements never actually attained.

**Definition 2.6** (Image). The *image* of  $f$  is the exact set of attained values,  $\{f(x) : x \in \text{dom}(f)\}$  – a subset of the codomain, not necessarily proper.

*Note.* CN deliberately avoids the word “range”, since usage is inconsistent (sometimes the image, sometimes the codomain). Rosen uses “range” for what CN calls the image.

*Note* (Why allow looseness at all?). The image is often hard, or even provably impossible, to compute exactly (e.g. a function whose value depends on uncomputable properties of programs), while a simple superset codomain is easy to state. So we specify a codomain we’re confident covers every value, rather than insisting on the exact image.

**Definition 2.7** (Image of a set, Rosen). For  $f : A \rightarrow B$  and  $S \subseteq A$ , the *image of  $S$  under  $f$*  is

$$f(S) = \{f(s) : s \in S\} \subseteq B.$$

(The image of  $f$  itself, above, is the special case  $f(A)$ .)

**Definition 2.8** (Onto / surjective, preview). If the codomain equals the image,  $f$  is *onto* (*surjective*), a *surjection*. (Formal treatment in §2.7.)

### 2.1.3 Rule

**Definition 2.9** (Rule). The *rule* of  $f$  specifies, for each  $x \in \text{dom}(f)$ , what  $f(x)$  is – it need not specify *how* to compute it (no algorithm required).

**Definition 2.10** (Graph of a function). The *graph* of  $f$  is  $\{(x, f(x)) : x \in \text{dom}(f)\}$ , the set of all argument–value pairs.

**Example 2.2.** A function on the finite domain  $\{1, 2, 3, 4\}$  can be given as a finite graph, e.g.  $\{(1, a), (2, a), (3, b), (4, c)\}$  (its image is  $\{a, b, c\}$ , a proper subset of a larger codomain). A function on an infinite domain typically needs a rule rather than a listed graph: the squaring function on  $\mathbb{Z}$  has graph  $\{(x, x^2) : x \in \mathbb{Z}\}$ .

## 2.2 Functions in computer science

### 2.2.1 Functions from analysis

*Note.* In software development, *analysis* produces a precise specification of *what* must be done – often expressible as a mathematical function. *Design* then produces an *algorithm* for computing it (*how*); *implementation* turns that algorithm into code.

### 2.2.2 Functions in mathematics and programming

*Note.* Two senses of “function” coexist: the *mathematical* sense (domain, codomain, rule – no algorithm implied), and the *programming* sense (a named block of code that computes something). Unless flagged otherwise (“Python function”, “algorithmic function”), we use the mathematical sense throughout.

## 2.3 Notation

**Definition 2.11** (Function declaration).

$$f : \text{domain} \rightarrow \text{codomain}, \quad f(x) = \text{expression in } x.$$

Equivalently, the rule may be given with a *mapping arrow*:  $x \mapsto \text{expression in } x$  (note:  $\mapsto$  for the rule,  $\rightarrow$  for domain/codomain – do not conflate them).

**Example 2.3.**  $f : \mathbb{R} \rightarrow \mathbb{R}$ ,  $f(x) = x^2$ , equivalently  $f : \mathbb{R} \rightarrow \mathbb{R}$ ,  $x \mapsto x^2$ . The codomain need not be the tightest possible:  $\mathbb{R}_0^+$  (the image) would also be valid, but a codomain like  $\mathbb{R} \cup \{0, -\sqrt{2}, -42\}$ , while technically valid, is needlessly cluttered.



## 2.4 Some special functions

**Definition 2.12** (Empty function).

$$\emptyset : \emptyset \rightarrow \emptyset$$

Empty domain, empty codomain, vacuous rule.

**Definition 2.13** (Identity function). For a set  $A$ :

$$i_A : A \rightarrow A, \quad i_A(x) = x.$$

**Definition 2.14** (Indicator (characteristic) function). For  $A \subseteq D$ :

$$\chi_A : D \rightarrow \{0, 1\}, \quad \chi_A(x) = \begin{cases} 1, & x \in A; \\ 0, & x \notin A. \end{cases}$$

Depends on  $D$ , not just  $A$ : different domains give different indicator functions.

**Example 2.4.**

$$\chi_{A \cup B}(x) = \max\{\chi_A(x), \chi_B(x)\} \quad \text{for all } x.$$

**Definition 2.15** (Constant function). For a domain  $D$  and object  $a$ :

$$c_a : D \rightarrow \{a\}, \quad c_a(x) = a.$$

**Definition 2.16** (Floor and ceiling, Rosen). For  $x \in \mathbb{R}$ :

$$\lfloor x \rfloor = \max\{n \in \mathbb{Z} : n \leq x\}, \quad \lceil x \rceil = \min\{n \in \mathbb{Z} : n \geq x\}.$$

**Definition 2.17** (Partial function). A *partial function*  $f$  from  $A$  to  $B$  assigns *at most one* element of  $B$  to each element of  $A$ ; the subset of  $A$  where  $f$  is actually defined is  $\text{dom}(f)$  (possibly a proper subset of  $A$ ).

*Note* (Contrast with CN's §2.1.1). CN's definition of a function already *is* what Rosen calls a *total* function:  $\text{dom}(f)$  equals the declared domain exactly, with no gaps. Rosen's "partial function from  $A$  to  $B$ " allows  $\text{dom}(f) \subsetneq A$  – a total function on the smaller set  $\text{dom}(f) \subseteq A$ .

**Example 2.5.**  $f : \mathbb{R} \rightarrow \mathbb{R}$ ,  $f(x) = 1/x$  is a partial function on  $\mathbb{R}$  (undefined at 0); as a *total* function it is  $f : \mathbb{R} \setminus \{0\} \rightarrow \mathbb{R}$  – same rule, different declared domain (CN's §2.1.1 view of the domain as an exact promise).

*Note* (Where partiality actually shows up – harder). Partial functions are essential in computability theory: a program  $P$  computing a function of its input may not halt on every input. The function  $f_P(x) = \text{"output of } P \text{ on input } x, \text{ if } P \text{ halts"}$  is a genuine partial function –  $\text{dom}(f_P)$  is exactly the set of inputs on which  $P$  halts, and this set can itself be uncomputable (the Halting Problem: no algorithm decides, for arbitrary  $P, x$ , whether  $x \in \text{dom}(f_P)$ ).

## 2.5 Functions with multiple arguments and values

**Definition 2.18** (Multiple arguments via Cartesian product). A function of two arguments  $f(x, y)$ ,  $x \in X, y \in Y$ , value in  $C$ , is formally  $f : X \times Y \rightarrow C$  – a function of *one* argument, the ordered pair  $(x, y)$ , with  $f(x, y)$  shorthand for  $f((x, y))$ . Extends to any number of arguments via  $X_1 \times \cdots \times X_n$ .

*Note* (Notation styles).  $f(x, y)$  is *prefix* notation (name before arguments). *Infix* (name between, e.g.  $x + y$ ) and *postfix* (name after) also occur.

**Definition 2.19** (Multiple (tuple-valued) outputs). A function returning a pair is  $f : X \rightarrow Y \times Z$ ; its value is a single element  $(y, z) \in Y \times Z$ , viewable as "two outputs." Extends to any number of returned values.

## 2.6 Restrictions

**Definition 2.20** (Restriction). For  $f : A \rightarrow B$  and  $X \subseteq A$ , the *restriction* of  $f$  to  $X$ , written  $f|_X$  (or  $f \upharpoonright_X$ ), is

$$f|_X : X \rightarrow B, \quad f|_X(x) = f(x) \text{ for all } x \in X.$$

*Note* (Why restrict?). A smaller domain means: a smaller stored representation (fewer ordered pairs to list); a smaller “promise” to keep (less testing if implemented); sometimes simpler analysis; and sometimes *stronger properties* unavailable to the original function.

**Example 2.6.** Let  $f : \mathbb{R} \rightarrow \mathbb{R}$ ,  $f(x) = x^2$ . Its restriction  $f|_{\mathbb{R}_0^+} : \mathbb{R}_0^+ \rightarrow \mathbb{R}$  is strictly increasing throughout its domain (unlike  $f$ , which decreases on  $\mathbb{R}_0^-$ ), and every  $y$  in its image has a *unique* preimage  $\sqrt{y}$  – whereas under  $f$  itself, each nonzero  $y$  in the image has two preimages,  $\pm\sqrt{y}$ . (This makes  $f|_{\mathbb{R}_0^+}$  invertible, unlike  $f$  – see §2.8.)

## 2.7 Injections, surjections, bijections

**Definition 2.21** (Injective / one-to-one).  $f : A \rightarrow B$  is *injective* (an *injection*, *one-to-one*) if distinct arguments always give distinct values:

$$\forall x_1, x_2 \in A, \quad x_1 \neq x_2 \implies f(x_1) \neq f(x_2).$$

*Note* (Rosen’s contrapositive phrasing). Equivalently (contrapositive):

$$\forall x_1, x_2 \in A, \quad f(x_1) = f(x_2) \implies x_1 = x_2.$$

This is the form most useful in proofs: assume  $f(x_1) = f(x_2)$  and derive  $x_1 = x_2$ .

*Note* (Injections preserve information). Every  $y$  in the image of an injection has a *unique* preimage, so knowing  $y$  determines  $x$  uniquely. A non-injective function is *lossy* (some inputs become indistinguishable from their output); an injective one is *lossless*.

**Definition 2.22** (Surjective / onto).  $f : A \rightarrow B$  is *surjective* (a *surjection*, *onto*) if

$$\forall y \in B, \exists x \in A \text{ such that } f(x) = y,$$

i.e. the image equals the codomain.

**Definition 2.23** (Bijective).  $f$  is *bijective* (a *bijection*, *one-to-one correspondence*) if it is both injective and surjective.

*Note* (Permutation). A bijection  $f : A \rightarrow A$  (domain = codomain) is also called a *permutation* of  $A$  – usually reserved for finite  $A$ .

**Theorem 2.1** (Finite equal-size domain/codomain). Let  $f : A \rightarrow B$  with  $A, B$  finite and  $|A| = |B|$ . Then  $f$  is injective  $\iff f$  is surjective  $\iff f$  is bijective.

*Proof by induction on  $n = |A| = |B|$ .* We show:  $f$  injective  $\implies f$  bijective. (The surjective case is symmetric, swapping the roles of  $A$  and  $B$  throughout.)

**Base case** ( $n = 0$ ):  $A = B = \emptyset$ . The unique function  $\emptyset \rightarrow \emptyset$  is vacuously injective and vacuously surjective, hence bijective.

**Inductive step:** Suppose the claim holds for all injective functions between finite sets of size  $n - 1$ . Let  $f : A \rightarrow B$  be injective with  $|A| = |B| = n \geq 1$ .

Pick any  $a \in A$  and let  $b = f(a)$ . Set  $A' = A \setminus \{a\}$ ,  $B' = B \setminus \{b\}$ , and let  $f' = f|_{A'}$  be the restriction of  $f$  to  $A'$ .

- $f'$  maps into  $B'$ : for  $x \in A'$  we have  $x \neq a$ , so by injectivity of  $f$ ,  $f(x) \neq f(a) = b$ ; hence  $f(x) \in B \setminus \{b\} = B'$ .
- $f'$  is injective: it is a restriction of the injective function  $f$ .
- $|A'| = |B'| = n - 1$ .

By the inductive hypothesis,  $f' : A' \rightarrow B'$  is bijective, in particular surjective onto  $B'$ .

Now take any  $y \in B$ . Either  $y = b$ , in which case  $y = f(a)$ ; or  $y \in B'$ , in which case surjectivity of  $f'$  gives some  $x \in A'$  with  $f(x) = f'(x) = y$ . Either way,  $y$  has a preimage under  $f$ . So  $f$  is surjective onto  $B$ , and being injective by hypothesis,  $f$  is bijective.

By induction, the claim holds for all  $n \geq 0$ . ■

*Note.* This theorem requires *finite*  $A, B$  of equal size – it fails for infinite sets (e.g.  $f : \mathbb{N} \rightarrow \mathbb{N}$ ,  $f(x) = 2x$  is injective but not surjective, despite  $|\mathbb{N}| = |\mathbb{N}|$  in the infinite-cardinality sense).

*Note* (Proof strategies for injectivity and surjectivity). To prove  $f : A \rightarrow B$  is **injective**: let  $x_1, x_2 \in A$  be arbitrary with  $f(x_1) = f(x_2)$ , and show  $x_1 = x_2$  follows (direct proof of the contrapositive form).

To prove  $f$  is **not injective**: exhibit specific  $x_1 \neq x_2 \in A$  with  $f(x_1) = f(x_2)$  (one counterexample suffices).

To prove  $f$  is **surjective**: let  $y \in B$  be arbitrary, and *construct* (or show the existence of) some  $x \in A$  with  $f(x) = y$  – typically by solving  $f(x) = y$  for  $x$  in terms of  $y$  and checking  $x \in A$ .

To prove  $f$  is **not surjective**: exhibit a specific  $y \in B$  for which no  $x \in A$  satisfies  $f(x) = y$  (one counterexample suffices).

## 2.8 Inverse functions

*Note* (Why uniqueness can fail). Let  $f : A \rightarrow B$ . For a given  $y \in B$ , trying to go “backwards” can fail in two distinct ways:

- *Non-uniqueness*: there exist  $x_1 \neq x_2 \in A$  with  $f(x_1) = f(x_2) = y$ .
- *Non-existence*:  $y$  lies in the codomain but not the image, so no  $x \in A$  satisfies  $f(x) = y$ .

Both failures are ruled out exactly when  $f$  is, respectively, injective and surjective.

**Definition 2.24** (Inverse function). If  $f : A \rightarrow B$  is such that every  $y \in B$  has a *unique*  $x \in A$  with  $f(x) = y$ , the *inverse function*  $f^{-1} : B \rightarrow A$  is defined by

$$f^{-1}(y) = \text{the unique } x \in A \text{ such that } f(x) = y.$$

Equivalently, as a set of ordered pairs (the graph of  $f$ , reversed):

$$f^{-1} = \{(y, x) : (x, y) \in f\} = \{(f(x), x) : x \in A\}.$$

**Theorem 2.2.**  $f : A \rightarrow B$  has an inverse function if and only if  $f$  is a bijection.

*Proof.* ( $\Rightarrow$ ) If  $f^{-1}$  exists, every  $y \in B$  has *some* preimage (so  $f$  is surjective) and *at most one* preimage (so  $f$  is injective, since two preimages of the same  $y$  would break the “unique  $x$ ” requirement in the definition of  $f^{-1}$ ). Hence  $f$  is a bijection.

( $\Leftarrow$ ) If  $f$  is a bijection, surjectivity guarantees every  $y \in B$  has *at least one* preimage, and injectivity guarantees *at most one*; together, exactly one. This unique preimage is exactly what  $f^{-1}(y)$  requires, so  $f^{-1}$  is well-defined. ■

*Note* (When invertibility fails). Neither the **Employer** function (not injective: two computers both mapped to Harvard College Observatory) nor the squaring function  $\mathbb{R} \rightarrow \mathbb{R}$  (not injective:  $2^2 = (-2)^2$ ; also not surjective onto all of  $\mathbb{R}$ ) is invertible.

**Theorem 2.3** ( $f^{-1}$  undoes  $f$ , and vice versa). *If  $f : A \rightarrow B$  is a bijection, then for all  $x \in A$  and all  $y \in B$ :*

$$f^{-1}(f(x)) = x, \quad f(f^{-1}(y)) = y.$$

*Proof.* Let  $x \in A$  and  $y = f(x)$ . By definition,  $f^{-1}(y)$  is the unique element of  $A$  mapping to  $y$  under  $f$  – and  $x$  is such an element, so  $f^{-1}(f(x)) = f^{-1}(y) = x$ . The second identity is symmetric: apply the same argument with the roles of  $f, f^{-1}$  swapped, using that  $f^{-1}$  is itself a bijection (below). ■

**Theorem 2.4.** *If  $f$  is a bijection, so is  $f^{-1}$ , and  $(f^{-1})^{-1} = f$ .*

*Proof.*  $f^{-1} : B \rightarrow A$  is injective and surjective by the symmetry of the defining property (unique  $x$  for each  $y \Leftrightarrow$  unique  $y$  for each  $x$ , since  $f$  itself is a bijection), hence  $f^{-1}$  is a bijection and has its own inverse. That inverse must send each  $x$  back to the  $y$  with  $f^{-1}(y) = x$ , i.e.  $y = f(x)$  – exactly  $f$ . So  $(f^{-1})^{-1} = f$ . ■

**Definition 2.25** (Preimage of a set, Rosen). For  $f : A \rightarrow B$  (not necessarily a bijection) and  $S \subseteq B$ , the *preimage* (or *inverse image*) of  $S$  under  $f$  is

$$f^{-1}(S) = \{a \in A : f(a) \in S\}.$$

*Note* (Overloaded notation – read carefully).  $f^{-1}(S)$  (preimage of a *set*, defined for *any* function  $f$ ) and  $f^{-1}(y)$  (the *inverse function* applied to a point, defined *only when  $f$  is a bijection*) use the same symbol  $f^{-1}$  but are different concepts. When  $f$  is a bijection,  $f^{-1}(\{y\}) = \{f^{-1}(y)\}$  – the set-level and point-level notions agree – but  $f^{-1}(S)$  makes sense even when no bijection (or even no inverse function) exists.

**Example 2.7.** For  $f : \mathbb{R} \rightarrow \mathbb{R}$ ,  $f(x) = x^2$  (not a bijection, so  $f^{-1}$  as a function does not exist), the preimage of  $S = [4, 9]$  is

$$f^{-1}([4, 9]) = \{x \in \mathbb{R} : x^2 \in [4, 9]\} = [-3, -2] \cup [2, 3].$$

## 2.9 Composition

**Definition 2.26** (Composition). For  $f : A \rightarrow B$  and  $g : B \rightarrow C$ , the *composition*  $g \circ f : A \rightarrow C$  is

$$(g \circ f)(x) = g(f(x)).$$

*Note* (Order of writing vs. order of application).  $g \circ f$  is read right-to-left in application ( $f$  first, then  $g$ ), matching the order inside  $g(f(x))$  – but this is the *reverse* of our usual left-to-right reading order. Read carefully.

*Note* (Compatibility requirement).  $g \circ f$  is defined only when  $\text{codomain}(f) = \text{domain}(g)$ : whatever  $f$  guarantees to produce must be exactly what  $g$  is guaranteed to handle. If this fails, the composition is simply undefined – not partially defined, not an error to patch around.

**Theorem 2.5** (Not commutative, but associative). *In general  $g \circ f \neq f \circ g$  (indeed one side may not even be defined). But for  $f : A \rightarrow B$ ,  $g : B \rightarrow C$ ,  $h : C \rightarrow D$ :*

$$h \circ (g \circ f) = (h \circ g) \circ f,$$

*so  $h \circ g \circ f$  is unambiguous without parentheses.*

*Proof.* Both sides are functions  $A \rightarrow D$ . For any  $x \in A$ :

$$(h \circ (g \circ f))(x) = h((g \circ f)(x)) = h(g(f(x))) = (h \circ g)(f(x)) = ((h \circ g) \circ f)(x).$$

Equal on every input, so the functions are equal. ■

**Theorem 2.6** (Role of the identity). *For  $f : A \rightarrow A$  a bijection:  $f \circ f^{-1} = i_A$  and  $f^{-1} \circ f = i_A$ . For any  $g : C \rightarrow D$ :  $g \circ i_C = g$  and  $i_D \circ g = g$ .*

**Definition 2.27** (Iterated composition). For  $f : A \rightarrow A$  and  $n \in \mathbb{Z}^+$ :

$$f^{(n)} = \underbrace{f \circ f \circ \cdots \circ f}_{n \text{ copies}}.$$

**Theorem 2.7** (Composition preserves injectivity/surjectivity, Rosen). *Let  $f : A \rightarrow B$ ,  $g : B \rightarrow C$ .*

(i) *If  $f, g$  are both injective, so is  $g \circ f$ .*

(ii) *If  $f, g$  are both surjective, so is  $g \circ f$ .*

(iii) *If  $f, g$  are both bijective, so is  $g \circ f$ .*

*Proof.* (i) Suppose  $(g \circ f)(x_1) = (g \circ f)(x_2)$ , i.e.  $g(f(x_1)) = g(f(x_2))$ . Since  $g$  is injective,  $f(x_1) = f(x_2)$ ; since  $f$  is injective,  $x_1 = x_2$ .

(ii) Let  $z \in C$ . Since  $g$  is surjective,  $z = g(y)$  for some  $y \in B$ ; since  $f$  is surjective,  $y = f(x)$  for some  $x \in A$ . Then  $(g \circ f)(x) = g(f(x)) = g(y) = z$ .

(iii) Immediate from (i) and (ii). ■

## 2.10 Cryptosystems

**Definition 2.28** (Cryptosystem). A *cryptosystem* consists of a message space  $M$ , cyphertext space  $C$ , keyspace  $K$ , an *encryption function*  $e : M \times K \rightarrow C$ , and a *decryption function*  $d : C \times K \rightarrow M$ , such that for every key  $k \in K$ , the single-argument functions

$$e_k : M \rightarrow C, \quad e_k(m) = e(m, k), \quad d_k : C \rightarrow M, \quad d_k(c) = d(c, k)$$

satisfy:

(i)  $e_k, d_k$  are bijective (onto their images), and

(ii)  $d_k = e_k^{-1}$ .

*Note.* Condition (ii) is the essential one: it forces  $d_k(e_k(m)) = m$  for all  $m \in M$  – decryption with  $k$  genuinely undoes encryption with  $k$ .

**Example 2.8** (Shift cipher, Rosen §4.6). Take  $M = C = \mathbb{Z}_{26} = \{0, 1, \dots, 25\}$  (letters as residues mod 26) and  $K = \mathbb{Z}_{26}$ . Define

$$e_k(p) = (p + k) \bmod 26, \quad d_k(c) = (c - k) \bmod 26.$$

Each  $e_k$  is a bijection on  $\mathbb{Z}_{26}$  with  $d_k = e_k^{-1}$ , so  $(M, C, K, e, d)$  is a cryptosystem. ( $k = 3$  is the classical Caesar cipher.)

**Definition 2.29** (Composition of cryptosystems). Given  $\mathcal{C} = (M, M, K, e, d)$  and  $\mathcal{C}' = (M, M, K', e', d')$  (same message space), the *keyed composition* of encryption functions is

$$e' \bullet e : M \times (K \times K') \rightarrow M, \quad (e' \bullet e)(m, (k, k')) = e'(e(m, k), k'),$$

and similarly  $d \bullet d'$  for decryption. The composed cryptosystem is  $\mathcal{C}' \circ \mathcal{C} = (M, M, K \times K', e' \bullet e, d \bullet d')$ . For each key pair  $(k, k')$ , the resulting single-argument maps are exactly the ordinary function compositions:

$$e_{(k, k')} = e'_{k'} \circ e_k, \quad d_{(k, k')} = d_k \circ d'_{k'}.$$

**Theorem 2.8.** *The composition of two cryptosystems is a cryptosystem.*

*Proof.* Fix  $(k, k') \in K \times K'$ . Since  $\mathcal{C}, \mathcal{C}'$  are cryptosystems,  $e_k, e'_{k'}, d_k, d'_{k'}$  are all bijections. By the composition-preserves-bijectivity theorem (§2.9),  $e_{(k, k')} = e'_{k'} \circ e_k$  is a bijection. It remains to check  $d_{(k, k')} = (e_{(k, k')})^{-1}$ : since  $(e'_{k'})^{-1} = d'_{k'}$  and  $(e_k)^{-1} = d_k$ , and the inverse of a composition reverses order,  $(e'_{k'} \circ e_k)^{-1} = e_k^{-1} \circ (e'_{k'})^{-1} = d_k \circ d'_{k'} = d_{(k, k')}$ , as required. ■

*Note* (Security is not guaranteed by composition). A good cryptosystem needs  $e_k, d_k$  easy to compute *with*  $k$ , but  $e_k$  hard to invert *without*  $k$ . Composing two hard-to-invert systems is often, but not always, harder still to break. Counterexample: composing two shift ciphers with keys  $k, k'$  gives  $e'_{k'}(e_k(p)) = (p + k + k') \bmod 26 = e_{k+k'}(p)$  – just another shift cipher, no stronger than a single shift, since the keyspace effectively collapses.

## 2.11 Loose composition

**Definition 2.30** (Loose composition). For  $f : A \rightarrow B$  and  $g : C \rightarrow D$  (no requirement that  $B = C$ ), the *loose composition*  $g \hat{\circ} f$  is

$$g \hat{\circ} f : \{x \in A : f(x) \in C\} \rightarrow D, \quad (g \hat{\circ} f)(x) = g(f(x)).$$

*Note* (Always defined, but harder to use). Unlike ordinary (“tight”) composition, loose composition is defined for *any*  $f, g$  – if  $B \cap C = \emptyset$ , it degenerates to the empty function. The cost: determining its domain requires inspecting the *rule* of  $f$  (which inputs land in  $C$ ), not just  $f$ ’s stated codomain – so it is harder to reason about mechanically. This course uses tight composition throughout.

## 2.12 Counting functions

**Theorem 2.9** (Counting all functions). *If  $|A| = m$ ,  $|B| = n$ , the number of functions  $f : A \rightarrow B$  is  $n^m$ .*

*Proof.* Each of the  $m$  elements of  $A$  independently has  $n$  possible images in  $B$ ; by the product rule,  $\underbrace{n \times n \times \cdots \times n}_m = n^m$ . ■

**Theorem 2.10** (Counting injections). *If  $|A| = m$ ,  $|B| = n$ , the number of injections  $f : A \rightarrow B$  is*

$$n(n-1)(n-2) \cdots (n-m+1) = \frac{n!}{(n-m)!} \quad (\text{Rosen: } P(n, m)).$$

*Proof.* Enumerate  $A = \{a_1, \dots, a_m\}$ .  $a_1$  has  $n$  choices in  $B$ ;  $a_2$  must avoid  $f(a_1)$ , so  $n-1$  choices;  $\dots$ ;  $a_m$  must avoid the  $m-1$  already-used values, so  $n-m+1$  choices. These choices are made in sequence, so by the product rule the total is  $n(n-1) \cdots (n-m+1)$ . ■

*Note.* If  $m > n$ , the product includes a factor of 0, correctly giving 0 injections – consistent with the pigeonhole principle.

**Theorem 2.11** (Counting bijections / permutations). *If  $|A| = |B| = n$ , the number of bijections  $f : A \rightarrow B$  is  $n!$ .*

*Proof.* By §2.7, for finite sets of equal size, bijective  $\iff$  injective. So this is the injection count with  $m = n$ :  $n(n-1)\cdots(n-n+1) = n!$ . ■

*Note.* In particular, the number of permutations of an  $n$ -element set (bijections  $A \rightarrow A$ ) is  $n!$ .

## 2.13 Binary relations

**Definition 2.31** (Binary relation). A *binary relation* from  $A$  to  $B$  is a subset  $R \subseteq A \times B$ . We write  $xRy$ ,  $R(x, y)$ , or  $(x, y) \in R$  interchangeably. A relation *on*  $A$  satisfies  $R \subseteq A \times A$ .

*Note* (Relations as “relaxed functions”). A function  $f : A \rightarrow B$  is exactly a relation  $R \subseteq A \times B$  satisfying: for every  $a \in A$ , there is a *unique*  $b \in B$  with  $(a, b) \in R$ . Dropping the uniqueness (and even the existence) requirement gives a general binary relation – so every function is a relation, but not conversely.

**Definition 2.32** (Domain, range/codomain of a relation). For  $R \subseteq A \times B$ :

$\{x \in A : (x, y) \in R \text{ for some } y \in B\}$  (elements actually related to something),

$\{y \in B : (x, y) \in R \text{ for some } x \in A\}$  (elements actually related to).

**Definition 2.33** ( $R(x)$ ,  $R^{-1}(y)$  – CN notation). For  $x \in A$ :  $R(x) = \{y \in B : xRy\}$  (a *set*, possibly empty – not a single value, unlike function notation). For  $y \in B$ :  $R^{-1}(y) = \{x \in A : xRy\}$ .

**Definition 2.34** (Inverse relation). For  $R \subseteq A \times B$ , the *inverse relation*  $R^{-1} \subseteq B \times A$  is

$$yR^{-1}x \iff xRy.$$

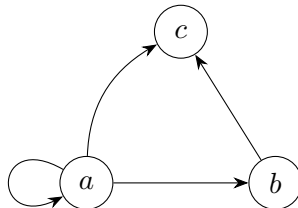
*Note.* When  $R$  happens to be a function, this agrees with the inverse function of §2.8 – *but only when that function is a bijection*. For a general function  $f$ ,  $f^{-1}$  as a *relation* always exists (just reverse the pairs); it is only a *function* in its own right when  $f$  is bijective.

**Definition 2.35** (Zero-one matrix of a relation). For finite  $A = \{a_1, \dots, a_m\}$ ,  $B = \{b_1, \dots, b_n\}$  and  $R \subseteq A \times B$ : the matrix  $M_R$  is  $m \times n$  with

$$(M_R)_{ij} = 1 \iff (a_i, b_j) \in R, \quad (M_R)_{ij} = 0 \text{ otherwise.}$$

*Note* (Properties as matrix conditions,  $R$  on  $A$ ). Reflexive: diagonal all 1s. Symmetric:  $M_R = M_R^t$ . Antisymmetric:  $(M_R)_{ij} = 1$  and  $i \neq j \implies (M_R)_{ji} = 0$ .

**Definition 2.36** (Directed graph of a relation). For  $R$  on  $A$ : vertices =  $A$ ; a directed edge  $a \rightarrow b$  exactly when  $(a, b) \in R$ .



Directed graph of  $R = \{(a, a), (a, b), (a, c), (b, c)\}$  – the loop at  $a$  shows reflexivity there

*Note* (Properties as graph conditions). Reflexive: a loop at every vertex. Symmetric: every edge  $a \rightarrow b$  has a matching edge  $b \rightarrow a$ . Transitive: whenever  $a \rightarrow b \rightarrow c$ , also  $a \rightarrow c$  directly.

*Note* (Boolean matrix product computes composition – harder, ties to §2.6). If  $M_R, M_S$  are zero-one matrices of  $R \subseteq A \times B, S \subseteq B \times C$ , then  $M_R \odot M_S = M_{S \circ R}$  (Boolean product, §2.6) – so relation composition (§2.15) reduces to matrix multiplication with  $\vee, \wedge$  in place of  $+, \times$ .

## 2.14 Properties of binary relations

**Definition 2.37** (Reflexive).  $R$  on  $A$  is *reflexive* iff

$$\forall a \in A, \quad (a, a) \in R.$$

**Definition 2.38** (Irreflexive).  $R$  on  $A$  is *irreflexive* iff

$$\forall a \in A, \quad (a, a) \notin R.$$

**Definition 2.39** (Symmetric).  $R$  on  $A$  is *symmetric* iff

$$\forall x, y \in A, \quad (x, y) \in R \implies (y, x) \in R.$$

**Definition 2.40** (Antisymmetric).  $R$  on  $A$  is *antisymmetric* iff

$$\forall x, y \in A, \quad [(x, y) \in R \wedge (y, x) \in R] \implies x = y.$$

**Definition 2.41** (Asymmetric).  $R$  on  $A$  is *asymmetric* iff

$$\forall x, y \in A, \quad (x, y) \in R \implies (y, x) \notin R.$$

**Definition 2.42** (Transitive).  $R$  on  $A$  is *transitive* iff

$$\forall x, y, z \in A, \quad [(x, y) \in R \wedge (y, z) \in R] \implies (x, z) \in R.$$

**Example 2.9.** On  $\mathbb{Z}$ :  $=, \leq, \geq$  are reflexive;  $<, >$  are irreflexive.  $=$  is symmetric;  $<, \leq, >, \geq$  are not.  $=, \leq, \geq$  are antisymmetric.  $<, >$  are asymmetric.  $=, <, \leq, >, \geq$  are all transitive;  $\neq$  is not.

**Theorem 2.12.**

$$R \text{ asymmetric} \implies R \text{ antisymmetric} \wedge R \text{ irreflexive}.$$

*The converse is false.*

*Proof.* Suppose  $R$  is asymmetric. If  $(x, y) \in R$  and  $(y, x) \in R$  simultaneously held for  $x \neq y$ , this would contradict asymmetry directly, so antisymmetry holds. If  $(a, a) \in R$  for some  $a$ , then taking  $x = y = a$  in the asymmetry condition gives  $(a, a) \in R \implies (a, a) \notin R$ , a contradiction; so  $R$  is irreflexive.

For the converse:  $\leq$  on  $\mathbb{Z}$  is antisymmetric (if  $x \leq y$  and  $y \leq x$  then  $x = y$ ) but not irreflexive ( $a \leq a$  holds for all  $a$ ), hence not asymmetric. ■

**Example 2.10** (Checking all properties on a concrete relation). Let  $A = \{a, b, c, d\}$  and

$$R = \{(a, a), (a, c), (a, d), (b, a), (b, b), (b, c), (b, d), (c, b), (c, c), (d, b), (d, d)\}.$$



- *Reflexive*: yes, since  $(a, a), (b, b), (c, c), (d, d) \in R$ .
- *Symmetric*: no, since  $(a, c) \in R$  but  $(c, a) \notin R$ .
- *Antisymmetric*: no, since  $(b, c), (c, b) \in R$  with  $b \neq c$ .
- *Transitive*: no, since  $(a, c), (c, b) \in R$  but  $(a, b) \notin R$ .

**Theorem 2.13** (Properties under union and intersection). *Let  $R, S$  be relations on  $A$ .*

$$\begin{aligned} R, S \text{ reflexive} &\implies R \cup S, R \cap S \text{ reflexive,} \\ R, S \text{ symmetric} &\implies R \cup S, R \cap S \text{ symmetric,} \\ R, S \text{ transitive} &\implies R \cap S \text{ transitive (but not necessarily } R \cup S). \end{aligned}$$

*Proof. Reflexive:*  $R, S \text{ reflexive} \implies \forall a \in A, (a, a) \in R \wedge (a, a) \in S \implies (a, a) \in R \cup S$  and  $(a, a) \in R \cap S$ .

*Symmetric:* If  $(x, y) \in R \cup S$ , then  $(x, y) \in R$  or  $(x, y) \in S$ ; symmetry of the relation containing it gives  $(y, x)$  in that same relation, hence  $(y, x) \in R \cup S$ . If  $(x, y) \in R \cap S$ , then  $(x, y)$  is in both, so by symmetry  $(y, x)$  is in both, hence  $(y, x) \in R \cap S$ .

*Transitive ( $\cap$  only):* If  $(x, y), (y, z) \in R \cap S$ , both pairs lie in  $R$ , so transitivity of  $R$  gives  $(x, z) \in R$ ; both pairs also lie in  $S$ , so  $(x, z) \in S$ ; hence  $(x, z) \in R \cap S$ . Union fails in general: a chain  $x \rightarrow y$  witnessed only by  $R$  and  $y \rightarrow z$  witnessed only by  $S$  need not put  $(x, z)$  in  $R$  or in  $S$  individually. ■

## 2.15 Combining binary relations

**Definition 2.43** (Union and intersection of relations). For  $R, S \subseteq A \times B$ :

$$R \cup S = \{(x, y) : xRy \text{ or } xSy\}, \quad R \cap S = \{(x, y) : xRy \text{ and } xSy\}.$$

**Definition 2.44** (Composition of relations). For  $R \subseteq A \times B$  and  $S \subseteq B \times C$ , the *composition*  $S \circ R \subseteq A \times C$  is

$$S \circ R = \{(x, z) : \exists y \in B \text{ such that } xRy \text{ and } ySz\}.$$

*Note.* When  $R, S$  happen to be functions, this is exactly function composition (§2.9). Relation composition generalizes it: no uniqueness of the intermediate  $y$  is required, and there may be several, or none.

**Theorem 2.14** (Composing  $R$  with its inverse). For  $R \subseteq A \times B$ :

$$(x, z) \in R^{-1} \circ R \iff \exists y \in B \text{ such that } xRy \text{ and } zRy.$$

*Proof.*

$$(x, z) \in R^{-1} \circ R \iff \exists y, xRy \wedge yR^{-1}z \iff \exists y, xRy \wedge zRy$$

using the defining property  $yR^{-1}z \iff zRy$ . ■

*Note.* So  $R^{-1} \circ R$  relates  $x, z \in A$  exactly when they share a common  $R$ -image in  $B$ ; symmetrically,  $R \circ R^{-1}$  relates elements of  $B$  that share a common preimage in  $A$ .

**Definition 2.45** (Iterated composition of a relation with itself). For  $R$  on  $A$  and  $n \in \mathbb{Z}^+$ :

$$R^{(n)} = \underbrace{R \circ R \circ \cdots \circ R}_{n \text{ copies}}.$$

**Example 2.11** (“Six degrees of separation”). For the relation knows on the set of all people, knows<sup>(2)</sup> relates two people with a mutual acquaintance; knows<sup>(6)</sup> is (under this popular hypothesis) the complete relation on all people.

**Definition 2.46** (Transitive closure). The *transitive closure* of  $R$  on  $A$  is the unique relation  $R^+$  on  $A$  such that, for all  $x, y, z \in A$ :

- (i)  $xRy \implies xR^+y$ ;
- (ii)  $xR^+y \wedge yRz \implies xR^+z$ ;
- (iii)  $xRy \wedge yR^+z \implies xR^+z$ .

*Note.* Condition (i) is essential (it seeds the closure); either (ii) or (iii) alone suffices together with (i) – both are not simultaneously required.

**Theorem 2.15.**

$$R^+ = \bigcup_{i=1}^{\infty} R^{(i)}.$$

$R^+$  is always transitive.

*Proof sketch.* Each  $R^{(i)}$  is contained in  $R^+$ :  $R^{(1)} = R \subseteq R^+$  by (i), and if  $R^{(i)} \subseteq R^+$ , then  $(x, z) \in R^{(i+1)}$  means  $\exists y, (x, y) \in R^{(i)} \subseteq R^+$  and  $yRz$ , so  $(x, z) \in R^+$  by (ii); induction gives  $R^{(i)} \subseteq R^+$  for all  $i$ , hence  $\bigcup_i R^{(i)} \subseteq R^+$ . Conversely,  $\bigcup_i R^{(i)}$  satisfies (i)-(iii) itself, and  $R^+$  is the *unique* relation satisfying them, so equality follows. Transitivity of  $R^+$  is then immediate: any finite chain of  $R^+$ -steps is absorbed into some  $R^{(i)} \subseteq R^+$ . ■

**Definition 2.47** (Closure, general). The *closure* of  $R$  under a property  $P$  is the smallest relation  $R' \supseteq R$  with property  $P$  (if it exists).

**Definition 2.48** (Reflexive closure).

$$r(R) = R \cup \Delta_A, \quad \Delta_A = \{(a, a) : a \in A\} \text{ (the diagonal relation).}$$

**Definition 2.49** (Symmetric closure).

$$s(R) = R \cup R^{-1}.$$

*Note.* Transitive closure  $R^+$  was already defined above:  $R^+ = \bigcup_{i=1}^{\infty} R^{(i)}$ .

**Theorem 2.16** (Warshall’s algorithm – harder, computes  $R^+$  efficiently). Let  $A = \{a_1, \dots, a_n\}$ ,  $M_0 = M_R$ . Define  $M_k$  from  $M_{k-1}$  by:

$$(M_k)_{ij} = (M_{k-1})_{ij} \vee ((M_{k-1})_{ik} \wedge (M_{k-1})_{kj}),$$

*i.e. allow paths that route through  $a_k$  as an intermediate vertex. Then  $M_n = M_{R^+}$ .*

---

**Algorithm 1:** Warshall's algorithm

---

**Input:** Zero-one matrix  $M$  of relation  $R$  on  $\{a_1, \dots, a_n\}$

**Output:** Zero-one matrix of  $R^+$

```
1 for  $k \leftarrow 1$  to  $n$  do
2   for  $i \leftarrow 1$  to  $n$  do
3     for  $j \leftarrow 1$  to  $n$  do
4        $M_{ij} \leftarrow M_{ij} \vee (M_{ik} \wedge M_{kj})$ ;
5     end
6   end
7 end
8 return  $M$ ;
```

---

*Proof idea.*  $(M_k)_{ij} = 1$  iff there is a path  $a_i \rightarrow a_j$  using only intermediate vertices from  $\{a_1, \dots, a_k\}$ . Induction on  $k$ : base case  $k = 0$  is exactly  $R$  itself; the update rule captures “either a path avoiding  $a_k$  already existed, or one exists routing through  $a_k$ .” At  $k = n$ ,  $(M_n)_{ij} = 1$  iff *any* path  $a_i \rightarrow a_j$  exists – exactly  $R^+$ . ■

*Note (Efficiency).* Computing  $R^+$  via  $\bigcup_i R^{(i)}$  directly can cost  $O(n^4)$ ; Warshall's algorithm computes the same  $R^+$  in  $O(n^3)$ .

## 2.16 Equivalence relations

**Definition 2.50** (Equivalence relation).  $R$  on  $A$  is an *equivalence relation* iff  $R$  is reflexive, symmetric, and transitive.

*Note* (Why not antisymmetric too?). Equality is reflexive, symmetric, *and* antisymmetric. But requiring antisymmetry of a general equivalence relation would force  $xRy \wedge yRx \implies x = y$  – collapsing “equivalent” into “identical,” which defeats the purpose of a looser notion of sameness.

**Example 2.12.** • Congruence modulo  $m$ :  $x \equiv y \pmod{m} \iff m \mid (x - y)$ .

- For any  $f : A \rightarrow B$ :  $x \sim_f y \iff f(x) = f(y)$  defines an equivalence relation on  $A$ .
- The transitive closure of any reflexive, symmetric relation is an equivalence relation.
- For any relation  $R$ :  $R \circ R^{-1}$  and  $R^{-1} \circ R$  are equivalence relations.

**Definition 2.51** (Equivalence class). For equivalence relation  $R$  on  $A$ , an *equivalence class* is a nonempty  $X \subseteq A$  such that:

$$(i) \forall x, y \in X, xRy; \quad (ii) \nexists x \in X, z \notin X \text{ such that } xRz.$$

Equivalently,  $X$  is a *maximal* subset of  $A$  on which  $R$  holds between every pair.

*Note.* For  $x \in A$ , the equivalence class containing  $x$  is  $[x]_R = \{y \in A : xRy\}$ .

**Theorem 2.17.** *The equivalence classes of an equivalence relation  $R$  on  $A$  form a partition of  $A$ .*

*Proof.* (a) **Every  $x \in A$  lies in some equivalence class.** Let  $X = \{y \in A : xRy\} = [x]_R$ .

- $X$  satisfies (i): for  $y, z \in X$ ,  $xRy$  and  $xRz$ ; by symmetry  $yRx$ , and with  $xRz$  and transitivity,  $yRz$ .

- $X$  satisfies (ii): suppose  $v \in X$ ,  $w \notin X$ ,  $vRw$ . Since  $v \in X$ :  $xRv$ ; combined with  $vRw$  and transitivity,  $xRw$ , so  $w \in X$  – contradiction. So no such  $v, w$  exist.
- $x \in X$  by reflexivity ( $xRx$ ).

So  $X$  is an equivalence class containing  $x$ .

**(b) Distinct equivalence classes are disjoint.** Suppose  $X, Y$  are equivalence classes with  $X \cap Y \neq \emptyset$ ; let  $x \in X \cap Y$ . We show  $X = Y$  by showing  $Y \setminus X = \emptyset$  and  $X \setminus Y = \emptyset$ .

If  $y \in Y \setminus X$ : since  $X$  is an equivalence class and  $y \notin X$ ,  $(x, y) \notin R$  (property (ii) for  $X$ , using  $x \in X$ ). But  $x, y \in Y$ , and  $Y$  is an equivalence class, so property (i) forces  $(x, y) \in R$  – contradiction. So  $Y \setminus X = \emptyset$ ; symmetrically  $X \setminus Y = \emptyset$ . Hence  $X = Y$ .

So every element of  $A$  lies in exactly one equivalence class – the classes are nonempty (by definition), pairwise disjoint (part (b)), and cover  $A$  (part (a)): a partition. ■

**Definition 2.52** (Partial order).  $R$  on  $A$  is a *partial order* iff reflexive, antisymmetric, and transitive.  $(A, R)$  is then a *partially ordered set* (*poset*), often written  $(A, \preceq)$ .

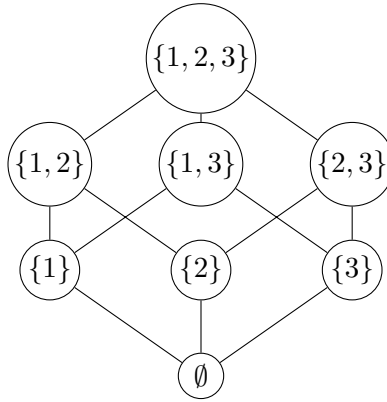
**Example 2.13.**  $(\mathbb{Z}, \leq)$ ;  $(\mathcal{P}(S), \subseteq)$  for any set  $S$  (§1.6);  $(\mathbb{Z}^+, |)$  where  $a | b$  means  $a$  divides  $b$ .

**Definition 2.53** (Comparable, total order, chain).  $a, b \in A$  are *comparable* if  $a \preceq b$  or  $b \preceq a$ . If every pair is comparable,  $\preceq$  is a *total* (or *linear*) order and  $A$  is a *chain*.

*Note.*  $(\mathbb{Z}, \leq)$  is a total order;  $(\mathcal{P}(S), \subseteq)$  is generally *not* (e.g.  $\{1\}$  and  $\{2\}$  are incomparable in  $\mathcal{P}(\{1, 2\})$ ) – exactly why §1.7’s maximal/maximum distinction was necessary.

**Definition 2.54** (Hasse diagram). A drawing of a finite poset: vertices =  $A$ , an edge drawn upward from  $a$  to  $b$  exactly when  $a \prec b$  and there is no  $c$  with  $a \prec c \prec b$  (a *covering* relation) – omitting all edges implied by reflexivity/transitivity.

**Example 2.14.** For  $\mathcal{P}(\{1, 2, 3\})$  under  $\subseteq$ :  $\emptyset$  at the bottom, the three singletons above it, the three pairs above those,  $\{1, 2, 3\}$  at the top.



Hasse diagram of  $(\mathcal{P}(\{1, 2, 3\}), \subseteq)$

**Definition 2.55** (Upper/lower bound, LUB, GLB – harder). For  $S \subseteq A$ :  $u \in A$  is an *upper bound* of  $S$  if  $s \preceq u$  for all  $s \in S$ ; a *least upper bound* (LUB, or *join*  $\vee$ ) is an upper bound  $\preceq$  every other upper bound. Dually for *lower bound* and *greatest lower bound* (GLB, *meet*  $\wedge$ ).

**Definition 2.56** (Lattice). A poset in which every pair  $a, b$  has both a LUB and a GLB.

**Example 2.15.**  $(\mathcal{P}(S), \subseteq)$  is a lattice:  $A \vee B = A \cup B$ ,  $A \wedge B = A \cap B$  (§1.12).

*Note.*  $(\mathbb{Z}^+, |)$  is also a lattice:  $\vee = \text{lcm}$ ,  $\wedge = \text{gcd}$ .

## 2.17 Relations

**Definition 2.57** (Ternary relation). A *ternary relation* on sets  $A, B, C$  is a subset of  $A \times B \times C$ : a set of triples  $(x, y, z)$  with  $x \in A$ ,  $y \in B$ ,  $z \in C$ .

**Definition 2.58** ( $n$ -ary relation). An  *$n$ -ary relation* on sets  $A_1, A_2, \dots, A_n$  is a subset

$$R \subseteq A_1 \times A_2 \times \dots \times A_n,$$

i.e. a set of  $n$ -tuples  $(x_1, \dots, x_n)$  with  $x_i \in A_i$  for each  $i$ . Each  $A_i$  is the  $i$ -th *domain*;  $n$  is the *degree*. An  $n$ -ary relation is also called an  *$n$ -ary predicate*.

**Example 2.16.** (first name, middle name, surname) triples form a ternary relation. (day, month, year) likewise. Points in  $\mathbb{R}^3$  form a ternary relation on  $\mathbb{R}$ .

*Note* (Connection to tables and databases, Rosen). An  $n$ -ary relation's tuples correspond to the *rows* of an  $n$ -column table; each domain  $A_i$  (analogous to a function's codomain – allowed to contain more than what actually appears) corresponds to a *column* or *attribute*. This correspondence is the basis of *relational databases*: a database table *is* an  $n$ -ary relation, its columns are attributes, and a *key* (or combination of attributes forming a key) is one that uniquely determines each tuple – analogous to how, in a binary relation that happens to be a function, the first coordinate uniquely determines the second.

**Definition 2.59** (Database, Rosen §9.2.2). A *database* is (in this model) an  $n$ -ary relation, its tuples called *records*, its domains  $A_1, \dots, A_n$  called *fields* or *attributes*. A *primary key* is a domain (or combination of domains) whose value(s) uniquely determine each record – i.e. no two distinct tuples agree on it.

**Definition 2.60** (Selection operator, Rosen §9.2.3). For an  $n$ -ary relation  $R$  and a condition  $C$  on  $n$ -tuples, the *selection operator*  $s_C$  maps  $R$  to

$$s_C(R) = \{(a_1, \dots, a_n) \in R : C(a_1, \dots, a_n) \text{ holds}\}.$$

(Rows satisfying the condition – SQL's **WHERE**.)

**Definition 2.61** (Projection, Rosen §9.2.3). For  $1 \leq i_1 < i_2 < \dots < i_m \leq n$ , the *projection*  $P_{i_1, \dots, i_m}$  maps each  $n$ -tuple to the  $m$ -tuple of its selected components:

$$P_{i_1, \dots, i_m}(a_1, \dots, a_n) = (a_{i_1}, a_{i_2}, \dots, a_{i_m}).$$

(Columns kept, duplicates then removed – SQL's **SELECT**, confusingly named the opposite of the relational-algebra term.)

*Note.* Projection can reduce the number of tuples: distinct  $n$ -tuples that agree on all  $m$  retained components collapse to one  $m$ -tuple.

**Definition 2.62** (Join, Rosen §9.2.3). Let  $R$  have degree  $m$ ,  $S$  have degree  $n$ , and fix  $p \leq m, n$ . The *join*  $J_p(R, S)$  is the relation of degree  $m + n - p$  consisting of all  $(m + n - p)$ -tuples

$$(a_1, \dots, a_{m-p}, c_1, \dots, c_p, b_1, \dots, b_{n-p})$$

such that  $(a_1, \dots, a_{m-p}, c_1, \dots, c_p) \in R$  and  $(c_1, \dots, c_p, b_1, \dots, b_{n-p}) \in S$  – i.e. tuples of  $R$  and  $S$  are merged wherever their last  $p$  / first  $p$  components agree.

*Note* (Applications, Rosen §9.2.4).  $n$ -ary relations underlie *data mining*: a store's transaction database (each transaction an  $n$ -ary tuple of purchased items) can be queried with selection/projection/join to find frequent itemsets, association rules, etc.

## 2.18 Counting relations

**Theorem 2.18** (Counting binary relations). *For finite  $A, B$  with  $|A| = m$ ,  $|B| = n$ :*

$$\#\{\text{binary relations from } A \text{ to } B\} = 2^{mn}.$$

*In particular, with  $A = B$ ,  $|A| = m$ :*

$$\#\{\text{binary relations on } A\} = 2^{m^2}.$$

*Proof.* A binary relation from  $A$  to  $B$  is exactly a subset of  $A \times B$ , so the count equals  $|\mathcal{P}(A \times B)| = 2^{|A \times B|}$  (§1.9). Since  $|A \times B| = mn$  (§1.14), this is  $2^{mn}$ . The case  $A = B$  substitutes  $n = m$ . ■

**Theorem 2.19** (Counting  $n$ -ary relations). *For finite  $A_1, \dots, A_n$  with  $|A_i| = m_i$ :*

$$\#\{n\text{-ary relations on } A_1, \dots, A_n\} = 2^{m_1 m_2 \cdots m_n}.$$

*If all  $A_i = A$  with  $|A| = m$ :  $2^{m^n}$ .*

*Proof.* Identical argument: an  $n$ -ary relation is a subset of  $A_1 \times \cdots \times A_n$ , of size  $\prod_i m_i$  (§1.14 generalized), so the count is  $2^{\prod_i m_i}$ . ■

**Example 2.17** (Counting relations with a given property, Rosen-style). Reflexive relations on  $A$  with  $|A| = m$ : the  $m$  diagonal pairs  $(a, a)$  are forced in, leaving  $m^2 - m$  off-diagonal pairs free to choose independently, so there are  $2^{m^2 - m}$  reflexive relations on  $A$ .

## 2.19 Matrices (Rosen §2.6 – extension, beyond CN syllabus)

**Definition 2.63** (Matrix). An  $m \times n$  matrix  $A = [a_{ij}]$  is a rectangular array with  $m$  rows,  $n$  columns, entries  $a_{ij}$  ( $1 \leq i \leq m$ ,  $1 \leq j \leq n$ ).  $A$  is *square* if  $m = n$ .

**Definition 2.64** (Addition, scalar multiplication). For same-size  $A, B$ :

$$(A + B)_{ij} = a_{ij} + b_{ij}.$$

For scalar  $c$ :

$$(cA)_{ij} = c a_{ij}.$$

Addition is defined only when  $A, B$  have identical dimensions.

**Definition 2.65** (Matrix multiplication). For  $A$  ( $m \times k$ ) and  $B$  ( $k \times n$ ), the product  $AB$  is  $m \times n$ , with entry  $(i, j)$  given by

$$(AB)_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + \cdots + a_{ik}b_{kj} = \sum_{l=1}^k a_{il}b_{lj}$$

– the dot product of row  $i$  of  $A$  with column  $j$  of  $B$ . Defined only when the number of columns of  $A$  equals the number of rows of  $B$ .

**Example 2.18.**

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix} = \begin{pmatrix} 1 \cdot 5 + 2 \cdot 7 & 1 \cdot 6 + 2 \cdot 8 \\ 3 \cdot 5 + 4 \cdot 7 & 3 \cdot 6 + 4 \cdot 8 \end{pmatrix} = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}.$$

**Theorem 2.20.** *Matrix multiplication is associative:  $(AB)C = A(BC)$  whenever the products are defined.*

*Proof.* Let  $A$  be  $m \times k$ ,  $B$  be  $k \times l$ ,  $C$  be  $l \times n$ . Both  $(AB)C$  and  $A(BC)$  are  $m \times n$ .

For entry  $(i, j)$  of  $(AB)C$ : first  $(AB)_{ip} = a_{i1}b_{1p} + \cdots + a_{ik}b_{kp}$  for each  $p$ , then

$$((AB)C)_{ij} = \sum_{p=1}^l (AB)_{ip} c_{pj} = \sum_{p=1}^l \left( \sum_{q=1}^k a_{iq} b_{qp} \right) c_{pj} = \sum_{q=1}^k \sum_{p=1}^l a_{iq} b_{qp} c_{pj}.$$

For entry  $(i, j)$  of  $A(BC)$ : first  $(BC)_{qj} = b_{q1}c_{1j} + \cdots + b_{ql}c_{lj}$  for each  $q$ , then

$$(A(BC))_{ij} = \sum_{q=1}^k a_{iq} (BC)_{qj} = \sum_{q=1}^k a_{iq} \left( \sum_{p=1}^l b_{qp} c_{pj} \right) = \sum_{q=1}^k \sum_{p=1}^l a_{iq} b_{qp} c_{pj}.$$

Both expand to the same triple sum  $\sum_{q,p} a_{iq} b_{qp} c_{pj}$  (finite sums may be freely reordered/regrouped), so  $(AB)C = A(BC)$ . ■

**Theorem 2.21.** *Matrix multiplication is not commutative in general.*

*Proof.*

$$\begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 2 & 0 \\ 0 & 0 \end{pmatrix}, \quad \begin{pmatrix} 1 & 0 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}.$$

Different results, so  $AB \neq BA$  here. ■

**Definition 2.66** (Identity matrix).  $I_n$  is  $n \times n$  with 1s on the diagonal, 0s elsewhere:  $(I_n)_{ij} = 1$  if  $i = j$ , else 0.

**Theorem 2.22.** For  $A$  ( $m \times n$ ):  $I_m A = A$  and  $A I_n = A$ .

**Definition 2.67** (Transpose).  $A^t$ :  $(A^t)_{ij} = a_{ji}$  – rows and columns swapped.

**Theorem 2.23.**  $(A + B)^t = A^t + B^t$ ;  $(AB)^t = B^t A^t$  (order reverses).

**Definition 2.68** (Symmetric matrix).  $A$  is *symmetric* if  $A = A^t$  (equivalently  $a_{ij} = a_{ji}$  for all  $i, j$ ; requires  $A$  square).

**Definition 2.69** (Zero-one matrices, meet/join/Boolean product, Rosen – harder, sets up §2.15/9.4). For matrices with entries in  $\{0, 1\}$ :

$$(A \wedge B)_{ij} = a_{ij} \wedge b_{ij} \quad (\text{meet}), \quad (A \vee B)_{ij} = a_{ij} \vee b_{ij} \quad (\text{join}).$$

Boolean product:

$$(A \odot B)_{ij} = (a_{i1} \wedge b_{1j}) \vee (a_{i2} \wedge b_{2j}) \vee \cdots \vee (a_{ik} \wedge b_{kj}) = \bigvee_{l=1}^k (a_{il} \wedge b_{lj}).$$

An entry is 1 iff *some*  $l$  has  $a_{il} = b_{lj} = 1$  – same shape as ordinary matrix multiplication with  $(\vee, \wedge)$  in place of  $(+, \times)$ .

**Example 2.19.**

$$A = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}, \quad B = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

$$A \wedge B = \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}, \quad A \vee B = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \quad A \odot B = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}.$$

For  $(A \odot B)_{11}$ :  $(a_{11} \wedge b_{11}) \vee (a_{12} \wedge b_{21}) = (1 \wedge 0) \vee (0 \wedge 1) = 0$ . For  $(A \odot B)_{21}$ :  $(1 \wedge 0) \vee (1 \wedge 1) = 1$ .

**Theorem 2.24** (Connection to relations, Rosen). *Let  $A$  be the zero-one matrix of relation  $R \subseteq X \times Y$  (§2.13:  $a_{ij} = 1 \iff (x_i, y_j) \in R$ ),  $B$  that of  $S \subseteq Y \times Z$ . Then  $A \odot B$  is exactly the matrix of  $S \circ R$  (§2.15).*

*Proof.*

$$(A \odot B)_{ij} = 1 \iff \exists l : a_{il} = 1 \wedge b_{lj} = 1 \iff \exists y_l : (x_i, y_l) \in R \wedge (y_l, z_j) \in S \iff (x_i, z_j) \in S \circ R,$$

matching the definition of relation composition exactly (§2.15). ■

*Note* (Iterating for closures). Writing  $A^{[n]} = A \odot A \odot \cdots \odot A$  ( $n$  copies) gives the matrix of  $R^{(n)}$ . For finite  $A$  ( $n \times n$  zero-one matrix), the join  $A \vee A^{[2]} \vee \cdots \vee A^{[n]}$  (only  $n$  terms needed, since further terms add nothing new) gives the matrix of the transitive closure  $R^+$  (§2.15) – this is the basis of *Warshall's algorithm*, computed far more efficiently than by literally forming all powers.



## 3 Proofs

### 3.1 Theorems and proofs

**Definition 3.1** (Theorem). A *theorem* is a mathematical statement that has been proved true.

**Definition 3.2** (Lemma, proposition, corollary – Rosen’s terminology). A *lemma* is a theorem whose main purpose is as a stepping stone in the proof of a more significant theorem. A *proposition* is a theorem of independent interest, typically less significant or easier to prove than the main theorems around it. A *corollary* is a theorem that follows almost immediately from one just proved.

**Definition 3.3** (Proof). A *proof* of a claim is a finite sequence of statements (*steps*), ending in the claim, where each step is one of:

- something already known: a *definition*, an *axiom* (a fundamental assumed property, e.g.  $x(y + z) = xy + xz$ ), or a previously proved theorem;
- an *assumption*, made as a starting point;
- a *logical consequence* of earlier steps.

The final step establishes the claim from what precedes it.

*Note* (Verifiability). A proof must be readable sequentially – each step justified only by what comes *before* it, never by reading ahead. The end of a proof is marked by ■ (or historically “Q.E.D.”, *quod erat demonstrandum*).

**Theorem 3.1.** For any sets  $A, B$ : if  $A \subseteq B$  then  $\mathcal{P}(A) \subseteq \mathcal{P}(B)$ .

*Proof.* Assume  $A \subseteq B$ . Let  $X \in \mathcal{P}(A)$ ; then  $X \subseteq A$  by definition of  $\mathcal{P}(A)$ . Since  $A \subseteq B$ , also  $X \subseteq B$ . Hence  $X \in \mathcal{P}(B)$  by definition of  $\mathcal{P}(B)$ . So  $X \in \mathcal{P}(A) \implies X \in \mathcal{P}(B)$ , i.e.  $\mathcal{P}(A) \subseteq \mathcal{P}(B)$ . ■

*Note* (Annotating each step). • “Assume  $A \subseteq B$ ” – an *assumption* (the theorem is conditional on it).

- “Let  $X \in \mathcal{P}(A)$ ” – a *definition*, naming an arbitrary element so we can reason about it generically.
- “ $X \subseteq A$ ” – a *logical consequence* of the previous line and the definition of  $\mathcal{P}(A)$ .
- “ $X \subseteq B$ ” – a consequence of  $X \subseteq A$  and the assumption  $A \subseteq B$ .
- “ $X \in \mathcal{P}(B)$ ” – a consequence of  $X \subseteq B$  and the definition of  $\mathcal{P}(B)$ .
- Final line – restates the whole chain as the subset relation  $\mathcal{P}(A) \subseteq \mathcal{P}(B)$ .

Every line either restates a definition/axiom, states an assumption, or follows logically from what came strictly before – exactly the requirement above.

### 3.2 Logical deduction

*Note* (Modus ponens). The core deduction rule: if  $P$  has been established, and  $P \implies Q$  has been established, then  $Q$  may be deduced. This is used constantly and rarely named explicitly, but it is the essence of every logical deduction step in a proof.

*Note* (Implication as a subset relation). In general, for statements  $P, Q$ , let  $\hat{P}, \hat{Q}$  be the sets of situations in which  $P$ , resp.  $Q$ , holds. Then

$$P \implies Q \text{ corresponds to } \hat{P} \subseteq \hat{Q}.$$

This generalizes the set-membership fact from §1.6:  $A \subseteq B \iff \forall x (x \in A \implies x \in B)$  – the correspondence holds even when  $P, Q$  are not phrased as membership statements at all (as the domino example shows).

*Note* (Vacuous truth). If  $X$  is false,  $X \implies Y$  is true regardless of  $Y$  – corresponding to  $\emptyset \subseteq Y$  for any set  $Y$  (§1.6, Theorem 1: the empty set is a subset of every set).

**Definition 3.4** (Converse). The *converse* of  $P \implies Q$  is  $Q \implies P$  (equivalently written  $P \Leftarrow Q$ ). An implication holding does *not* entail its converse holds: implication is not symmetric, as both examples above show ( $P \implies Q$  but not  $Q \implies P$ ;  $X \implies Y$  but not  $Y \implies X$ ).

**Definition 3.5** (Biconditional). When both  $P \implies Q$  and its converse  $Q \implies P$  hold, we write  $P \iff Q$  (“ $P$  if and only if  $Q$ ”;  $P, Q$  are *logically equivalent*:  $\hat{P} = \hat{Q}$  as sets of situations).

**Definition 3.6** (Contrapositive and inverse, Rosen). For  $P \implies Q$ : the *contrapositive* is  $\neg Q \implies \neg P$ ; the *inverse* is  $\neg P \implies \neg Q$ .

**Theorem 3.2.**  $P \implies Q$  is logically equivalent to its contrapositive  $\neg Q \implies \neg P$ . It is not, in general, equivalent to its converse or its inverse (which are themselves contrapositives of each other).

*Proof.* Both  $P \implies Q$  and  $\neg Q \implies \neg P$  are false in exactly one case ( $P$  true,  $Q$  false) and true otherwise – identical truth tables, hence logically equivalent. The converse  $Q \implies P$  and inverse  $\neg P \implies \neg Q$  each fail in the case  $P$  false,  $Q$  true (for the converse) or  $P$  false,  $Q$  true (checking directly) – a different failure case from the original, so neither need agree with  $P \implies Q$  in general; the domino example ( $P \implies Q$  true,  $Q \implies P$  false) is a concrete witness. ■

*Note* (Common logical fallacies, Rosen). *Affirming the consequent*: from  $P \implies Q$  and  $Q$ , wrongly concluding  $P$  (this is the *converse* error from earlier in this section). *Denying the antecedent*: from  $P \implies Q$  and  $\neg P$ , wrongly concluding  $\neg Q$  (this is the *inverse* error). *Begging the question / circular reasoning*: using the claim being proved (or something equivalent to it) as a premise in its own proof.

**Example 3.1** (Affirming the consequent, in the wild). “If it rains, the ground is wet. The ground is wet. Therefore it rained.” Invalid: the ground could be wet from a sprinkler.

### 3.3 Proofs of existential and universal statements

**Definition 3.7** (Existential statement). An *existential statement* asserts  $\exists x \in D, P(x)$  for some domain  $D$  and predicate  $P$ : that *something* with the stated property exists. It may be true or false.

**Theorem 3.3.** *English has a palindrome.*

*Proof.* ‘rotator’ is an English word, and it reads the same forwards and backwards. ■

*Note* (Proving  $\exists x, P(x)$ ). A single *witness* suffices: exhibit one  $x \in D$  and verify  $P(x)$  holds. No need to search further once found.

**Definition 3.8** (Universal statement). A *universal statement* asserts  $\forall x \in D, P(x)$ : *everything* in  $D$  has the property.

**Theorem 3.4.** *Every English word has a vowel or a ‘y’.*

*Proof sketch.* One could, in principle, check every English word individually (‘aardvark’ has a vowel; ‘aardwolf’ has a vowel; ...; ‘syzygy’ has a ‘y’; ...) – but  $D$  here has tens of thousands of members, making case-by-case verification impractical (and hopeless for an infinite  $D$ ). ■

*Note* (Proving  $\forall x \in D, P(x)$ ). Checking every case individually only works for small, finite  $D$  – and not even then, practically. Instead: let  $x$  be an *arbitrary, generic* element of  $D$  (no properties assumed beyond  $x \in D$ ), and show  $P(x)$  follows from that alone. Since the argument uses nothing specific to the particular  $x$  chosen, it applies to *every* element of  $D$  simultaneously. This is the technique already used, e.g., in §1.6 (“let  $x \in A$ , show  $x \in B$ ”) and in §3.1’s Theorem 13 proof (“let  $X \in \mathcal{P}(A)$ ”).

*Note* (Disproof is the mirror image). To *disprove* a universal statement  $\forall x \in D, P(x)$  (i.e. prove its negation  $\exists x \in D, \neg P(x)$ ), a single *counterexample* suffices – disproof of a universal is existential proof of its negation. Conversely, disproving an existential statement  $\exists x \in D, P(x)$  requires proving the universal  $\forall x \in D, \neg P(x)$  – generally the harder direction, since no single case can rule out all possibilities.

**Theorem 3.5** (Uniqueness example). *There is exactly one real number  $x$  such that  $2x + 3 = 7$ .*

*Proof. Existence:*  $x = 2$  satisfies  $2(2) + 3 = 7$ .

**Uniqueness:** suppose  $x_1, x_2$  both satisfy  $2x + 3 = 7$ . Then  $2x_1 + 3 = 2x_2 + 3$ , so  $2x_1 = 2x_2$ , so  $x_1 = x_2$ .

So exactly one such  $x$  exists. ■

*Note* (The uniqueness pattern). Assume two witnesses  $x_1, x_2$  both satisfy the property; derive  $x_1 = x_2$ . Combined with a separate existence argument, this proves “exactly one.”

**Example 3.2** (A plausible-looking false universal claim). Claim: “every prime  $p$  satisfies  $2^p - 1$  is prime” (a *Mersenne prime* for every prime exponent).

*Disproof by counterexample.*  $p = 11$  is prime, but  $2^{11} - 1 = 2047 = 23 \times 89$ , not prime. So the universal claim is false – a single counterexample suffices, despite the pattern holding for  $p = 2, 3, 5, 7$  ( $2^p - 1 = 3, 7, 31, 127$ , all prime). ■

*Note.* This is a genuine cautionary example: checking several small cases can be misleading.  $p = 2, 3, 5, 7$  all work, and it’s tempting to conjecture the pattern continues – but  $p = 11$  already breaks it.

### 3.4 Finding proofs

*Note* (No algorithm for finding proofs). There is no systematic method for discovering proofs, for deep theoretical reasons (Gödel 1931, Church 1936, Turing 1936). Finding proofs draws on logical reasoning, familiarity with the objects involved, practice, creativity, and persistence – not a fixed recipe.

*Note* (Strategy: proving  $A \subseteq B$ ). 1. Name a general member: “Let  $x \in A$ .”

2. Unpack what the definition of  $A$  tells you about  $x$ .

3. Follow the logical consequences.

4. Aim to conclude  $x$  satisfies the definition of  $B$ .

*Note* (Strategy: proving  $A = B$  (sets)). Prove  $A \subseteq B$  and  $A \supseteq B$  separately (§1.6). Each direction uses the subset strategy above.

*Note* (Strategy: proving  $A = B$  (numbers)). If direct symbolic manipulation (algebraic rules) can transform expression  $A$  into expression  $B$ , use that. Otherwise, prove  $A \leq B$  and  $A \geq B$  separately – the numerical analogue of the double-inclusion technique.

*Note* (Without loss of generality (WLOG), Rosen). When a proof is symmetric in two variables (swapping them leaves the claim unchanged), it suffices to prove one case and invoke the symmetry for the other – written “WLOG, assume  $x \leq y$ ” etc. Using WLOG when the situation is *not* actually symmetric is a common error – check the symmetry claim carefully before invoking it.

*Note* (Forward vs. backward reasoning, Rosen). *Forward*: start from known facts/assumptions, derive consequences, until reaching the goal. *Backward*: start from the goal, ask “what would suffice to prove this?”, and work backward to something already known. Backward reasoning is often how a proof is *discovered*; it is then usually rewritten forward for presentation.

### 3.5 Types of proofs

*Note* (Proof types). • *Direct proof / proof by symbolic manipulation* (§3.6): assume  $P$ , derive  $Q$  via a direct chain of implications or equations.

- *Proof by construction* (§3.7, also called *proof by example*): exhibit a specific object and verify it works – a *constructive* existence proof, producing an explicit witness (contrast: *nonconstructive* existence proofs establish that something exists, e.g. via contradiction or a counting argument, without exhibiting it).
- *Proof by cases* (§3.8): split the domain into exhaustive cases, prove the claim in each.
- *Proof by contradiction* (§3.9): assume the negation of the claim, derive a contradiction (some statement and its negation both true).
- *Proof by contraposition*: to prove  $P \implies Q$ , instead prove the equivalent  $\neg Q \implies \neg P$  (§3.2’s contrapositive theorem) – useful when  $\neg Q$  gives more to work with than  $P$  does.
- *Vacuous / trivial proof*: prove  $P \implies Q$  by showing  $P$  is always false (vacuous) or  $Q$  is always true (trivial).
- *Proof by (mathematical) induction* (§3.10).
- *Uniqueness proofs*: show existence, then show any two witnesses must coincide.

This list is not exhaustive, and real proofs frequently mix several of these – e.g. a proof by cases where one case is handled by contradiction and another by direct construction.

**Example 3.3** (Vacuous proof). “For all  $x \in \emptyset$ ,  $x^2 < 0$ .” True vacuously:  $\emptyset$  has no members, so the implication “ $x \in \emptyset \implies x^2 < 0$ ” is never tested against an actual counterexample.

**Example 3.4** (Trivial proof). “If  $n > 0$ , then  $n^2 + 1 > 0$ .” The conclusion  $n^2 + 1 > 0$  holds for *every* real  $n$  regardless of the hypothesis, so the implication is trivially true.

**Theorem 3.6** (Contraposition example). *If  $n^2$  is even, then  $n$  is even.*

*Proof by contraposition.* We prove the contrapositive: if  $n$  is odd, then  $n^2$  is odd. Suppose  $n = 2k + 1$  for some  $k \in \mathbb{Z}$ . Then

$$n^2 = (2k + 1)^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1,$$

which is odd. By §3.2's contrapositive theorem, this proves the original statement. ■

*Note.* Contraposition was the right tool here because “ $n$  is odd” gives a concrete algebraic form  $(2k + 1)$  to compute with, while “ $n^2$  is even” does not directly hand you a useful form for  $n$  itself.

### 3.6 Proof by symbolic manipulation

*Note* (The pattern). A chain of equations, each step justified by a known law (axiom or previously proved fact), linking the two sides of the claim.

**Theorem 3.7** (Difference of two squares).

$$(x - y)(x + y) = x^2 - y^2.$$

*Proof.*

$$\begin{aligned} (x - y)(x + y) &= x^2 + xy - yx - y^2 && \text{(distributive law)} \\ &= x^2 + xy - xy - y^2 && \text{(commutativity of multiplication)} \\ &= x^2 - y^2 && \text{(additive cancellation).} \end{aligned}$$

■

*Note* (Direction is arbitrary). The proof above started from the *right*-hand side and worked toward the left – equally valid as starting from the left. Pick whichever direction is more natural for the specific identity.

*Note* (Examples already seen in this note). This is exactly the technique behind: the De Morgan's-law proof via set-builder notation (§1.11), the chained-identity proof  $\overline{A \cup (B \cap C)} = (\overline{C} \cup \overline{B}) \cap \overline{A}$  (§1.11), and the symmetric-difference identities  $A \triangle B = \overline{A \triangle B}$  etc. (§1.13) – each is a sequence of equalities, each step citing a specific law.

### 3.7 Proof by construction

*Note* (Pattern). Describe a specific object precisely, and show it exists and satisfies the required conditions – a *constructive* proof of an existential statement (§3.3). Theorem 14 (‘rotator’ is a palindrome) is an example.

*Note* (Two mistakes to avoid). • Using construction to attempt a *universal* statement: exhibiting one object with a property never establishes that *every* object has it.

- Mistaking an illustrative example for a proof: an example can clarify a proof's ideas, but is not itself a proof unless the statement being proved is genuinely existential.

**Theorem 3.8** (Ramanujan's taxicab number, Rosen). *There is a positive integer expressible as the sum of two positive cubes in two genuinely different ways.*

*Proof.*

$$1729 = 10^3 + 9^3 = 1000 + 729 = 12^3 + 1^3 = 1728 + 1.$$

Both decompositions use positive integers, and  $\{10, 9\} \neq \{12, 1\}$ , so these are two different ways. ■

*Note.* This is the smallest such number – hence its fame as the “taxicab number,” from the (possibly apocryphal) anecdote of Hardy visiting Ramanujan in a taxi numbered 1729.

### 3.8 Proof by cases

*Note (Pattern).* Identify finitely many cases covering every possibility, and prove the theorem separately in each – each case-proof is a subproof. Cases may overlap (though this can signal redundant effort), but together they must be *exhaustive*: every object the theorem concerns must fall into at least one case.

*Note (Typical shape).* Often a theorem concerns an *infinite* set, split into a small number of cases, at least one of which still covers infinitely many objects – the reasoning within that case must be general enough to apply to all of them, not just verify a few instances.

**Theorem 3.9** (Nonconstructive existence via cases, Rosen). *There exist irrational numbers  $x, y$  such that  $x^y$  is rational.*

*Proof.* Consider  $\sqrt{2}^{\sqrt{2}}$ . Either it is rational or it is irrational – these are the only two (exhaustive) cases.

**Case 1:**  $\sqrt{2}^{\sqrt{2}}$  is rational. Take  $x = y = \sqrt{2}$ , both irrational, with  $x^y$  rational by assumption. Done.

**Case 2:**  $\sqrt{2}^{\sqrt{2}}$  is irrational. Take  $x = \sqrt{2}^{\sqrt{2}}$  (irrational, by this case’s assumption) and  $y = \sqrt{2}$  (irrational). Then

$$x^y = \left( \sqrt{2}^{\sqrt{2}} \right)^{\sqrt{2}} = \sqrt{2}^{\sqrt{2} \cdot \sqrt{2}} = \sqrt{2}^2 = 2,$$

which is rational. Done.

Either way, such  $x, y$  exist. ■

*Note (Strikingly nonconstructive).* This proves the statement true *without ever determining which case actually holds* – we don’t know, from this proof alone, whether  $\sqrt{2}^{\sqrt{2}}$  itself is rational or irrational. (It is, in fact, irrational – even transcendental, by the Gelfond–Schneider theorem – but that fact is not needed here.) This is the clearest illustration of the constructive/nonconstructive distinction from §3.5.

**Definition 3.9** (Chomp). A rectangular grid of cookies; players alternate taking a cookie and every cookie above-and-to-the-right of it (relative to a fixed corner); the top-left cookie is poisoned; whoever takes it loses.

**Example 3.5** (Chomp – Rosen §1.8, nonconstructive existence via strategy-stealing). On any rectangular Chomp board with more than one cookie, the *first* player has a winning strategy.

*Proof.* Consider the first player’s opening move: take just the bottom-right cookie.

**Case 1:** this opening move is part of *some* winning strategy for the first player. Then the first player has a winning strategy – done.

**Case 2:** this opening move is *not* part of any winning strategy for the first player. Then the second player, facing the resulting position, must have a winning response  $r$ . But now imagine the first player had opened with exactly that move  $r$  instead (a legal first move, since it was a legal move for the second player from a position that only has more cookies removed than the empty-except-poison start). By the same argument, whatever position  $r$  leads to, the first player is now in the position the second player would have been in Case 2 – with a winning strategy available.

Either way, the first player wins. ■

*Note* (Why this is nonconstructive). The proof establishes a winning strategy *exists* for the first player without ever revealing what it is – indeed, for most board sizes, no explicit winning strategy is known at all. This is the sharpest possible illustration of the constructive/nonconstructive distinction (§3.5): the theorem is proved, but the object it asserts exists (the strategy) remains, in general, unknown.

*Note* (Strategy-stealing, the technique). The argument never analyzes what the winning strategy actually *does* – it only argues that *if* the second player had a winning response, the first player could “steal” it by playing that same move first. This strategy-stealing pattern appears throughout combinatorial game theory whenever a game has the property that having extra pieces on the board never hurts a player (true for Chomp, since eaten cookies only shrink future options for both sides equally).

### 3.9 Proof by contradiction

*Note* (Pattern). Assume the *negation* of the claim; reason from it to a contradiction (some statement and its negation both derived as true); conclude the assumption was false, so the original claim holds.

**Theorem 3.10** (Euclid). *There are infinitely many prime numbers.*

*Proof.* Suppose, for contradiction, that there are only finitely many primes  $p_1, p_2, \dots, p_n$ . Let

$$q = p_1 p_2 \cdots p_n + 1.$$

Since  $q > p_i$  for every  $i$ ,  $q$  is not itself prime, so  $q$  is composite and hence divisible by some prime. But for each  $p_i$ , dividing  $q$  by  $p_i$  leaves remainder 1 (since  $p_i$  divides  $p_1 \cdots p_n$  exactly), so  $p_i \nmid q$ . So  $q$  is divisible by no prime at all – contradicting that  $q$  is composite. Hence the assumption was false: there are infinitely many primes. ■

*Note* (A useful auxiliary fact used implicitly). The argument above (and many proofs by contradiction over  $\mathbb{N}$  or  $\mathbb{Z}^+$ ) relies on: any nonempty set of natural numbers has a smallest element (well-ordering). This lets you say things like “let  $n$  be the smallest counterexample” and derive a contradiction from its minimality.

**Theorem 3.11.**  $\sqrt{2}$  is irrational.

*Proof.* Suppose, for contradiction, that  $\sqrt{2}$  is rational:  $\sqrt{2} = a/b$  for integers  $a, b$  with no common factor (in lowest terms). Squaring,  $2 = a^2/b^2$ , so  $a^2 = 2b^2$  – hence  $a^2$  is even, hence  $a$  is even (an odd number squares to an odd number), say  $a = 2c$ . Substituting:  $4c^2 = 2b^2$ , so  $b^2 = 2c^2$  – hence  $b^2$  is even, hence  $b$  is even. But then  $a$  and  $b$  are *both* even, contradicting that  $a/b$  was in lowest terms. So the assumption was false:  $\sqrt{2}$  is irrational. ■

*Note.* Alongside Euclid’s proof, this is the other canonical proof-by-contradiction example in mathematics – both hinge on deriving a property (divisibility, in each case) that eventually contradicts an assumption baked into the setup (finiteness of the prime list; lowest-terms form of the fraction).

*Note* (Open problems, Rosen). Not every true mathematical statement has a known proof. An *open problem* is a precisely stated claim believed true (or at least undecided) but not yet proved or disproved.

**Example 3.6** (The Collatz conjecture). Define, for  $n \in \mathbb{Z}^+$ :

$$\text{Collatz}(n) = \begin{cases} n/2, & n \text{ even,} \\ 3n + 1, & n \text{ odd.} \end{cases}$$

*Conjecture:* iterating Collatz from any starting  $n \in \mathbb{Z}^+$  eventually reaches 1.

*Note.* Verified by computer for enormous ranges of starting values, but no proof (or disproof) is known – an active open problem since 1937. Contrast with Euclid’s infinitude-of-primes proof (§3.9 above): that claim, once conjectured, was fully settled by a short contradiction argument; Collatz has resisted proof for nearly a century despite its simple statement.

### 3.10 Proof by mathematical induction

*Note* (Principle of mathematical induction). To prove  $S(n)$  holds for all  $n \in \mathbb{N}$  (or all  $n \geq n_0$ ):

- **Base case:** prove  $S(1)$  (or  $S(n_0)$ ).
- **Inductive step:** prove, for all  $k$ :

$$S(k) \implies S(k+1)$$

(the assumption  $S(k)$  here is the *inductive hypothesis*).

Conclude  $S(n)$  holds for all  $n$ .

*Note* (Why this works).  $S(1)$  holds (base case). Then:

$$S(1) \implies S(2), \quad S(2) \implies S(3), \quad S(3) \implies S(4), \quad \dots$$

Induction packages this infinite chain of modus ponens applications into a single finite argument, rather than an informal “and so on forever.”

**Theorem 3.12** (Generalized De Morgan’s law, CN Thm 18). *For all  $n \in \mathbb{N}$  and sets  $A_1, \dots, A_n$ :*

$$\overline{A_1 \cup A_2 \cup \dots \cup A_n} = \overline{A_1} \cap \overline{A_2} \cap \dots \cap \overline{A_n}.$$

*Proof.* Let  $S(n)$  be this identity for a given  $n$ .

**Base case** ( $n = 1$ ):

$$\overline{A_1} = \overline{A_1}.$$

**Inductive step:** assume  $S(k)$ , i.e.

$$\overline{A_1 \cup \dots \cup A_k} = \overline{A_1} \cap \dots \cap \overline{A_k}.$$

Then

$$\overline{A_1 \cup \dots \cup A_k \cup A_{k+1}} = \overline{(A_1 \cup \dots \cup A_k) \cup A_{k+1}} = \overline{(A_1 \cup \dots \cup A_k)} \cap \overline{A_{k+1}}$$

by ordinary De Morgan’s law (§1.11). Applying  $S(k)$  to the first factor:

$$= (\overline{A_1} \cap \dots \cap \overline{A_k}) \cap \overline{A_{k+1}} = \overline{A_1} \cap \dots \cap \overline{A_{k+1}},$$

which is  $S(k+1)$ .

By induction,  $S(n)$  holds for all  $n$ . ■



**Theorem 3.13** (Divisibility example). *For all  $n \in \mathbb{N}$ :  $3 \mid (n^3 - n)$ .*

*Proof.* **Base case** ( $n = 0$ ):  $n^3 - n = 0$ , and  $3 \mid 0$ .

**Inductive step:** assume  $3 \mid (k^3 - k)$  for some  $k \geq 0$ , i.e.  $k^3 - k = 3m$  for some  $m \in \mathbb{Z}$ . Consider  $n = k + 1$ :

$$(k+1)^3 - (k+1) = k^3 + 3k^2 + 3k + 1 - k - 1 = (k^3 - k) + 3k^2 + 3k = 3m + 3k^2 + 3k = 3(m + k^2 + k).$$

This is 3 times an integer, so  $3 \mid ((k+1)^3 - (k+1))$ .

By induction,  $3 \mid (n^3 - n)$  for all  $n \in \mathbb{N}$ . ■

**Definition 3.10** (Well-ordering property). Every nonempty subset of  $\mathbb{N}$  has a least element.

**Theorem 3.14** (Well-ordering  $\implies$  mathematical induction). *If the well-ordering property holds for  $\mathbb{N}$ , then the principle of mathematical induction is valid.*

*Proof.* Suppose  $S(1)$  holds and  $S(k) \implies S(k+1)$  for all  $k \geq 1$ . Suppose, for contradiction,  $S(n)$  fails for some  $n$ . Let

$$F = \{n \in \mathbb{Z}^+ : S(n) \text{ is false}\}.$$

$F \neq \emptyset$  by assumption, so by well-ordering,  $F$  has a least element  $m$ . Since  $S(1)$  holds,  $m \neq 1$ , so  $m \geq 2$  and  $m-1 \geq 1$ . Since  $m$  is the *least* element of  $F$ ,  $m-1 \notin F$ , i.e.  $S(m-1)$  holds. But then the inductive step gives  $S(m-1) \implies S(m)$ , so  $S(m)$  holds – contradicting  $m \in F$ .

So  $F = \emptyset$ :  $S(n)$  holds for all  $n$ . ■

*Note.* This is why induction and well-ordering are often called *equivalent* principles over  $\mathbb{N}$  – each can be used to derive the other, and most texts (Rosen included) simply take one as an axiom and prove the other as a consequence.

**Definition 3.11** (Principle of strong induction, formal statement). To prove  $S(n)$  for all  $n \geq n_0$ :

- **Base case:** prove  $S(n_0)$ .
- **Inductive step:** prove, for all  $k \geq n_0$ : if  $S(n_0), S(n_0+1), \dots, S(k)$  all hold, then  $S(k+1)$  holds.

*Note* (Strong vs. weak induction – equivalent in power). Strong induction’s hypothesis is more generous (all of  $S(n_0), \dots, S(k)$ , not just  $S(k)$ ), but this doesn’t let you prove anything weak induction can’t – any strong-induction proof can be mechanically rewritten as a weak-induction proof of the stronger statement  $T(n) = S(n_0) \wedge \dots \wedge S(n)$ . Strong induction is a convenience, not a logically stronger tool. See §3.11 for worked examples of both.

### 3.11 Induction: more examples

**Theorem 3.15** (Sum of the first  $n$  positive integers, CN Thm 19). *For all  $n \geq 1$ :*

$$1 + 2 + \dots + n = \frac{n(n+1)}{2}.$$

*Proof.* **Base case** ( $n = 1$ ): LHS = 1, RHS =  $1 \cdot 2/2 = 1$ .

**Inductive step:** assume  $1 + \dots + k = k(k+1)/2$  for some  $k \geq 1$ . Then

$$1 + \dots + k + (k+1) = \frac{k(k+1)}{2} + (k+1) = (k+1) \left( \frac{k}{2} + 1 \right) = \frac{(k+1)(k+2)}{2},$$

which is the formula for  $n = k+1$ .

By induction, the formula holds for all  $n \geq 1$ . ■

**Theorem 3.16** ( $2^n > n^2$  for  $n \geq 5$ , Rosen – inequality with nontrivial base). *For every integer  $n \geq 5$ :  $2^n > n^2$ .*

*Proof.* **Base case** ( $n = 5$ ):  $2^5 = 32 > 25 = 5^2$ .

**Inductive step:** let  $k \geq 5$ , assume  $2^k > k^2$ . We want  $2^{k+1} > (k+1)^2$ .

$$2^{k+1} = 2 \cdot 2^k > 2k^2 \quad (\text{inductive hypothesis}).$$

It suffices to show  $2k^2 \geq (k+1)^2$  for  $k \geq 5$ :

$$2k^2 - (k+1)^2 = k^2 - 2k - 1 = (k-1)^2 - 2,$$

which is positive whenever  $(k-1)^2 > 2$ , i.e. certainly for  $k \geq 5$ . So  $2k^2 > (k+1)^2$ , hence  $2^{k+1} > 2k^2 > (k+1)^2$ .

By induction,  $2^n > n^2$  for all  $n \geq 5$ . ■

*Note* (Why the base case matters here). The inequality is *false* for  $n = 1, 2, 3, 4$ . Getting the base case right – and matching it to where the inductive step’s algebra actually works – is essential; a careless choice of base case can make the whole proof invalid even though the inductive step itself is correctly argued.

**Theorem 3.17** (Bernoulli’s inequality, Rosen). *For all  $x \geq -1$  and all integers  $n \geq 0$ :*

$$(1+x)^n \geq 1+nx.$$

*Proof.* **Base case** ( $n = 0$ ):  $(1+x)^0 = 1 = 1 + 0 \cdot x$ .

**Inductive step:** assume  $(1+x)^k \geq 1+kx$  for some  $k \geq 0$ . Since  $x \geq -1$ , we have  $1+x \geq 0$ , so multiplying both sides of the inductive hypothesis by  $(1+x)$  preserves the inequality:

$$(1+x)^{k+1} = (1+x)^k(1+x) \geq (1+kx)(1+x) = 1+kx+x+kx^2 = 1+(k+1)x+kx^2.$$

Since  $kx^2 \geq 0$ :

$$(1+x)^{k+1} \geq 1+(k+1)x.$$

By induction, the inequality holds for all  $n \geq 0$ . ■

*Note.* The condition  $x \geq -1$  is exactly what’s needed for  $1+x \geq 0$ , which is what allows multiplying an inequality through by  $(1+x)$  without flipping its direction.

**Theorem 3.18** (Harmonic numbers, Rosen). *Let  $H_m = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{m}$ . Then for every  $n \geq 0$ :*

$$H_{2^n} \geq 1 + \frac{n}{2}.$$

*Proof.* **Base case** ( $n = 0$ ):  $H_{2^0} = H_1 = 1 \geq 1 + 0$ .

**Inductive step:** assume  $H_{2^k} \geq 1 + \frac{k}{2}$  for some  $k \geq 0$ . Then

$$H_{2^{k+1}} = H_{2^k} + \frac{1}{2^k + 1} + \frac{1}{2^k + 2} + \cdots + \frac{1}{2^{k+1}}.$$

There are  $2^k$  terms in this added block, each  $\geq \frac{1}{2^{k+1}}$ , so the block sums to at least  $2^k \cdot \frac{1}{2^{k+1}} = \frac{1}{2}$ . Hence

$$H_{2^{k+1}} \geq \left(1 + \frac{k}{2}\right) + \frac{1}{2} = 1 + \frac{k+1}{2}.$$

By induction,  $H_{2^n} \geq 1 + \frac{n}{2}$  for all  $n \geq 0$ . ■

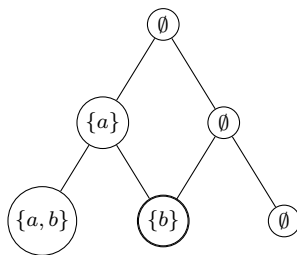
*Note.* Since  $1 + n/2 \rightarrow \infty$  as  $n \rightarrow \infty$ , this proves the harmonic series diverges.

**Theorem 3.19** (Counting subsets, Rosen – revisits §1.8 with a picture). *A set with  $n$  elements has  $2^n$  subsets.*

*Proof.* **Base case** ( $n = 0$ ):  $\emptyset$  has exactly  $1 = 2^0$  subset (itself).

**Inductive step:** assume a  $k$ -element set has  $2^k$  subsets. A  $(k+1)$ -element set  $S = T \cup \{x\}$  (with  $|T| = k$ ,  $x \notin T$ ) has subsets falling into two kinds: those not containing  $x$  (exactly the  $2^k$  subsets of  $T$ ), and those containing  $x$  (each is  $\{x\} \cup U$  for a subset  $U$  of  $T$ , again  $2^k$  of them). Total:  $2^k + 2^k = 2^{k+1}$ .

By induction, an  $n$ -element set has  $2^n$  subsets. ■



Building all subsets of  $\{a, b\}$  by deciding “include  $a$ ?” then “include  $b$ ?” – each level doubles the count

**Definition 3.12** (Depth of a set of intervals, Rosen). For lectures given as time intervals, the *depth* is the maximum number that mutually overlap at any single instant.

**Theorem 3.20** (Greedy lecture-scheduling algorithm is optimal, Rosen). *Sorting lectures by start time and greedily assigning each to the first available room needs exactly depth rooms – optimal, since no schedule can use fewer than depth rooms.*

---

**Algorithm 2:** Greedy lecture scheduling

---

**Input:** Lectures as intervals  $[s_1, f_1], \dots, [s_n, f_n]$ **Output:** Room assignment using depth rooms

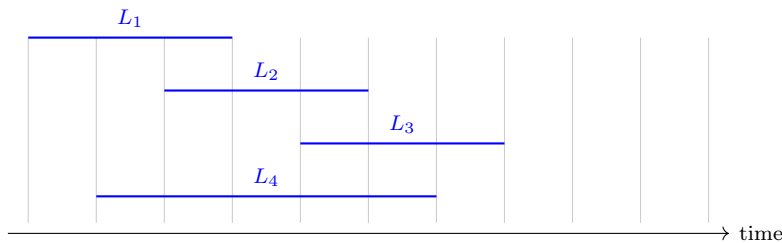
```
1 sort lectures by increasing start time  $s_i$ ;  
2 rooms  $\leftarrow$  empty list;  
3 foreach lecture  $L$  in sorted order do  
4   if some room in rooms is free at  $L$ 's start time then  
5     | assign  $L$  to any such free room;  
6   else  
7     | open a new room, assign  $L$  to it;  
8   end  
9 end
```

---

*Proof sketch, induction on the number of lectures scheduled.* **Base case:** the first lecture needs 1 room;  $1 \leq \text{depth}$ .

**Inductive step:** suppose the first  $k$  lectures (in start-time order) have been assigned using at most depth rooms. When lecture  $k+1$  arrives, if all currently-open rooms are busy at that moment, the number of busy rooms equals the number of lectures overlapping at that instant – at most depth. If exactly depth rooms are busy, opening one more room suffices and keeps the total at depth; if fewer are busy, an existing free room is reused.

By induction, the greedy algorithm never needs more than depth rooms – matching the lower bound, hence optimal. ■



$L_1$  and  $L_4$  overlap  $L_2$ ;  $L_4$  overlaps  $L_3$  too – depth = 2 at every instant, so 2 rooms suffice

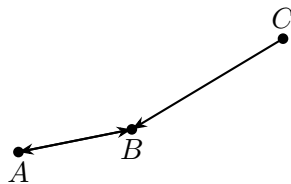
**Theorem 3.21** (Odd pie fight – creative induction). *An odd number  $n$  of people stand at pairwise distinct distances. Each simultaneously throws a pie at their nearest neighbor. Then at least one person is not hit.*

*Proof.* Induction on odd  $n$  (i.e.  $n = 2m + 1$ , induct on  $m \geq 1$ ).

**Base case** ( $n = 3$ ): let  $A, B, C$  be the three people. Some pair – say  $A, B$  – is the *closest* pair overall. Then  $A, B$  pie each other;  $C$ 's pie goes to whichever of  $A, B$  is closer to  $C$ . Either way,  $A$  or  $B$  is hit twice and  $C$  isn't hit – a survivor exists.

**Inductive step:** assume the claim for  $n = 2k + 1$  people, and consider  $n = 2k + 3$ . Find the overall closest pair  $A, B$  – they pie each other. Remove  $A, B$ : the other  $2k + 1$  people still throw pies among themselves at mutually distinct distances – a valid instance of the same problem. By the inductive hypothesis, this group has a survivor, who also survives in the full group.

By induction, every odd  $n$  has a survivor. ■



Base case  $n = 3$ :  $A, B$  are the closest pair and pie each other;  $C$  pies  $B - A$  survives

*Note.* False for even  $n$ : e.g. 4 people can be arranged in two mutual-nearest-neighbor pairs, hitting everyone.

**Theorem 3.22** (Deficient chessboard tiling, ordinary induction – Rosen §5.1). *For every  $n \geq 1$ , a  $2^n \times 2^n$  chessboard with any one square removed can be exactly tiled by L-trominoes.*

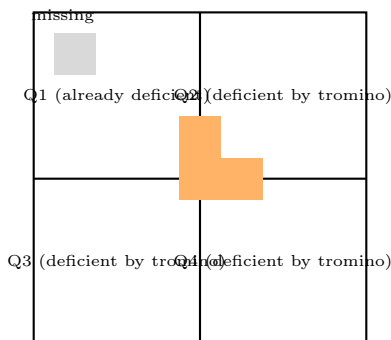
*Proof.* **Base case** ( $n = 1$ ): a  $2 \times 2$  board minus one square is exactly one L-tromino.

**Inductive step:** assume every deficient  $2^k \times 2^k$  board can be tiled. Consider a deficient  $2^{k+1} \times 2^{k+1}$  board, split into four  $2^k \times 2^k$  quadrants.

The missing square lies in exactly one quadrant – deficient, tileable by the inductive hypothesis. Place one L-tromino at the very center, covering the corner cell of *each* of the other three quadrants nearest the center – making each of those quadrants deficient too.

One central tromino plus four tiled quadrants tiles the whole board.

By induction (using only  $P(k) \Rightarrow P(k+1)$  – this is *ordinary*, not strong, induction, despite the recursive flavor), the claim holds for all  $n \geq 1$ . ■



One tromino at the center makes the other three quadrants deficient too

**Theorem 3.23** (Postage stamp problem, Rosen – strong induction). *Every integer  $n \geq 12$  can be formed using only 4-cent and 5-cent stamps.*

*Proof.* **Base cases** ( $n = 12, 13, 14, 15$ ):

$$12 = 4 + 4 + 4, \quad 13 = 4 + 4 + 5, \quad 14 = 4 + 5 + 5, \quad 15 = 5 + 5 + 5.$$

**Inductive step:** let  $k \geq 15$ , and suppose every integer from 12 to  $k$  can be formed. Consider  $k+1$ . Since  $k+1 \geq 16$ , we have  $k+1-4 \geq 12$ , so by the strong inductive hypothesis,  $k+1-4$  can be formed. Add one more 4-cent stamp to form  $k+1$ .

By strong induction, every  $n \geq 12$  can be formed. ■

*Note* (Why strong induction, not weak). The step for  $k + 1$  reaches back to  $k + 1 - 4$ , which for small  $k$  is *not* simply  $k$  or  $k - 1$  – ordinary induction, which only hands you  $S(k)$ , isn't enough here.

**Theorem 3.24** (Existence of prime factorization – classic strong induction example). *Every integer  $n \geq 2$  can be written as a product of one or more primes.*

*Proof.* **Base case** ( $n = 2$ ): 2 itself is prime.

**Inductive step:** let  $k \geq 2$ , and suppose every integer from 2 to  $k$  has a prime factorization. Consider  $n = k + 1$ .

**Case 1:**  $n$  is prime. Then  $n$  itself is its factorization.

**Case 2:**  $n$  is composite, so  $n = ab$  for integers  $2 \leq a, b \leq n - 1 = k$ . By the strong inductive hypothesis, both  $a$  and  $b$  have prime factorizations; concatenating them gives a prime factorization of  $n = ab$ .

Either way,  $n$  has a prime factorization.

By strong induction, every  $n \geq 2$  has one. ■

*Note.* This needs *strong* induction: the factors  $a, b$  of composite  $n$  can be much smaller than  $n - 1$ , so the hypothesis must cover *all* smaller values.

**Theorem 3.25** (Division algorithm – existence part, via well-ordering). *For  $a \in \mathbb{Z}$ ,  $d \in \mathbb{Z}^+$ , there exist  $q, r \in \mathbb{Z}$  with  $a = dq + r$  and  $0 \leq r < d$ .*

*Proof.* Consider  $S = \{a - dq : q \in \mathbb{Z}, a - dq \geq 0\}$ .  $S \subseteq \mathbb{N}$  and  $S \neq \emptyset$ . By well-ordering,  $S$  has a least element  $r = a - dq_0$ .

We claim  $r < d$ . If not,  $r - d = a - d(q_0 + 1) \in S$ , but  $r - d < r$ , contradicting minimality of  $r$ .

So  $r < d$ , and  $q = q_0$  gives  $a = dq + r$  with  $0 \leq r < d$ . ■

**Example 3.7** (Game-strategy example, Rosen – strong induction, non-numeric). Two players alternately remove 1 or 2 stones from a pile; the player taking the last stone wins. Claim: the first player wins whenever  $n \not\equiv 0 \pmod{3}$ , and loses whenever  $n \equiv 0 \pmod{3}$ .

*Proof sketch, strong induction on  $n$ .* **Base cases:**  $n = 1, 2$ : first player wins.  $n = 3$ : whatever the first player takes, the opponent wins – first player loses.

**Inductive step:** assume the claim for all pile sizes less than  $n$ . If  $n \equiv 0 \pmod{3}$ : any move leaves  $n - 1$  or  $n - 2$ , neither  $\equiv 0 \pmod{3}$ , so the opponent wins – first player loses. If  $n \not\equiv 0 \pmod{3}$ : the first player can always leave a multiple of 3, putting the opponent in a losing position. ■

**Definition 3.13** (Simple polygon, diagonal). A *simple polygon* is a polygon whose sides do not cross. A *diagonal* is a line segment connecting two non-adjacent vertices that lies entirely inside the polygon.

**Lemma 3.1** (Every simple polygon with  $\geq 4$  sides has a diagonal).

*Proof sketch.* Take the leftmost vertex  $v$  and its two neighbors  $u, w$ . If segment  $uw$  lies inside the polygon, it is a diagonal. Otherwise, some vertex lies inside triangle  $uvw$ ; take the one farthest from line  $uw$  – the segment from  $v$  to that vertex is a diagonal. ■

**Theorem 3.26** (Triangulation of simple polygons – Rosen, strong induction). *Every simple polygon with  $n \geq 3$  sides can be triangulated into exactly  $n - 2$  triangles using  $n - 3$  non-crossing diagonals.*

*Proof.* **Base case** ( $n = 3$ ): a triangle is already  $1 = 3 - 2$  triangle, using  $0 = 3 - 3$  diagonals.

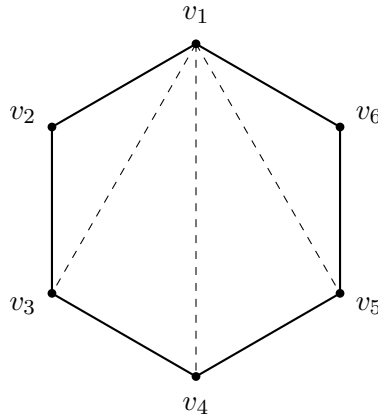
**Inductive step:** let  $k \geq 3$ , and suppose every simple polygon with fewer than  $k + 1$  sides can be triangulated as claimed. Let  $P$  have  $n = k + 1 \geq 4$  sides.

By the lemma,  $P$  has a diagonal  $d$ , splitting  $P$  into  $P_1, P_2$  sharing edge  $d$ , with  $n_1 + n_2 = n + 2$ .

Since  $3 \leq n_1, n_2 \leq n - 1$ , the strong inductive hypothesis applies to both:  $P_1$  triangulates into  $n_1 - 2$  triangles,  $P_2$  into  $n_2 - 2$ .

Together:  $(n_1 - 2) + (n_2 - 2) = n - 2$  triangles, using  $1 + (n_1 - 3) + (n_2 - 3) = n - 3$  diagonals.

By strong induction, the claim holds for all  $n \geq 3$ . ■



A hexagon triangulated by 3 diagonals into 4 triangles

**Theorem 3.27** (The matchstick game – Rosen §5.2, Example 4, strong induction). *Two piles start with  $n$  matches each. Players alternate turns; each turn, remove any positive number of matches from one pile. The player who removes the last match wins. Then the second player can always force a win, for every  $n \geq 1$ .*

*Proof.* Let  $P(n)$ : “if both piles start with  $n$  matches, the second player can force a win.”

**Base case** ( $n = 1$ ): the first player must remove the single match from one pile; the second takes the other and wins.

**Inductive step:** let  $k \geq 1$ , suppose  $P(1), \dots, P(k)$  hold. Whatever the first player does – removing  $r$  matches from one pile – the second mirrors on the other pile.

- If  $r = k + 1$ : the second empties the other pile, winning immediately.
- If  $r < k + 1$ : both piles now have equal size  $k + 1 - r \leq k$  – by  $P(k + 1 - r)$ , the second player can force a win from there.

Either way, the second player wins. By strong induction,  $P(n)$  holds for all  $n \geq 1$ . ■

**Theorem 3.28** (Round-robin tournament ranking – classic strong induction example). *In a round-robin tournament with  $n$  players, the players can be labeled  $p_1, \dots, p_n$  such that  $p_i$  beats  $p_{i+1}$  for every  $i$ .*

*Proof.* **Base case** ( $n = 1$ ): trivial.

**Inductive step:** let  $k \geq 1$ , suppose the claim holds for at most  $k$  players. Consider  $k+1$  players. Pick any player  $x$ ; let  $B =$  players who beat  $x$ ,  $L =$  players  $x$  beats, so  $|B|, |L| \leq k$ .

By the inductive hypothesis,  $B$  has a chain ordering  $b_1, \dots, b_{|B|}$ , and  $L$  has  $\ell_1, \dots, \ell_{|L|}$ . Concatenate:

$$b_1, \dots, b_{|B|}, x, \ell_1, \dots, \ell_{|L|}.$$

The joins  $b_{|B|} \rightarrow x$  and  $x \rightarrow \ell_1$  hold by definition of  $B, L$ .

By strong induction, the claim holds for all  $n \geq 1$ . ■

### 3.12 Induction: extended example

*Note (Setup).*  $m$  job positions  $A$  ( $|A| = m$ ),  $n$  applicants  $B$  ( $|B| = n$ ), a *shortlist relation*  $S \subseteq A \times B$ .

For  $X \subseteq A$ , write

$$S(X) = \bigcup_{a \in X} S(a)$$

(§2.13 notation) for the combined shortlist of positions in  $X$ .

A valid assignment (every position filled, no applicant doubled) is exactly an *injection*  $A \rightarrow B$  contained in  $S$ .

**Theorem 3.29** (Necessity, CN Thm 20). *If  $S$  contains an injection with domain  $A$ , then  $|X| \leq |S(X)|$  for every  $X \subseteq A$ .*

*Proof.* An injection assigns distinct applicants to distinct positions, so the positions in  $X$  alone already require  $|X|$  distinct applicants, all drawn from  $S(X)$ . ■

**Theorem 3.30** (Sufficiency, CN Thm 21 – the converse, by induction on  $|A|$ ). *If  $|X| \leq |S(X)|$  for every  $X \subseteq A$ , then  $S$  contains an injection with domain  $A$ .*

*Proof.* Induction on  $m = |A|$ .

**Base case** ( $m = 1$ ):

$A = \{a\}$ ; the condition with  $X = \{a\}$  gives  $|S(a)| \geq 1$ , so pick any  $(a, b) \in S$  – this single pair is an injection with domain  $A$ .

**Inductive step:** assume the theorem for all relations with domain-size  $\leq m$ . Let  $|A| = m + 1$  and suppose  $|X| \leq |S(X)|$  for all  $X \subseteq A$ .

**Case 1:** every nonempty proper  $X \subsetneq A$  has *strict slack*,

$$|X| \leq |S(X)| - 1.$$

Pick any  $(a, b) \in S$ . Restrict  $S$  to

$$T = S \cap ((A \setminus \{a\}) \times (B \setminus \{b\})),$$

i.e. drop  $a$  and  $b$ . For  $X \subseteq A \setminus \{a\}$ , the strict slack gives  $|T(X)| \geq |X|$  even after excluding  $b$ .

Since  $|A \setminus \{a\}| = m$ , the inductive hypothesis applies:  $T$  contains an injection  $g$  with domain  $A \setminus \{a\}$ .

Then  $g \cup \{(a, b)\}$  is an injection with domain  $A$ , contained in  $S$ .



**Case 2:** some nonempty proper  $X \subsetneq A$  has *exact* fit,

$$|X| = |S(X)|.$$

Apply the inductive hypothesis to  $S$  restricted to  $X \times S(X)$  (domain size  $|X| \leq m$ ): get an injection  $g : X \rightarrow S(X)$ .

For the rest,  $A \setminus X$ : for any  $Z \subseteq A \setminus X$ , a counting argument using  $|X \cup Z| \leq |S(X \cup Z)|$  and  $|S(X)| = |X|$  shows

$$|Z| \leq |S(Z) \setminus S(X)|.$$

So the relation  $U = S \cap ((A \setminus X) \times (B \setminus S(X)))$  satisfies the same slack condition on domain  $A \setminus X$  (size  $\leq m$ ), so by the inductive hypothesis,  $U$  contains an injection  $h$  with domain  $A \setminus X$ .

Since  $g, h$  have disjoint domains ( $X, A \setminus X$ ) and disjoint codomains ( $S(X), B \setminus S(X)$ ),  $g \cup h$  is an injection with domain  $A$ , contained in  $S$ .

Cases 1 and 2 are exhaustive, completing the inductive step. By induction, the theorem holds for all  $m$ . ■

**Corollary 3.1** (CN Cor. 22).  $S \subseteq A \times B$  contains an injection with domain  $A \iff |X| \leq |S(X)|$  for every  $X \subseteq A$ .

*Note* (Why this matters). This gives easily *verifiable* evidence in both directions: a “yes” is witnessed by the injection itself (checkable directly); a “no” is witnessed by a single offending set  $X$  with  $|X| > |S(X)|$  – no need to search the whole space of possible assignments. This is a defect-form statement of *Hall’s marriage theorem*.

### 3.13 Mathematical induction and statistical induction

*Note. Statistical induction* (drawing general conclusions from data, typically under randomness, tolerating some error) is unrelated to *mathematical induction* despite the shared name. Statistical induction is not logical deduction and cannot serve as a proof step; mathematical induction is a rigorous deductive technique. Don’t conflate the two.

### 3.14 Programs and proofs

*Note* (The analogy, point by point). • **Statements:** a program is a sequence of statements in a programming language; a proof is a sequence of statements in mathematical language (augmented by precise natural language).

- **Variables:** both use named references to objects; program variables have allocated memory, proof variables do not.
- **Sequential dependence:** a program statement uses only the most recently computed value of a variable, never a later one; a proof statement is justified only by *earlier* statements, never later ones.
- **Branching:** an if-statement picks a branch by a logical condition; proof by cases (§3.8) covers all situations via a finite set of logical conditions.
- **Calling other code:** a program calls other functions/programs; a proof invokes previously proved theorems.

- **Repetition:** a program repeats via loops or recursion; a proof repeats (in effect) via mathematical induction.
- **Bugs:** a program with a bug may crash on some inputs but still work on others; a proof with a genuine gap is *not a proof at all* – it must be repaired, or the claim may simply be false.

*Note* (Loops  $\leftrightarrow$  induction, Rosen §5.5). Proving a loop computes the correct result typically uses a *loop invariant*: a statement  $I$  that (i) holds before the loop begins, and (ii) if true before an iteration, remains true after it – exactly the base case and inductive step of mathematical induction, applied to “iteration number.” If  $I$  also implies the desired postcondition once the loop terminates, this establishes *partial correctness*.

*Note* (Recursion  $\leftrightarrow$  structural induction, Rosen §5.3-5.4). Proving a recursive function correct mirrors proving a recursively-defined structure has some property: verify the function is correct on the base case(s) of the recursion, then verify that, *assuming* correctness on smaller/simpler inputs (the recursive calls), the function is correct on the current input – the direct analogue of an inductive hypothesis.

*Note* (Programming paradigms). Just as learning multiple programming paradigms (procedural, object-oriented, logic, functional) improves programming skill generally, learning to write rigorous proofs – a distinct “paradigm” of precise reasoning – transfers back to writing better, more carefully-reasoned programs.

## 4 Propositional Logic

### 4.1 Truth values

**Definition 4.1** (Truth values). Logic (classical, two-valued) is built on two *truth values*: *True* ( $T$ ) and *False* ( $F$ ).

*Note.* Physically, truth values may correspond to presence/absence of electrical current, magnetisation, a switch's on/off state, etc. – the abstraction lets us reason about logic independently of implementation. The *bit* (0 or 1) is a closely related abstraction:  $F \leftrightarrow 0, T \leftrightarrow 1$ .

### 4.2 Boolean variables

**Definition 4.2** (Boolean / propositional variable). A *Boolean variable* (or *propositional variable*) is a variable whose value is  $T$  or  $F$ .

### 4.3 Propositions

**Definition 4.3** (Proposition, Rosen). A *proposition* is a declarative sentence that is either true or false, but not both.

**Example 4.1** (Propositions).  $1 + 1 = 2$  (true); “The earth is flat.” (false); “It will rain tomorrow.” (a proposition – has a definite truth value, even if unknown to us right now).

**Example 4.2** (Not propositions). Questions (“Come and work for us!”), nonsensical or non-declarative sentences (Lewis Carroll’s “’Twas brillig...”), and self-referential sentences with no consistent truth value (“This statement is false.”) are not propositions.

*Note* (Naming a proposition). A proposition may be named by a Boolean variable, e.g. let  $X$  be the proposition  $1 + 1 = 2$ ; then  $X$ 's truth value is  $T$ .

### 4.4 Logical operations

|                                      |             |            |
|--------------------------------------|-------------|------------|
|                                      | Not         | $\neg$     |
|                                      | And         | $\wedge$   |
| <i>Note</i> (The basic connectives). | Or          | $\vee$     |
|                                      | Implies     | $\implies$ |
|                                      | Equivalence | $\iff$     |

Logical operations combining propositions are also called *connectives*; in hardware they are implemented as *logic gates*.

### 4.5 Negation

**Definition 4.4** (Negation). For proposition  $P$ ,  $\neg P$  is true iff  $P$  is false.

|     |          |
|-----|----------|
| $P$ | $\neg P$ |
| $F$ | $T$      |
| $T$ | $F$      |

**Theorem 4.1** (Double negation).

$$\neg\neg P = P.$$

*Note* (Analogy with set complement). Negation mirrors complementation (§1.11): both give the “completely opposite” outcome, and applying either twice returns the original. If  $P$  is the proposition  $x \in A$ , then  $\neg P$  is  $x \in \bar{A}$ : e.g.  $\neg(\sqrt{2} \in \mathbb{Q})$  is the same as  $\sqrt{2} \in \bar{\mathbb{Q}}$  (equivalently  $\sqrt{2} \notin \mathbb{Q}$ ).

## 4.6 Conjunction

**Definition 4.5** (Conjunction). For propositions  $P, Q$ :  $P \wedge Q$  (“ $P$  and  $Q$ ”) is true iff *both*  $P$  and  $Q$  are true.

| $P$ | $Q$ | $P \wedge Q$ |
|-----|-----|--------------|
| $F$ | $F$ | $F$          |
| $F$ | $T$ | $F$          |
| $T$ | $F$ | $F$          |
| $T$ | $T$ | $T$          |

*Note* (Analogy with set intersection). For object  $x$  and sets  $A, B$ :

$$(x \in A) \wedge (x \in B) \iff x \in A \cap B, \quad A \cap B = \{x : x \in A \wedge x \in B\}$$

(restating §1.12’s definition using  $\wedge$ ).

**Definition 4.6** (Generalized (n-ary) conjunction, Rosen). For propositions  $P_1, \dots, P_n$ :

$$P_1 \wedge P_2 \wedge \dots \wedge P_n$$

is true iff *every*  $P_i$  is true.

**Example 4.3** (Translating English “and”, Rosen). “You can access the internet from campus only if you are a computer science major or you are not a freshman” involves nested connectives; more simply, “It is below freezing and snowing” translates directly as  $P \wedge Q$  for  $P$  = “it is below freezing”,  $Q$  = “it is snowing”. Care is needed when English sentences bury an implicit “and” inside a single clause (e.g. “ $x$  is between 2 and 5” is really  $(x > 2) \wedge (x < 5)$ ).

## 4.7 Disjunction

**Definition 4.7** (Disjunction). For propositions  $P, Q$ :  $P \vee Q$  (“ $P$  or  $Q$ ”) is true iff *at least one* of  $P, Q$  is true.

| $P$ | $Q$ | $P \vee Q$ |
|-----|-----|------------|
| $F$ | $F$ | $F$        |
| $F$ | $T$ | $T$        |
| $T$ | $F$ | $T$        |
| $T$ | $T$ | $T$        |

*Note* (Inclusive, not exclusive). This is the *inclusive-or*: true even when *both*  $P, Q$  are true. Contrast with *exclusive-or* (§4.11), true precisely when *exactly one* holds.

**Example 4.4** (English “or” is ambiguous, Rosen). “Students who have taken CS100 or CS200 may take this class” – inclusive: having taken both still qualifies. “Soup or salad comes with this entree” – exclusive in ordinary restaurant use: you get one, not both, even though the sentence uses the same word “or.” Mathematical  $\vee$  always means *inclusive-or* unless stated otherwise; natural language is not reliable here.

*Note* (Analogy with set union). For object  $x$  and sets  $A, B$ :

$$(x \in A) \vee (x \in B) \iff x \in A \cup B, \quad A \cup B = \{x : x \in A \vee x \in B\}$$

(restating §1.12’s definition using  $\vee$ ).

*Note* (Operator precedence so far, Rosen). Among the connectives introduced up to this point, precedence (highest to lowest) is:

$$\neg > \wedge > \vee.$$

So  $\neg P \vee Q$  means  $(\neg P) \vee Q$ , and  $P \wedge Q \vee R$  means  $(P \wedge Q) \vee R$  – parenthesize explicitly whenever there’s any doubt.

## 4.8 De Morgan’s Laws

**Theorem 4.2** (De Morgan’s Laws for propositions).

$$\neg(P \vee Q) = \neg P \wedge \neg Q, \quad \neg(P \wedge Q) = \neg P \vee \neg Q.$$

*Proof of the first law, by truth table.*

| $P$ | $Q$ | $P \vee Q$ | $\neg(P \vee Q)$ | $\neg P$ | $\neg Q$ | $\neg P \wedge \neg Q$ |
|-----|-----|------------|------------------|----------|----------|------------------------|
| $F$ | $F$ | $F$        | $T$              | $T$      | $T$      | $T$                    |
| $F$ | $T$ | $T$        | $F$              | $T$      | $F$      | $F$                    |
| $T$ | $F$ | $T$        | $F$              | $F$      | $T$      | $F$                    |
| $T$ | $T$ | $T$        | $F$              | $F$      | $F$      | $F$                    |

The columns for  $\neg(P \vee Q)$  and  $\neg P \wedge \neg Q$  agree in every row, so the two expressions are *logically equivalent* – true or false together, for every assignment of truth values to  $P, Q$ . ■

*Note* (Building the table incrementally). The table is constructed left to right, each column using only earlier columns: start with all combinations of  $P, Q$ ; build  $P \vee Q$ ; negate it to get  $\neg(P \vee Q)$ ; separately build  $\neg P$  and  $\neg Q$ ; conjoin them to get  $\neg P \wedge \neg Q$ . Comparing the two target columns (highlighted) completes the proof.

*Note* (Deriving the second law from the first). Rather than repeating the truth-table argument, the second law follows from the first by substitution: replace  $P, Q$  with  $\neg P, \neg Q$  in the first law and simplify using double negation (§4.5).

**Theorem 4.3** (Generalized De Morgan’s law, CN Thm 23). *For all  $n$ :*

$$\neg(P_1 \vee \cdots \vee P_n) = \neg P_1 \wedge \cdots \wedge \neg P_n.$$

*Proof.*

$$\begin{aligned} \text{LHS is true} &\iff P_1 \vee \cdots \vee P_n \text{ is false} \\ &\iff P_1, \dots, P_n \text{ are all false} \\ &\iff \neg P_1, \dots, \neg P_n \text{ are all true} \\ &\iff \text{RHS is true.} \end{aligned}$$

■

*Note.* This argument works for *every*  $n$  at once – unlike the truth-table method, which would need a separate (and infinite) family of tables, one per  $n$ .

*Note* (Correspondence with the set-theoretic De Morgan's laws). Compare directly with §1.11's Theorem (De Morgan's laws for sets):  $\overline{A \cup B} = \overline{A} \cap \overline{B}$  and  $\overline{A \cap B} = \overline{A} \cup \overline{B}$  are exactly the propositional laws above, applied to the propositions  $x \in A$  and  $x \in B$ .

*Note* (Rosen: naming logical equivalence). Two compound propositions are *logically equivalent* iff their truth tables are identical in every row – written  $P \equiv Q$  or  $P \iff Q$  (a *tautology*, true under every assignment; formalized further in §4.12).

## 4.9 Implication

**Definition 4.8** (Implication).  $P \implies Q$  (“if  $P$  then  $Q$ ”) is false only when  $P$  is true and  $Q$  is false.

| $P$ | $Q$ | $P \implies Q$ |
|-----|-----|----------------|
| $F$ | $F$ | $T$            |
| $F$ | $T$ | $T$            |
| $T$ | $F$ | $F$            |
| $T$ | $T$ | $T$            |

*Note* (Rosen's phrasings).  $P \implies Q$  reads: “if  $P$  then  $Q$ ”; “ $P$  only if  $Q$ ”; “ $P$  is sufficient for  $Q$ ”; “ $Q$  is necessary for  $P$ ”; “ $Q$  whenever  $P$ ”.

## 4.10 Equivalence

**Definition 4.9** (Equivalence / biconditional).  $P \iff Q$  is true iff  $P, Q$  have the same truth value.

| $P$ | $Q$ | $P \iff Q$ |
|-----|-----|------------|
| $F$ | $F$ | $T$        |
| $F$ | $T$ | $F$        |
| $T$ | $F$ | $F$        |
| $T$ | $T$ | $T$        |

**Theorem 4.4.**

$$P \iff Q \equiv (P \implies Q) \wedge (Q \implies P).$$

*Note* (Rosen's phrasing).  $P \iff Q$  reads: “ $P$  if and only if  $Q$ ”; “ $P$  is necessary and sufficient for  $Q$ ”.

*Note* (Analogy with set equality).  $x \in A \iff x \in B$  for all  $x$  is exactly  $A = B$  (§1.6).

## 4.11 Exclusive-or

**Definition 4.10** (Exclusive-or).  $P \oplus Q$  is true iff exactly one of  $P, Q$  is true.

| $P$ | $Q$ | $P \oplus Q$ |
|-----|-----|--------------|
| $F$ | $F$ | $F$          |
| $F$ | $T$ | $T$          |
| $T$ | $F$ | $T$          |
| $T$ | $T$ | $F$          |

**Theorem 4.5.**

$$P \oplus Q = \neg(P \iff Q).$$

**Theorem 4.6** (Generalized XOR parity, CN Thm 24). *For all  $n$  and propositions  $P_1, \dots, P_n$ :  $P_1 \oplus \dots \oplus P_n$  is true iff an odd number of the  $P_i$  are true.*

*Proof.* Induction on  $n$ .

**Base case** ( $n = 1$ ):  $P_1$  alone is true iff exactly 1 (odd) of it is true.

**Inductive step:** assume the claim for  $k$  propositions. Then

$$P_1 \oplus \dots \oplus P_{k+1} = (P_1 \oplus \dots \oplus P_k) \oplus P_{k+1}.$$

By the inductive hypothesis, the first factor is  $T$  iff an odd number of  $P_1, \dots, P_k$  are true, else  $F$ . Checking all four combinations of (that factor,  $P_{k+1}$ ) against  $\oplus$ 's truth table shows the result is  $T$  exactly when the *total* count of true propositions among  $P_1, \dots, P_{k+1}$  is odd.

By induction, the claim holds for all  $n$ . ■

*Note* (Analogy with symmetric difference).  $x \in A_1 \triangle \dots \triangle A_n \iff (x \in A_1) \oplus \dots \oplus (x \in A_n)$  (§1.13).

**Example 4.5** (Knights and knaves, Rosen – a harder puzzle). On an island, every inhabitant is either a *knight* (always tells the truth) or a *knave* (always lies).  $A$  says: “ $B$  is a knave.”  $B$  says: “ $A$  and  $I$  are of opposite types.”

Let  $a$  = “ $A$  is a knight”,  $b$  = “ $B$  is a knight.”  $A$ 's statement, if  $A$  is a knight, must be true, and if  $A$  is a knave, must be false – i.e.  $a \iff \neg b$ .  $B$ 's statement “ $A, B$  are opposite types” is  $a \oplus b$ ; by the same reasoning,  $b \iff (a \oplus b)$ .

From  $a \iff \neg b$ : exactly one of  $a, b$  is true. Substitute into  $b \iff (a \oplus b)$ : since exactly one of  $a, b$  is true,  $a \oplus b$  is true, so this forces  $b$  true, hence  $a$  false. So  $A$  is a knave,  $B$  is a knight.

## 4.12 Tautologies and logical equivalence

**Definition 4.11** (Tautology). A proposition that is true under every assignment of truth values (every row of its truth table is  $T$ ).

**Definition 4.12** (Logical equivalence).  $P \equiv Q$  (written  $P = Q$ ) iff their truth tables are identical, iff  $P \iff Q$  is a tautology.

*Note* (Key equivalences).

$$\begin{aligned} \neg\neg P &= P, & \neg(P \vee Q) &= \neg P \wedge \neg Q, & \neg(P \wedge Q) &= \neg P \vee \neg Q, \\ P \implies Q &= \neg P \vee Q, & P \iff Q &= (P \implies Q) \wedge (Q \implies P), \\ P \oplus Q &= \neg(P \iff Q) = P \iff \neg Q. \end{aligned}$$

**Theorem 4.7** (Exportation law, Rosen – harder derivation).

$$(P \wedge Q) \implies R \equiv P \implies (Q \implies R).$$

*Proof.* Using  $X \implies Y = \neg X \vee Y$  repeatedly:

$$\begin{aligned} (P \wedge Q) \implies R &= \neg(P \wedge Q) \vee R = (\neg P \vee \neg Q) \vee R && \text{(De Morgan's)} \\ &= \neg P \vee (\neg Q \vee R) && \text{(associativity of } \vee) \\ &= \neg P \vee (Q \implies R) \\ &= P \implies (Q \implies R). \end{aligned}$$

■

### 4.13 History

### 4.14 Distributive Laws

**Theorem 4.8** (Distributive laws).

$$P \wedge (Q \vee R) = (P \wedge Q) \vee (P \wedge R), \quad P \vee (Q \wedge R) = (P \vee Q) \wedge (P \vee R).$$

**Theorem 4.9** (Absorption laws, Rosen – derived, not axiomatic).

$$P \vee (P \wedge Q) = P, \quad P \wedge (P \vee Q) = P.$$

*Proof.*

$$\begin{aligned} P \vee (P \wedge Q) &= (P \wedge T) \vee (P \wedge Q) && \text{(identity: } P = P \wedge T) \\ &= P \wedge (T \vee Q) && \text{(distributive law)} \\ &= P \wedge T && \text{(domination: } T \vee Q = T) \\ &= P. \end{aligned}$$

The second identity is the dual (swap  $\wedge \leftrightarrow \vee$ ,  $T \leftrightarrow F$  throughout). ■

### 4.15 Laws of Boolean algebra

*Note* (Full table, dual pairs).

$$\begin{aligned} \neg\neg P &= P \\ \neg T &= F, \quad \neg F = T \\ P \wedge Q &= Q \wedge P, \quad P \vee Q = Q \vee P && \text{(commutative)} \\ (P \wedge Q) \wedge R &= P \wedge (Q \wedge R), \quad (P \vee Q) \vee R = P \vee (Q \vee R) && \text{(associative)} \\ P \wedge P &= P, \quad P \vee P = P && \text{(idempotent)} \\ P \wedge \neg P &= F, \quad P \vee \neg P = T && \text{(complement)} \\ P \wedge T &= P, \quad P \vee F = P && \text{(identity)} \\ P \wedge F &= F, \quad P \vee T = T && \text{(domination)} \\ P \wedge (Q \vee R) &= (P \wedge Q) \vee (P \wedge R), \quad P \vee (Q \wedge R) = (P \vee Q) \wedge (P \vee R) && \text{(distributive)} \\ \neg(P \vee Q) &= \neg P \wedge \neg Q, \quad \neg(P \wedge Q) = \neg P \vee \neg Q && \text{(De Morgan's)} \end{aligned}$$

**Example 4.6** (Algebraic proof, Rosen – harder chain). Show  $\neg(P \implies Q) \equiv P \wedge \neg Q$ .

$$\begin{aligned} \neg(P \implies Q) &= \neg(\neg P \vee Q) \\ &= \neg\neg P \wedge \neg Q && \text{(De Morgan's)} \\ &= P \wedge \neg Q && \text{(double negation).} \end{aligned}$$



## 4.16 Disjunctive Normal Form

**Definition 4.13** (Literal). A logical variable, unnegated or negated once ( $X$  or  $\neg X$ ).

**Definition 4.14** (DNF). A disjunction of conjunctions of literals.

*Note* (Constructing DNF from a truth table). For each row where the expression is  $T$ : form the conjunction of literals matching that row ( $X$  if  $X = T$  in that row,  $\neg X$  if  $X = F$ ); disjoin all such conjunctions.

**Example 4.7** (Majority function of 3 variables, Rosen-style – harder).  $\text{Maj}(X, Y, Z)$  is true iff at least two of  $X, Y, Z$  are true.

| $X$ | $Y$ | $Z$ | Maj |
|-----|-----|-----|-----|
| $F$ | $F$ | $F$ | $F$ |
| $F$ | $F$ | $T$ | $F$ |
| $F$ | $T$ | $F$ | $F$ |
| $F$ | $T$ | $T$ | $T$ |
| $T$ | $F$ | $F$ | $F$ |
| $T$ | $F$ | $T$ | $T$ |
| $T$ | $T$ | $F$ | $T$ |
| $T$ | $T$ | $T$ | $T$ |

DNF (four true rows):

$$\text{Maj}(X, Y, Z) = (\neg X \wedge Y \wedge Z) \vee (X \wedge \neg Y \wedge Z) \vee (X \wedge Y \wedge \neg Z) \vee (X \wedge Y \wedge Z).$$

*Note* (Cost of DNF). Every expression has an equivalent DNF, but the number of parts can be as large as  $2^k$  for  $k$  variables – exponential blow-up. DNF makes *satisfiability* easy to certify (any one part gives a satisfying assignment) but gives no efficient way to certify a *tautology*.

## 4.17 Conjunctive Normal Form

**Definition 4.15** (CNF). A conjunction of *clauses*, each a disjunction of literals.

*Note* (Duality with DNF, via De Morgan's). If  $P$  is in CNF,  $\neg P$  – negation of a conjunction of disjunctions – becomes, via repeated De Morgan's and double negation, a disjunction of conjunctions of literals: DNF. So CNF and DNF are dual representations.

*Note* (Converting truth table  $\rightarrow$  CNF). Negate the output column; build DNF for  $\neg P$ ; negate that expression; apply De Morgan's to push the negation inward into CNF form.

*Note* (3-SAT, Rosen/CS – harder, beyond CN). When every CNF clause has exactly 3 literals, satisfiability is the classical *3-SAT* problem – the canonical NP-complete problem (Cook–Levin theorem). This is why CNF, not DNF, is the standard input format for SAT solvers: checking DNF satisfiability is trivial, but CNF satisfiability is computationally hard in general, which is exactly the structure that makes CNF expressive enough to encode real combinatorial problems.

## 4.18 Satisfiability

**Definition 4.16** (Satisfiable, unsatisfiable). A compound proposition is *satisfiable* if some assignment of truth values to its variables makes it true (a *satisfying assignment*); otherwise it is *unsatisfiable*.

*Note* (Satisfiability and normal forms). In DNF (§4.16), satisfiability is trivial to check: any one disjunct with no variable appearing both negated and unnegated gives a satisfying assignment directly. In CNF (§4.17), determining satisfiability in general is the SAT problem – NP-complete (§4.17's note on 3-SAT).

**Example 4.8** (The  $n$ -queens problem as satisfiability – CN Ex. 4.8 / Rosen §1.3). Place  $n$  queens on an  $n \times n$  chessboard so that no two attack each other (same row, column, or diagonal). Variables  $x_{i,j}$  ( $1 \leq i, j \leq n$ ): true iff a queen occupies row  $i$ , column  $j$ .

**Clause**  $\Leftrightarrow$  **at-most-one, symbolically.** For any  $x_1, \dots, x_m$ ,

$$\bigwedge_{1 \leq j < k \leq m} (\neg x_j \vee \neg x_k) \equiv \neg \exists j \neq k (x_j \wedge x_k) \equiv \text{"at most one } x_i \text{ is true"},$$

since each pairwise clause  $\neg x_j \vee \neg x_k \equiv \neg(x_j \wedge x_k)$  rules out that particular pair, and ranging over all  $\binom{m}{2}$  pairs rules out every pair.

**Exactly one queen per row  $i$ :**

$$\underbrace{x_{i,1} \vee \dots \vee x_{i,n}}_{\text{at least one}} \quad \wedge \quad \bigwedge_{1 \leq j < k \leq n} \underbrace{(\neg x_{i,j} \vee \neg x_{i,k})}_{\text{at most one}}.$$

Clause count per row:  $1 + \binom{n}{2}$ . Over  $n$  rows:

$$n \binom{n}{2} = n \cdot \frac{n(n-1)}{2} = \frac{n^3 - n^2}{2} = \Theta(n^3).$$

**At most one queen per column  $j$ :**

$$\bigwedge_{j=1}^n \bigwedge_{1 \leq i < k \leq n} (\neg x_{i,j} \vee \neg x_{k,j}), \quad n \binom{n}{2} = \Theta(n^3) \text{ clauses.}$$

*Why no "at least one" clause is needed (pigeonhole, formalised).* Let  $S = \{(i, j) : x_{i,j} = \text{true}\}$  under any satisfying assignment. The row constraints give a bijection  $\sigma : \{1, \dots, n\} \rightarrow \{1, \dots, n\}$ ,  $\sigma(i) = j$  where  $(i, j) \in S$  (well-defined and total since each row has exactly one true variable). The column constraints say  $\sigma$  is injective. A total injective function on a finite set of size  $n$  to itself is a bijection, so  $\sigma$  is surjective too – every column is hit exactly once, with no separate clause required.

**At most one queen per diagonal:** for  $(i, j) \neq (k, l)$  with  $|i - k| = |j - l|$ ,

$$\neg x_{i,j} \vee \neg x_{k,l}.$$

Fix a diagonal direction and offset  $c = i - j \in \{-(n-1), \dots, n-1\}$ ; the diagonal  $\{(i, j) : i - j = c\}$  has  $n - |c|$  cells, contributing  $\binom{n-|c|}{2}$  clauses. Summing over both directions:

$$2 \sum_{c=-(n-1)}^{n-1} \binom{n-|c|}{2} = 2 \sum_{m=1}^n \binom{m}{2} = 2 \binom{n+1}{3} = \Theta(n^3),$$

using  $\sum_{m=1}^n \binom{m}{2} = \binom{n+1}{3}$  (hockey-stick identity).

**Size.**  $n^2$  variables,  $\Theta(n^3)$  clauses total.  $\Phi_n$  satisfiable  $\iff$  a solution exists, with both directions computable in  $O(n^2)$ ; checking a candidate against  $\Phi_n$  is  $O(n^3)$ . Hence  $n$ -queens  $\in NP$ .

*Note* (Sequential at-most-one encoding – beyond Rosen). Introduce  $s_1, \dots, s_{n-1}$  with intended meaning  $s_i \equiv \bigvee_{k \leq i} x_k$ , enforced by

$$(\neg x_i \vee s_i), \quad (\neg s_i \vee s_{i+1}), \quad (\neg x_{i+1} \vee \neg s_i) \quad (1 \leq i \leq n-1).$$

*Correctness.* By induction on  $i$ : the first clause gives  $x_i \Rightarrow s_i$ ; the second propagates  $s_i \Rightarrow s_{i+1}$ , so by induction  $s_i \Rightarrow \bigvee_{k \leq i} x_k$  is forced tight (no clause forces  $s_i$  true without some  $x_k$ ,  $k \leq i$ , true, since  $s_i$  appears only positively in clauses whose other literal is  $\neg x_i$  or  $s_{i-1}$ ). The third clause gives  $x_{i+1} \wedge s_i \Rightarrow \perp$ , i.e.  $x_{i+1} \Rightarrow \neg \bigvee_{k \leq i} x_k$ : no  $x_{i+1}$  can be true while an earlier  $x_k$  is. Together this is exactly "at most one  $x_i$  true".

*Cost.*  $3(n-1) = \Theta(n)$  clauses and  $n-1 = \Theta(n)$  auxiliary variables, versus  $\binom{n}{2} = \Theta(n^2)$  pairwise clauses. Applied to the row/column groups this drops their total from  $\Theta(n^3)$  to  $\Theta(n^2)$ . The diagonal group does not benefit, since it is a union of many unequal-length "at most one" constraints on overlapping variable sets, not a single ordered list.

**Example 4.9** (Sudoku as satisfiability – symbolic derivation, cf. Example 4.8). A  $9 \times 9$  grid, digits 1–9, each row/column/ $3 \times 3$  box containing every digit exactly once, some cells pre-filled. Variables  $x_{i,j,k}$  ( $1 \leq i, j, k \leq 9$ ): true iff cell  $(i, j)$  contains digit  $k$ .

**Exactly one digit per cell  $(i, j)$ :**

$$\underbrace{x_{i,j,1} \vee \dots \vee x_{i,j,9}}_{\text{at least one}} \quad \wedge \quad \bigwedge_{1 \leq k < l \leq 9} \underbrace{(\neg x_{i,j,k} \vee \neg x_{i,j,l})}_{\text{at most one}}.$$

Over all 81 cells: 81 long clauses, plus

$$81 \binom{9}{2} = 81 \cdot \frac{9 \cdot 8}{2} = 81 \cdot 36 = 2916$$

short clauses.

**Each row  $i$  contains every digit  $k$  at least once:**

$$\bigwedge_{i=1}^9 \bigwedge_{k=1}^9 \left( x_{i,1,k} \vee x_{i,2,k} \vee \dots \vee x_{i,9,k} \right), \quad 9 \cdot 9 = 81 \text{ clauses.}$$

Why no "at most once per row-digit" clause is needed (pigeonhole, exactly as in Example 4.8). Fix row  $i$ . The cell constraints give a total function

$$\tau : \{1, \dots, 9\} \rightarrow \{1, \dots, 9\}, \quad \tau(j) = k \text{ where } (i, j, k) \in S,$$

well-defined since each cell  $(i, j)$  has exactly one  $k$  with  $x_{i,j,k}$  true. The row-digit clauses just proved say: for each  $k$ , some  $j$  has  $\tau(j) = k$ , i.e.  $\tau$  is *surjective*. A total surjective function between finite sets of equal size 9 is a bijection, hence also *injective*: no digit repeats in row  $i$ . So uniqueness in the row is a *consequence* of the clauses already written, not an extra constraint.

**Each column  $j$  contains every digit  $k$  at least once** (indices swapped):

$$\bigwedge_{j=1}^9 \bigwedge_{k=1}^9 \left( x_{1,j,k} \vee x_{2,j,k} \vee \cdots \vee x_{9,j,k} \right), \quad 81 \text{ clauses.}$$

The same bijection argument (columns in place of rows) shows no "at most once" clause is needed.

**Each  $3 \times 3$  box contains every digit  $k$  at least once:** for box  $B_{p,q} = \{(i, j) : \lceil i/3 \rceil = p, \lceil j/3 \rceil = q\}$ ,  $1 \leq p, q \leq 3$ ,

$$\bigwedge_{p,q=1}^3 \bigwedge_{k=1}^9 \bigvee_{(i,j) \in B_{p,q}} x_{i,j,k}, \quad 9 \cdot 9 = 81 \text{ clauses.}$$

Each box has 9 cells and 9 digits, so the identical bijection argument applies: at-least-one-per-digit plus cell exactly-one forces at-most-one-per-digit within the box too.

**Given clues:** for each pre-filled cell  $(i, j) = k_0$ , the unit clause

$$x_{i,j,k_0}, \quad \text{at most 81 clauses.}$$

**Size.**  $9^3 = 729$  variables. Total clauses:

$$\underbrace{81}_{\text{cell, at least}} + \underbrace{2916}_{\text{cell, at most}} + \underbrace{81}_{\text{row}} + \underbrace{81}_{\text{column}} + \underbrace{81}_{\text{box}} + \underbrace{\leq 81}_{\text{clues}} = 3240 + (\text{clues}),$$

dominated by the  $\Theta(n^4)$  cell at-most-one group (writing  $n = 9$  for the grid dimension, since  $n^2 \binom{n}{2} = \Theta(n^4)$ ).

A satisfying assignment is exactly a solved Sudoku grid consistent with the clues: the conjunction  $\Psi$  of all clause groups is satisfiable iff a solution exists, by the same two-way polynomial correspondence (assignment  $\leftrightarrow$  grid) as in Example 4.8. Checking a candidate grid against  $\Psi$  is  $O(n^4)$ , so (the decision version of) Sudoku is in  $NP$ .

*Note.* Both encodings turn a combinatorial puzzle into a pure question about propositional satisfiability – this is exactly how modern SAT solvers are used to solve such puzzles in practice, rather than hand-coding a puzzle-specific search algorithm.

#### 4.19 Representing logical statements

*Note* (Top-down / bottom-up modelling). **Top-down:** identify the top-level conjunction of required conditions. **Bottom-up:** identify the atomic Boolean variables (simplest possible true/false assertions) before worrying about their eventual truth values.

**Example 4.10** (CNF from natural-language rules). Variables: one Boolean per guest. Rules:

$$\begin{aligned} & (\text{Harry} \vee \text{Ron} \vee \text{Hermione} \vee \text{Ginny}) \\ & \wedge (\neg \text{Hagrid} \vee \text{Norberta}) \\ & \wedge (\neg \text{Fred} \vee \text{George}) \wedge (\text{Fred} \vee \neg \text{George}) \\ & \wedge (\neg \text{Voldemort} \vee \neg \text{Bellatrix}) \wedge (\neg \text{Voldemort} \vee \neg \text{Dolores}) \wedge (\neg \text{Bellatrix} \vee \neg \text{Dolores}). \end{aligned}$$

“At least one of”  $\rightarrow$  disjunction; “ $A$  only if  $B$ ”  $\rightarrow$  implication  $\rightarrow \neg A \vee B$ ; “none or both”  $\rightarrow$  biconditional  $\rightarrow$  two implications; “no more than one of a set”  $\rightarrow$  conjunction of pairwise “not both” clauses.

## 4.20 Statements about how many variables are true

**Definition 4.17** (At most  $k$  of  $n$  variables true). For variables  $x_1, \dots, x_n$ :

$$\text{“at most } k \text{ true”} \equiv \bigwedge_{\substack{S \subseteq \{1, \dots, n\} \\ |S|=k+1}} \bigvee_{i \in S} \neg x_i.$$

*Why this is correct.* “At most  $k$  true” fails iff at least  $k+1$  variables are true, i.e. iff some set  $S$  of size  $k+1$  has *all* its variables true, i.e. iff some  $S$  has  $\bigvee_{i \in S} \neg x_i$  false. So “at most  $k$  true” holds iff *every* size- $(k+1)$  set  $S$  has  $\bigvee_{i \in S} \neg x_i$  true – exactly the conjunction above. There are  $\binom{n}{k+1}$  such sets  $S$ . ■

**Definition 4.18** (At least  $k$  of  $n$  variables true).

$$\text{“at least } k \text{ true”} \equiv \bigwedge_{\substack{S \subseteq \{1, \dots, n\} \\ |S|=n-k+1}} \bigvee_{i \in S} x_i.$$

*Why this is correct.* “At least  $k$  true” fails iff at most  $k-1$  are true, iff at least  $n-k+1$  are false, iff some set  $S$  of size  $n-k+1$  has *all* its variables false, iff some  $S$  has  $\bigvee_{i \in S} x_i$  false. So “at least  $k$  true” holds iff every size- $(n-k+1)$  set  $S$  has  $\bigvee_{i \in S} x_i$  true. There are  $\binom{n}{n-k+1}$  such sets. ■

**Definition 4.19** (Exactly  $k$ ).

$$\text{“exactly } k \text{ true”} \equiv \text{“at least } k \text{”} \wedge \text{“at most } k \text{”}.$$

**Example 4.11.** At most 2 of  $w, x, y, z$  true ( $n = 4, k = 2$ , so  $\binom{4}{3} = 4$  conjuncts, one per 3-subset of  $\{w, x, y, z\}$ ):

$$(\neg w \vee \neg x \vee \neg y) \wedge (\neg w \vee \neg x \vee \neg z) \wedge (\neg w \vee \neg y \vee \neg z) \wedge (\neg x \vee \neg y \vee \neg z).$$

*Note* (Cost, and a linear-size alternative). The encodings above have size  $\Theta(\binom{n}{k+1})$  or  $\Theta(\binom{n}{n-k+1})$  – exponential in the worst case ( $k \approx n/2$ ). Sinz (2005) gives a *sequential-counter* encoding using  $O(nk)$  auxiliary variables and  $O(nk)$  clauses: auxiliary variables  $s_{i,j}$  track “at least  $j$  of  $x_1, \dots, x_i$  are true,” updated via

$$s_{i,j} \implies s_{i+1,j}, \quad (x_{i+1} \wedge s_{i,j}) \implies s_{i+1,j+1},$$

avoiding the combinatorial blow-up of the direct encoding above.

*Note* (Reference). C. Sinz, “Towards an Optimal CNF Encoding of Boolean Cardinality Constraints,” *Principles and Practice of Constraint Programming (CP 2005)*, LNCS 3709, pp. 827–831.

## 4.21 Universal sets of operations

**Definition 4.20** (Universal set of operations).  $X$  is *universal* if every logical expression is equivalent to one using only operations from  $X$ .

**Theorem 4.10.**  $\{\wedge, \vee, \neg\}$  is universal (immediate from DNF/CNF, §4.16-4.17).

**Theorem 4.11.**  $\{\wedge, \neg\}$  and  $\{\vee, \neg\}$  are each universal.

*Proof.* De Morgan’s laws express  $\vee$  in terms of  $\wedge, \neg$  (and vice versa), so either can be dropped while keeping  $\neg$ . ■

*Note* ( $\{\wedge, \vee\}$  is not universal). Without  $\neg$ , every expression built from  $\wedge, \vee$  is *monotone*: increasing any input from  $F$  to  $T$  can never make the output go from  $T$  to  $F$ . Since  $\neg P$  itself is not monotone, it cannot be expressed – so  $\{\wedge, \vee\}$  fails to be universal.

**Definition 4.21** (Sheffer stroke (NAND), Rosen – single-operator universal set).

$$P \mid Q = \neg(P \wedge Q).$$

**Theorem 4.12** ( $\{\mid\}$  alone is universal, Rosen).

$$\neg P = P \mid P, \quad P \wedge Q = (P \mid Q) \mid (P \mid Q), \quad P \vee Q = (P \mid P) \mid (Q \mid Q).$$

*Proof.* Check each directly by truth table, or:  $P \mid P = \neg(P \wedge P) = \neg P$ ; then  $(P \mid Q) \mid (P \mid Q) = \neg(P \mid Q) = \neg\neg(P \wedge Q) = P \wedge Q$ ; and  $(P \mid P) \mid (Q \mid Q) = \neg P \mid \neg Q = \neg(\neg P \wedge \neg Q) = P \vee Q$  (De Morgan's). Since  $\{\wedge, \vee, \neg\}$  is universal and all three are expressible via  $\mid$  alone,  $\{\mid\}$  is universal. ■

*Note.* This is why NAND gates alone suffice to build any digital circuit – a genuinely important fact in computer engineering, well beyond what CN's syllabus covers here.

## 4.22 Boolean algebra (Rosen Ch. 12 – new section, not in CN)

### 4.22.1 Boolean functions

**Definition 4.22** (Boolean algebra, axiomatic). A *Boolean algebra* is a set  $B$  with operations  $+, \cdot, \bar{\phantom{x}}$  and elements  $0, 1$  satisfying, for all  $x, y, z \in B$ :

$$\begin{aligned} x + 0 &= x, \quad x \cdot 1 = x && \text{(identity)} \\ x + \bar{x} &= 1, \quad x \cdot \bar{x} = 0 && \text{(complement)} \\ (x + y) + z &= x + (y + z), \quad (x \cdot y) \cdot z = x \cdot (y \cdot z) && \text{(associative)} \\ x + y &= y + x, \quad x \cdot y = y \cdot x && \text{(commutative)} \\ x + (y \cdot z) &= (x + y) \cdot (x + z), \quad x \cdot (y + z) = (x \cdot y) + (x \cdot z) && \text{(distributive)} \end{aligned}$$

*Note* (Two familiar instances).  $B = \{0, 1\}$  with  $+=\vee$ ,  $\cdot=\wedge$ ,  $\bar{x}=\neg x$  (propositional logic, §4.15) is a Boolean algebra; so is  $B = \mathcal{P}(S)$  with  $+=\cup$ ,  $\cdot=\cap$ ,  $\bar{X}=S \setminus X$  (§1.11-1.12). Every law in §4.15's table has a direct set-theoretic mirror in §1.11-1.12, and vice versa – both are instances of the same abstract structure.

**Theorem 4.13** (Duality principle). *Every identity in a Boolean algebra remains true if  $+\leftrightarrow\cdot$  and  $0\leftrightarrow 1$  are swapped throughout (its dual).*

*Proof.* The axioms themselves come in  $+/\cdot$ -swapped pairs (identity, complement, associative, commutative, distributive each list *both* versions together). So any proof built from the axioms, with every step's dual also an axiom instance, remains valid after swapping  $+\leftrightarrow\cdot$ ,  $0\leftrightarrow 1$  throughout. ■

**Example 4.12** (Deriving absorption from the axioms alone – harder).

$$x + (x \cdot y) = x.$$

*Proof.*

$$\begin{aligned}
x + (x \cdot y) &= (x \cdot 1) + (x \cdot y) && \text{(identity)} \\
&= x \cdot (1 + y) && \text{(distributive)} \\
&= x \cdot 1 && \text{(since } 1 + y = 1, \text{ itself provable from identity+complement laws)} \\
&= x && \text{(identity).}
\end{aligned}$$

By duality,  $x \cdot (x + y) = x$  follows immediately without a separate proof. ■

**Definition 4.23** (Boolean function). An  $n$ -ary *Boolean function* is  $f : \{0, 1\}^n \rightarrow \{0, 1\}$ .

**Theorem 4.14.** *There are  $2^{2^n}$  Boolean functions of  $n$  variables.*

*Proof.*  $\{0, 1\}^n$  has  $2^n$  elements (§1.14's power notation); a function to  $\{0, 1\}$  is one of  $2^{|\{0, 1\}^n|} = 2^{2^n}$  choices (§2.12's counting-functions theorem,  $n^m$  with  $n = 2$ ,  $m = 2^n$ ). ■

#### 4.22.2 Representing Boolean functions

**Definition 4.24** (Literal, minterm). A *literal* is  $x$  or  $\bar{x}$ ; a *minterm* of  $n$  variables is a product (AND) of  $n$  literals, one per variable, each chosen unnegated or negated.

**Theorem 4.15** (Sum-of-products form always exists, Rosen – proves what §4.16 only demonstrated). *Every Boolean function  $f \not\equiv 0$  can be written as a sum (OR) of minterms.*

*Proof.* For each input  $(a_1, \dots, a_n) \in \{0, 1\}^n$  with  $f(a_1, \dots, a_n) = 1$ , form the minterm  $m = y_1 \cdot y_2 \cdots y_n$  where  $y_i = x_i$  if  $a_i = 1$ , else  $y_i = \bar{x}_i$ . This  $m$  equals 1 exactly at that one input (every other input disagrees with  $a_i$  in some coordinate, making that literal 0, hence the whole product 0). Summing (OR-ing) all such minterms, one per input where  $f = 1$ : the result is 1 exactly at those inputs and 0 elsewhere – matching  $f$  everywhere. ■

*Note.* This is exactly §4.16's DNF, restated with "minterm" as the formal name for each DNF disjunct, and now with a proof that the construction always succeeds (§4.16 only worked one example).

#### 4.22.3 Logic gates

**Definition 4.25** (Logic gate). A *logic gate* is a physical/schematic device computing one basic Boolean operation; a *circuit* wires gates together, each gate's output feeding into other gates' inputs.

$$\text{AND: } x \cdot y \quad \text{OR: } x + y \quad \text{NOT: } x \rightarrow \bar{x}$$

AND, OR, NOT gates

**Example 4.13** (Half adder). Adds two bits  $x, y$ , producing sum bit  $s$  and carry bit  $c$ :

$$s = x \oplus y, \quad c = x \cdot y.$$

**Example 4.14** (Full adder – harder, three-input circuit). Adds bits  $x, y$  plus an incoming carry  $c_{\text{in}}$ , producing sum  $s$  and outgoing carry  $c_{\text{out}}$ :

$$s = x \oplus y \oplus c_{\text{in}}, \quad c_{\text{out}} = (x \cdot y) + (c_{\text{in}} \cdot (x \oplus y)).$$

*Note* (Chaining full adders). An  $n$ -bit adder chains  $n$  full adders, feeding each one's  $c_{\text{out}}$  into the next's  $c_{\text{in}}$  (*ripple-carry adder*) – this is exactly how integer addition is implemented at the hardware level, and it is a direct physical realization of §6.6/6.7's recursive definitions:  $s_i, c_{i+1}$  each depend only on  $x_i, y_i, c_i$ , exactly a recurrence in  $i$ .

#### 4.22.4 Minimization of circuits

*Note* (Motivation). Different sum-of-products expressions for the same function can need very different numbers of gates – minimization finds an equivalent expression with fewest literals/terms, directly reducing circuit cost.

**Definition 4.26** (Karnaugh map (K-map)). A grid of cells, one per minterm, arranged so that *adjacent* cells (including wraparound) differ in exactly one variable. Circling a rectangular block of  $2^k$  adjacent 1-cells corresponds to a single product term with  $k$  fewer literals than a minterm.

|      |    | $yz$ |    |    |    |
|------|----|------|----|----|----|
|      |    | 00   | 01 | 11 | 10 |
| $wx$ | 00 | 1    | 0  | 0  | 1  |
|      | 01 | 1    | 0  | 0  | 1  |
|      | 11 | 0    | 0  | 0  | 0  |
|      | 10 | 0    | 0  | 0  | 0  |

4-variable K-map: red block gives  $\bar{x}\bar{y}\bar{z}$ , blue block gives  $\bar{x}yz$  – both size-2 blocks (one variable eliminated each)

*Note* (Reading off blocks). A block of size  $2^k$  eliminates exactly  $k$  variables from its minterm (the  $k$  variables that differ across the block's cells); the surviving literals are read from whichever value (0  $\rightarrow$  negated, 1  $\rightarrow$  plain) is constant across the whole block.

**Definition 4.27** (Don't-care condition). An input combination that (by problem constraints) never actually occurs; marked 'd' or 'x' on the K-map, and can be treated as either 0 or 1 – whichever helps form a larger block.

*Note* (Quine-McCluskey algorithm, Rosen – harder, systematic alternative). K-maps become impractical beyond 4–5 variables (grids too large to draw/read visually). The *Quine-McCluskey algorithm* systematizes the same idea.

---

#### Algorithm 3: Quine-McCluskey algorithm

---

**Input:** List of minterms as bit strings

**Output:** Set of prime implicants covering every minterm

```

1 terms  $\leftarrow$  minterms, each unmarked;
2 while some pair in terms differs in exactly one bit do
3   foreach pair  $(t_1, t_2)$  differing in exactly one bit do
4     mark  $t_1, t_2$  as combined;
5     add  $t_1$  with that bit replaced by “–” to newTerms;
6   end
7   terms  $\leftarrow$  newTerms (duplicates removed);
8 end
9 primeImplicants  $\leftarrow$  all terms never marked combined;
10 select a minimal subset of primeImplicants covering every original minterm (set-cover);
11 return selected subset;
```

---



## 5 Predicate Logic

### 5.1 Relations, predicates, and truth-valued functions

**Definition 5.1** (Predicate). A *predicate* is a relation (§2.13, §2.17), used with intent to do logic with it. A predicate with  $k$  arguments is *k-ary*.

**Definition 5.2** (Predicate as a truth-valued function, Rosen). A  $k$ -ary relation  $R \subseteq A_1 \times \cdots \times A_k$  can equivalently be viewed as a function

$$R : A_1 \times \cdots \times A_k \rightarrow \{T, F\}, \quad R(a_1, \dots, a_k) = T \iff (a_1, \dots, a_k) \in R.$$

Rosen also calls this a *propositional function*.

*Note* (Substitution yields a proposition). Giving specific values to every argument of a predicate produces a proposition (definite truth value).

*Note* (Equality is always available).  $=$  is a predicate assumed available for *every* domain – without it, no domain of objects could even be meaningfully discussed.

**Definition 5.3** (Property). A unary predicate (*property*)  $P(x)$  on domain  $D$  corresponds to the set  $\{x \in D : P(x)\}$ .

### 5.2 Variables and constants

**Definition 5.4** (Variable). A name for an object, without specifying it exactly, used to reason about an unknown or arbitrary member of its *domain*.

*Note* (Domain). *Domain* also call *domain of discourse* or *universe of discourse*.

**Definition 5.5** (Constant). An object belonging to the domain of some variable – a specific, named member of that domain.

*Note* (Every variable has a domain). E.g. “let  $x \in \mathbb{Z}$ ” declares  $x$  with domain  $\mathbb{Z}$ ; a Boolean variable always has domain  $\{T, F\}$ .

*Note* (Math vs. programming variables, Rosen). A programming variable occupies memory and is restricted to finite/representable values; a mathematical variable has no memory association and its domain may be infinite.

### 5.3 Predicates and variables

**Definition 5.6** (Free variable). A variable in a predicate expression not (yet) assigned a value.

*Note* (Truth value requires all arguments fixed). An expression with  $\geq 1$  free variable is not a proposition (no truth value). Only when *every* free variable is assigned a value from its domain does the expression become a proposition.

*Note* (Domain matching is mandatory). Any value or variable placed into an argument of a predicate must belong to that argument’s declared domain – mismatched domains make the expression ill-formed, not just false.

**Example 5.1.**  $\text{Parent}(\text{Alan Turing}, X)$  with  $X$  a free variable, domain  $\mathbb{P}$  (people): not a proposition. Substituting  $X = \text{“Sara Turing”}$  gives the proposition  $\text{Parent}(\text{Alan Turing}, \text{Sara Turing})$ , true.

*Note* (Prefix vs. infix notation, Rosen). Newly defined predicates typically use prefix notation,  $R(a, b)$ ; predicates for ordering/equality/equivalence conventionally use infix notation,  $a < b$ ,  $a = b$  (§2.5).

## 5.4 Arguments of predicates

**Definition 5.7** (Term, recursive). A *term* of domain  $D$  is one of:

- a constant from  $D$ ;
- a variable with domain  $D$ ;
- a function with codomain  $D$ , each of whose arguments is given a term of matching domain.

*Note* (Recursion bottoms out). Terms are built from constants/variables/functions to arbitrary finite depth (composition, nested function calls), but never infinitely – every term has a finite construction.

**Example 5.2.** Let  $a, b$  be variables with domain  $\mathbb{R}$ . The expression

$$a^2 + b^2$$

is a term of domain  $\mathbb{R}$ .

To see this, first consider  $a^2$ . The squaring function

$$\text{square} : \mathbb{R} \rightarrow \mathbb{R}$$

has codomain  $\mathbb{R}$ , and its argument  $a$  is a term of domain  $\mathbb{R}$ . Therefore,

$$a^2 = \text{square}(a)$$

is a term of domain  $\mathbb{R}$ .

Similarly, since  $b$  is a term of domain  $\mathbb{R}$ ,

$$b^2 = \text{square}(b)$$

is also a term of domain  $\mathbb{R}$ .

Finally, consider the real addition function

$$+ : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}.$$

Its two arguments must each be terms of domain  $\mathbb{R}$ . Since both  $a^2$  and  $b^2$  are terms of domain  $\mathbb{R}$ , they can be given to the two arguments of the addition function. Hence,

$$a^2 + b^2$$

is a term of domain  $\mathbb{R}$ .

**Example 5.3.** Let  $\mathbb{P}$  be the domain of people, and suppose that

$$\text{FatherOf} : \mathbb{P} \rightarrow \mathbb{P}$$

and

$$\text{MotherOf} : \mathbb{P} \rightarrow \mathbb{P}.$$

If  $Y$  is a variable with domain  $\mathbb{P}$ , then

$$\text{FatherOf}(Y)$$

is a term of domain  $\mathbb{P}$ . Consequently,

$$\text{MotherOf}(\text{FatherOf}(Y))$$

is also a term of domain  $\mathbb{P}$ .

**Example 5.4** (Not terms).  $2 + 3 +$  (malformed),  $x++$  (undefined operation here),  $\log x$  (if  $\log$  is not among the available functions),  $z$  (if  $z$  is neither a declared variable nor constant).

*Note.* Only terms whose domain matches an argument's declared domain may be plugged into that argument – of a function *or* a predicate; the same matching rule applies uniformly at every level of nesting.

## 5.5 Building logical expressions with predicates

**Definition 5.8** (Predicate logic expression, recursive). •  $T, F$  are predicate logic expressions.

- A predicate applied to terms of matching domains is a predicate logic expression.
- If  $E, F$  are predicate logic expressions, so are  $(E)$ ,  $\neg E$ ,  $E \wedge F$ ,  $E \vee F$ ,  $E \implies F$ ,  $E \iff F$ .

*Note* (Free variables propagate). A predicate logic expression's free variables are the union of the free variables of its terms/sub-expressions – combining expressions never removes a free variable (only quantification, §5.6-5.9, does that).

**Example 5.5.**  $(1 < i) \wedge (i < n)$  has free variables  $i, n$ .  $\text{Parent}(X, Y) \iff (Y = \text{MotherOf}(X)) \vee (Y = \text{FatherOf}(X))$  has free variables  $X, Y$ .

*Note* (Boolean algebra applies unchanged). Every law from Chapter 4 (De Morgan's, distributive, etc.) applies to predicate logic expressions exactly as to propositions – since fixing values for all free variables reduces any predicate expression to an ordinary proposition, and the laws hold for *every* such assignment.

## 5.6 Existential quantifier

**Definition 5.9** (Existential quantifier). For predicate  $P(x)$  on domain  $D$ :

$$\exists x P(x)$$

is true iff  $P(a)$  is true for *at least one*  $a \in D$ .

*Note* (Punctuation).  $\exists x : P(x)$ ,  $\exists x. P(x)$ ,  $\exists x P(x)$  are all used; the colon/period is read “such that”.

**Definition 5.10** (Bound variable). Once quantified,  $x$  is *bound*: no longer available to receive a specific value. A fully-quantified expression has no free variables and is a genuine proposition, with a definite truth value.

*Note* (Domain matters).  $\exists W \in \mathbb{N} : W < 0$  is false;  $\exists W \in \mathbb{Z} : W < 0$  is true – same predicate, different domain, different truth value.

*Note* (Analogy with disjunction – and where it breaks).  $\exists x \in D$ ,  $P(x)$  behaves like the disjunction  $\bigvee_{a \in D} P(a)$  – true iff at least one disjunct is true. But this is only an analogy: disjunctions must be *finite*, while  $D$  may be infinite, so existential quantification genuinely extends what propositional logic alone can express.

*Note* (Vacuous quantification). If  $x$  does not appear in  $P$ ,  $\exists x P$  is equivalent to  $P$  itself (the quantifier is inert): e.g.  $\exists w : z < 0$  is just  $z < 0$ .

*Note.* Quantifiers apply only to variables, never to constants:  $\exists 5$  or  $\exists \text{Annie}$  are ill-formed.

**Example 5.6** (Rosen – harder: nested quantifiers and negation). Let  $P(x, y)$  mean “ $x + y = 0$ ” on domain  $\mathbb{R}$ . Consider  $\exists x \forall y P(x, y)$  vs.  $\forall y \exists x P(x, y)$ .

$\exists x \forall y P(x, y)$  : one fixed  $x$  that works for *every*  $y$  – false (no single  $x$  satisfies  $x + y = 0$  for all  $y$ ).

$\forall y \exists x P(x, y)$  : for each  $y$ , some  $x$  (possibly depending on  $y$ ) works – true, take  $x = -y$ .

This shows quantifier *order* genuinely matters – covered fully in §5.10.

## 5.7 Restricting existentially quantified variables

*Note* (Restricting  $\exists$  uses  $\wedge$ ). To assert “some  $X$  satisfying  $R(X)$  also satisfies  $P(X)$ ”:

$$\exists X : R(X) \wedge P(X).$$

**Example 5.7** (Correct vs. a common wrong attempt). “Some computer is human”:

correct:  $\exists X : \text{computer}(X) \wedge \text{human}(X)$ ,      wrong:  $\exists X : \text{computer}(X) \implies \text{human}(X)$ .

The wrong version says “something is not a computer, or is human” – true of any non-computer, regardless of computers at all, so it fails to capture the intended restriction.

*Note* (Rosen’s restricted-quantifier shorthand). Rosen writes  $\exists X \in D [\dots]$  or  $\exists X < 0 [\dots]$  directly as shorthand for the conjunction above, folding the restriction into the quantifier notation itself rather than writing it as an explicit  $\wedge$ .

## 5.8 Universal quantifier

**Definition 5.11** (Universal quantifier). For predicate  $P(x)$  on domain  $D$ :

$$\forall x P(x)$$

is true iff  $P(a)$  is true for *every*  $a \in D$ .

*Note*. As with  $\exists$ : the quantified variable becomes bound; an irrelevant quantifier (variable not appearing in the body) can be dropped.

*Note* (Rosen – vacuous truth generalizes).  $\forall x \in \emptyset, P(x)$  is vacuously true for *any*  $P$ , exactly generalizing  $\emptyset \subseteq A$  (§1.6 Theorem 1) to arbitrary predicates, not just set membership.

## 5.9 Restricting universally quantified variables

*Note* (Restricting  $\forall$  uses  $\implies$ ). To assert “every  $X$  satisfying  $R(X)$  also satisfies  $P(X)$ ”:

$$\forall X : R(X) \implies P(X).$$

**Example 5.8** (Correct vs. a common wrong attempt). “Every computer is human”:

correct:  $\forall X : \text{computer}(X) \implies \text{human}(X)$ ,      wrong:  $\forall X : \text{computer}(X) \wedge \text{human}(X)$ .

The wrong version says “everything is both a computer and human” – forces non-computers to also be human computers, which was never intended.

*Note* (The mirror-image trap, worth memorizing).  $\exists$  pairs naturally with  $\wedge$ ;  $\forall$  pairs naturally with  $\implies$ . Swapping these (using  $\implies$  with  $\exists$ , or  $\wedge$  with  $\forall$ ) is one of the most common predicate-logic errors – always double check which connective a restricted quantifier needs.

**Theorem 5.1** (De Morgan’s for quantifiers, Rosen – preview of §5.13).

$$\neg \forall x P(x) \equiv \exists x \neg P(x), \quad \neg \exists x P(x) \equiv \forall x \neg P(x).$$

*Note*. Consequence: negating  $\forall X : R(X) \implies P(X)$  gives  $\exists X : R(X) \wedge \neg P(X)$  – exactly matching the  $\exists/\wedge$  pairing above, since  $\neg(R \implies P) \equiv R \wedge \neg P$  (§4.15).

## 5.10 Multiple quantifiers

*Note* (Same-type quantifiers commute).

$$\exists X \exists Y P = \exists Y \exists X P = \exists(X, Y) P, \quad \forall X \forall Y P = \forall Y \forall X P = \forall(X, Y) P.$$

*Note* (Mixed quantifiers do *not* commute).

$$\exists X \forall Y P, \quad \forall X \exists Y P, \quad \exists Y \forall X P, \quad \forall Y \exists X P$$

are, in general, four genuinely different statements.

**Example 5.9** (hasVisited( $P, C$ ): person  $P$  has visited country  $C$ ).

$\exists P \forall C$  : someone has visited every country,  $\forall C \exists P$  : every country has been visited by someone.

The first is a strong claim about one exceptional traveler; the second is far weaker (each country just needs *some* visitor, possibly a different one each time).

**Example 5.10** (Rosen – classic order-matters example). Let  $D(x, y)$  mean “ $x$  has dated  $y$ ” on the domain of all people.

$\exists x \forall y D(x, y)$  : someone has dated everyone.

$\forall y \exists x D(x, y)$  : everyone has been dated by someone.

The first implies the second (that one exceptional dater witnesses every  $y$ ), but not conversely – the second only needs, for each  $y$ , *some*  $x$ , possibly different for each  $y$ .

**Theorem 5.2** ( $\exists \forall \implies \forall \exists$ , general fact, Rosen).

$$\exists x \forall y P(x, y) \implies \forall y \exists x P(x, y).$$

*The converse implication does not hold in general.*

*Proof.* Suppose  $\exists x \forall y P(x, y)$ : fix such an  $x_0$ , so  $P(x_0, y)$  holds for every  $y$ . Then for any  $y$ , taking  $x = x_0$  witnesses  $\exists x P(x, y)$  – so  $\forall y \exists x P(x, y)$  holds. The dating example above (with  $D$  not symmetric enough to reverse) shows the converse can fail. ■

**Quantifier–connective pitfall.\*\*\***  $\forall$  pairs with  $\implies$ ;  $\exists$  pairs with  $\wedge$  — the other way round lets a trivial witness make the statement true for the wrong reason.

## 5.11 Predicate logic expressions

**Definition 5.12** (Predicate logic expression, complete recursive grammar). •  $T, F$  are predicate logic expressions.

- Terms of matching domains plugged into a predicate give a predicate logic expression.
- If  $E, F$  are predicate logic expressions:  $(E)$ ,  $\neg E$ ,  $E \wedge F$ ,  $E \vee F$ ,  $E \implies F$ ,  $E \iff F$ .
- If  $E$  is a predicate logic expression and  $X$  is a free variable of  $E$ :  $\exists X : E$  and  $\forall X : E$  are predicate logic expressions, and  $X$  is *no longer free* in either.

*Note* (Free variables shrink under quantification). If  $V$  is the free-variable set of  $E$ , then  $\exists X : E$  and  $\forall X : E$  each have free-variable set  $V \setminus \{X\}$ .

## 5.12 Doing logic with quantifiers

*Note* (Instantiation and generalization).

$$(\forall X P(X)) \implies P(\text{obj}), \quad P(\text{obj}) \implies (\exists X P(X))$$

for any specific obj in  $X$ 's domain.

**Theorem 5.3** ( $\forall$  distributes over  $\wedge$ ;  $\exists$  distributes over  $\vee$ ).

$$\forall X (P(X) \wedge Q(X)) \equiv (\forall X P(X)) \wedge (\forall X Q(X)),$$

$$\exists X (P(X) \vee Q(X)) \equiv (\exists X P(X)) \vee (\exists X Q(X)).$$

*Note* ( $\forall$  over  $\vee$ , and  $\exists$  over  $\wedge$ , only give one-way implications, Rosen).

$$(\forall X P(X)) \vee (\forall X Q(X)) \implies \forall X (P(X) \vee Q(X)),$$

$$\exists X (P(X) \wedge Q(X)) \implies (\exists X P(X)) \wedge (\exists X Q(X)),$$

but neither converse holds in general – e.g. every integer is even or odd ( $\forall X, \text{even}(X) \vee \text{odd}(X)$ ), yet it is false that every integer is even, or every integer is odd.

**Definition 5.13** (Scope of a quantified variable). The sub-expression from the quantifier to the end of its innermost enclosing parentheses (or the end of the whole expression, if unparenthesized).

**Example 5.11** (Two disjoint scopes, same variable name). In  $(\forall X P(X)) \wedge (\forall X Q(X))$ , the two  $X$ 's are entirely independent – like distinct local variables in a program that happen to share a name.

**At least / at most / exactly  $n$  (predicate logic).** At least  $n$ :

$$\exists x_1 \cdots x_n \left( \bigwedge_{i < j} x_i \neq x_j \wedge \bigwedge_i P(x_i) \right)$$

At most  $n$ :

$$\forall x_1 \cdots x_{n+1} \left( \bigwedge_i P(x_i) \Rightarrow \bigvee_{i < j} x_i = x_j \right)$$

Exactly  $n$ : conjunction of both.

**The uniqueness quantifier  $\exists!$ .**  $\exists! x P(x)$  means “there is a unique  $x$  such that  $P(x)$ .” Without “!”:

$$\exists! x P(x) \equiv \exists x \left( P(x) \wedge \forall z (P(z) \Rightarrow z = x) \right)$$

General template for “exactly one thing satisfies  $P$ ”: existence ( $P(x)$ )  $\wedge$  uniqueness (anything satisfying  $P$  equals  $x$ ). (*CN Ch.5 Problem 4; Rosen §1.5 Ex. 52.*)

## 5.13 Duality between quantifiers

**Theorem 5.4** (De Morgan's laws for quantifiers).

$$\neg \forall X P(X) \equiv \exists X \neg P(X), \quad \neg \exists X P(X) \equiv \forall X \neg P(X).$$

**Example 5.12** (“Not all stars are visible” = “some star is invisible”).

$$\begin{aligned} \neg \forall X (\text{star}(X) \implies \text{visible}(X)) &= \exists X \neg (\text{star}(X) \implies \text{visible}(X)) \\ &= \exists X \neg (\neg \text{star}(X) \vee \text{visible}(X)) \\ &= \exists X (\text{star}(X) \wedge \neg \text{visible}(X)). \end{aligned}$$

*Note.*  $\neg \forall Y \neg = \exists Y$ , and  $\neg \exists Y \neg = \forall Y$  (apply the laws twice, using double negation).

## 5.14 Some examples

**Example 5.13** (“Infinitely many primes”, in predicate logic). With  $\text{Prime}(n)$  on domain  $\mathbb{N}$  and predicate  $\leq$ :

$$\forall b \exists n \text{ Prime}(n) \wedge \neg(n \leq b).$$

Read: for every bound  $b$ , some prime exceeds it – correctly capturing “infinitely many primes” without needing an infinite conjunction.

**Example 5.14** (Swapping the quantifiers gives a different, false, statement).

$$\exists n \forall b \text{ Prime}(n) \wedge \neg(n \leq b).$$

*Why this is false – step by step.* Distribute  $\forall b$  over the conjunction (§5.12):

$$\forall b \text{ Prime}(n) \wedge \neg(n \leq b) \equiv (\forall b \text{ Prime}(n)) \wedge (\forall b \neg(n \leq b)).$$

Since  $\text{Prime}(n)$  does not involve  $b$ , the quantifier is superfluous (§5.9):

$$\forall b \text{ Prime}(n) \equiv \text{Prime}(n).$$

Substituting back:

$$\exists n (\text{Prime}(n) \wedge \forall b \neg(n \leq b)) \equiv \exists n \in \{\text{primes}\} \forall b n > b.$$

This asserts a single prime exceeding *every* number – false. So the two quantifier orderings genuinely differ in meaning, one true and one false. ■

**Example 5.15** (Rosen – translating a nested-quantifier English sentence, harder). “Every real number except 0 has a multiplicative inverse.” With domain  $= \mathbb{R}$ :

$$\forall x (x \neq 0 \implies \exists y (xy = 1)).$$

Note the restricted- $\forall$  pattern from §5.9 ( $x \neq 0 \implies \dots$ ) nested with a plain  $\exists y$  inside –  $y$ ’s existence is allowed to depend on  $x$ , which is essential: no single  $y$  works for every nonzero  $x$ .

**Example 5.16** (Limit of a function, nested quantifiers – Rosen, requires calculus).  $\lim_{x \rightarrow a} f(x) = L$  means: for every real number  $\epsilon > 0$  there exists a real number  $\delta > 0$  such that  $|f(x) - L| < \epsilon$  whenever  $0 < |x - a| < \delta$ .

In quantifiers, with  $\epsilon, \delta$  ranging over *all* reals and the positivity built into the body:

$$\forall \epsilon \exists \delta \forall x (0 < |x - a| < \delta \implies |f(x) - L| < \epsilon),$$

where the domain for  $\epsilon, \delta$  is understood to already be “positive reals,” and for  $x$  is all reals.

Equivalently, using *restricted* quantifiers (§5.7, §5.9) to make the positivity explicit rather than implicit in the domain:

$$\forall \epsilon > 0 \exists \delta > 0 \forall x (0 < |x - a| < \delta \implies |f(x) - L| < \epsilon),$$

now with  $\epsilon, \delta$  ranging over *all* real numbers, and the restriction “ $> 0$ ” carried by the quantifier notation itself instead of by the choice of domain.

*Note* (Order is essential here). The quantifier order  $\forall\epsilon\exists\delta\forall x$  matters critically:  $\delta$  is allowed to depend on  $\epsilon$  (chosen *after*  $\epsilon$  is fixed), but must work for *all*  $x$  satisfying the closeness condition. Swapping to  $\exists\delta\forall\epsilon$  would wrongly demand a single  $\delta$  working for every  $\epsilon$  simultaneously – a much stronger, generally false, claim.

**Example 5.17** (“Limit does not exist,” via negation of nested quantifiers – Rosen, requires calculus).  $\lim_{x \rightarrow a} f(x)$  does not exist means: for every real number  $L$ ,  $\lim_{x \rightarrow a} f(x) \neq L$ .

Using the restricted-quantifier form above,  $\lim_{x \rightarrow a} f(x) \neq L$  is the negation of

$$\forall\epsilon > 0 \exists\delta > 0 \forall x \left( 0 < |x - a| < \delta \implies |f(x) - L| < \epsilon \right),$$

i.e.

$$\neg\forall\epsilon > 0 \exists\delta > 0 \forall x \left( 0 < |x - a| < \delta \implies |f(x) - L| < \epsilon \right).$$

Push the negation inward one quantifier at a time, using §5.13’s De Morgan’s laws for quantifiers at each step:

$$\begin{aligned} & \neg\forall\epsilon > 0 \exists\delta > 0 \forall x (0 < |x - a| < \delta \implies |f(x) - L| < \epsilon) \\ \equiv & \exists\epsilon > 0 \neg\exists\delta > 0 \forall x (0 < |x - a| < \delta \implies |f(x) - L| < \epsilon) \\ \equiv & \exists\epsilon > 0 \forall\delta > 0 \neg\forall x (0 < |x - a| < \delta \implies |f(x) - L| < \epsilon) \\ \equiv & \exists\epsilon > 0 \forall\delta > 0 \exists x \neg(0 < |x - a| < \delta \implies |f(x) - L| < \epsilon) \\ \equiv & \exists\epsilon > 0 \forall\delta > 0 \exists x \left( 0 < |x - a| < \delta \wedge |f(x) - L| \geq \epsilon \right). \end{aligned}$$

The last step uses  $\neg(P \implies Q) \equiv P \wedge \neg Q$  (§4.15), applied with  $P = (0 < |x - a| < \delta)$ ,  $Q = (|f(x) - L| < \epsilon)$ , and  $\neg(|f(x) - L| < \epsilon) \equiv |f(x) - L| \geq \epsilon$ .

Since “the limit does not exist” means  $\forall L, \lim_{x \rightarrow a} f(x) \neq L$ , prepend  $\forall L$ :

$$\forall L \exists\epsilon > 0 \forall\delta > 0 \exists x \left( 0 < |x - a| < \delta \wedge |f(x) - L| \geq \epsilon \right).$$

Read aloud: “for every candidate limit value  $L$ , there is some  $\epsilon > 0$  such that, no matter how small a  $\delta > 0$  is chosen, some  $x$  within  $\delta$  of  $a$  still has  $f(x)$  at least  $\epsilon$  away from  $L$ ” – exactly capturing that no  $L$  can serve as the limit.

*Note.* Each of the four inner negation steps flips  $\forall \leftrightarrow \exists$  and pushes  $\neg$  past exactly one quantifier, mirroring the definitions in §5.13 one at a time; only the final step differs, converting the negated implication via ordinary propositional logic rather than a quantifier law.

**Example 5.18** (Negating a triple-nested quantifier statement – Rosen, harder). “There is no woman who has flown on every airline in the world.” Let  $P(w, f)$ : “woman  $w$  has taken flight  $f$ ”;  $Q(f, a)$ : “flight  $f$  is on airline  $a$ .” “Some woman has flown on every airline” is

$$\exists w \forall a \exists f (P(w, f) \wedge Q(f, a)),$$

so the negation is  $\neg\exists w \forall a \exists f (P(w, f) \wedge Q(f, a))$ . Pushing the negation inward, one quantifier at a time:

$$\begin{aligned} & \neg\exists w \forall a \exists f (P(w, f) \wedge Q(f, a)) \\ \equiv & \forall w \neg\forall a \exists f (P(w, f) \wedge Q(f, a)) \\ \equiv & \forall w \exists a \neg\exists f (P(w, f) \wedge Q(f, a)) \\ \equiv & \forall w \exists a \forall f \neg(P(w, f) \wedge Q(f, a)) \\ \equiv & \forall w \exists a \forall f (\neg P(w, f) \vee \neg Q(f, a)). \end{aligned}$$



Reading it back: “for every woman, there is some airline such that, for every flight, either she hasn’t taken that flight or that flight isn’t on that airline.”

*Note.* Three nested quantifiers, three applications of the same rule from §5.13 – the pattern doesn’t change with depth, only the bookkeeping does.

### 5.15 Summary of rules for logic with quantifiers

*Note* (Full rule list).

$$\exists x \in A P(x) \equiv \exists x (x \in A \wedge P(x)) \quad (\S 5.7)$$

$$\forall x \in A P(x) \equiv \forall x (x \in A \implies P(x)) \quad (\S 5.9)$$

$$\exists x R \equiv R, \quad \forall x R \equiv R \quad (x \text{ not free in } R; \S 5.7, 5.9)$$

$$\exists x \exists y P(x, y) \equiv \exists y \exists x P(x, y) \quad (\S 5.10)$$

$$\forall x \forall y P(x, y) \equiv \forall y \forall x P(x, y) \quad (\S 5.10)$$

$$\forall x P(x) \implies P(\text{obj}), \quad P(\text{obj}) \implies \exists x P(x) \quad (\S 5.12)$$

$$\exists x (P(x) \vee Q(x)) \equiv (\exists x P(x)) \vee (\exists x Q(x)) \quad (\S 5.12)$$

$$\forall x (P(x) \wedge Q(x)) \equiv (\forall x P(x)) \wedge (\forall x Q(x)) \quad (\S 5.12)$$

$$\neg \exists x P(x) \equiv \forall x \neg P(x), \quad \neg \forall x P(x) \equiv \exists x \neg P(x) \quad (\S 5.13)$$

### 5.16 Rules of inference (Rosen §1.6 – extension, beyond CN syllabus)

**Definition 5.14** (Rule of inference). A *rule of inference* is an argument form: premises above a line, conclusion below, valid because the conjunction of the premises implying the conclusion is a tautology.

*Note* (Basic rules for propositional logic).

$$\text{Modus ponens: } \frac{P, P \implies Q}{\therefore Q} \quad \text{Modus tollens: } \frac{\neg Q, P \implies Q}{\therefore \neg P}$$

$$\text{Hypothetical syllogism: } \frac{P \implies Q, Q \implies R}{\therefore P \implies R} \quad \text{Disjunctive syllogism: } \frac{P \vee Q, \neg P}{\therefore Q}$$

$$\text{Addition: } \frac{P}{\therefore P \vee Q} \quad \text{Simplification: } \frac{P \wedge Q}{\therefore P} \quad \text{Conjunction: } \frac{P, Q}{\therefore P \wedge Q}$$

$$\text{Resolution: } \frac{P \vee Q, \neg P \vee R}{\therefore Q \vee R}$$

**Theorem 5.5.** *Each rule above is valid: the conjunction of its premises implies its conclusion is a tautology.*

*Verification for modus tollens.* Suppose  $\neg Q$  and  $P \implies Q$  both hold. If  $P$  were true,  $P \implies Q$  would force  $Q$  true, contradicting  $\neg Q$ . So  $P$  is false, i.e.  $\neg P$  holds. (The others verify similarly, by truth table or direct argument – §4.15’s laws are exactly what’s needed.) ■

*Note* (Rules of inference for quantified statements).

Universal instantiation:  $\frac{\forall x P(x)}{\therefore P(c)}$  ( $c$  any specific element of the domain)

Universal generalization:  $\frac{P(c) \text{ for arbitrary, generic } c}{\therefore \forall x P(x)}$

Existential instantiation:  $\frac{\exists x P(x)}{\therefore P(c)}$  ( $c$  some element, not otherwise specified)

Existential generalization:  $\frac{P(c) \text{ for a specific } c}{\therefore \exists x P(x)}$

*Note* (These match CN's own principles). Universal instantiation and existential generalization are exactly the two implications from §5.12; universal generalization is exactly the “let  $x$  be arbitrary, generic” technique from §3.3/§3.4 for proving universal statements.

**Example 5.19** (Chained formal proof – harder, Rosen-style). Premises: (1)  $\forall x (\text{Student}(x) \implies \text{Studies}(x))$ ; (2)  $\forall x (\text{Studies}(x) \implies \text{Passes}(x))$ ; (3)  $\neg \text{Passes}(\text{Sam})$ ; (4)  $\text{Student}(\text{Sam})$ .

Conclusion to derive: a contradiction (showing the premises are jointly unsatisfiable).

- |                                                                       |                                                     |
|-----------------------------------------------------------------------|-----------------------------------------------------|
| (5) $\text{Student}(\text{Sam}) \implies \text{Studies}(\text{Sam})$  | (universal instantiation on (1), $c = \text{Sam}$ ) |
| (6) $\text{Studies}(\text{Sam}) \implies \text{Passes}(\text{Sam})$   | (universal instantiation on (2), $c = \text{Sam}$ ) |
| (7) $\text{Student}(\text{Sam}) \implies \text{Passes}(\text{Sam})$   | (hypothetical syllogism, (5)+(6))                   |
| (8) $\text{Passes}(\text{Sam})$                                       | (modus ponens, (4)+(7))                             |
| (9) $\text{Passes}(\text{Sam}) \wedge \neg \text{Passes}(\text{Sam})$ | (conjunction, (8)+(3))                              |

Line (9) is a contradiction, so the four premises cannot all hold simultaneously.

## 6 Sequences & Series

### 6.1 Definitions and notation

**Definition 6.1** (Sequence). A *sequence* is a function whose domain is  $\mathbb{N}$  (an *infinite sequence*) or  $\{1, \dots, n\}$  for some  $n$  (a *finite sequence*, equivalently an ordered  $n$ -tuple, §1.14).

**Definition 6.2** (Term,  $n$ -th term notation). For a sequence  $f$  over a set  $A$  (i.e.  $f : \mathbb{N} \rightarrow A$  or similar),  $f(n)$  (also written  $f_n$ ) is the  $n$ -th *term*. The *terms* of  $f$  are the members of its image (§2.1.2), listed in the order given by the domain.

**Definition 6.3** (Length). The *length* of a finite sequence is the size of its domain – its number of terms.

*Note* (Rosen's phrasing). Rosen defines a sequence identically: a function from a subset of  $\mathbb{Z}$  (typically  $\{0, 1, 2, \dots\}$  or  $\{1, 2, 3, \dots\}$ ) to a set  $S$ ;  $a_n$  denotes the image of  $n$ , called a *term*.

*Note* (Ways to specify a sequence). • List all terms explicitly (only practical if short).

- A formula for the  $n$ -th term:  $(n^2 : n \in \mathbb{N})$  or  $(n^2)_{n=1}^\infty$ , matching set-builder notation (§1.2) but with parentheses since order matters.

- A recurrence relation (§6.2): each term defined using previous term(s).

A sequence is *not* well-defined merely by listing a few terms and trusting a "pattern" – infinitely many different formulas can match any finite initial segment (see §6.2's exercise on this danger, and the counterexample below).

**Example 6.1** (A pattern-guessing trap, Rosen – harder). The sequence  $1, 2, 4, 8, 16, \dots$  (five terms shown) matches  $a_n = 2^{n-1}$  – but it also matches

$$a_n = \binom{n-1}{0} + \binom{n-1}{1} + \binom{n-1}{2} + \binom{n-1}{3} + \binom{n-1}{4},$$

which equals  $1, 2, 4, 8, 16$  for  $n = 1, \dots, 5$  but gives 31, not 32, at  $n = 6$  (since the  $\binom{n-1}{5}$  term, absent from this formula, would otherwise contribute). A finite list of terms never uniquely determines a sequence.

*Note* (Number sequences). Sequences over  $\mathbb{N}, \mathbb{Z}, \mathbb{Q}, \mathbb{R}$  (*number sequences*) are especially important – not only for direct computation, but for measuring properties of computation itself: running time, memory use, input/output size, and so on.

### 6.2 Recursive definitions of sequences

**Definition 6.4** (Recurrence relation). A *recurrence relation* for  $(a_n)$  expresses  $a_n$  in terms of one or more earlier terms ( $a_{n-1}$ , or several), together with enough *base cases* (initial terms given explicitly) to make every term uniquely determined.

*Note* (General shape of a recursive definition, Rosen §5.3). • **Base case:** explicitly define finitely many of the simplest objects.

- **Recursive case:** define a general object in terms of simpler ones already in the family.

This is the same two-part shape as recursively-defined sets/functions (§5.3 generalizes it beyond sequences – see the note below).

**Example 6.2.**

$$f_1 = 1, f_n = f_{n-1} + 2 \Rightarrow 1, 3, 5, 7, \dots \text{ (odd numbers);}$$

$$f_1 = 0, f_n = f_{n-1} + 2 \Rightarrow 0, 2, 4, 6, \dots \text{ (even numbers) – same rule, different base case, different sequence;}$$

$$f_1 = 1, f_n = n f_{n-1} \Rightarrow 1, 2, 6, 24, 120, \dots \text{ (factorials, i.e. } n!).$$

*Note* (Multiple base cases).  $f_n = 4f_{n-2}$  (depending on the term *two* back) needs *two* base cases,  $f_1, f_2$ , to determine the whole sequence – one base case alone only fixes every other term. E.g.  $f_1 = 2, f_2 = 0, f_n = 4f_{n-2}$  gives  $2, 0, 8, 0, 32, 0, \dots$

**Example 6.3** (Structural induction preview, Rosen §5.3 – harder). Recursively-defined *sets* follow the same base/recursive pattern: e.g. the set  $S$  of "balanced parenthesis strings" is defined by (i)  $\varepsilon \in S$  (base case); (ii) if  $w \in S$ , then  $(w) \in S$  and, if  $u, v \in S$ , then  $uv \in S$  (recursive case). Proving a property  $P$  holds for every  $w \in S$  then uses *structural induction*: prove  $P$  for the base case, and prove that the recursive case preserves  $P$  (assuming it holds for the smaller pieces used to build  $w$ ) – the direct set-theoretic analogue of induction on sequences.

**6.3 Arithmetic sequences**

**Definition 6.5** (Arithmetic sequence). A sequence with constant difference  $d$  between consecutive terms:  $f_1 = a, f_n = f_{n-1} + d$ . The  $n$ -th term is  $a + (n - 1)d$ .

*Note* (Rosen's phrasing). Rosen calls this an *arithmetic progression*:  $a_n = a + dn$  (indexing from  $n = 0$ ) or equivalently  $a_n = a_{n-1} + d$  – same sequence, differing only in whether the first term is indexed 0 or 1.

**6.4 Geometric sequences**

**Definition 6.6** (Geometric sequence). A sequence with constant ratio  $r$  between consecutive terms:  $f_1 = a, f_n = f_{n-1} \cdot r$ . The  $n$ -th term is  $ar^{n-1}$ .

*Note* (Rosen's phrasing). Rosen calls this a *geometric progression*:  $a_n = ar^n$  (indexing from 0) or  $a_n = a_{n-1} \cdot r$ .

**6.5 Harmonic sequences**

**Definition 6.7** (Harmonic sequence).  $(a_n)$  is *harmonic* iff  $(1/a_n)$  is arithmetic.

*Note*.  $(1/n : n \in \mathbb{N})$  is the key example – harmonic since  $\mathbb{N}$  itself is arithmetic. See §6.15 for its properties.

**6.6 From recursive definitions to formulas**

*Note* (Explore – conjecture – prove). 1. **Explore**: compute the first several terms.

2. **Conjecture**: look for a pattern; propose a closed-form formula.

3. **Prove**: verify the formula for all  $n$ , typically by induction.

**Example 6.4.**

$$f_1 = 1, \quad f_n = 2f_{n-1} + 1.$$

**Explore:**  $f_1 = 1, f_2 = 3, f_3 = 7, f_4 = 15$ .

**Conjecture:** each term is one less than a power of 2:  $f_n = 2^n - 1$ .

**Prove** (induction on  $n$ ):

*Base case* ( $n = 1$ ):  $2^1 - 1 = 1 = f_1$ .

*Inductive step:* assume  $f_k = 2^k - 1$  for some  $k \geq 1$ . Then

$$f_{k+1} = 2f_k + 1 = 2(2^k - 1) + 1 = 2^{k+1} - 2 + 1 = 2^{k+1} - 1.$$

By induction,  $f_n = 2^n - 1$  for all  $n \geq 1$ .

**Definition 6.8** (Linear homogeneous recurrence with constant coefficients, Rosen §8.2 – systematic method).

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \cdots + c_k a_{n-k}, \quad c_k \neq 0,$$

with  $k$  initial conditions  $a_0, \dots, a_{k-1}$  given. Its *characteristic equation* is

$$x^k - c_1 x^{k-1} - c_2 x^{k-2} - \cdots - c_k = 0.$$

**Theorem 6.1** (Distinct roots case, Rosen). *If the characteristic equation has  $k$  distinct roots  $r_1, \dots, r_k$ , then every solution has the form*

$$a_n = \alpha_1 r_1^n + \alpha_2 r_2^n + \cdots + \alpha_k r_k^n$$

for constants  $\alpha_1, \dots, \alpha_k$  determined by the initial conditions.

*Note* (Why this is really linear algebra). The set of all sequences satisfying this recurrence forms a vector space: if  $(b_n)$  and  $(c_n)$  both satisfy it, so does any combination  $\beta b_n + \gamma c_n$  (the recurrence is *linear*). This space has dimension  $k$  – a solution is pinned down entirely by its  $k$  initial values  $a_0, \dots, a_{k-1}$ .

Each geometric sequence  $r_i^n$  is itself a solution (plugging it into the recurrence and using  $r_i^k = c_1 r_i^{k-1} + \cdots + c_k$  confirms it). When the  $r_i$  are distinct, these  $k$  solutions are *linearly independent* – this is exactly a Vandermonde-determinant fact from linear algebra: distinct roots  $\Rightarrow$  nonzero Vandermonde determinant  $\Rightarrow$  independence.  $k$  independent vectors in a  $k$ -dimensional space form a *basis*, so every solution is a unique linear combination of them – which is exactly the formula above, with  $\alpha_1, \dots, \alpha_k$  solved for using whatever initial conditions are given.

**Example 6.5** (Solving a second-order recurrence – harder, worked example).

$$a_n = 5a_{n-1} - 6a_{n-2}, \quad a_0 = 1, \quad a_1 = 4.$$

**Characteristic equation:**  $x^2 - 5x + 6 = 0$ , i.e.  $(x - 2)(x - 3) = 0$ , roots  $r_1 = 2, r_2 = 3$ .

**General solution:**  $a_n = \alpha_1 2^n + \alpha_2 3^n$ .

**Solve for  $\alpha_1, \alpha_2$**  using initial conditions:

$$a_0 = 1 : \alpha_1 + \alpha_2 = 1, \quad a_1 = 4 : 2\alpha_1 + 3\alpha_2 = 4.$$

From the first equation,  $\alpha_1 = 1 - \alpha_2$ ; substituting:  $2(1 - \alpha_2) + 3\alpha_2 = 4 \Rightarrow \alpha_2 = 2, \alpha_1 = -1$ .

**Closed form:**

$$a_n = -2^n + 2 \cdot 3^n.$$

*Note* (Why this beats guess-and-check). The explore-conjecture-prove method above needs a pattern to be *visible* in the first few terms – it can fail entirely on recurrences with less obvious closed forms (this one,  $a_n = 5a_{n-1} - 6a_{n-2}$ , produces 1, 4, 14, 46,  $\dots$ , no obvious pattern by inspection). The characteristic-equation method is systematic: it always works, whenever the roots can be found. This is exactly the machinery behind the exact Fibonacci formula in §6.7.4.

**Definition 6.9** (Recursively defined set, Rosen §5.3). A set  $S$  is defined recursively by:

- **Basis step:** specify an initial collection of elements in  $S$ .
- **Recursive step:** rules that construct new elements of  $S$  from existing ones.
- (Implicitly)  $S$  contains *only* elements obtainable this way.

**Example 6.6** (Balanced parenthesis strings). Define  $B$  recursively: (i)  $\varepsilon \in B$  (empty string); (ii) if  $w \in B$ , then  $(w) \in B$ ; (iii) if  $u, v \in B$ , then  $uv \in B$ .

**Definition 6.10** (Recursively defined function, Rosen §5.3). A function  $f$  on a recursively defined set  $S$  is defined by: giving  $f$ 's value(s) on the basis elements, and giving a rule for  $f$ 's value on a constructed element in terms of  $f$ 's value(s) on the piece(s) it was built from.

**Example 6.7.**  $n!$  on  $\mathbb{N}$  (basis:  $0! = 1$ ; recursive:  $n! = n \cdot (n-1)!$ ) fits this exact pattern – a recursively defined function on the recursively defined set  $\mathbb{N}$ .

**Theorem 6.2** (Structural induction, Rosen §5.3). *To prove property  $P$  holds for every element of a recursively defined set  $S$ :*

- **Basis step:** prove  $P$  holds for every basis element.
- **Recursive step:** prove that, if  $P$  holds for the element(s) a recursive rule builds from,  $P$  holds for the newly constructed element too.

**Example 6.8** (Structural induction on balanced parentheses – harder). Prove: every  $w \in B$  has an equal number of '(' and ')'.  
*Proof.* **Basis step:**  $\varepsilon$  has 0 of each – equal.

**Recursive step, rule (ii):** suppose  $w \in B$  has equal counts, say  $k$  each. Then  $(w)$  has  $k+1$  of each – still equal.

**Recursive step, rule (iii):** suppose  $u, v \in B$  have equal counts  $k_u, k_v$  respectively. Then  $uv$  has  $k_u + k_v$  of each – still equal.

By structural induction, every  $w \in B$  has equal counts of '(' and ')'. ■

*Note* (Structural induction  $\equiv$  ordinary induction, in disguise). Structural induction is not a genuinely new principle – it is ordinary (or strong) induction on the "construction depth" of an element (how many recursive steps were used to build it), applied to the recursively defined set instead of to  $\mathbb{N}$  directly.

**Example 6.9** (Rabbit population, Rosen §8.1 – the origin of Fibonacci). A pair of rabbits produces one new pair each month starting from their second month of life; no rabbit dies. Let  $f_n$  = number of pairs after  $n$  months, starting from one newborn pair. Then  $f_1 = 1, f_2 = 1$ , and for  $n \geq 3$ : pairs present at month  $n$  = pairs already alive at month  $n - 1$ , plus new pairs born (one per pair that was already alive and mature at month  $n - 2$ ):

$$f_n = f_{n-1} + f_{n-2},$$

exactly the Fibonacci recurrence (§6.7).

**Example 6.10** (Tower of Hanoi, Rosen §8.1).  $n$  disks, decreasing size, on peg  $A$ ; move all to peg  $C$  (via peg  $B$ ), one disk at a time, never placing a larger disk on a smaller one. Let  $H_n$  = minimum number of moves.

*Note* (Recurrence). Move the top  $n - 1$  disks  $A \rightarrow B$  ( $H_{n-1}$  moves), move the largest disk  $A \rightarrow C$  (1 move), move the  $n - 1$  disks  $B \rightarrow C$  ( $H_{n-1}$  moves):

$$H_1 = 1, \quad H_n = 2H_{n-1} + 1.$$

**Theorem 6.3.**  $H_n = 2^n - 1$ .

*Proof.* Exactly the recurrence and proof of §6.6's worked example (same recurrence shape,  $a = 1, b = 1$  shifted), by induction: base case  $H_1 = 1 = 2^1 - 1$ ; inductive step  $H_{k+1} = 2H_k + 1 = 2(2^k - 1) + 1 = 2^{k+1} - 1$ . ■

**Definition 6.11** (Linear nonhomogeneous recurrence, Rosen §8.2).

$$a_n = c_1 a_{n-1} + \cdots + c_k a_{n-k} + F(n), \quad F(n) \not\equiv 0.$$

The associated *homogeneous* recurrence drops  $F(n)$ .

**Theorem 6.4** (General solution shape). *Every solution has the form*

$$a_n = a_n^{(p)} + a_n^{(h)},$$

where  $a_n^{(p)}$  is any one particular solution to the nonhomogeneous recurrence, and  $a_n^{(h)}$  is the general solution of the associated homogeneous recurrence.

*Proof.* If  $a_n^{(p)}$  solves the nonhomogeneous recurrence and  $(b_n)$  is any solution to it, then  $b_n - a_n^{(p)}$  satisfies the *homogeneous* recurrence (subtracting the two nonhomogeneous equations cancels  $F(n)$ ). So  $b_n - a_n^{(p)} = a_n^{(h)}$  for some homogeneous solution, i.e.  $b_n = a_n^{(p)} + a_n^{(h)}$ . ■

*Note* (Finding a particular solution – trial forms). When  $F(n)$  is a polynomial times  $s^n$ , try  $a_n^{(p)}$  of the same shape (polynomial of the same degree times  $s^n$ ) – unless  $s$  is itself a root of the characteristic equation, in which case multiply by  $n$  (or  $n^m$  for a root of multiplicity  $m$ ).

**Example 6.11** (Worked nonhomogeneous example – harder).

$$a_n = 3a_{n-1} + 2^n, \quad a_0 = 1.$$

**Homogeneous part:** characteristic equation  $x - 3 = 0$ , so  $a_n^{(h)} = \alpha \cdot 3^n$ .

**Particular solution:** try  $a_n^{(p)} = A \cdot 2^n$  ( $s = 2$  is not a root of  $x - 3 = 0$ ). Substituting:

$$A \cdot 2^n = 3A \cdot 2^{n-1} + 2^n \Rightarrow 2A = 3A + 2 \Rightarrow A = -2.$$

So  $a_n^{(p)} = -2 \cdot 2^n = -2^{n+1}$ .

**General solution:**  $a_n = \alpha \cdot 3^n - 2^{n+1}$ .

**Match**  $a_0 = 1$ :  $\alpha - 2 = 1 \Rightarrow \alpha = 3$ .

**Closed form:**  $a_n = 3^{n+1} - 2^{n+1}$ .

**Theorem 6.5** (General solution, distinct and repeated roots combined). *Let the characteristic equation of a linear homogeneous recurrence with constant coefficients (Definition 6.12) factor as*

$$x^k - c_1 x^{k-1} - \cdots - c_k = (x - r_1)^{m_1} (x - r_2)^{m_2} \cdots (x - r_t)^{m_t},$$

where  $r_1, \dots, r_t$  are the distinct roots with multiplicities  $m_1, \dots, m_t$  and  $m_1 + m_2 + \cdots + m_t = k$ . Then every solution has the form

$$a_n = \sum_{i=1}^t \left( \alpha_{i,0} + \alpha_{i,1}n + \alpha_{i,2}n^2 + \cdots + \alpha_{i,m_i-1}n^{m_i-1} \right) r_i^n,$$

for constants  $\alpha_{i,j}$  determined by the  $k$  initial conditions  $a_0, \dots, a_{k-1}$ .

**Corollary 6.1** (Single repeated root,  $t = 1$ ). *If the characteristic equation has a single root  $r$  of multiplicity  $m = k$ , the sum above collapses to one term, so the general solution is*

$$a_n = (\alpha_0 + \alpha_1 n + \alpha_2 n^2 + \cdots + \alpha_{m-1} n^{m-1}) r^n$$

instead of a single term  $\alpha r^n$ .

*Note.* This statement subsumes both special cases already given: if every root is simple ( $m_i = 1$  for all  $i$ , so  $t = k$ ), each bracket collapses to the single constant  $\alpha_{i,0}$  and the formula reduces to Theorem 6.13; if  $k = 2$  with a single double root ( $t = 1, m_1 = 2$ ), it reduces to Theorem 6.27's  $(\alpha_0 + \alpha_1 n)r^n$ . In general each root contributes exactly  $m_i$  degrees of freedom, and  $\sum m_i = k$  guarantees the total matches the dimension of the solution space (see the Note after Theorem 6.13).

**Example 6.12.**  $a_n = 4a_{n-1} - 4a_{n-2}$ : characteristic equation  $x^2 - 4x + 4 = (x - 2)^2 = 0$ , double root  $r = 2$ . General solution:  $a_n = (\alpha_0 + \alpha_1 n)2^n$ .

**Definition 6.12** (Divide-and-conquer recurrence, Rosen §8.3). An algorithm that splits a size- $n$  problem into  $a$  subproblems of size  $n/b$ , spending  $g(n)$  extra work to combine results, has running time

$$f(n) = a f(n/b) + g(n).$$

**Example 6.13** (Merge sort). Split into  $a = 2$  halves ( $b = 2$ ), merge in  $g(n) = n$  (linear) time:

$$f(n) = 2f(n/2) + n.$$



**Theorem 6.6** (Master theorem, special case  $g(n) = cn^d$ , Rosen). For  $f(n) = af(n/b) + cn^d$  ( $a \geq 1$ ,  $b > 1$ ,  $c, d \geq 0$  real), when  $n$  is a power of  $b$ :

$$f(n) = \Theta \left( \begin{cases} n^d, & a < b^d, \\ n^d \log n, & a = b^d, \\ n^{\log_b a}, & a > b^d. \end{cases} \right)$$

*Proof sketch, unrolling the recurrence.* Let  $n = b^k$ . Unroll once:  $f(n) = af(n/b) + cn^d$ . Unroll again:  $f(n) = a^2 f(n/b^2) + ac(n/b)^d + cn^d$ . After  $k$  unrollings:

$$f(n) = a^k f(1) + cn^d \sum_{i=0}^{k-1} \left( \frac{a}{b^d} \right)^i.$$

The geometric sum  $\sum (a/b^d)^i$  behaves differently depending on whether the ratio  $a/b^d$  is  $< 1$ ,  $= 1$ , or  $> 1$  – giving the three cases (via the finite geometric series formula, §6.13, and  $a^k = a^{\log_b n} = n^{\log_b a}$ ). ■

**Example 6.14** (Applying it to merge sort).  $a = 2$ ,  $b = 2$ ,  $d = 1$ :  $a = b^d$  ( $2 = 2^1$ ), so  $f(n) = \Theta(n \log n)$ .

**Definition 6.13** (Generating function, Rosen §8.4). For sequence  $(a_n)_{n=0}^\infty$ , its *generating function* is the formal power series

$$G(x) = \sum_{n=0}^{\infty} a_n x^n.$$

**Example 6.15.**  $(1, 1, 1, \dots)$ :  $G(x) = \sum x^n = \frac{1}{1-x}$  (geometric series, §6.14).  $\binom{n}{k}$  for fixed  $k$ , varying  $n$ : related to  $(1-x)^{-(k+1)}$ .

*Note* (Solving recurrences via generating functions – the technique). 1. Multiply the recurrence  $a_n = c_1 a_{n-1} + \dots$  by  $x^n$  and sum over  $n$ .

2. Recognize each resulting sum as a shifted/scaled copy of  $G(x) = \sum a_n x^n$ .

3. Solve the resulting algebraic equation for  $G(x)$ .

4. Expand  $G(x)$  back into a power series (e.g. partial fractions) to read off  $a_n$  as the coefficient of  $x^n$ .

**Example 6.16** (Fibonacci via generating functions – harder, alternate route to §6.7.4's formula).  $f_0 = 0$ ,  $f_1 = 1$ ,  $f_n = f_{n-1} + f_{n-2}$  for  $n \geq 2$ . Let  $G(x) = \sum_{n \geq 0} f_n x^n$ .

$$G(x) = f_0 + f_1 x + \sum_{n \geq 2} f_n x^n = x + \sum_{n \geq 2} (f_{n-1} + f_{n-2}) x^n = x + x \sum_{n \geq 2} f_{n-1} x^{n-1} + x^2 \sum_{n \geq 2} f_{n-2} x^{n-2}.$$

The two sums are  $G(x)$  itself (shifted indices, re-summed from  $n = 1$  and  $n = 0$  respectively, matching since  $f_0 = 0$  doesn't affect the first):

$$G(x) = x + xG(x) + x^2 G(x) \Rightarrow G(x)(1 - x - x^2) = x \Rightarrow G(x) = \frac{x}{1 - x - x^2}.$$

Partial-fraction expanding  $\frac{x}{1-x-x^2}$  (using the roots of  $1 - x - x^2 = 0$ , related to the golden ratio) and reading off the coefficient of  $x^n$  reproduces exactly the closed form from §6.7.4.

## 6.7 The Fibonacci sequence

**Definition 6.14** (Fibonacci sequence).

$$f_1 = 1, \quad f_2 = 1, \quad f_n = f_{n-1} + f_{n-2} \quad (n \geq 3).$$

$$1, 1, 2, 3, 5, 8, 13, 21, 34, 55, \dots$$

*Note* (Explore-conjecture-prove, applied here). No formula is obvious from the recurrence alone. The ratio  $f_{n+1}/f_n$  appears to settle down (e.g.  $f_{10}/f_9 = 55/34 \approx 1.617$ ), suggesting exponential growth at some rate – explored further below.

### 6.7.1 Upper bounds

**Theorem 6.7.**  $f_n \leq 2^n$  for all  $n \geq 1$ .

*Proof.* Strong induction, base cases  $n = 1, 2$  ( $f_1 = 1 \leq 2$ ,  $f_2 = 1 \leq 4$ ); inductive step ( $k \geq 2$ ):  $f_{k+1} = f_k + f_{k-1} \leq 2^k + 2^{k-1} < 2^{k+1}$ . ■

*Note* (Tightening the bound). The proof only used  $2^k + 2^{k-1} \leq 2^{k+1}$ , i.e. (dividing by  $2^{k-1}$ )  $2+1 \leq 2^2$ . Any  $r$  with  $r+1 \leq r^2$  would work the same way in place of 2 – so the smallest such  $r$  gives the tightest bound of this form.

**Theorem 6.8** (Sharpest bound of this form). Let  $r_1 = \frac{1+\sqrt{5}}{2}$  (the positive root of  $r^2 - r - 1 = 0$ ). Then  $f_n \leq r_1^n$  for all  $n \geq 1$ .

*Proof.* Same induction, using  $r_1^k + r_1^{k-1} = r_1^{k+1}$  exactly (since  $r_1$  satisfies  $r_1 + 1 = r_1^2$ , multiply through by  $r_1^{k-1}$ ), and base cases  $f_1, f_2 = 1 < r_1, r_1^2$ . ■

### 6.7.2 Lower bounds

**Theorem 6.9.**  $f_n \geq r_1^{n-2}$  for all  $n \geq 1$ , where  $r_1$  is as above.

*Proof sketch.* Same strategy in reverse: seek the largest  $r \leq r_1$  with  $r+1 \geq r^2$ ; the quadratic  $r^2 - r - 1 = 0$ 's roots are exactly the boundary, so  $r_1$  itself (the larger root) works with equality in the recurrence step, giving the tightest such lower bound after matching the base cases  $n = 1, 2$ . ■

### 6.7.3 Asymptotic behaviour

*Note* (Squeezing  $f_n$  between bounds). Combining both bounds:

$$r_1^{n-2} \leq f_n \leq r_1^n \quad \implies \quad r_1^{-2} \leq \frac{f_n}{r_1^n} \leq 1.$$

So  $f_n/r_1^n$  is trapped between two constants, independent of  $n$  – meaning  $f_n$  grows *exactly* like  $r_1^n$  up to a bounded factor.  $r_1$  is the *golden ratio*, usually written  $\varphi$ .

#### 6.7.4 An exact formula

*Note* (Why two roots are needed – motivating Theorem 6.10). It is tempting to seek an exact formula of the single-term form  $\alpha_1 r_1^n$ , using only the larger root  $r_1 = \frac{1+\sqrt{5}}{2}$  of  $r^2 - r - 1 = 0$  from §???. Any such  $r_1^n$  does satisfy the Fibonacci recurrence itself, since

$$r_1^n = r_1^{n-1} + r_1^{n-2}$$

follows directly from  $r_1^2 = r_1 + 1$  (multiply through by  $r_1^{n-2}$ ). But matching the *initial conditions*  $f_1 = f_2 = 1$  requires satisfying two equations, while the single-term form has only one free constant  $\alpha_1$  to work with – one degree of freedom cannot in general solve two independent equations exactly. This is exactly why §???’s bounds only ever achieved an *inequality* in one base case, never equality in both simultaneously.

The fix is to bring back the smaller root  $r_2 = \frac{1-\sqrt{5}}{2}$ , previously discarded. Since  $r_2$  also satisfies  $r^2 - r - 1 = 0$ , the same argument gives

$$r_2^n = r_2^{n-1} + r_2^{n-2},$$

so  $r_2^n$  *also* solves the recurrence. By linearity of the homogeneous recurrence (any linear combination of solutions is again a solution – cf. the vector-space viewpoint in the Note after Theorem ??), the two-parameter family

$$f_n = \alpha_1 r_1^n + \alpha_2 r_2^n$$

still satisfies the recurrence for *any* choice of  $\alpha_1, \alpha_2$ , but now has exactly two free constants – enough to match both initial conditions  $f_1 = 1, f_2 = 1$  exactly. Solving the resulting  $2 \times 2$  linear system for  $\alpha_1, \alpha_2$  is precisely the content of the proof of Theorem 6.10 below.

**Theorem 6.10** (Binet’s formula). *For all  $n \geq 1$ :*

$$f_n = \frac{\varphi^n - \psi^n}{\sqrt{5}}, \quad \varphi = \frac{1 + \sqrt{5}}{2}, \quad \psi = \frac{1 - \sqrt{5}}{2}.$$

*Proof.* The characteristic equation of  $f_n = f_{n-1} + f_{n-2}$  is  $x^2 - x - 1 = 0$ , with roots exactly  $\varphi, \psi$  (the two roots identified in §6.7.1–6.7.2). Since these are distinct, the general solution (§6.6’s Distinct roots theorem) is

$$f_n = \alpha \varphi^n + \beta \psi^n.$$

Match initial conditions  $f_1 = f_2 = 1$ :

$$\alpha \varphi + \beta \psi = 1, \quad \alpha \varphi^2 + \beta \psi^2 = 1.$$

Solving this  $2 \times 2$  linear system (using  $\varphi - \psi = \sqrt{5}$ ) gives  $\alpha = \frac{1}{\sqrt{5}}, \beta = -\frac{1}{\sqrt{5}}$ , yielding

$$f_n = \frac{1}{\sqrt{5}} \varphi^n - \frac{1}{\sqrt{5}} \psi^n = \frac{\varphi^n - \psi^n}{\sqrt{5}}.$$

■

*Note* (Consistency check with §6.7.3). Since  $|\psi| < 1$ ,  $\psi^n \rightarrow 0$ , so  $f_n \approx \varphi^n / \sqrt{5}$  for large  $n$  – consistent with the squeeze  $r_1^{-2} \leq f_n / r_1^n \leq 1$  obtained purely by induction, without ever solving the characteristic equation.

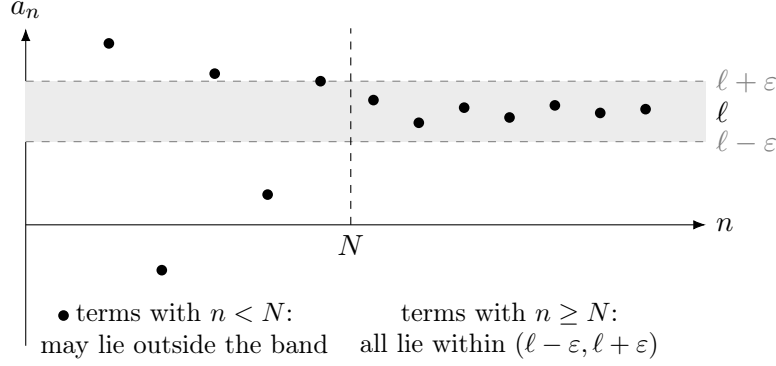
*Note* (A genuinely surprising fact).  $f_n$  is always an integer, yet the formula involves  $\sqrt{5}$  throughout – the irrational parts exactly cancel in  $\varphi^n - \psi^n$  for every  $n$ . In fact  $f_n$  is the *nearest integer* to  $\varphi^n / \sqrt{5}$ , since  $|\psi^n / \sqrt{5}| < \frac{1}{2}$  for all  $n \geq 1$ .

## 6.8 Limits of infinite sequences

**Definition 6.15** (Limit of a sequence,  $\varepsilon$ - $N$  definition).  $(a_n)$  has *limit*  $\ell$ , written  $\lim_{n \rightarrow \infty} a_n = \ell$  (or  $a_n \rightarrow \ell$ ), iff

$$\forall \varepsilon > 0 \exists N \in \mathbb{N} \forall n \geq N \quad |a_n - \ell| < \varepsilon.$$

If such  $\ell$  exists,  $(a_n)$  is *convergent*; otherwise *divergent*.



**Example 6.17.**  $\lim_{n \rightarrow \infty} \frac{1}{n} = 0$ : given  $\varepsilon > 0$ , take  $N > 1/\varepsilon$ ; then  $n \geq N \Rightarrow 1/n \leq 1/N < \varepsilon$ , so  $|1/n - 0| < \varepsilon$ .

**Example 6.18.**  $\lim_{n \rightarrow \infty} \frac{n}{n+1} = 1$ : given  $\varepsilon > 0$ ,

$$\left| \frac{n}{n+1} - 1 \right| = \frac{1}{n+1} < \varepsilon \iff n > \frac{1}{\varepsilon} - 1,$$

so take  $N$  any integer  $> \frac{1}{\varepsilon} - 1$ .

**Theorem 6.11** (Uniqueness of the limit). *If  $a_n \rightarrow \ell_1$  and  $a_n \rightarrow \ell_2$ , then  $\ell_1 = \ell_2$ .*

*Proof.* Suppose  $\ell_1 \neq \ell_2$ ; let  $\varepsilon = |\ell_1 - \ell_2|/2 > 0$ . By the two limit definitions, there exist  $N_1, N_2$  with  $|a_n - \ell_1| < \varepsilon$  for  $n \geq N_1$  and  $|a_n - \ell_2| < \varepsilon$  for  $n \geq N_2$ . For  $n \geq \max(N_1, N_2)$ , the triangle inequality gives

$$|\ell_1 - \ell_2| \leq |\ell_1 - a_n| + |a_n - \ell_2| < \varepsilon + \varepsilon = |\ell_1 - \ell_2|,$$

a contradiction. So  $\ell_1 = \ell_2$ . ■

**Theorem 6.12** (Convergent sequences are bounded). *If  $(a_n)$  converges, then  $(a_n)$  is bounded:  $\exists M > 0$ ,  $|a_n| \leq M$  for all  $n$ .*

*Proof.* Let  $a_n \rightarrow \ell$ . Taking  $\varepsilon = 1$  in the definition, there is  $N$  with  $|a_n - \ell| < 1$  for all  $n \geq N$ , so  $|a_n| < |\ell| + 1$  for those  $n$ . Let

$$M = \max(|a_1|, \dots, |a_{N-1}|, |\ell| + 1).$$

Then  $|a_n| \leq M$  for every  $n$ . ■

**Theorem 6.13** (Algebra of limits). *If  $a_n \rightarrow a$  and  $b_n \rightarrow b$ , then:*

$$a_n + b_n \rightarrow a + b, \quad a_n - b_n \rightarrow a - b, \quad a_n b_n \rightarrow ab, \quad \frac{a_n}{b_n} \rightarrow \frac{a}{b} \quad (b \neq 0, b_n \neq 0).$$

*Proof of the product rule, as a representative example.* Since  $(a_n)$  converges, it is bounded (previous theorem):  $|a_n| \leq M$  for all  $n$ . Given  $\varepsilon > 0$ , choose  $N_1$  so that  $n \geq N_1 \Rightarrow |a_n - a| < \frac{\varepsilon}{2(|b| + 1)}$ , and  $N_2$  so that  $n \geq N_2 \Rightarrow |b_n - b| < \frac{\varepsilon}{2M}$ . For  $n \geq \max(N_1, N_2)$ :

$$|a_n b_n - ab| = |a_n b_n - a_n b + a_n b - ab| \leq |a_n| |b_n - b| + |b| |a_n - a| < M \cdot \frac{\varepsilon}{2M} + |b| \cdot \frac{\varepsilon}{2(|b| + 1)} < \varepsilon.$$

■

**Theorem 6.14** (Squeeze theorem). *If  $a_n \leq c_n \leq b_n$  for all  $n$  (eventually), and  $a_n \rightarrow \ell$ ,  $b_n \rightarrow \ell$ , then  $c_n \rightarrow \ell$ .*

*Proof.* Given  $\varepsilon > 0$ , choose  $N_1, N_2$  with  $|a_n - \ell| < \varepsilon$  for  $n \geq N_1$  and  $|b_n - \ell| < \varepsilon$  for  $n \geq N_2$ . For  $n \geq \max(N_1, N_2)$ :

$$\ell - \varepsilon < a_n \leq c_n \leq b_n < \ell + \varepsilon,$$

so  $|c_n - \ell| < \varepsilon$ .

■

**Example 6.19** (Applying the squeeze theorem – revisits §6.7.3). The bounds  $r_1^{-2} \leq f_n/r_1^n \leq 1$  established for Fibonacci (§6.7.3) do *not* by themselves give  $\lim f_n/r_1^n$  – the squeeze theorem needs *both* bounding sequences to converge to the *same* limit, which  $r_1^{-2}$  (constant) and 1 (constant) do not (they’re different constants) – so this only shows  $f_n/r_1^n$  stays bounded, not that it converges. (It does in fact converge, to  $1/\sqrt{5}$ , by Binet’s formula, §6.7.4 – but that requires more than the squeeze theorem alone.)

**Theorem 6.15** (Monotone convergence theorem). *Every bounded, monotonic sequence converges.*

*Note* (Proof idea). Relies on the completeness of  $\mathbb{R}$  (every bounded set of reals has a supremum): if  $(a_n)$  is increasing and bounded above,  $\ell = \sup\{a_n\}$  exists, and one shows  $a_n \rightarrow \ell$  directly from the definition of supremum. This is genuinely a deeper real-analysis fact (completeness of  $\mathbb{R}$ ), not provable from the algebraic properties of  $\mathbb{R}$  alone – it fails, for instance, over  $\mathbb{Q}$ .

**Definition 6.16** (Subsequence). Given  $(a_n)_{n=1}^\infty$  and a strictly increasing sequence of indices  $n_1 < n_2 < n_3 < \dots$ , the sequence  $(a_{n_k})_{k=1}^\infty$  is a *subsequence* of  $(a_n)$ .

**Theorem 6.16** (Bolzano-Weierstrass theorem). *Every bounded sequence in  $\mathbb{R}$  has a convergent subsequence.*

*Proof sketch, bisection.* Let  $(a_n) \subseteq [A, B]$ . Bisect  $[A, B]$  into two closed halves; at least one half contains  $a_n$  for infinitely many  $n$  – call it  $I_1$ , and pick any  $a_{n_1} \in I_1$ . Bisect  $I_1$  similarly, get  $I_2 \subset I_1$  (half the length) still containing infinitely many terms, pick  $a_{n_2} \in I_2$  with  $n_2 > n_1$ . Repeating gives nested intervals  $I_1 \supset I_2 \supset \dots$  with lengths  $\rightarrow 0$ , and a subsequence  $a_{n_k} \in I_k$ . By the Nested Interval Property (a consequence of completeness of  $\mathbb{R}$ ),  $\bigcap_k I_k$  is a single point  $\ell$ , and  $a_{n_k} \rightarrow \ell$  since  $|a_{n_k} - \ell| \leq \text{length}(I_k) \rightarrow 0$ .

■

**Definition 6.17** (Cauchy sequence).  $(a_n)$  is *Cauchy* iff

$$\forall \varepsilon > 0 \exists N \in \mathbb{N} \forall m, n \geq N \quad |a_n - a_m| < \varepsilon.$$

## 6.9 Big-O notation

**Definition 6.18** (Big-O). For  $(a_n)$  nonnegative and  $f : \mathbb{N} \rightarrow \mathbb{R}$ :

$$a_n = O(f(n)) \iff \exists c, N \forall n \geq N \quad a_n \leq c \cdot f(n).$$

**Example 6.20.** For  $t_n = 100n + 10n^2 + 2n + \log n$ : for  $n \geq 10$ , each of  $100n, 10n^2, \log n \leq 2^n$ , so

$$t_n \leq 4 \cdot 2^n \quad (n \geq 10), \quad \text{hence } t_n = O(2^n).$$

*Note* (Big-O swallows constant factors).  $t_n = O(2^n)$ ,  $O(4 \cdot 2^n)$ ,  $O(0.001 \cdot 2^n)$  are all correct – since  $2^n = O(0.001 \cdot 2^n)$  too (take  $c = 1000$ ). So constants inside the  $O(\cdot)$  are pointless; state  $O(2^n)$ , not  $O(4 \cdot 2^n)$ .

*Note* (Base matters, unlike constant factors).  $t_n = O(3^n)$  is also true (looser, since  $2^n \leq 3^n$ ), but  $t_n = O(1.9^n)$  is *false* – no constant  $c$  makes  $c \cdot 1.9^n$  eventually dominate  $2^n$ . Exponential *bases* are not interchangeable, unlike constant multipliers.

**Example 6.21** (Dominant-term technique).

$$\frac{1}{8}n^3 + \frac{1}{4}n^2 + \frac{1}{2}n + 1 = O(n^3), \quad \log n + \log \log n = O(\log n), \quad (\sqrt{n} + 3n + 9^{1/n})(17 + 12 \log n) = O(n \log n).$$

**Definition 6.19** (Big-Omega, Big-Theta, Rosen §3.2).

$$f(n) = \Omega(g(n)) \iff \exists c, N \forall n \geq N \quad f(n) \geq cg(n) \quad (\text{lower bound; } g = O(f)),$$

$$f(n) = \Theta(g(n)) \iff f(n) = O(g(n)) \text{ and } f(n) = \Omega(g(n)) \quad (\text{matching bounds both ways}).$$

*Note.*  $O$  alone (CN's focus) only gives an upper bound – e.g.  $n = O(n^2)$  is true but loose.  $\Theta$  pins growth down exactly (up to constants) –  $t_n = \Theta(2^n)$ , but *not*  $t_n = \Theta(3^n)$ , since  $3^n$  is not  $O(t_n)$ .

**Definition 6.20** (Time complexity of an algorithm, Rosen §3.3). The *worst-case time complexity* of an algorithm is  $f(n) = O(g(n))$  if the number of operations it performs on *any* input of size  $n$  is  $O(g(n))$ .

**Example 6.22** (Linear search, Rosen – harder, complete algorithm analysis). Search a list of  $n$  items for a target, comparing one at a time until found or the list ends.

- *Worst case*: target absent (or last),  $n$  comparisons:  $\Theta(n)$ .
- *Best case*: target first, 1 comparison:  $\Theta(1)$ .
- *Average case* (target present, each position equally likely):  $\frac{1+2+\dots+n}{n} = \frac{n+1}{2} = \Theta(n)$  (§6.11's sum formula).

**Example 6.23** (Binary search, Rosen). On a sorted list of  $n$  items, each comparison halves the remaining search space:  $T(n) = T(n/2) + O(1)$ , giving  $T(n) = O(\log n)$  (§6.6/§8.3's divide-and-conquer machinery, master theorem case  $a = 1, b = 2, d = 0$ :  $a = b^d$ ).

*Note* (Why worst-case dominates in practice). CN's emphasis on upper bounds (via  $O$  rather than  $\Theta$  or average-case analysis) matches how complexity is usually reported: a guaranteed ceiling on resource use is often more actionable for budgeting than an exact or average figure, especially when the exact behavior is too complex to pin down precisely (CN's own motivation for introducing  $O$  at all).

**Definition 6.21** (Polynomial-time algorithm, tractability). An algorithm is *polynomial-time* if its worst-case complexity is  $O(n^k)$  for some fixed  $k$  – called *tractable*. Complexity like  $O(2^n)$  or  $O(n!)$  is *intractable*: even modest  $n$  makes it infeasible to run.

**Example 6.24** (Why the distinction matters – concrete numbers). At  $10^9$  operations/second:  $n = 100$ ,  $O(n^3)$  ( $10^6$  ops) takes a fraction of a second;  $O(2^n)$  ( $2^{100} \approx 10^{30}$  ops) takes vastly longer than the age of the universe. The gap between polynomial and exponential dwarfs any constant-factor speedup from faster hardware.

**Definition 6.22** (Class P, class NP, Rosen – harder, beyond CN).  $P$  = decision problems solvable by a polynomial-time algorithm.  $NP$  = decision problems whose "yes" answers have a polynomial-time *verifiable* certificate (a solution can be checked quickly, even if it might be hard to *find*).

*Note.*  $P \subseteq NP$ : a polynomial-time solver is trivially also a polynomial-time verifier (just re-solve and compare). Whether  $P = NP$  – whether every efficiently-*verifiable* problem is also efficiently *solvable* – is the most famous open problem in computer science (a Clay Millennium Prize problem), unresolved despite decades of effort.

*Note* (Connection to earlier material). The  $n$ -queens / Sudoku satisfiability encodings (§4's Satisfiability section) are instances of  $NP$ : a proposed solution (a truth assignment) is checked in polynomial time by just evaluating the CNF formula, but no polynomial-time algorithm for *finding* a satisfying assignment is known for general CNF – SAT is, in fact, *NP-complete*: every problem in  $NP$  can be reduced to it in polynomial time, so a polynomial-time SAT solver would imply  $P = NP$ .

**Definition 6.23** (NP-hard, Garey & Johnson 1979 / cf. Cormen et al., *Introduction to Algorithms*). A problem  $H$  is *NP-hard* if every problem  $L \in NP$  is polynomial-time reducible to  $H$ , i.e. for every  $L \in NP$  there is a function  $f$ , computable in polynomial time, such that for every input  $x$ ,

$$x \in L \iff f(x) \in H.$$

$H$  itself need not belong to  $NP$ .

**Definition 6.24** (NP-complete, Cook 1971 / Levin 1973; standard formulation, e.g. Garey & Johnson 1979, Sipser *Introduction to the Theory of Computation*). A problem  $L$  is *NP-complete* if

1.  $L \in NP$ , and
2.  $L$  is NP-hard.

*Note* (Consequence of NP-completeness). If any single NP-complete problem admits a polynomial-time algorithm, then every problem in  $NP$  does (compose that algorithm with the polynomial-time reduction), giving  $P = NP$ . Conversely, if any problem in  $NP$  can be shown to require superpolynomial time, then  $P \neq NP$ . This is why NP-complete problems are the natural target of the  $P$  vs.  $NP$  question: resolving the status of a single one resolves it for the whole class.

*Note* (Cook–Levin theorem, S. Cook, *The Complexity of Theorem-Proving Procedures*, STOC 1971; independently L. Levin, 1973). SAT was the first problem proven NP-complete (Cook, 1971; independently Levin). The proof shows directly that every problem in  $NP$  reduces to SAT, by simulating the computation of an arbitrary polynomial-time verifier as a Boolean formula. Once one NP-complete problem is known, further problems are shown NP-complete by reduction *from* an already-known NP-complete problem (rather than from the definition directly): to show  $L$  is NP-complete, it suffices to show  $L \in NP$  and exhibit a polynomial-time reduction from some known

NP-complete problem to  $L$ . Thousands of problems have been classified this way, including 3-SAT, vertex cover, independent set, Hamiltonian cycle, graph colouring, subset sum, and the decision version of the travelling salesman problem.

*Note* (The defining condition, and how the pieces fit together).

The single line that defines NP-hardness is:

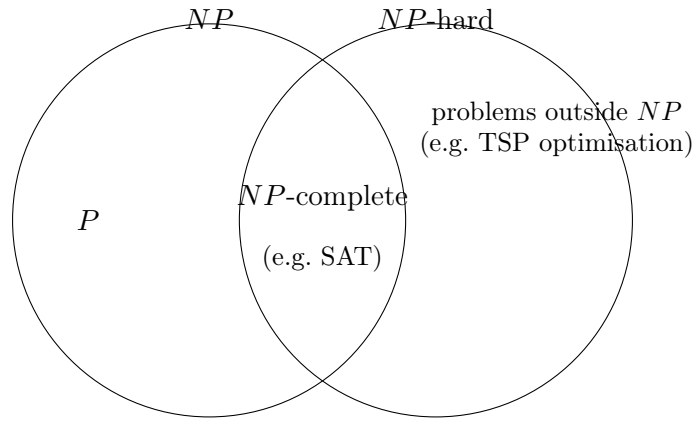
$$H \text{ is NP-hard} \iff \forall L \in NP, L \text{ is polynomial-time reducible to } H.$$

Three things in this line all matter simultaneously:

- “every problem  $L \in NP$ ” – a universal quantifier: not just some problem, but all of them.
- “polynomial-time” – the reduction  $f$  itself must be computable in polynomial time, or it cannot transfer tractability from  $H$  to  $L$ .
- “reducible to  $H$ ” – the direction matters: everything else reduces *into*  $H$ , making  $H$  a hardest problem, not the other way around.

Overall relationship:

$$NP\text{-complete} = NP \cap NP\text{-hard}.$$



The left circle is  $NP$  (decision problems with polynomial-time-verifiable certificates); the right circle is  $NP$ -hard (everything at least as hard as every  $NP$  problem, via reduction).  $H$  need not lie in  $NP$ , so the right circle extends beyond the left one. Their intersection – problems that are both easy to verify and at least as hard as anything in  $NP$  – is exactly  $NP$ -complete.

## 6.10 Sums and summation

*Note* (Notation).

$$\sum_{k=1}^n a_k = a_1 + a_2 + \cdots + a_n.$$

**Example 6.25** (Double sum, swapping order). Consider the array of values  $j$  indexed by row



$i = 1, \dots, n$  and column  $j = 1, \dots, i$ :

| $i \backslash j$ | 1        | 2        | 3 | $\dots$  | $n$ |
|------------------|----------|----------|---|----------|-----|
| 1                | 1        |          |   |          |     |
| 2                | 1        | 2        |   |          |     |
| 3                | 1        | 2        | 3 |          |     |
| $\vdots$         | $\vdots$ | $\vdots$ |   | $\ddots$ |     |
| $n$              | 1        | 2        | 3 | $\dots$  | $n$ |

Summing *row by row* gives the original sum; summing *column by column* gives the swapped form. Column  $j$  contains the value  $j$  repeated once for every row  $i = j, \dots, n$ , i.e.  $n - j + 1$  times:

$$\sum_{i=1}^n \sum_{j=1}^i j = \underbrace{(1 + \dots + 1)}_{j=1, n \text{ copies}} + \underbrace{(2 + \dots + 2)}_{j=2, n-1 \text{ copies}} + \dots + \underbrace{(n)}_{j=n, 1 \text{ copy}}.$$

Each column's total is  $j \times (n - j + 1)$  (e.g. column  $j = 2$ :  $2 \times (n - 2 + 1) = 2(n - 1)$ , matching  $n - 1$  copies of 2). Summing over columns instead of rows swaps the order of summation:

$$\sum_{i=1}^n \sum_{j=1}^i j = \sum_{j=1}^n \sum_{i=j}^n j = \sum_{j=1}^n j(n - j + 1).$$

The first equality swaps summation order (the region  $1 \leq j \leq i \leq n$  described either "by rows" or "by columns"); the second collapses the inner sum since  $j$  is constant over  $i = j, \dots, n$ , contributing  $n - j + 1$  copies.

**Example 6.26** (Same sum, expanded directly). Writing out the inner sum  $\sum_{j=1}^i j = 1 + 2 + \dots + i$  for each  $i$  before summing over  $i$  gives the same total from a different angle:

$$\sum_{i=1}^n \sum_{j=1}^i j = \sum_{i=1}^n (1 + 2 + \dots + i) = 1 + (1 + 2) + (1 + 2 + 3) + \dots + (1 + \dots + n).$$

This makes the triangular shape of the summation region visible directly: the  $i$ -th block has length  $i$ , so the blocks grow row by row – the same picture that the swapped form in the example above sums up column by column instead.

**Example 6.27** (Telescoping sum – harder).

$$\sum_{k=1}^n \frac{1}{k(k+1)} = \sum_{k=1}^n \left( \frac{1}{k} - \frac{1}{k+1} \right) = 1 - \frac{1}{n+1},$$

since every interior term cancels:  $(1 - \frac{1}{2}) + (\frac{1}{2} - \frac{1}{3}) + \dots + (\frac{1}{n} - \frac{1}{n+1})$ , leaving only the first and last pieces.

**Theorem 6.17** (Abel summation (summation by parts) – beyond Rosen, genuinely advanced). *For sequences  $(a_k), (b_k)$ , let  $A_n = \sum_{k=1}^n a_k$ . Then*

$$\sum_{k=1}^n a_k b_k = A_n b_n - \sum_{k=1}^{n-1} A_k (b_{k+1} - b_k).$$

*Proof.* Write  $a_k = A_k - A_{k-1}$  (with  $A_0 = 0$ ). Then

$$\sum_{k=1}^n a_k b_k = \sum_{k=1}^n (A_k - A_{k-1}) b_k = \sum_{k=1}^n A_k b_k - \sum_{k=1}^n A_{k-1} b_k.$$

Re-index the second sum ( $j = k - 1$ ):  $\sum_{j=0}^{n-1} A_j b_{j+1} = \sum_{j=1}^{n-1} A_j b_{j+1}$  (since  $A_0 = 0$ ). So

$$\sum_{k=1}^n a_k b_k = A_n b_n + \sum_{k=1}^{n-1} A_k (b_k - b_{k+1}) = A_n b_n - \sum_{k=1}^{n-1} A_k (b_{k+1} - b_k).$$

■

*Note* (The discrete analogue of integration by parts).  $A_n$  plays the role of an antiderivative of  $a_k$ ;  $b_{k+1} - b_k$  is a discrete derivative –  $\int u dv = uv - \int v du$  with sums replacing integrals. This underlies proving convergence of series like  $\sum \frac{(-1)^k}{k}$  that don't converge absolutely.

## 6.11 Finite series

*Note.* A *finite series* is the sum of a finite sequence's terms; a *partial sum*  $S_n = \sum_{k=1}^n a_k$  of an infinite sequence.

## 6.12 Finite arithmetic series

**Theorem 6.18.**

$$\sum_{k=1}^n (a + (k-1)d) = n \cdot \frac{2a + (n-1)d}{2}.$$

**Example 6.28** (Deriving  $\sum k^2$  by telescoping, before proving it – zhangzhusheng-style construction). Start from the identity

$$(k+1)^3 - k^3 = 3k^2 + 3k + 1.$$

Sum both sides for  $k = 1, \dots, n$ . The left side telescopes:

$$\sum_{k=1}^n [(k+1)^3 - k^3] = (n+1)^3 - 1^3,$$

since every intermediate term cancels:  $(2^3 - 1^3) + (3^3 - 2^3) + \dots + ((n+1)^3 - n^3)$ .

The right side sums termwise:

$$\sum_{k=1}^n (3k^2 + 3k + 1) = 3 \sum_{k=1}^n k^2 + 3 \sum_{k=1}^n k + n = 3 \sum_{k=1}^n k^2 + 3 \cdot \frac{n(n+1)}{2} + n.$$

Equating and solving for  $\sum k^2$ :

$$(n+1)^3 - 1 = 3 \sum_{k=1}^n k^2 + \frac{3n(n+1)}{2} + n \implies \sum_{k=1}^n k^2 = \frac{(n+1)^3 - 1 - n - \frac{3n(n+1)}{2}}{3}.$$

Simplifying the numerator (common denominator 2, expand  $(n+1)^3 = n^3 + 3n^2 + 3n + 1$ ):

$$(n+1)^3 - 1 - n - \frac{3n(n+1)}{2} = n^3 + 3n^2 + 3n - n - \frac{3n^2 + 3n}{2} = \frac{2n^3 + 6n^2 + 4n - 3n^2 - 3n}{2} = \frac{2n^3 + 3n^2 + n}{2} = \frac{n(n+1)(2n+1)}{2}.$$

Dividing by 3:

$$\sum_{k=1}^n k^2 = \frac{n(n+1)(2n+1)}{6}.$$

*Note* (Why this method matters). Unlike the induction proof below (which needs the formula *guessed* first), this telescoping method *derives* the formula from scratch – the same technique, using  $(k+1)^4 - k^4$ , derives  $\sum k^3$ , and in general  $(k+1)^{m+1} - k^{m+1}$  derives  $\sum k^m$  for any  $m$ , always reducing to lower-power sums already known.

### 6.13 Finite geometric series

**Theorem 6.19.** For  $r \neq 1$ :

$$\sum_{k=0}^n ar^k = a \cdot \frac{r^{n+1} - 1}{r - 1}.$$

**Example 6.29** (Sum  $\sum k \cdot 2^k$  – via differentiating a generating function, harder). Start from  $\sum_{k=0}^n x^k = \frac{x^{n+1} - 1}{x - 1}$ . Differentiate both sides with respect to  $x$ :

$$\sum_{k=0}^n kx^{k-1} = \frac{(n+1)x^n(x-1) - (x^{n+1} - 1)}{(x-1)^2}.$$

Multiply by  $x$  and substitute  $x = 2$ :

$$\sum_{k=0}^n k2^k = 2[(n+1)2^n \cdot 1 - (2^{n+1} - 1)] = (n-1)2^{n+1} + 2.$$

### 6.14 Infinite geometric series

**Theorem 6.20** (Convergence condition, via §6.8's rigorous limit machinery).

$$\sum_{k=0}^{\infty} ar^k = \frac{a}{1-r} \iff |r| < 1.$$

*Proof.* The partial sums  $S_n = a \frac{1-r^{n+1}}{1-r}$ . By §6.8's example ( $r^n \rightarrow 0$  for  $|r| < 1$ , established rigorously via  $\varepsilon$ - $N$ ),  $S_n \rightarrow \frac{a}{1-r}$ . For  $|r| \geq 1$ ,  $r^n$  does not  $\rightarrow 0$  (it diverges, or stays  $\geq 1$  in absolute value), so  $S_n$  has no finite limit. ■

### 6.15 Harmonic numbers

**Definition 6.25** (Harmonic numbers).

$$H_n = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}.$$

**Theorem 6.21** (Divergence of the harmonic series).  $H_n \rightarrow \infty$  as  $n \rightarrow \infty$ , despite  $1/n \rightarrow 0$ .

*Classical proof, grouping by powers of 2.* Group terms:

$$H_{2^k} = 1 + \frac{1}{2} + \left(\frac{1}{3} + \frac{1}{4}\right) + \left(\frac{1}{5} + \cdots + \frac{1}{8}\right) + \cdots + \left(\frac{1}{2^{k-1}+1} + \cdots + \frac{1}{2^k}\right).$$

The block from  $\frac{1}{2^{j-1}+1}$  to  $\frac{1}{2^j}$  has  $2^{j-1}$  terms, each  $\geq \frac{1}{2^j}$ , so the block sums to  $\geq 2^{j-1} \cdot \frac{1}{2^j} = \frac{1}{2}$ . There are  $k$  such blocks (for  $j = 1, \dots, k$ ), so

$$H_{2^k} \geq 1 + \frac{k}{2}.$$

Since  $1 + k/2 \rightarrow \infty$ ,  $H_{2^k} \rightarrow \infty$ , and  $H_n$  is increasing, so  $H_n \rightarrow \infty$ . ■

*Note.* This is exactly the same bound proved earlier by induction (§3.11's harmonic-numbers theorem,  $H_{2^n} \geq 1 + n/2$ ) – here derived directly, without induction, by explicit grouping.

*Note* (Contrast: convergent vs. divergent zero-limit sequences).  $(1/n^2)$  and  $(1/2^n)$  both  $\rightarrow 0$  with *finite* sum (the latter by §6.14's geometric series).  $(1/n)$  is the borderline case: it decays just slowly enough to have infinite sum – among decaying sequences,  $1/n$  marks roughly where convergence of the sum starts to fail (the  $p$ -series  $\sum 1/n^p$  converges for  $p > 1$ , diverges for  $p \leq 1$ ).

*Note* (Euler's constant).

$$\lim_{n \rightarrow \infty} (H_n - \ln n) = \gamma = 0.5772\dots,$$

so  $H_n \approx \ln n + \gamma$  for large  $n$ . Whether  $\gamma$  is rational is unknown – an open problem.

## 7 Number Theory

### 7.1 Multiples and divisors

**Definition 7.1** (Divisibility). For  $d \in \mathbb{Z} \setminus \{0\}$  and  $n \in \mathbb{Z}$ , we say  $d$  divides  $n$ , written  $d \mid n$ , if

$$\exists q \in \mathbb{Z} \quad n = qd.$$

Then  $d$  is a *divisor* (or *factor*) of  $n$ , and  $n$  is a *multiple* of  $d$ . We write  $d \nmid n$  otherwise. The set of all multiples of  $d$  is

$$d\mathbb{Z} = \{kd : k \in \mathbb{Z}\}.$$

A divisor of  $n$  is *proper* if it is neither 1 nor  $n$ .

*Note.*  $d \mid 0$  for every  $d \neq 0$ , since

$$0 = 0 \cdot d.$$

Zero must often be treated as a special case.

**Theorem 7.1** (Linearity of divisibility). For all  $d, m, n, x, y \in \mathbb{Z}$  with  $d \neq 0$ ,

$$d \mid m \wedge d \mid n \implies d \mid (xm + yn).$$

In particular  $d \mid (m + n)$  and  $d \mid (m - n)$ .

*Proof.* Write

$$m = q_1d, \quad n = q_2d.$$

Then

$$xm + yn = (xq_1 + yq_2)d,$$

and  $xq_1 + yq_2 \in \mathbb{Z}$ . ■

*Note.* The converse fails:  $2 \mid (3 + 5)$  but  $2 \nmid 3$  and  $2 \nmid 5$ .

**Example 7.1** (Divisibility by a product of consecutive integers – harder). Prove

$$6 \mid n(n + 1)(n + 2)$$

for all  $n \in \mathbb{Z}$ .

Among any three consecutive integers, at least one is even, so  $2 \mid n(n + 1)(n + 2)$ .

Writing  $n \equiv r \pmod{3}$  with  $r \in \{0, 1, 2\}$ :

- if  $r = 0$  then  $3 \mid n$ ;
- if  $r = 1$  then  $3 \mid (n + 2)$ ;
- if  $r = 2$  then  $3 \mid (n + 1)$ .

So  $3 \mid n(n + 1)(n + 2)$  in every case. Since  $\gcd(2, 3) = 1$ , Theorem 7.14 gives  $6 \mid n(n + 1)(n + 2)$ .

**Example 7.2** (Divisibility tests, beyond CN – Rosen §4.1). Let  $n$  have decimal digits  $a_k \cdots a_1 a_0$ , i.e.

$$n = \sum_{i=0}^k a_i 10^i.$$

Since  $10^i \equiv 0 \pmod{2^j}$  for  $i \geq j$ , the residue  $n \bmod 2^j$  depends only on the last  $j$  digits:

$$2^j \mid n \iff 2^j \mid \sum_{i=0}^{j-1} a_i 10^i.$$

This is why  $4 \mid n$  is decided by the last two digits and  $8 \mid n$  by the last three. The same argument with  $5^j$  gives the tests for 5, 25, 125.

In general, divisibility by  $d$  is decidable from a bounded number of trailing decimal digits precisely when  $d \mid 10^j$  for some  $j$ , i.e. when  $d = 2^a 5^b$ .

## 7.2 Prime numbers

**Definition 7.2** (Prime, composite). An integer  $p > 1$  is *prime* if it has no proper divisors. An integer  $n > 1$  that is not prime is *composite*.

**Theorem 7.2** (Existence of prime factorisation). *Every integer  $n \geq 2$  is a product of primes.*

*Proof.* By strong induction on  $n$ .

**Basis.**  $n = 2$  is prime, hence a product of one prime.

**Inductive step.** Let  $n \geq 2$  and assume every  $k$  with  $2 \leq k \leq n$  is a product of primes. Consider  $n + 1$ .

- If  $n + 1$  is prime, it is a product of one prime.
- If  $n + 1$  is composite, it has a divisor  $a$  with  $1 < a < n + 1$ . Set

$$b = \frac{n+1}{a} \in \mathbb{Z}.$$

Then  $1 < b < n + 1$  as well: if  $b = 1$  then  $a = n + 1$ , and if  $b = n + 1$  then  $a = 1$ , both contradicting the choice of  $a$ . Since  $a, b \leq n$ , the inductive hypothesis applies to each, so both are products of primes, hence so is  $n + 1 = ab$ .

By strong induction the claim holds for all  $n \geq 2$ . ■

**Theorem 7.3** (Infinitude of primes, Euclid – Rosen §4.3). *There are infinitely many primes.*

*Proof.* Suppose the primes are exactly  $p_1, \dots, p_k$ . Let

$$N = p_1 p_2 \cdots p_k + 1.$$

By Theorem 7.2,  $N$  has a prime divisor  $p_i$  for some  $i$ . But  $p_i \mid p_1 \cdots p_k$ , so by Theorem 7.1

$$p_i \mid (N - p_1 \cdots p_k) = 1,$$

which is impossible for a prime. Hence no finite list contains all primes. ■

**Theorem 7.4** (Trial division bound – Rosen §4.3). *If  $n > 1$  is composite, then  $n$  has a prime divisor  $p \leq \sqrt{n}$ .*

*Proof.* Write  $n = ab$  with  $1 < a \leq b < n$ . Then

$$a^2 \leq ab = n, \quad \text{so} \quad a \leq \sqrt{n}.$$

By Theorem 7.2,  $a$  has a prime divisor  $p \leq a \leq \sqrt{n}$ , and  $p \mid a \mid n$ . ■

*Note* (Extra: density of primes). The Prime Number Theorem states that  $\pi(x)$ , the number of primes  $\leq x$ , satisfies

$$\pi(x) \sim \frac{x}{\ln x}.$$

So primes thin out, but only logarithmically: a random integer near  $x$  is prime with probability roughly  $1/\ln x$ . This is what makes random search for large primes (as needed in §7.18) practical.

**Example 7.3** (A composite family – harder, Sophie Germain identity). Show that  $n^4 + 4$  is composite for every integer  $n > 1$ .

Complete the square twice:

$$n^4 + 4 = (n^4 + 4n^2 + 4) - 4n^2 = (n^2 + 2)^2 - (2n)^2 = (n^2 - 2n + 2)(n^2 + 2n + 2).$$

For  $n > 1$  we have

$$n^2 - 2n + 2 = (n - 1)^2 + 1 \geq 2, \quad n^2 + 2n + 2 > n^2 - 2n + 2,$$

so both factors are proper. Hence  $n^4 + 4$  is composite.

### 7.3 Remainders and the mod operation

**Theorem 7.5** (Division algorithm – Rosen §4.1, more rigorous than CN’s statement: uniqueness included). *For all  $n \in \mathbb{Z}$  and  $d \in \mathbb{N}$  there exist unique  $q, r \in \mathbb{Z}$  with*

$$n = qd + r, \quad 0 \leq r \leq d - 1.$$

*Proof.* **Existence.** Let

$$S = \{n - kd : k \in \mathbb{Z}, n - kd \geq 0\}.$$

$S$  is nonempty (take  $k = -|n|$ , giving  $n + |n|d \geq 0$ ), so by well-ordering it has a least element  $r = n - qd \geq 0$ . If  $r \geq d$  then

$$r - d = n - (q + 1)d \geq 0$$

lies in  $S$  and is smaller, a contradiction. So  $0 \leq r \leq d - 1$ .

**Uniqueness.** Suppose  $n = q_1d + r_1 = q_2d + r_2$  with  $0 \leq r_1, r_2 \leq d - 1$ . Then

$$r_1 - r_2 = (q_2 - q_1)d,$$

so  $d \mid (r_1 - r_2)$ . But  $|r_1 - r_2| \leq d - 1 < d$ , forcing  $r_1 = r_2$ , and then  $q_1 = q_2$  since  $d \neq 0$ . ■

**Definition 7.3** (The mod operation). The unique  $r$  of Theorem 7.5 is written  $n \bmod d$ , and  $q = \lfloor n/d \rfloor$ . Thus

$$\frac{n}{d} = \left\lfloor \frac{n}{d} \right\rfloor + \frac{n \bmod d}{d}, \quad \frac{n \bmod d}{d} \in [0, 1).$$

*Note.* The divisor  $d$  is always taken positive;  $n$  may be negative. For example,

$$-6 \bmod 5 = 4,$$

since the largest multiple of 5 that is  $\leq -6$  is  $-10$  (so  $q = -2$ ) and  $-10 + 4 = -6$ . Programming languages’ `%` operator need not agree with `mod` on negative arguments.

## 7.4 Parity

**Definition 7.4** (Parity). The *parity* of  $x \in \mathbb{Z}$  is  $x \bmod 2$ ;  $x$  is *even* if this is 0 and *odd* if it is 1.

*Note* (Parity as Boolean algebra). Writing parities as 0, 1, the addition and multiplication tables mod 2 are

$$\begin{array}{c|cc} + & 0 & 1 \\ \hline 0 & 0 & 1 \\ 1 & 1 & 0 \end{array} \qquad \begin{array}{c|cc} \times & 0 & 1 \\ \hline 0 & 0 & 0 \\ 1 & 0 & 1 \end{array}$$

These are exactly the truth tables of  $\oplus$  (exclusive-or) and  $\wedge$  (conjunction) with 0 = False, 1 = True. So  $\mathbb{Z}_2$  with  $+, \times$  is propositional logic with  $\oplus, \wedge$ , up to renaming – an *isomorphism*. This identification underlies the use of logic gates to perform arithmetic.

**Example 7.4** (Parity argument – harder). Prove  $\sqrt{2} \notin \mathbb{Q}$ .

Suppose

$$\sqrt{2} = \frac{a}{b}, \quad a, b \in \mathbb{N}, \gcd(a, b) = 1.$$

Then  $a^2 = 2b^2$ , so  $a^2$  is even. Since the square of an odd number is odd,  $a$  must be even; write  $a = 2c$ . Then

$$4c^2 = 2b^2 \implies b^2 = 2c^2,$$

and by the same argument  $b$  is even. Then  $2 \mid \gcd(a, b) = 1$ , a contradiction.

## 7.5 The greatest common divisor

**Definition 7.5** (gcd, lcm). For  $a, b \in \mathbb{Z}$  not both 0,  $\gcd(a, b)$  is the greatest  $d$  with  $d \mid a$  and  $d \mid b$ . For  $a, b \in \mathbb{N}$ ,  $\text{lcm}(a, b)$  is the least positive common multiple.

*Note.*  $\gcd(a, b)$  is well defined: the set of common divisors is nonempty (it contains 1) and bounded above (since  $d \mid n \neq 0 \implies d \leq |n|$ ), and every bounded nonempty set of integers has a greatest element. If  $a = b = 0$  every integer is a common divisor, so  $\gcd(0, 0)$  is undefined.

**Theorem 7.6** (Subtraction invariance). For  $a, b \in \mathbb{Z}$  not both 0,

$$\gcd(a, b) = \gcd(b, a - b).$$

*Proof.* Let

$$\text{CD}(x, y) = \{d : d \mid x \wedge d \mid y\}.$$

We show  $\text{CD}(a, b) = \text{CD}(b, a - b)$ .

( $\subseteq$ ) If  $d \mid a$  and  $d \mid b$  then  $d \mid (a - b)$  by Theorem 7.1, so  $d \in \text{CD}(b, a - b)$ .

( $\supseteq$ ) If  $d \mid b$  and  $d \mid (a - b)$  then

$$d \mid ((a - b) + b) = a,$$

so  $d \in \text{CD}(a, b)$ .

Both sets are nonempty and bounded above, hence each has a greatest element; equal sets have equal greatest elements. ■

**Theorem 7.7** (gcd-lcm identity – Rosen §4.3, beyond CN). For  $a, b \in \mathbb{N}$ ,

$$\gcd(a, b) \cdot \text{lcm}(a, b) = ab.$$



*Proof.* Write  $a = \prod_i p_i^{e_i}$  and  $b = \prod_i p_i^{f_i}$  over a common list of primes (Theorem 7.15). Then

$$\gcd(a, b) = \prod_i p_i^{\min(e_i, f_i)}, \quad \text{lcm}(a, b) = \prod_i p_i^{\max(e_i, f_i)},$$

and  $\min(e_i, f_i) + \max(e_i, f_i) = e_i + f_i$  for each  $i$ . ■

## 7.6 The Euclidean algorithm

**Definition 7.6** (Euclidean Algorithm (EA)). 1. Input  $a, b \in \mathbb{N}$ ; if  $a < b$ , swap so that  $a \geq b$ .

2. If  $b \mid a$ , output  $b$ .

3. Let

$$q := \left\lfloor \frac{a}{b} \right\rfloor;$$

output

$$\gcd(b, a - qb) = \gcd(b, a \bmod b).$$

*Note.* Repeated subtraction (Theorem 7.6) is replaced by a single remainder computation, since subtracting  $b$  from  $a$  until the result drops below  $b$  is exactly division with remainder.

**Theorem 7.8** (Lamé's theorem – Rosen §4.3, beyond CN). *The number of divisions used by the Euclidean algorithm on  $a > b \geq 1$  is at most 5 times the number of decimal digits of  $b$ .*

*Proof sketch.* Let the algorithm perform  $n$  divisions on  $(r_0, r_1) = (a, b)$ , producing

$$r_0 > r_1 > \cdots > r_n > r_{n+1} = 0.$$

Each step gives  $r_{k-1} = q_k r_k + r_{k+1}$  with  $q_k \geq 1$ , so  $r_{k-1} \geq r_k + r_{k+1}$ . Comparing with the Fibonacci recurrence and  $r_n \geq 1 = f_2$  gives  $b = r_1 \geq f_{n+1}$  by induction. By Binet's formula (Theorem 6.10),

$$f_{n+1} > \varphi^{n-1},$$

so  $n - 1 < \log_\varphi b$ . Since  $\log_{10} \varphi > 1/5$ , we get  $n < 5 \log_{10} b + 1$ , i.e.  $n$  is at most 5 times the digit count of  $b$ . ■

**Example 7.5** (Worst case of the EA – harder). Consecutive Fibonacci numbers are the worst case:  $\gcd(f_{n+1}, f_n)$  takes exactly  $n - 1$  divisions. For example, for  $(55, 34)$ :

$$\begin{aligned} 55 &= 1 \cdot 34 + 21, & 34 &= 1 \cdot 21 + 13, & 21 &= 1 \cdot 13 + 8, & 13 &= 1 \cdot 8 + 5, \\ 8 &= 1 \cdot 5 + 3, & 5 &= 1 \cdot 3 + 2, & 3 &= 1 \cdot 2 + 1, & 2 &= 2 \cdot 1 + 0, \end{aligned}$$

so  $\gcd(55, 34) = 1$ . Every quotient is 1 (except the last), which is the slowest possible descent – each remainder is the previous Fibonacci number. Consecutive Fibonacci numbers are therefore always coprime.

## 7.7 The gcd and integer linear combinations

**Definition 7.7** (Integer linear combinations). For  $m, n \in \mathbb{Z}$ ,

$$m\mathbb{Z} + n\mathbb{Z} = \{xm + yn : x, y \in \mathbb{Z}\}.$$

**Theorem 7.9.** Let  $m, n \in \mathbb{N}$  and let  $\Delta$  be the least positive element of  $m\mathbb{Z} + n\mathbb{Z}$ . Then

$$m\mathbb{Z} + n\mathbb{Z} = \Delta\mathbb{Z}.$$

*Proof.* ( $\supseteq$ ) Write  $\Delta = x_0m + y_0n$ . For  $k \in \mathbb{Z}$ ,

$$k\Delta = (kx_0)m + (ky_0)n \in m\mathbb{Z} + n\mathbb{Z}.$$

( $\subseteq$ ) Let  $z = xm + yn$  and write  $z = q\Delta + r$  with  $0 \leq r < \Delta$  (Theorem 7.5). Since  $q\Delta \in m\mathbb{Z} + n\mathbb{Z}$  and  $m\mathbb{Z} + n\mathbb{Z}$  is closed under subtraction,

$$r = z - q\Delta \in m\mathbb{Z} + n\mathbb{Z}.$$

If  $r > 0$  this contradicts minimality of  $\Delta$ . So  $r = 0$  and  $\Delta \mid z$ . ■

**Theorem 7.10** (Bézout). For  $m, n \in \mathbb{N}$ ,

$$\gcd(m, n) = \min\{xm + yn : x, y \in \mathbb{Z}, xm + yn > 0\}.$$

Equivalently, there exist  $x, y \in \mathbb{Z}$  with

$$\gcd(m, n) = xm + yn.$$

*Proof.* With  $\Delta$  as in Theorem 7.9,  $\Delta$  divides every element of  $m\mathbb{Z} + n\mathbb{Z}$ , in particular  $m$  and  $n$ , so  $\Delta$  is a common divisor. Conversely any common divisor  $d$  of  $m, n$  divides  $xm + yn$  for all  $x, y$  (Theorem 7.1), in particular  $d \mid \Delta$ , so  $d \leq \Delta$ . Hence  $\Delta = \gcd(m, n)$ . ■

*Note.* Bézout coefficients are never unique. Since  $nm - mn = 0$ , adding  $k \cdot (n, -m)$  to any solution  $(x, y)$  gives another:

$$(x + kn)m + (y - km)n = xm + yn.$$

So each element of  $m\mathbb{Z} + n\mathbb{Z}$  arises in infinitely many ways.

## 7.8 The extended Euclidean algorithm

**Definition 7.8** (Extended Euclidean Algorithm (EEA)). 1. Input  $m, n \in \mathbb{N}$ ; if  $m < n$ , swap so that  $m \geq n$ .

2. Initialise

$$(a, x, y) := (m, 1, 0), \quad (b, z, w) := (n, 0, 1).$$

3. If  $b = 0$ , output  $(a, x, y)$ .

4. Let

$$q := \left\lfloor \frac{a}{b} \right\rfloor.$$

Update

$$(a, x, y)_{\text{new}} := (b, z, w), \quad (b, z, w)_{\text{new}} := (a, x, y) - q \cdot (b, z, w),$$

using vector subtraction on the old values. Return to step 3.

**Theorem 7.11** (Correctness of the EEA). *At every stage the triples satisfy  $a = xm + yn$  and  $b = zm + wn$ . On termination  $a = \gcd(m, n)$ , so the final  $a$ -triple gives Bézout coefficients.*

*Proof.* The invariant holds initially:

$$m = 1 \cdot m + 0 \cdot n, \quad n = 0 \cdot m + 1 \cdot n.$$

If it holds before an update, then the new  $b$ -triple satisfies

$$a - qb = (x - qz)m + (y - qw)n,$$

which is the required form, and the new  $a$ -triple is the old  $b$ -triple. The first components evolve exactly as in the EA, so termination gives  $a = \gcd(m, n)$ . ■

**Example 7.6** (EEA with a nontrivial gcd – harder). Compute  $\gcd(1180, 482)$  and express it as an integer linear combination. Each row  $(t, x, y)$  satisfies  $t = 1180x + 482y$ :

|      |     |      |                                  |
|------|-----|------|----------------------------------|
| 1180 | 1   | 0    |                                  |
| 482  | 0   | 1    |                                  |
| 216  | 1   | -2   | $(\lfloor 1180/482 \rfloor = 2)$ |
| 50   | -2  | 5    | $(\lfloor 482/216 \rfloor = 2)$  |
| 16   | 9   | -22  | $(\lfloor 216/50 \rfloor = 4)$   |
| 2    | -29 | 71   | $(\lfloor 50/16 \rfloor = 3)$    |
| 0    | 241 | -590 | $(\lfloor 16/2 \rfloor = 8)$     |

So  $\gcd(1180, 482) = 2$  and

$$2 = -29 \cdot 1180 + 71 \cdot 482.$$

**Check.**  $-29 \cdot 1180 = -34220$ ,  $71 \cdot 482 = 34222$ , difference 2. The final row checks out too:

$$241 \cdot 1180 = 284380 = 590 \cdot 482.$$

Note the zig-zag of signs down the last two columns – a useful error check.

## 7.9 Coprimality

**Definition 7.9** (Coprime).  $a, b \in \mathbb{Z}$  are *coprime* (relatively prime) if  $\gcd(a, b) = 1$ .

*Note.* Coprimality is not primality: 21 and 25 are coprime, neither is prime. Coprimality is easy to test (run the EA); primality testing is a genuinely different and harder problem.

**Theorem 7.12** (Bézout characterisation of coprimality).  $m, n \in \mathbb{Z}$  are coprime if and only if there exist  $x, y \in \mathbb{Z}$  with

$$xm + yn = 1.$$

*Proof.* By Theorem 7.10,  $\gcd(m, n)$  is the least positive element of  $m\mathbb{Z} + n\mathbb{Z}$ . Since no positive integer is smaller than 1, this least element equals 1 iff  $1 \in m\mathbb{Z} + n\mathbb{Z}$ . ■

**Theorem 7.13** (Euclid's lemma). *Let  $p$  be prime and  $a, b \in \mathbb{Z}$ . Then*

$$p \mid ab \implies p \mid a \vee p \mid b.$$

*Proof.* Assume  $p \mid ab$  and  $p \nmid a$ . Since  $p$  is prime and  $p \nmid a$ ,  $\gcd(p, a) = 1$ , so by Theorem 7.12 there are  $x, y \in \mathbb{Z}$  with

$$xp + ya = 1.$$

Multiply by  $b$ :

$$xpb + yab = b.$$

The first summand is a multiple of  $p$ ; the second is a multiple of  $ab$ , hence of  $p$  by assumption. So  $p \mid b$ . ■

**Theorem 7.14** (Coprime divisor rule – Rosen §4.3). *If  $a \mid n$ ,  $b \mid n$  and  $\gcd(a, b) = 1$ , then  $ab \mid n$ .*

*Proof.* Write  $xa + yb = 1$  (Theorem 7.12) and  $n = a\alpha = b\beta$ . Then

$$n = n(xa + yb) = xan + ybn = xa(b\beta) + yb(a\alpha) = ab(x\beta + y\alpha).$$

■

**Theorem 7.15** (Fundamental Theorem of Arithmetic). *Every positive integer has a factorisation into primes that is unique up to order.*

*Proof.* Existence is Theorem 7.2. For uniqueness, suppose some integer has two distinct prime factorisations and let  $n$  be the smallest such. Write, over the primes  $p_1, \dots, p_k$  appearing in either factorisation,

$$n = p_1^{e_1} \cdots p_k^{e_k} = p_1^{f_1} \cdots p_k^{f_k},$$

with  $e_i, f_i \geq 0$  not both zero for each  $i$ , and  $e_i \neq f_i$  for some  $i$ .

Suppose some  $p_j$  has  $e_j > 0$  and  $f_j > 0$ . Put

$$g_j = \min(e_j, f_j) > 0$$

and divide both factorisations by  $p_j^{g_j}$ . Some prime still has unequal exponents in the two results (if  $e_j \neq f_j$  then  $p_j$  does; otherwise  $p_i$  does), so  $n/p_j^{g_j} < n$  has two distinct prime factorisations, contradicting minimality of  $n$ .

Hence the two factorisations use disjoint sets of primes. Let  $p$  occur in the first. Then  $p \mid n$ , and  $n$  is a product of primes  $q_1, \dots, q_t$  none equal to  $p$ . Repeated application of Theorem 7.13 gives  $p \mid q_t$  for some  $t$ , so  $p = q_t$  – a contradiction. ■

## 7.10 Modular arithmetic

**Definition 7.10** (Congruence). For  $n \in \mathbb{N}$  and  $a, b \in \mathbb{Z}$ ,  $a$  and  $b$  are *congruent modulo  $n$* , written  $a \equiv b \pmod{n}$ , if  $n \mid (a - b)$ .

**Theorem 7.16.**

$$a \equiv b \pmod{n} \iff a \bmod n = b \bmod n.$$

*Proof.* Write

$$a = q_a n + r_a, \quad b = q_b n + r_b, \quad 0 \leq r_a, r_b \leq n - 1.$$

Then

$$a - b = (q_a - q_b)n + (r_a - r_b),$$

so  $n \mid (a - b)$  iff  $n \mid (r_a - r_b)$ . But  $|r_a - r_b| \leq n - 1 < n$ , so this holds iff  $r_a = r_b$ . ■

**Theorem 7.17.** For every  $n \in \mathbb{N}$ , congruence modulo  $n$  is an equivalence relation on  $\mathbb{Z}$ .

*Proof.* **Reflexive:**  $a - a = 0 = 0 \cdot n$ .

**Symmetric:** if  $a - b = kn$  then  $b - a = (-k)n$ .

**Transitive:** if  $a - b = kn$  and  $b - c = ln$  then  $a - c = (k + l)n$ . ■

**Definition 7.11** (Residue classes and  $\mathbb{Z}_n$ ). The equivalence classes of congruence mod  $n$  are the *residue classes*

$$[a] = a + n\mathbb{Z} = \{a + kn : k \in \mathbb{Z}\},$$

and there are exactly  $n$  of them:  $[0], [1], \dots, [n-1]$ . We write  $\mathbb{Z}_n = \{0, 1, \dots, n-1\}$ , equipped with  $+, -, \times$  followed by reduction mod  $n$ .

**Theorem 7.18** (Congruences respect arithmetic – Rosen §4.1, stated more generally than CN). *If  $a \equiv b \pmod{n}$  and  $c \equiv d \pmod{n}$ , then*

$$a + c \equiv b + d \pmod{n}, \quad a - c \equiv b - d \pmod{n}, \quad ac \equiv bd \pmod{n}.$$

*Proof.* Write  $a - b = kn$ ,  $c - d = ln$ . Then

$$(a + c) - (b + d) = (k + l)n, \quad (a - c) - (b - d) = (k - l)n.$$

For the product,

$$ac - bd = ac - bc + bc - bd = c(a - b) + b(c - d) = (ck + bl)n. \quad \blacksquare$$

*Note.* Theorem 7.18 justifies reducing mod  $n$  at every step of a computation involving only  $+, -, \times$ , rather than only at the end. Division and exponentiation need separate treatment (§7.11, §7.14).

**Example 7.7** (Casting out nines – harder). Since  $10 \equiv 1 \pmod{9}$ , we have  $10^i \equiv 1 \pmod{9}$  for all  $i$ , so for  $n = \sum a_i 10^i$ ,

$$n \equiv \sum_i a_i = \text{ds}(n) \pmod{9},$$

where ds is the digital sum. Hence a claimed product can be checked cheaply: the alleged identity  $84 \times 32 = 2678$  fails because

$$(84 \times 32) \bmod 9 = (3 \times 5) \bmod 9 = 15 \bmod 9 = 6, \quad 2678 \bmod 9 = 5.$$

The check is one-sided: it detects some errors, never certifies correctness – it is blind to any error that is a multiple of 9, e.g. transposed digits. The same argument with  $10 \equiv -1 \pmod{11}$  gives  $n \equiv \text{ads}(n) \pmod{11}$  for the alternating digital sum.

**Example 7.8** (Large modular power without Euler – harder). Find the last two decimal digits of  $7^{1000}$ , i.e.  $7^{1000} \bmod 100$ .

Note

$$7^2 = 49, \quad 7^4 = 49^2 = 2401 \equiv 1 \pmod{100}.$$

So the powers of 7 cycle with period 4 mod 100, and  $1000 = 4 \cdot 250$ :

$$7^{1000} = (7^4)^{250} \equiv 1^{250} = 1 \pmod{100}.$$

The last two digits are 01.

## 7.11 Modular inverses

**Definition 7.12** (Modular inverse,  $\mathbb{Z}_n^*$ ).  $x^{-1} \in \mathbb{Z}_n$  is a *multiplicative inverse* of  $x \in \mathbb{Z}_n$  if  $xx^{-1} \equiv 1 \pmod{n}$ . We write  $\mathbb{Z}_n^*$  for the set of elements of  $\mathbb{Z}_n$  possessing an inverse.

**Theorem 7.19** (Existence of inverses).  $x \in \mathbb{Z}_n$  has an inverse in  $\mathbb{Z}_n$  if and only if  $\gcd(x, n) = 1$ .

*Proof.* ( $\Rightarrow$ ) If  $xx^{-1} \equiv 1 \pmod{n}$  then  $xx^{-1} + kn = 1$  for some  $k \in \mathbb{Z}$ , so  $x$  and  $n$  are coprime by Theorem 7.12.

( $\Leftarrow$ ) If  $\gcd(x, n) = 1$ , Theorem 7.12 gives  $y, z \in \mathbb{Z}$  with

$$yx + zn = 1.$$

Let  $y' = y \bmod n$ , so  $y = ln + y'$ . Then

$$y'x + (lx + z)n = 1,$$

so  $y'x \equiv 1 \pmod{n}$  with  $y' \in \mathbb{Z}_n$ . ■

*Note.* The proof is constructive: run the EEA on  $(x, n)$  to obtain  $y, z$  with  $yx + zn = 1$ ; then  $x^{-1} = y \bmod n$ . This is the principal practical use of the EEA.

**Example 7.9** (Inverse computation – harder). Find  $17^{-1}$  in  $\mathbb{Z}_{101}$ . Rows  $(t, x, y)$  satisfy  $t = 101x + 17y$ :

|     |    |      |                                  |
|-----|----|------|----------------------------------|
| 101 | 1  | 0    |                                  |
| 17  | 0  | 1    |                                  |
| 16  | 1  | -5   | ( $\lfloor 101/17 \rfloor = 5$ ) |
| 1   | -1 | 6    | ( $\lfloor 17/16 \rfloor = 1$ )  |
| 0   | 17 | -101 | ( $\lfloor 16/1 \rfloor = 16$ )  |

So

$$1 = -1 \cdot 101 + 6 \cdot 17,$$

giving  $17^{-1} \equiv 6 \pmod{101}$ . Check:  $6 \cdot 17 = 102 \equiv 1 \pmod{101}$ .

**Theorem 7.20** (Linear congruences – Rosen §4.4, beyond CN). Let  $d = \gcd(a, n)$ . The congruence  $ax \equiv b \pmod{n}$  has a solution if and only if  $d \mid b$ , in which case it has exactly  $d$  solutions in  $\mathbb{Z}_n$ .

*Proof.*  $ax \equiv b \pmod{n}$  has a solution iff  $b \in a\mathbb{Z} + n\mathbb{Z}$ , which by Theorem 7.9 equals  $d\mathbb{Z}$ ; so solvability is exactly  $d \mid b$ . If  $d \mid b$ , divide through:

$$\frac{a}{d}x \equiv \frac{b}{d} \pmod{n/d},$$

where  $\gcd(a/d, n/d) = 1$ , so by Theorem 7.19 there is a unique solution  $x_0 \bmod n/d$ . Its lifts  $x_0, x_0 + n/d, \dots, x_0 + (d-1)n/d$  are the  $d$  solutions in  $\mathbb{Z}_n$ . ■

**Theorem 7.21** (Chinese Remainder Theorem – Rosen §4.4, beyond CN). Let  $n_1, \dots, n_k$  be pairwise coprime and  $N = n_1 \cdots n_k$ . Then the system

$$x \equiv a_i \pmod{n_i}, \quad i = 1, \dots, k,$$

has a unique solution modulo  $N$ .

*Proof.* Let  $N_i = N/n_i$ . Since the moduli are pairwise coprime,  $\gcd(N_i, n_i) = 1$ , so by Theorem 7.19 there is  $M_i$  with  $N_i M_i \equiv 1 \pmod{n_i}$ . Set

$$x = \sum_{i=1}^k a_i N_i M_i.$$

For each  $j$ , every term with  $i \neq j$  has  $n_j \mid N_i$ , so

$$x \equiv a_j N_j M_j \equiv a_j \pmod{n_j}.$$

For uniqueness, if  $x, x'$  both solve the system then  $n_i \mid (x - x')$  for all  $i$ , and pairwise coprimality gives  $N \mid (x - x')$  by repeated use of Theorem 7.14. ■

**Example 7.10** (CRT – harder). Solve

$$x \equiv 2 \pmod{3}, \quad x \equiv 3 \pmod{5}, \quad x \equiv 2 \pmod{7}.$$

Here  $N = 105$ , and  $N_1 = 35$ ,  $N_2 = 21$ ,  $N_3 = 15$ . Inverses:

$$35 \equiv 2 \pmod{3}, \quad 2^{-1} \equiv 2 \pmod{3}; \quad 21 \equiv 1 \pmod{5}, \quad 1^{-1} \equiv 1; \quad 15 \equiv 1 \pmod{7}, \quad 1^{-1} \equiv 1.$$

So

$$x = 2 \cdot 35 \cdot 2 + 3 \cdot 21 \cdot 1 + 2 \cdot 15 \cdot 1 = 140 + 63 + 30 = 233 \equiv 23 \pmod{105}.$$

Check:

$$23 = 3 \cdot 7 + 2, \quad 23 = 5 \cdot 4 + 3, \quad 23 = 7 \cdot 3 + 2. \checkmark$$

## 7.12 The Euler totient function

**Definition 7.13** (Euler totient function). For  $n \in \mathbb{N}$ ,

$$\varphi(n) = |\{x : 0 < x < n, \gcd(x, n) = 1\}| = |\mathbb{Z}_n^*|,$$

the second equality by Theorem 7.19.

**Theorem 7.22** (Totient of a prime power). For  $p$  prime and  $m \in \mathbb{N}$ ,

$$\varphi(p^m) = p^{m-1}(p - 1).$$

In particular  $\varphi(p) = p - 1$ .

*Proof.* An integer  $x$  with  $0 < x < p^m$  fails to be coprime to  $p^m$  exactly when  $p \mid x$ . The multiples of  $p$  below  $p^m$  are  $p, 2p, \dots, (p^{m-1} - 1)p$ , of which there are  $p^{m-1} - 1$ . Hence

$$\varphi(p^m) = (p^m - 1) - (p^{m-1} - 1) = p^m - p^{m-1} = p^{m-1}(p - 1). \quad \blacksquare$$

**Theorem 7.23** (Multiplicativity). If  $\gcd(a, b) = 1$  then

$$\varphi(ab) = \varphi(a)\varphi(b).$$

*Note* (Extra: why multiplicativity holds). The Chinese Remainder Theorem (Theorem 7.21) supplies a bijection

$$\mathbb{Z}_{ab} \rightarrow \mathbb{Z}_a \times \mathbb{Z}_b, \quad x \mapsto (x \bmod a, x \bmod b).$$

Under it,  $x$  is coprime to  $ab$  iff  $x \bmod a$  is coprime to  $a$  and  $x \bmod b$  is coprime to  $b$ , so the bijection restricts to  $\mathbb{Z}_{ab}^* \rightarrow \mathbb{Z}_a^* \times \mathbb{Z}_b^*$ , giving  $\varphi(ab) = \varphi(a)\varphi(b)$ .

**Theorem 7.24** (General formula). *If  $n = p_1^{m_1} \cdots p_k^{m_k}$  with distinct primes  $p_i$ , then*

$$\varphi(n) = \prod_{i=1}^k p_i^{m_i-1} (p_i - 1) = n \prod_{i=1}^k \left(1 - \frac{1}{p_i}\right).$$

*In particular, for distinct primes  $p, q$ :*

$$\varphi(pq) = (p-1)(q-1).$$

*Proof.* Distinct prime powers are coprime, so Theorem 7.23 applies repeatedly, and each factor is evaluated by Theorem 7.22. The second form follows from

$$p_i^{m_i-1} (p_i - 1) = p_i^{m_i} \left(1 - \frac{1}{p_i}\right). \quad \blacksquare$$

*Note.* Computing  $\varphi(n)$  by this formula requires the prime factorisation of  $n$ . No method is known for computing  $\varphi(n)$  from scratch that is essentially easier than factorising  $n$  – a fact the RSA cryptosystem depends on.

### 7.13 Fast exponentiation

**Definition 7.14** (Binary (square-and-multiply) exponentiation). Write  $m = \sum_i b_i 2^i$  in binary. Then

$$a^m = \prod_{i: b_i=1} a^{2^i},$$

where the values  $a^{2^i}$  are obtained by repeated squaring:

$$a^{2^{i+1}} = \left(a^{2^i}\right)^2.$$

**Theorem 7.25** (Cost). *Binary exponentiation computes  $a^m$  using at most  $2\lfloor \log_2 m \rfloor$  multiplications.*

*Proof.* There are  $\lfloor \log_2 m \rfloor$  squarings to produce  $a^2, a^4, \dots, a^{2^{\lfloor \log_2 m \rfloor}}$ , and at most  $\lfloor \log_2 m \rfloor$  further multiplications to combine those  $a^{2^i}$  with  $b_i = 1$ .  $\blacksquare$

**Example 7.11** (Square-and-multiply – harder). Compute  $3^{36}$ . Since  $36 = 100100_2 = 2^5 + 2^2$ ,

$$3^{36} = 3^{2^5} \cdot 3^{2^2} = \left( (((((3^2)^2)^2)^2)^2) \right) \cdot (3^2)^2.$$

The squarings give

$$9, 81, 6561, 43046721, 1853020188851841,$$

and one final multiplication by 81 yields  $3^{36}$ . Six multiplications, versus 35 for the naive method.

### 7.14 Modular exponentiation

*Note.* Modular exponentiation is exponentiation in  $\mathbb{Z}_n$ . Two independent savings apply: reduce each intermediate product mod  $n$  (Theorem 7.18), and reduce the *exponent* mod  $\varphi(n)$  (Theorem 7.26 below).

**Theorem 7.26** (Euler). *If  $\gcd(x, n) = 1$ , then*

$$x^{\varphi(n)} \equiv 1 \pmod{n}.$$



*Proof.* Let  $\mathbb{Z}_n^* = \{y_1, \dots, y_{\varphi(n)}\}$ . Consider  $xy_1, \dots, xy_{\varphi(n)}$ . These are distinct mod  $n$ : if  $xy_i \equiv xy_j$  then multiplying by  $x^{-1}$  (which exists, by Theorem 7.19) gives  $y_i \equiv y_j$ , hence  $y_i = y_j$ . They also lie in  $\mathbb{Z}_n^*$ , since a product of elements coprime to  $n$  is coprime to  $n$ . So multiplication by  $x$  permutes  $\mathbb{Z}_n^*$ , and the two lists have equal products:

$$y_1 y_2 \cdots y_{\varphi(n)} \equiv (xy_1)(xy_2) \cdots (xy_{\varphi(n)}) = x^{\varphi(n)} y_1 y_2 \cdots y_{\varphi(n)} \pmod{n}.$$

The product  $y_1 \cdots y_{\varphi(n)}$  lies in  $\mathbb{Z}_n^*$ , hence is invertible; multiplying both sides by its inverse gives

$$x^{\varphi(n)} \equiv 1 \pmod{n}. \quad \blacksquare$$

**Corollary 7.1** (Fermat's Little Theorem). *If  $p$  is prime and  $p \nmid x$ , then*

$$x^{p-1} \equiv 1 \pmod{p}.$$

*Proof.* Take  $n = p$  in Theorem 7.26 and use  $\varphi(p) = p - 1$  (Theorem 7.22). ■

*Note* (Group-theoretic reading, beyond CN).  $\mathbb{Z}_n^*$  is a group under multiplication: it is closed, associative, has identity 1, and every element is invertible (Theorem 7.19). Lagrange's theorem gives  $g^{|G|} = e$  for any  $g$  in a finite group  $G$ ; Theorem 7.26 is exactly this for  $G = \mathbb{Z}_n^*$ , where  $|G| = \varphi(n)$ .

**Example 7.12** (Exponent reduction – harder). Compute  $3^{36} \bmod 25$ . First

$$\varphi(25) = \varphi(5^2) = 5 \cdot 4 = 20,$$

and  $\gcd(3, 25) = 1$ , so by Theorem 7.26 the exponent may be reduced mod 20:

$$3^{36} \equiv 3^{36 \bmod 20} = 3^{16} \pmod{25}.$$

Now square repeatedly mod 25:

$$3^2 = 9, \quad 3^4 = 81 \equiv 6, \quad 3^8 \equiv 6^2 = 36 \equiv 11, \quad 3^{16} \equiv 11^2 = 121 \equiv 21 \pmod{25}.$$

So  $3^{36} \equiv 21 \pmod{25}$  – four multiplications rather than six, and no number ever exceeds 121.

*Note* (Extra: pseudoprimes and Carmichael numbers – Rosen §4.4). The converse of Corollary 7.1 fails, so it gives only a *probable*-primality test. A composite  $n$  with  $b^{n-1} \equiv 1 \pmod{n}$  is a *pseudoprime to base  $b$* ; e.g.  $341 = 11 \cdot 31$  is a pseudoprime to base 2. Worse, a *Carmichael number* is composite and passes for *every* base coprime to it – the smallest is  $561 = 3 \cdot 11 \cdot 17$ . There are infinitely many, so the Fermat test alone cannot certify primality; practical tests (Miller–Rabin) strengthen it.

## 7.15 Primitive roots

**Definition 7.15** (Primitive root). Let  $\gcd(x, n) = 1$ . Then  $x$  is a *primitive root* of  $n$  if

$$x^k \not\equiv 1 \pmod{n} \quad \text{for all } k < \varphi(n),$$

so that  $\varphi(n)$  is the least positive exponent with  $x^{\varphi(n)} \equiv 1$ . Equivalently,  $x, x^2, x^3, \dots$  run through all of  $\mathbb{Z}_n^*$ ; we say  $x$  *generates*  $\mathbb{Z}_n^*$ .

**Example 7.13** (Primitive root and a non-example). For  $n = 7$ ,  $\varphi(7) = 6$  and  $\mathbb{Z}_7^* = \{1, \dots, 6\}$ . Powers of 3:

| $k$           | 1 | 2 | 3 | 4 | 5 | 6 |
|---------------|---|---|---|---|---|---|
| $3^k \bmod 7$ | 3 | 2 | 6 | 4 | 5 | 1 |

All six elements appear, and 1 is first reached at  $k = 6 = \varphi(7)$ , so 3 is a primitive root of 7.

By contrast  $2^1, 2^2, 2^3 \equiv 2, 4, 1$ , reaching 1 at  $k = 3 < 6$ ; so 2 generates only  $\{1, 2, 4\}$  and is not a primitive root.

**Theorem 7.27** (Which moduli have primitive roots).  *$n$  has a primitive root if and only if  $n \in \{1, 2, 4\}$  or  $n = p^k$  or  $n = 2p^k$  for an odd prime  $p$  and  $k \in \mathbb{N}$ .*

**Theorem 7.28** (How many). *If  $n$  has a primitive root, then it has exactly  $\varphi(\varphi(n))$  of them.*

*Note* (Why  $\varphi(\varphi(n))$ ). Let  $a$  be a primitive root of  $n$ . Every element of  $\mathbb{Z}_n^*$  is  $a^k$  for a unique  $k \in \{1, \dots, \varphi(n)\}$ , and  $a^k$  is itself a primitive root iff  $\gcd(k, \varphi(n)) = 1$ : if  $d = \gcd(k, \varphi(n)) > 1$  then

$$(a^k)^{\varphi(n)/d} = (a^{\varphi(n)})^{k/d} \equiv 1 \pmod{n}$$

with exponent  $\varphi(n)/d < \varphi(n)$ , so  $a^k$  falls short. Counting admissible  $k$  gives  $\varphi(\varphi(n))$ .

For  $n = 7$ :

$$\varphi(\varphi(7)) = \varphi(6) = \varphi(2)\varphi(3) = 2,$$

and indeed the primitive roots are  $3^1 = 3$  and  $3^5 \equiv 5$ , while  $3^2 \equiv 2$ ,  $3^3 \equiv 6$ ,  $3^4 \equiv 4$  are not (their exponents 2, 3, 4 each share a factor with 6).

**Example 7.14** (No primitive root – harder). Show 8 has no primitive root. Here  $\varphi(8) = 4$  and  $\mathbb{Z}_8^* = \{1, 3, 5, 7\}$ . But

$$3^2 = 9 \equiv 1, \quad 5^2 = 25 \equiv 1, \quad 7^2 = 49 \equiv 1 \pmod{8},$$

so every element of  $\mathbb{Z}_8^*$  satisfies  $x^2 \equiv 1$ , reaching 1 at exponent  $2 < 4 = \varphi(8)$ . Elements outside  $\mathbb{Z}_8^*$  cannot help: powers of 2 give 2, 4, 0, 0, ... and never reach 1. So  $\mathbb{Z}_8^*$  has no generator – consistent with Theorem 7.27, since  $8 = 2^3$  is a power of the *even* prime.

*Note.* No fast algorithm is known for finding a primitive root of a given  $n$ .

## 7.16 One-way functions

**Definition 7.16** (One-way function, informal). A function  $f$  is *one-way* if it is easy to compute but hard to invert, for almost all inputs.

*Note.* No function has been *proved* one-way; doing so would resolve  $P \neq NP$  (§??). Several functions are believed to be one-way and are used as such.

*Note* (Application: password storage). Rather than storing passwords  $P_i$ , a system stores pairs  $(U_i, f(P_i))$  for a one-way  $f$ . A login attempt with password  $P$  is accepted iff

$$f(P) = f(P_i),$$

which is fast, since  $f$  is easy to compute. An attacker who obtains  $f(P_i)$  faces the inversion problem. Strictly this is not encryption: no key is involved and no party is meant to recover  $P_i$ .

## 7.17 Modular exponentiation with fixed base

**Definition 7.17** (Modular exponentiation with fixed base). Fix  $n \in \mathbb{N}$  and a base  $a < n$ . The map is

$$x \mapsto a^x \bmod n, \quad x < \varphi(n).$$

**Definition 7.18** (Discrete Logarithm problem). Fix  $n \in \mathbb{N}$  and a primitive root  $a$  of  $n$ . Given  $y \in \mathbb{Z}_n^*$ , find  $x \leq \varphi(n)$  with

$$a^x \equiv y \pmod{n}.$$

We write  $x = \log_a y$ .

*Note.* The base is chosen to be a primitive root precisely because such a base gives the widest possible range of values (§7.15), maximising the search space for an attacker. Discrete Logarithm is believed to have no fast algorithm – empirically of comparable difficulty to integer factorisation. Since  $\varphi(n)$  is maximised when  $n$  is prime, the best candidate one-way function takes a large prime  $p$  and a primitive root  $a$  of  $p$ .

## 7.18 Diffie–Hellman key agreement

**Definition 7.19** (Diffie–Hellman scheme). Public system-wide parameters: a large prime  $p$  and a primitive root  $a$  of  $p$ . Each user picks a private random  $x \in \{1, \dots, p-1\}$  and publishes

$$y = a^x \bmod p.$$

For users  $A, B$  with private numbers  $x_A, x_B$  and public numbers  $y_A, y_B$ , each computes

$$k_{AB} := y_B^{x_A} \bmod p, \quad k_{BA} := y_A^{x_B} \bmod p.$$

**Theorem 7.29** (Agreement).

$$k_{AB} = k_{BA}.$$

*Proof.* In  $\mathbb{Z}_p^*$ ,

$$k_{AB} = y_B^{x_A} = (a^{x_B})^{x_A} = a^{x_B x_A} = a^{x_A x_B} = (a^{x_A})^{x_B} = y_A^{x_B} = k_{BA}. \quad \blacksquare$$

**Definition 7.20** (Diffie–Hellman problem). Given  $p, a, a^{x_A} \bmod p$  and  $a^{x_B} \bmod p$ , find

$$a^{x_A x_B} \bmod p.$$

**Theorem 7.30.** Any algorithm for Discrete Logarithm yields an algorithm for the Diffie–Hellman problem, with only polynomial extra work.

*Proof.* Feed  $p, a, a^{x_A}$  to the Discrete Log algorithm to recover  $x_A$ . Then compute

$$(a^{x_B})^{x_A} = a^{x_A x_B} \bmod p$$

by fast modular exponentiation (§7.14). ■

*Note* (Open problem). Whether the reduction goes the other way – whether an efficient algorithm for the Diffie–Hellman problem yields one for Discrete Log – is widely believed but unproven.

*Note.* Diffie–Hellman is a key *agreement* scheme, not a cryptosystem: neither party can use it to send a message of their choosing. Its output  $k_{AB}$  is then used as a key in a conventional secret-key cryptosystem. Its significance is that no secure channel is needed to establish the shared key, and neither party alone determines it.

**Example 7.15** (Diffie–Hellman with small numbers). Take  $p = 11$ ,  $a = 2$ , a primitive root of 11. Alice picks  $x_A = 3$ , Bob picks  $x_B = 6$ :

$$y_A = 2^3 \bmod 11 = 8, \quad y_B = 2^6 \bmod 11 = 64 \bmod 11 = 9.$$

They exchange  $y_A, y_B$  and compute

$$k_{AB} = 9^3 \bmod 11 = 729 \bmod 11 = 3, \quad k_{BA} = 8^6 \bmod 11 = 262144 \bmod 11 = 3.$$

Both equal

$$a^{x_A x_B} = 2^{18} \equiv 3 \pmod{11},$$

which neither could compute alone.

**Example 7.16** (RSA – beyond CN, Rosen §4.6). Diffie–Hellman rests on Discrete Log; RSA rests on factorisation and on Theorem 7.26.

**Setup.** Choose distinct large primes  $p, q$ ; set  $n = pq$ , so

$$\varphi(n) = (p-1)(q-1)$$

by Theorem 7.24. Choose  $e$  coprime to  $\varphi(n)$  and compute  $d = e^{-1} \bmod \varphi(n)$  by the EEA. Publish  $(n, e)$ ; keep  $d$  (and  $p, q$ ) secret.

**Encryption/decryption.**

$$C = M^e \bmod n, \quad M = C^d \bmod n.$$

**Why it inverts.** Since  $ed \equiv 1 \pmod{\varphi(n)}$ , write  $ed = 1 + k\varphi(n)$ . For  $M$  coprime to  $n$ , Theorem 7.26 gives

$$C^d = M^{ed} = M^{1+k\varphi(n)} = M \cdot (M^{\varphi(n)})^k \equiv M \cdot 1^k = M \pmod{n}.$$

For  $M$  not coprime to  $n$  the same conclusion follows via the CRT applied mod  $p$  and mod  $q$  separately.

**Security.** Recovering  $d$  from  $(n, e)$  requires  $\varphi(n)$ , which requires factorising  $n$  – believed hard. Note that anyone who learns  $\varphi(n)$  can factor  $n$ :  $p$  and  $q$  are the roots of

$$t^2 - (n - \varphi(n) + 1)t + n = 0,$$

since  $p + q = n - \varphi(n) + 1$  and  $pq = n$ .

## 8 Counting & Combinatoric

### 8.1 Counting by addition

**Theorem 8.1** (Sum rule). *If  $A, B$  are disjoint finite sets, then  $|A \cup B| = |A| + |B|$ . More generally, for pairwise disjoint  $A_1, \dots, A_m$ :*

$$|A_1 \cup A_2 \cup \dots \cup A_m| = |A_1| + |A_2| + \dots + |A_m|.$$

*Note.* Already established set-theoretically at §1.12. Combinatorially: choosing one item from  $m$  pairwise disjoint option-sets has  $\sum_i |A_i|$  total possibilities.

**Example 8.1.** Two disjoint lists, lengths  $r, c$ ; concatenating and summing all entries costs  $r + c$  additions.

### 8.2 Counting by multiplication

**Theorem 8.2** (Product rule). *For finite  $A, B$ :  $|A \times B| = |A| \cdot |B|$ . More generally, for a task done in  $m$  independent steps, with  $n_i$  options at step  $i$ :*

$$\text{total options} = n_1 \cdot n_2 \cdot \dots \cdot n_m.$$

*Note.* Already established at §1.14. "Independent" is essential: the count of options at step  $i$  must not depend on earlier choices.

**Example 8.2.** An  $r \times c$  table: summing every entry via nested loops costs  $rc$  additions – exactly  $|\{1, \dots, r\} \times \{1, \dots, c\}|$ .

**Definition 8.1** (Pigeonhole principle, Rosen §6.2). *If  $k + 1$  or more objects are placed into  $k$  boxes, some box contains at least 2 objects.*

*Proof.* Suppose, for contradiction, every box has at most 1 object. Then the total is at most  $k \cdot 1 = k$ , contradicting  $k + 1$  or more objects placed. ■

**Example 8.3.** Among any 13 people, two share a birth month.

**Theorem 8.3** (Generalized pigeonhole principle, Rosen). *If  $N$  objects are placed into  $k$  boxes, some box contains at least  $\lceil N/k \rceil$  objects.*

*Proof.* Suppose every box has at most  $\lceil N/k \rceil - 1$  objects. Then

$$N \leq k(\lceil N/k \rceil - 1) < k\left(\frac{N}{k} + 1 - 1\right) = N,$$

a contradiction ( $\lceil N/k \rceil - 1 < N/k$  since  $\lceil N/k \rceil < N/k + 1$ ). ■

**Example 8.4.** Among 100 people, some birth month has  $\lceil 100/12 \rceil = 9$  or more people.

**Example 8.5** (Ramsey number  $R(3, 3)$  – Rosen, genuinely hard). Among any 6 people, either 3 mutually know each other, or 3 mutually don't.

*Proof.* Fix person  $P$ . Among the other 5, by the pigeonhole principle, at least  $\lceil 5/2 \rceil = 3$  are related to  $P$  the same way (all "know  $P$ ", or all "don't know  $P$ ") – say 3 people  $X, Y, Z$  all know  $P$  (the other case is symmetric).

If any pair among  $X, Y, Z$  know each other – say  $X, Y$  – then  $\{P, X, Y\}$  mutually know each other. Otherwise, no pair among  $X, Y, Z$  knows each other, so  $\{X, Y, Z\}$  mutually don't know each other.

Either way, a mutually-know or mutually-don't-know triple exists. ■

*Note* (This is  $R(3, 3) = 6$ ). The Ramsey number  $R(3, 3)$  is the smallest  $n$  guaranteeing this property among  $n$  people; the proof shows  $R(3, 3) \leq 6$ . That 5 people do *not* suffice (a 5-cycle of acquaintance has neither) shows  $R(3, 3) > 5$ , so  $R(3, 3) = 6$  exactly. Ramsey numbers grow explosively –  $R(5, 5)$  is still not exactly known, only bounded between 43 and 48 (as of recent results), despite the problem being simply stated.

### 8.3 Inclusion-exclusion

**Theorem 8.4** (Two sets).

$$|A \cup B| = |A| + |B| - |A \cap B|.$$

*Note.* Already proved at §1.12.  $|A| + |B|$  double-counts  $A \cap B$ ; subtracting once corrects it.

**Theorem 8.5** (Three sets).

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|.$$

*Note* (Why the correction pattern). Summing  $|A| + |B| + |C|$  counts elements in exactly two sets twice, elements in all three sets three times. Subtracting all pairwise intersections fixes the double-count but now over-corrects triple-members (removed three times, once per pair, instead of the needed two). Adding back  $|A \cap B \cap C|$  restores them to count 1.

**Theorem 8.6** (General inclusion-exclusion, Rosen). *For finite  $A_1, \dots, A_n$ :*

$$\left| \bigcup_{i=1}^n A_i \right| = \sum_{k=1}^n (-1)^{k+1} \sum_{1 \leq i_1 < \dots < i_k \leq n} |A_{i_1} \cap \dots \cap A_{i_k}|.$$

*Proof by induction on  $n$ .* **Base case** ( $n = 1$ ): both sides equal  $|A_1|$ .

**Inductive step:** assume the formula for  $n$  sets. Write  $U = A_1 \cup \dots \cup A_n$ . Then

$$\left| \bigcup_{i=1}^{n+1} A_i \right| = |U \cup A_{n+1}| = |U| + |A_{n+1}| - |U \cap A_{n+1}|$$

by the two-set formula. Since  $U \cap A_{n+1} = \bigcup_{i=1}^n (A_i \cap A_{n+1})$  (distributive law, §1.12), the inductive hypothesis applies to *both*  $|U|$  and  $|U \cap A_{n+1}|$  (the latter as a union of  $n$  sets  $A_i \cap A_{n+1}$ ):

$$\begin{aligned} |U| &= \sum_{k=1}^n (-1)^{k+1} \sum_{i_1 < \dots < i_k \leq n} |A_{i_1} \cap \dots \cap A_{i_k}|, \\ |U \cap A_{n+1}| &= \sum_{k=1}^n (-1)^{k+1} \sum_{i_1 < \dots < i_k \leq n} |A_{i_1} \cap \dots \cap A_{i_k} \cap A_{n+1}|. \end{aligned}$$

The first sum ranges over all size- $k$  subsets of  $\{1, \dots, n\}$  (excluding  $A_{n+1}$ ); the second, negated, ranges over all size- $(k+1)$  subsets of  $\{1, \dots, n+1\}$  that *include*  $A_{n+1}$  (re-indexing  $k \rightarrow k-1$ ). Together with the standalone  $|A_{n+1}|$  term (the  $k=1$ , subset  $\{n+1\}$  case), every subset of  $\{1, \dots, n+1\}$  of every size  $k=1, \dots, n+1$  is covered exactly once, with sign  $(-1)^{k+1}$ :

$$\left| \bigcup_{i=1}^{n+1} A_i \right| = \sum_{k=1}^{n+1} (-1)^{k+1} \sum_{i_1 < \dots < i_k \leq n+1} |A_{i_1} \cap \dots \cap A_{i_k}|.$$

■

**Theorem 8.7** (Dual form – intersection via unions, Rosen).

$$\left| \bigcap_{i=1}^n A_i \right| = \sum_{k=1}^n (-1)^{k+1} \sum_{1 \leq i_1 < \dots < i_k \leq n} |A_{i_1} \cup \dots \cup A_{i_k}|.$$

*Note* (Proof strategy). Either repeat the induction above with  $\cup \leftrightarrow \cap$  swapped throughout, or apply the theorem above to  $\overline{A_1}, \dots, \overline{A_n}$  and De Morgan's laws (§1.11).

**Example 8.6** (Counting integers avoiding several divisors – Rosen, harder). How many integers in  $\{1, \dots, 1000\}$  are divisible by none of 2, 3, 5?

*Solution.* Let  $A_i = \{n \leq 1000 : i \mid n\}$  for  $i = 2, 3, 5$ . Then

$|A_2| = 500$ ,  $|A_3| = 333$ ,  $|A_5| = 200$ ,  $|A_2 \cap A_3| = 166$ ,  $|A_2 \cap A_5| = 100$ ,  $|A_3 \cap A_5| = 66$ ,  $|A_2 \cap A_3 \cap A_5| = 33$  (each =  $\lfloor 1000/(\text{product}) \rfloor$ ). By the three-set formula:

$$|A_2 \cup A_3 \cup A_5| = 500 + 333 + 200 - 166 - 100 - 66 + 33 = 734.$$

Divisible by none:  $1000 - 734 = 266$ .

■

## 8.4 Inclusion-exclusion: derangements

**Definition 8.2** (Derangement). A *derangement* of  $\{1, \dots, n\}$  is a permutation  $\sigma$  with  $\sigma(i) \neq i$  for every  $i$  (no fixed points).

**Theorem 8.8** (Derangement count).

$$D_n = n! \sum_{k=0}^n \frac{(-1)^k}{k!}.$$

*Proof.* Let  $A_i =$  permutations of  $\{1, \dots, n\}$  with  $\sigma(i) = i$ . Then  $D_n = n! - |A_1 \cup \dots \cup A_n|$ .

For any  $k$  indices  $i_1 < \dots < i_k$ :  $|A_{i_1} \cap \dots \cap A_{i_k}| =$  permutations fixing those  $k$  positions, i.e. permutations of the remaining  $n-k$  elements:  $(n-k)!$ . There are  $\binom{n}{k}$  such  $k$ -subsets.

By inclusion-exclusion:

$$|A_1 \cup \dots \cup A_n| = \sum_{k=1}^n (-1)^{k+1} \binom{n}{k} (n-k)!.$$

Since  $\binom{n}{k} (n-k)! = \frac{n!}{k!}$ :

$$D_n = n! - \sum_{k=1}^n (-1)^{k+1} \frac{n!}{k!} = n! + \sum_{k=1}^n (-1)^k \frac{n!}{k!} = n! \sum_{k=0}^n \frac{(-1)^k}{k!}$$

(the  $k=0$  term contributes  $n!$  back in).

■

**Theorem 8.9** (Recurrence, Rosen – harder, alternate derivation).

$$D_n = (n-1)(D_{n-1} + D_{n-2}), \quad D_0 = 1, \quad D_1 = 0.$$

*Proof sketch.* In a derangement of  $\{1, \dots, n\}$ , element 1 maps to some  $k \neq 1$  –  $n-1$  choices. Split by what happens to  $k$ :

**Case A:**  $\sigma(k) = 1$  (a 2-cycle  $1 \leftrightarrow k$ ). The remaining  $n-2$  elements must derange among themselves:  $D_{n-2}$  ways.

**Case B:**  $\sigma(k) \neq 1$ . Treat this as deranging  $\{1, \dots, n\} \setminus \{1\}$  where  $k$  is forbidden from mapping to  $k$  and forbidden from mapping to... (relabelling 1's role onto  $k$  as the "new position  $k$  must avoid") – this is exactly a derangement of  $n-1$  elements:  $D_{n-1}$  ways.

Summing over the  $n-1$  choices of  $k$ , each splitting into these two cases:

$$D_n = (n-1)(D_{n-2} + D_{n-1}).$$

■

**Theorem 8.10** (Limiting probability, Rosen).

$$\lim_{n \rightarrow \infty} \frac{D_n}{n!} = \frac{1}{e}.$$

*Proof.*

$$\frac{D_n}{n!} = \sum_{k=0}^n \frac{(-1)^k}{k!},$$

the  $n$ -th partial sum of the Taylor series  $e^{-1} = \sum_{k=0}^{\infty} \frac{(-1)^k}{k!}$  (via §6.8's limit machinery, this partial sum converges to  $e^{-1}$  as  $n \rightarrow \infty$ ). ■

*Note* (Interpretation). The fraction of permutations with *no* fixed point converges rapidly to  $1/e \approx 0.368$  – already accurate to 4 decimal places by  $n = 7$ . This underlies the classic "hat-check problem": the probability nobody gets their own hat back, for  $n$  people with hats mixed uniformly at random, is  $D_n/n! \rightarrow 1/e$ , essentially independent of  $n$  once  $n$  is even moderately large.

## 8.5 Selection

*Note* (Four cases).

|                     | order matters                   | order irrelevant                  |
|---------------------|---------------------------------|-----------------------------------|
| with replacement    | ordered, w/ replacement (§8.6)  | unordered, w/ replacement (§8.9)  |
| without replacement | ordered, w/o replacement (§8.7) | unordered, w/o replacement (§8.8) |

## 8.6 Ordered selection with replacement

**Theorem 8.11.** *The number of length- $r$  sequences over an  $n$ -element set  $A$  is*

$$|A \times \dots \times A| = n^r$$

(§1.14, §1.5 for the string special case, §2.12 for counting functions).



## 8.7 Ordered selection without replacement

**Definition 8.3** (Falling factorial, permutations).

$$(n)_r = n(n-1)(n-2)\cdots(n-r+1) = \frac{n!}{(n-r)!}, \quad \text{also written } {}_nP_r \text{ or } P(n, r).$$

*Derivation.* Choice  $k$  (for  $k = 1, \dots, r$ ) has  $n - (k - 1)$  options remaining (the  $k - 1$  prior choices excluded). By the product rule (§8.2):

$$n \cdot (n-1) \cdots (n-r+1).$$

■

*Note.* Already the counting-injections theorem of §2.12.

## 8.8 Unordered selection without replacement

**Theorem 8.12.**

$$\binom{n}{r} = \frac{n!}{(n-r)!r!} = \frac{(n)_r}{r!}.$$

*Note.* Already §1.10; each  $r$ -subset corresponds to  $r!$  ordered sequences (§8.7), collapsed by dividing.

**Theorem 8.13** (Pascal's identity, combinatorial proof, Rosen).

$$\binom{n}{r} = \binom{n-1}{r-1} + \binom{n-1}{r}.$$

*Proof.* Fix element  $x \in A$ ,  $|A| = n$ . Every  $r$ -subset of  $A$  either contains  $x$  (choose the remaining  $r - 1$  from  $A \setminus \{x\}$ :  $\binom{n-1}{r-1}$  ways) or excludes  $x$  (choose all  $r$  from  $A \setminus \{x\}$ :  $\binom{n-1}{r}$  ways) – exhaustive, mutually exclusive cases. ■

**Theorem 8.14** (Vandermonde's identity, Rosen – harder).

$$\binom{m+n}{r} = \sum_{k=0}^r \binom{m}{k} \binom{n}{r-k}.$$

*Proof.* Let  $A, B$  be disjoint,  $|A| = m, |B| = n$ . An  $r$ -subset of  $A \cup B$  is determined by choosing  $k$  elements from  $A$  and  $r - k$  from  $B$ , for some  $k = 0, \dots, r$  (mutually exclusive over  $k$ , by how many come from  $A$ ). Summing over  $k$  (sum rule, §8.1) gives the total count both ways. ■

**Theorem 8.15** (Binomial theorem, Rosen – full proof).

$$(x+y)^n = \sum_{k=0}^n \binom{n}{k} x^k y^{n-k}.$$

*Proof.* Expanding  $(x+y)^n = \underbrace{(x+y)(x+y)\cdots(x+y)}_{n \text{ factors}}$  by distributivity picks, from each of the  $n$  factors, either  $x$  or  $y$ ; a term  $x^k y^{n-k}$  arises once for every way of choosing which  $k$  of the  $n$  factors contribute  $x$  – exactly  $\binom{n}{k}$  ways. ■

**Example 8.7** (Application – deriving  $\sum \binom{n}{k} = 2^n$  again, differently). Setting  $x = y = 1$ :  $(1+1)^n = \sum_k \binom{n}{k} = 2^n$  – matching §1.10's theorem, now via the binomial theorem instead of direct combinatorial counting.

## 8.9 Unordered selection with replacement

**Theorem 8.16** (Stars and bars). *The number of  $(x_1, \dots, x_n)$  with each  $x_i \geq 0$  integer and  $x_1 + \dots + x_n = r$  is*

$$\binom{r+n-1}{n-1} = \binom{r+n-1}{r}.$$

*Proof.* Substitute  $y_i = x_i + 1 \geq 1$ , so  $\sum y_i = r + n$ . Represent  $r + n$  as  $r + n$  ones in a row; the values  $y_1, \dots, y_n$  correspond to  $n - 1$  barrier positions splitting them into  $n$  groups, each barrier placed in one of the  $r + n - 1$  gaps *between* consecutive ones (no barrier before the first or after the last, since each  $y_i \geq 1$ ), with at most one barrier per gap (since each  $y_i \geq 1$ , not  $\geq 0$ ).

This is exactly choosing  $n - 1$  of the  $r + n - 1$  gap-positions, without replacement, unordered (§8.8):

$$\binom{r+n-1}{n-1}.$$

■

*Note* (Why  $n^r$ ,  $(n)_r$ ,  $\binom{n}{r}$ ,  $\binom{r+n-1}{r}$  are genuinely different counts). All four answer "choose  $r$  from  $n$ ," differing only in order-matters and replacement-allowed – yet the formulas look structurally unrelated at first glance. The stars-and-bars bijection above is what connects the fourth case back to ordinary binomial coefficients.

## 8.10 Permutations with repetition, and distributing objects into boxes

**Theorem 8.17** (Permutations of a multiset). *The number of distinct arrangements of  $n$  objects, with  $n_1$  of type 1,  $n_2$  of type 2,  $\dots$ ,  $n_k$  of type  $k$  ( $n_1 + \dots + n_k = n$ ), is*

$$\frac{n!}{n_1! n_2! \dots n_k!}.$$

*Proof.* Temporarily label identical objects as distinguishable:  $n!$  orderings. Each genuinely distinct arrangement is then counted  $n_1! n_2! \dots n_k!$  times (permuting each type's labels among its own occupied slots). Divide. ■

**Example 8.8.** Arrangements of MISSISSIPPI ( $n = 11$ ; M:1, I:4, S:4, P:2):

$$\frac{11!}{1! 4! 4! 2!} = 34650.$$

*Note* (Distributing  $n$  objects into  $k$  boxes – Rosen's four cases).

|                         | objects distinguishable    | objects indistinguishable             |
|-------------------------|----------------------------|---------------------------------------|
| boxes distinguishable   | $k^n$                      | $\binom{n+k-1}{k-1}$                  |
| boxes indistinguishable | Stirling numbers $S(n, k)$ | partitions of $n$ into $\leq k$ parts |

*Note* (Case 1: distinguishable objects, distinguishable boxes). Each object independently chooses one of  $k$  boxes:  $k^n$  (§8.6, ordered-with-replacement in disguise).

*Note* (Case 2: indistinguishable objects, distinguishable boxes). Exactly §8.9's stars-and-bars count,  $\binom{n+k-1}{k-1}$ , with "box  $i$  gets  $x_i$  objects" playing the role of  $x_i$ .

**Definition 8.4** (Stirling number of the second kind, Rosen – harder).  $S(n, k)$  = number of ways to partition an  $n$ -element set into exactly  $k$  nonempty, unlabeled subsets.

**Theorem 8.18** (Recurrence).

$$S(n, k) = k \cdot S(n-1, k) + S(n-1, k-1), \quad S(n, n) = S(n, 1) = 1, \quad S(n, 0) = 0 \quad (n \geq 1).$$

*Proof sketch.* Consider where the  $n$ -th object goes. Either it joins one of the  $k$  existing groups formed by partitioning the first  $n-1$  objects into  $k$  groups ( $k \cdot S(n-1, k)$  ways), or it forms its own new singleton group, with the first  $n-1$  objects partitioned into the remaining  $k-1$  groups ( $S(n-1, k-1)$  ways). ■

*Note* (Case 3: distinguishable objects, indistinguishable boxes). Distributing  $n$  distinguishable objects into  $k$  indistinguishable nonempty boxes is exactly  $S(n, k)$ ; allowing empty boxes too sums  $\sum_{j=1}^k S(n, j)$ .

*Note* (Case 4: indistinguishable objects, indistinguishable boxes). Equivalent to counting integer partitions of  $n$  into at most  $k$  parts – no simple closed form exists (unlike the other three cases); computed via its own recurrence or generating function, genuinely the hardest of the four.

## 8.11 Generating permutations and combinations

**Definition 8.5** (Lexicographic order on permutations). For permutations of  $\{1, \dots, n\}$  written as sequences:  $\pi < \sigma$  iff, at the first position where they differ,  $\pi$  has the smaller value.

*Note* (Next-permutation algorithm, Rosen). Given permutation  $a_1 a_2 \dots a_n$  (not the largest,  $n, n-1, \dots, 1$ ):

1. Find the largest index  $j$  with  $a_j < a_{j+1}$  (the last "ascent").
2. Find the largest index  $k > j$  with  $a_k > a_j$  (guaranteed to exist, since  $a_{j+1} > a_j$  qualifies).
3. Swap  $a_j, a_k$ .
4. Reverse the suffix  $a_{j+1}, \dots, a_n$  (now guaranteed to be in decreasing order).

The result is the immediate lexicographic successor of  $a_1 \dots a_n$ .

**Example 8.9.**  $362541 \rightarrow 363214$ ? Let's trace: last ascent at  $j = 3$  ( $a_3 = 2 < a_4 = 5$ ). Largest  $k > 3$  with  $a_k > 2$ :  $a_6 = 1$ ? No,  $1 < 2$ ; check  $a_5 = 4 > 2$ : yes, and it's the last such (since  $a_6 = 1 \not> 2$ ), so  $k = 5$ . Swap  $a_3, a_5$ :  $365241 \rightarrow$  wait, swap positions 3, 5:  $364521$ . Reverse suffix from position 4:  $5, 2, 1 \rightarrow 1, 2, 5$ : result  $364125$ .

*Note* (Generating  $r$ -combinations in lex order, Rosen). Represent an  $r$ -combination of  $\{1, \dots, n\}$  as increasing  $a_1 < a_2 < \dots < a_r$ . To find the next:

1. Find the largest  $i$  such that  $a_i \neq n - r + i$  (i.e. not already at its maximum possible value).
2. Replace  $a_i$  with  $a_i + 1$ .
3. Set  $a_j = a_i + (j - i)$  for each  $j = i + 1, \dots, r$  (the smallest valid values following the new  $a_i$ ).

**Example 8.10.**  $n = 5, r = 3$ , current  $\{2, 3, 5\}$ : check from the right –  $a_3 = 5 = n - r + 3 = 5$ , so skip;  $a_2 = 3$ , is it  $= n - r + 2 = 4$ ? No ( $3 \neq 4$ ), so  $i = 2$ . Increment:  $a_2 = 4$ . Set  $a_3 = a_2 + 1 = 5$ . Next combination:  $\{2, 4, 5\}$ .

*Note* (Why this matters computationally). These algorithms let software enumerate all  $n!$  permutations or all  $\binom{n}{r}$  combinations one at a time, in a fixed, predictable order, using only the previous one – no need to store or generate the whole list at once, essential when  $n!$  or  $\binom{n}{r}$  is far too large to hold in memory.

## 9 Discrete Probability I

### 9.1 The nature of randomness

*Note.* Randomness may reflect: a genuinely unpredictable physical process (quantum measurement); a deterministic process too complex to model exactly (a coin toss); or an observer's own ignorance of a fixed but unknown outcome. Probability theory quantifies all three uniformly, regardless of the underlying cause.

### 9.2 Probability

**Definition 9.1** (Sample space, event). An *experiment* has a set  $U$  of all possible outcomes, the *sample space*. An *event* is  $A \subseteq U$ .

**Definition 9.2** (Probability, equally-likely case). If  $U$  is finite and every outcome equally likely:

$$\Pr(A) = \frac{|A|}{|U|}.$$

**Definition 9.3** (Probability, general case). Assign  $\Pr(x) \in [0, 1]$  to each  $x \in U$  with

$$\sum_{x \in U} \Pr(x) = 1, \quad \Pr(A) = \sum_{x \in A} \Pr(x).$$

*Note.*  $\Pr(\emptyset) = 0$ ,  $\Pr(U) = 1$ . The equally-likely case is the special case  $\Pr(x) = 1/|U|$  for all  $x$ .

**Example 9.1** (Infinite sample space – harder).  $U = \mathbb{N}$ ,  $\Pr(n) = 1/2^n$ . Check normalization:

$$\sum_{n=1}^{\infty} \frac{1}{2^n} = \frac{1/2}{1 - 1/2} = 1$$

(§6.14, geometric series). Then

$$\Pr(\text{odd}) = \sum_{k=1}^{\infty} \Pr(2k-1) = \sum_{k=1}^{\infty} \frac{1}{2^{2k-1}} = \frac{1/2}{1 - 1/4} = \frac{2}{3}.$$

### 9.3 Choice of sample space

*Note* (Outcomes must be equally likely, not merely distinguishable). Modelling two dice as an *unordered* pair  $\{d_1, d_2\}$  gives 21 outcomes, *not* equally likely ( $\{1, 2\}$  arises two ways,  $\{1, 1\}$  one way) – the equally-likely formula  $|A|/|U|$  then gives wrong answers. Modelling as an *ordered* pair  $(d_1, d_2)$  gives 36 genuinely equally-likely outcomes. Choosing a sample space with equally-likely outcomes, even at the cost of "over-specifying," is essential before applying §9.2's formula.

### 9.4 Mutually exclusive events

**Definition 9.4** (Mutually exclusive).  $A, B$  are *mutually exclusive* iff  $A \cap B = \emptyset$ .

**Theorem 9.1.** If  $A, B$  mutually exclusive:  $\Pr(A \cup B) = \Pr(A) + \Pr(B)$ .

*Proof.*  $\Pr(A \cup B) = \sum_{x \in A \cup B} \Pr(x) = \sum_{x \in A} \Pr(x) + \sum_{x \in B} \Pr(x)$  (§8.1, sum rule, since  $A, B$  disjoint)  $= \Pr(A) + \Pr(B)$ . ■

## 9.5 Operations on events

**Theorem 9.2** (Complement).

$$\Pr(\overline{A}) = 1 - \Pr(A).$$

*Proof.*  $A, \overline{A}$  mutually exclusive,  $A \cup \overline{A} = U$ :  $\Pr(A) + \Pr(\overline{A}) = \Pr(U) = 1$ . ■

**Theorem 9.3** (Monotonicity).

$$A \subseteq B \implies \Pr(A) \leq \Pr(B).$$

*Proof.*  $B = A \cup (B \setminus A)$ , mutually exclusive, so  $\Pr(B) = \Pr(A) + \Pr(B \setminus A) \geq \Pr(A)$  (probabilities nonnegative). ■

## 9.6 Inclusion-Exclusion for probabilities

**Theorem 9.4.**

$$\Pr(A \cup B) = \Pr(A) + \Pr(B) - \Pr(A \cap B).$$

*Proof.* Same argument as §8.3, with  $|\cdot|/|U|$  in place of  $|\cdot|$ , or directly:  $A \cup B = A \cup (B \setminus A)$ , mutually exclusive, and  $B \setminus A = B \setminus (A \cap B)$  has  $\Pr(B \setminus A) = \Pr(B) - \Pr(A \cap B)$  (monotonicity argument above, since  $A \cap B \subseteq B$ ). ■

**Theorem 9.5** (General case).

$$\Pr\left(\bigcup_{i=1}^n A_i\right) = \sum_{k=1}^n (-1)^{k+1} \sum_{i_1 < \dots < i_k} \Pr(A_{i_1} \cap \dots \cap A_{i_k}).$$

*Note.* Identical proof to §8.3's Theorem, with  $\Pr$  in place of  $|\cdot|/|U|$  throughout.

## 9.7 Independent events

**Definition 9.5** (Independent).  $A, B$  are *independent* iff

$$\Pr(A \cap B) = \Pr(A) \cdot \Pr(B).$$

**Example 9.2.** Two fair dice,  $A$  = "first die is 3,"  $B$  = "sum is 7":  $\Pr(A) = 1/6$ ,  $\Pr(B) = 6/36 = 1/6$ ,  $\Pr(A \cap B) = \Pr(\{(3, 4)\}) = 1/36 = (1/6)(1/6)$  – independent.

**Definition 9.6** (Mutual independence,  $n$  events, Rosen – harder).  $A_1, \dots, A_n$  are *mutually independent* iff for *every* subset  $\{i_1, \dots, i_k\} \subseteq \{1, \dots, n\}$ :

$$\Pr(A_{i_1} \cap \dots \cap A_{i_k}) = \Pr(A_{i_1}) \cdots \Pr(A_{i_k}).$$

**Example 9.3** (Pairwise independent but not mutually independent – classic counterexample). Toss two fair coins.  $A$  = "1st is Heads,"  $B$  = "2nd is Heads,"  $C$  = "the two tosses agree." Each pair:

$$\Pr(A) = \Pr(B) = \Pr(C) = \frac{1}{2}, \quad \Pr(A \cap B) = \Pr(A \cap C) = \Pr(B \cap C) = \frac{1}{4},$$

so every pair is independent. But

$$\Pr(A \cap B \cap C) = \Pr(\{HH\}) = \frac{1}{4} \neq \frac{1}{8} = \Pr(A) \Pr(B) \Pr(C).$$

Pairwise independence does not imply mutual independence.

## 9.8 Conditional probability

**Definition 9.7** (Conditional probability). For  $\Pr(B) > 0$ :

$$\Pr(A \mid B) = \frac{\Pr(A \cap B)}{\Pr(B)}.$$

**Theorem 9.6** (Independence restated).  $A, B$  independent (with  $\Pr(B) > 0$ )  $\iff \Pr(A \mid B) = \Pr(A)$ .

*Proof.*  $\Pr(A \mid B) = \Pr(A) \iff \Pr(A \cap B) / \Pr(B) = \Pr(A) \iff \Pr(A \cap B) = \Pr(A) \Pr(B)$ . ■

**Theorem 9.7** (Multiplication rule).

$$\Pr(A \cap B) = \Pr(B) \cdot \Pr(A \mid B).$$

## 9.9 Bayes' Theorem

**Theorem 9.8** (Law of total probability). If  $B_1, \dots, B_n$  partition  $U$  (§1.15), each  $\Pr(B_i) > 0$ :

$$\Pr(A) = \sum_{i=1}^n \Pr(A \mid B_i) \Pr(B_i).$$

*Proof.*  $A = \bigcup_i (A \cap B_i)$ , mutually exclusive (since  $B_i$  pairwise disjoint), so  $\Pr(A) = \sum_i \Pr(A \cap B_i) = \sum_i \Pr(A \mid B_i) \Pr(B_i)$  (multiplication rule). ■

**Theorem 9.9** (Bayes' Theorem).

$$\Pr(B_j \mid A) = \frac{\Pr(A \mid B_j) \Pr(B_j)}{\sum_{i=1}^n \Pr(A \mid B_i) \Pr(B_i)}.$$

*Proof.*

$$\Pr(B_j \mid A) = \frac{\Pr(A \cap B_j)}{\Pr(A)} = \frac{\Pr(A \mid B_j) \Pr(B_j)}{\Pr(A)},$$

then substitute the law of total probability for  $\Pr(A)$ . ■

**Example 9.4** (Base rate fallacy – Rosen, harder, counterintuitive). A disease affects 0.1% of a population. A test is 99% accurate (both sensitivity and specificity). Given a positive result, what is  $\Pr(\text{disease} \mid \text{positive})$ ?

*Solution.* Let  $D$  = disease,  $T$  = positive test.  $\Pr(D) = 0.001$ ,  $\Pr(T \mid D) = 0.99$ ,  $\Pr(T \mid \bar{D}) = 0.01$ .

$$\Pr(D \mid T) = \frac{\Pr(T \mid D) \Pr(D)}{\Pr(T \mid D) \Pr(D) + \Pr(T \mid \bar{D}) \Pr(\bar{D})} = \frac{0.99 \times 0.001}{0.99 \times 0.001 + 0.01 \times 0.999} = \frac{0.00099}{0.01098} \approx 0.090.$$

*Note.* Despite 99% test accuracy, a positive result means only  $\approx 9\%$  chance of actually having the disease – because the disease is rare, false positives from the large healthy population outnumber true positives. This is the classic argument for why screening-test results must always be combined with the base rate, not read in isolation.

## 10 Discrete Probability II

### 10.1 Random variables

**Definition 10.1** (Random variable). A *random variable* on sample space  $U$  is a function  $X : U \rightarrow \mathbb{R}$ .

**Example 10.1.** Two dice,  $X = \text{sum of the two values}$ :  $X : U \rightarrow \{2, \dots, 12\}$ ,  $U = \{1, \dots, 6\}^2$ .

*Note* (Notation).  $\{X = k\}$  abbreviates the event  $\{u \in U : X(u) = k\}$ , so  $\Pr(X = k)$  means  $\Pr(\{X = k\})$ .

**Definition 10.2** (Independent random variables, Rosen).  $X, Y$  are *independent* iff for all  $r, s \in \mathbb{R}$ :

$$\Pr(X = r \text{ and } Y = s) = \Pr(X = r) \cdot \Pr(Y = s).$$

*Note.* Distinct from independence of *events* (§9.7) only in that it must hold for *every* pair of values  $r, s$  simultaneously.

### 10.2 Probability distributions

**Definition 10.3** (Probability mass function). For  $X : U \rightarrow \mathbb{R}$  with countable image:

$$p(k) = \Pr(X = k), \quad \sum_k p(k) = 1.$$

**Example 10.2.** Sum of two fair dice:  $p(2) = 1/36$ ,  $p(7) = 6/36$ , symmetric about 7.

### 10.3 Expectation

**Definition 10.4** (Expectation).

$$E[X] = \sum_{u \in U} X(u) \Pr(u) = \sum_k k \cdot \Pr(X = k).$$

**Theorem 10.1** (Linearity of expectation).

$$E[aX + b] = aE[X] + b, \quad E[X + Y] = E[X] + E[Y].$$

*Proof.*

$$E[X + Y] = \sum_{u \in U} (X(u) + Y(u)) \Pr(u) = \sum_u X(u) \Pr(u) + \sum_u Y(u) \Pr(u) = E[X] + E[Y].$$

■

*Note* (No independence required). This holds for *any*  $X, Y$ , independent or not – the proof never uses independence.

**Example 10.3** (Hat-check problem via linearity – Rosen, harder, revisits §8.4).  $n$  people's hats mixed uniformly at random; find  $E[\text{number of people who get their own hat}]$ .



*Solution.* Let  $X_i = 1$  if person  $i$  gets their own hat, else 0;  $X = \sum_i X_i$ .  $\Pr(X_i = 1) = 1/n$  (uniform), so  $E[X_i] = 1/n$ . By linearity:

$$E[X] = \sum_{i=1}^n E[X_i] = n \cdot \frac{1}{n} = 1.$$

■

*Note.* Remarkably simple – computing this directly via the derangement-based distribution of  $X$  would require §8.4’s full machinery; linearity bypasses it entirely, and works regardless of  $n$ .

## 10.4 Median

**Definition 10.5** (Median).  $m$  is a *median* of  $X$  iff  $\Pr(X \leq m) \geq \frac{1}{2}$  and  $\Pr(X \geq m) \geq \frac{1}{2}$ .

## 10.5 Mode

**Definition 10.6** (Mode). A *mode* of  $X$  is a value  $k$  maximizing  $\Pr(X = k)$ .

## 10.6 Variance

**Definition 10.7** (Variance, standard deviation).

$$\text{Var}(X) = E[(X - E[X])^2], \quad \sigma_X = \sqrt{\text{Var}(X)}.$$

**Theorem 10.2.**

$$\text{Var}(X) = E[X^2] - E[X]^2.$$

*Proof.* Let  $\mu = E[X]$ .

$$E[(X - \mu)^2] = E[X^2 - 2\mu X + \mu^2] = E[X^2] - 2\mu E[X] + \mu^2 = E[X^2] - 2\mu^2 + \mu^2 = E[X^2] - \mu^2.$$

■

**Theorem 10.3** (Variance of independent sum, Rosen). *If  $X, Y$  independent:*

$$\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y).$$

*Proof.*

$$E[(X + Y)^2] = E[X^2] + 2E[XY] + E[Y^2].$$

Independence gives  $E[XY] = E[X]E[Y]$  (sum over  $r, s$  of  $rs \Pr(X = r, Y = s) = rs \Pr(X = r) \Pr(Y = s)$ , factoring). So

$$\text{Var}(X + Y) = E[X^2] + 2E[X]E[Y] + E[Y^2] - (E[X] + E[Y])^2 = (E[X^2] - E[X]^2) + (E[Y^2] - E[Y]^2).$$

■

**Theorem 10.4** (Chebyshev’s inequality, Rosen – harder). *For  $a > 0$ :*

$$\Pr(|X - E[X]| \geq a) \leq \frac{\text{Var}(X)}{a^2}.$$

*Proof.* Let  $\mu = E[X]$ . Then

$$\text{Var}(X) = \sum_k (k - \mu)^2 \Pr(X = k) \geq \sum_{k: |k - \mu| \geq a} (k - \mu)^2 \Pr(X = k) \geq \sum_{k: |k - \mu| \geq a} a^2 \Pr(X = k) = a^2 \Pr(|X - \mu| \geq a).$$

Divide by  $a^2$ . ■

*Note.* Applies to *any* distribution with finite variance – no shape assumption needed, unlike most probability bounds.

## 10.7 Uniform distribution

**Definition 10.8** (Discrete uniform).  $X$  uniform on  $\{1, \dots, n\}$ :  $p(k) = 1/n$  for each  $k$ .

**Theorem 10.5.**

$$E[X] = \frac{n+1}{2}, \quad \text{Var}(X) = \frac{n^2-1}{12}.$$

*Sketch.*  $E[X] = \frac{1}{n} \sum_{k=1}^n k = \frac{1}{n} \cdot \frac{n(n+1)}{2}$  (§6.12). Variance via  $E[X^2] = \frac{1}{n} \sum k^2$  (§6.12's sum-of-squares formula) and the theorem above. ■

## 10.8 Binomial distribution

**Definition 10.9** (Bernoulli trial, Rosen). A *Bernoulli trial* has two outcomes, "success" (probability  $p$ ) and "failure" (probability  $1 - p$ ).

**Definition 10.10** (Binomial distribution).  $X$  = number of successes in  $n$  independent Bernoulli trials:

$$\Pr(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}.$$

*Note.*  $\binom{n}{k}$  counts which  $k$  of the  $n$  trials succeed (§8.8);  $p^k (1 - p)^{n-k}$  is that specific outcome's probability, by independence.

**Theorem 10.6.**

$$E[X] = np, \quad \text{Var}(X) = np(1 - p).$$

*Proof.* Let  $X_i = 1$  if trial  $i$  succeeds, else 0;  $X = \sum X_i$ ,  $E[X_i] = p$ . By linearity:  $E[X] = np$ . Trials independent,  $\text{Var}(X_i) = E[X_i^2] - E[X_i]^2 = p - p^2 = p(1 - p)$ ; by the independent-sum theorem:  $\text{Var}(X) = np(1 - p)$ . ■

## 10.9 Poisson distribution

**Definition 10.11** (Poisson distribution).

$$\Pr(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}, \quad k = 0, 1, 2, \dots$$

**Theorem 10.7** (Poisson as a limit of binomial). Fix  $\lambda > 0$ . As  $n \rightarrow \infty$  with  $p = \lambda/n$ :

$$\binom{n}{k} p^k (1 - p)^{n-k} \rightarrow \frac{\lambda^k e^{-\lambda}}{k!}.$$

*Proof.*

$$\binom{n}{k} \left(\frac{\lambda}{n}\right)^k \left(1 - \frac{\lambda}{n}\right)^{n-k} = \frac{n!}{k!(n-k)!} \cdot \frac{\lambda^k}{n^k} \cdot \left(1 - \frac{\lambda}{n}\right)^{n-k}.$$

As  $n \rightarrow \infty$ :  $\frac{n!}{(n-k)!n^k} = \frac{n(n-1)\cdots(n-k+1)}{n^k} \rightarrow 1$  (each factor  $\rightarrow 1$ ), and  $\left(1 - \frac{\lambda}{n}\right)^{n-k} \rightarrow e^{-\lambda}$

(by the standard limit  $(1 + x/n)^n \rightarrow e^x$ ). So the whole expression  $\rightarrow \frac{\lambda^k}{k!} e^{-\lambda}$ . ■

*Note.* Models rare events over many trials (typos per page, calls per minute) where  $n$  is large,  $p$  small,  $np = \lambda$  moderate.

**Theorem 10.8.**  $E[X] = \lambda$ ,  $\text{Var}(X) = \lambda$ .

## 10.10 Geometric distribution

**Definition 10.12** (Geometric distribution).  $X$  = number of trials until first success, success probability  $p$ :

$$\Pr(X = k) = (1 - p)^{k-1} p, \quad k = 1, 2, 3, \dots$$

**Theorem 10.9.**

$$E[X] = \frac{1}{p}.$$

*Proof.*

$$E[X] = \sum_{k=1}^{\infty} k(1-p)^{k-1} p = p \sum_{k=1}^{\infty} k(1-p)^{k-1}.$$

Using  $\sum_{k=1}^{\infty} kx^{k-1} = \frac{1}{(1-x)^2}$  (differentiate the geometric series  $\sum x^k = \frac{1}{1-x}$ , §6.14) with  $x = 1 - p$ :

$$E[X] = p \cdot \frac{1}{p^2} = \frac{1}{p}.$$

■

## 10.11 The coupon collector's problem

**Theorem 10.10** (Coupon collector).  $n$  coupon types, each draw uniform over all  $n$ , independent draws. Let  $T$  = number of draws until all  $n$  types collected. Then

$$E[T] = nH_n = n \left(1 + \frac{1}{2} + \cdots + \frac{1}{n}\right).$$

*Proof.* Let  $T_i$  = number of draws to go from  $i-1$  to  $i$  distinct types collected,  $i = 1, \dots, n$ ;  $T = \sum_i T_i$ .

Once  $i-1$  types are collected, each draw is "new" with probability  $\frac{n-i+1}{n}$  – geometric with

$p = \frac{n-i+1}{n}$ , so

$$E[T_i] = \frac{n}{n-i+1}.$$

By linearity:

$$E[T] = \sum_{i=1}^n \frac{n}{n-i+1} = n \sum_{j=1}^n \frac{1}{j} = nH_n$$

(substituting  $j = n - i + 1$ ). ■

*Note.* Combines §10.3's linearity, §10.10's geometric expectation, and §6.15's harmonic numbers – collecting all  $n$  types takes  $\Theta(n \log n)$  draws on average (§6.15's  $H_n \approx \ln n$ ), not  $\Theta(n)$ : the last few coupons are disproportionately expensive to find.

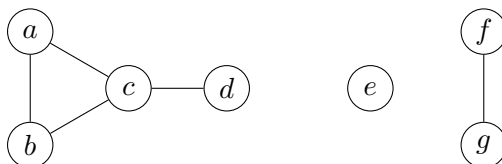
# 11 Graph Theory I

## 11.1 Basic definitions

**Definition 11.1** (Graph). A graph  $G = (V, E)$ :  $V$  a set of *vertices*,  $E$  a set of unordered pairs of vertices, *edges*.

**Definition 11.2** (Adjacent, neighbour, incident).  $v \sim w$  iff  $\{v, w\} \in E$ . Then  $v, w$  are *adjacent*, each a *neighbour* of the other, and each is *incident* with edge  $\{v, w\}$ . The *neighbourhood*  $N(v)$  is the set of  $v$ 's neighbours.

**Definition 11.3** (Isolated vertex, leaf).  $v$  is *isolated* iff  $N(v) = \emptyset$ ; a *leaf* iff  $|N(v)| = 1$ .



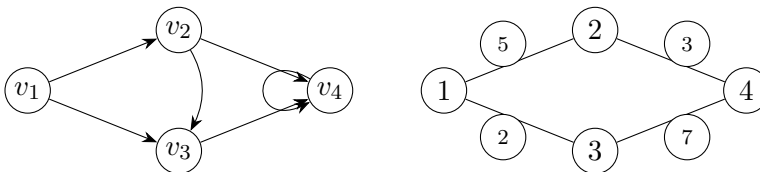
$e$  is isolated;  $d, f, g$  are leaves;  $\{a, b, c, d\}$ ,  $\{e\}$ ,  $\{f, g\}$  are the three components (§11.12)

*Note* (Common graph models, Rosen §10.1). *Acquaintanceship graph*: people as vertices, edge iff they know each other. *Influence graph*: directed, edge  $u \rightarrow v$  iff  $u$  influences  $v$ . *Call graph*: directed multigraph, edge per phone call. *Precedence graph*: directed, edge  $u \rightarrow v$  iff task  $u$  must precede task  $v$  (used in scheduling). *Niche overlap graph*: species as vertices, edge iff they compete for the same food source.

## 11.2 Types of graphs

**Definition 11.4** (Simple graph).  $G$  is *simple* iff no loops (edge  $\{v, v\}$ ) and no multiple edges.

*Note* (Relaxations, Rosen's classification). *Multigraph*: multiple edges allowed. *Pseudograph*: loops also allowed. *Directed graph*: edges are ordered pairs  $(v, w) \in V \times V$  (*arcs*). *Weighted graph*: each edge carries a number.



Directed graph (left, with a self-loop at  $v_4$ ) and weighted graph (right)

**Definition 11.5** (Complement, Rosen). For simple  $G = (V, E)$ :  $\overline{G} = (V, \overline{E})$ ,  $\overline{E} = \{\{v, w\} : v \neq w, \{v, w\} \notin E\}$ .

**Definition 11.6** (Wheel graph, Rosen).  $W_n$ :  $C_n$  plus one hub vertex adjacent to all  $n$  rim vertices.

**Theorem 11.1.**

$$|E(\overline{G})| = \binom{n}{2} - |E(G)|.$$

### 11.3 Graphs and relations

**Theorem 11.2.** *Simple graphs correspond exactly to irreflexive symmetric binary relations.*

*Proof.* Given simple  $G = (V, E)$ , define  $R \subseteq V \times V$  by  $(v, w) \in R \iff \{v, w\} \in E$ .  $R$  is irreflexive (no loops) and symmetric ( $\{v, w\} = \{w, v\}$ ). Conversely, any irreflexive symmetric  $R$  on  $V$  gives  $E = \{\{v, w\} : (v, w) \in R\}$ , a simple graph. ■

*Note.* Dropping irreflexivity allows loops; dropping symmetry gives directed graphs (§2.13's general relations, unrestricted).

### 11.4 Representing graphs

#### 11.4.1 Edge list

**Definition 11.7.** The list of all edges,  $\{v, w\}$  pairs, explicitly.

#### 11.4.2 Adjacency matrix

**Definition 11.8.** For  $V = \{v_1, \dots, v_n\}$ :  $A_{ij} = 1$  if  $v_i \sim v_j$ , else 0.

$$A = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

Adjacency matrix of §11.1's graph, restricted to  $\{a, b, c, d, e\}$ , in that vertex order – row/column 5 ( $e$ ) all zero, matching that  $e$  is isolated

*Note.*  $A$  is symmetric (undirected); diagonal all 0 (simple). Exactly §2.6's zero-one matrix of the adjacency relation.

#### 11.4.3 Adjacency list

**Definition 11.9.** For each  $v$ , the list  $N(v)$ .

#### 11.4.4 Incidence matrix

**Definition 11.10.** For  $V = \{v_1, \dots, v_n\}$ ,  $E = \{e_1, \dots, e_m\}$ :  $M_{ij} = 1$  if  $v_i$  incident with  $e_j$ , else 0.

*Note.* Each column has exactly two 1s (an edge has two endpoints).

**Theorem 11.3** (Counting walks via matrix powers, Rosen). *For adjacency matrix  $A$  of  $G$ :  $(A^r)_{ij}$  = number of length- $r$  walks from  $v_i$  to  $v_j$ .*

*Induction on  $r$ .* **Base case** ( $r = 1$ ):  $(A^1)_{ij} = A_{ij} \in \{0, 1\}$  counts length-1 walks (edges)  $v_i \rightarrow v_j$  correctly.

**Inductive step:** assume  $(A^r)_{ij}$  counts length- $r$  walks. A length- $(r+1)$  walk  $v_i \rightarrow v_j$  is a length- $r$  walk  $v_i \rightarrow v_k$  followed by an edge  $v_k \rightarrow v_j$ , for some intermediate  $v_k$ :

$$(A^{r+1})_{ij} = \sum_k (A^r)_{ik} A_{kj},$$

exactly ordinary matrix multiplication, and exactly the count of length- $(r+1)$  walks (summing over all valid last-step choices  $k$ ). ■

## 11.5 Graph isomorphism

**Definition 11.11** (Isomorphism).  $G_1 = (V_1, E_1)$ ,  $G_2 = (V_2, E_2)$  are *isomorphic* iff  $\exists$  bijection  $f : V_1 \rightarrow V_2$  with  $\{u, v\} \in E_1 \iff \{f(u), f(v)\} \in E_2$ .

*Note* (Necessary invariants). Isomorphic graphs agree on:  $|V|$ ,  $|E|$ , degree sequence (sorted list of degrees), connectivity. Disagreement on any  $\Rightarrow$  not isomorphic.

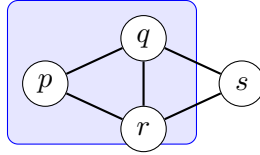
**Example 11.1** (Same degree sequence, not isomorphic – harder). Two graphs on 6 vertices, both with degree sequence  $(2, 2, 2, 2, 2, 2)$ :  $C_6$  (one hexagon) vs. two disjoint copies of  $C_3$ . Not isomorphic ( $C_6$  connected, the other isn't) – degree sequence is necessary, not sufficient.

*Note* (Computational status, Rosen – genuinely open). No polynomial-time algorithm for graph isomorphism is known, yet it is also not known to be NP-complete – one of very few natural problems whose complexity class remains unresolved (though quasi-polynomial algorithms exist, Babai 2015).

## 11.6 Subgraphs

**Definition 11.12** (Subgraph).  $H = (V_H, E_H)$  is a *subgraph* of  $G = (V, E)$  iff  $V_H \subseteq V$ ,  $E_H \subseteq E$  (with  $E_H$ 's endpoints in  $V_H$ ).

**Definition 11.13** (Induced subgraph). The subgraph *induced* by  $S \subseteq V$ : vertices  $S$ , edges  $\{v, w\} \in E$  with  $v, w \in S$  (*all* such edges).



The subgraph induced by  $\{p, q, r\}$  (shaded): includes *every* edge among them,  $\{p, q\}$ ,  $\{p, r\}$ ,  $\{q, r\}$  – not just some

## 11.7 Some special graphs

**Definition 11.14** (Standard families).

$K_n$  (complete) : every pair adjacent.       $C_n$  (cycle) :  $v_1 \sim v_2 \sim \dots \sim v_n \sim v_1$ .

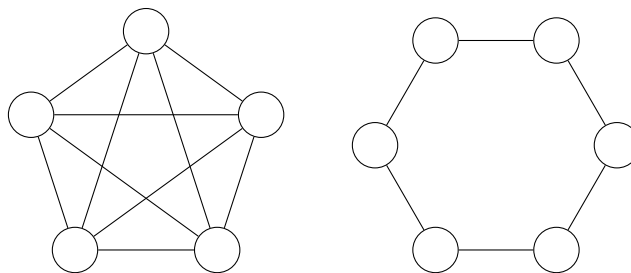
$K_{m,n}$  (complete bipartite) :  $V = A \sqcup B$ ,  $|A| = m$ ,  $|B| = n$ ,  $v \sim w \iff v \in A, w \in B$ .

$Q_n$  (hypercube) :  $V = \{0, 1\}^n$ ,  $v \sim w \iff v, w$  differ in exactly one coordinate.

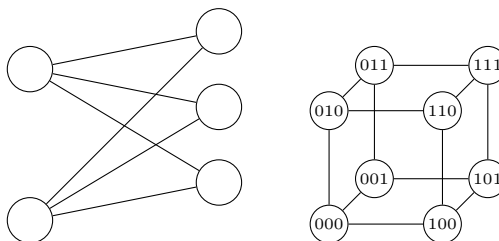
**Theorem 11.4.**

$$|E(K_n)| = \binom{n}{2}, \quad |E(Q_n)| = n \cdot 2^{n-1}.$$

$Q_n$  case. Each vertex has degree  $n$  (flip any one of  $n$  coordinates); by §11.8's handshaking lemma,  $|E| = \frac{n \cdot 2^n}{2} = n2^{n-1}$ . ■



$K_5$  (left) and  $C_6$  (right)



$K_{2,3}$  (left) and  $Q_3$  (right)

## 11.8 Degree

**Definition 11.15** (Degree).  $\deg(v) = |N(v)|$ .

**Theorem 11.5** (Handshaking lemma).

$$\sum_{v \in V} \deg(v) = 2|E|.$$

*Proof.* Each edge  $\{v, w\}$  contributes exactly 1 to  $\deg(v)$  and 1 to  $\deg(w)$  – 2 total across the sum. Summing over all edges:  $2|E|$ . ■

**Corollary 11.1.** *The number of odd-degree vertices is even.*

*Proof.*  $\sum_v \deg(v) = 2|E|$  is even. Split the sum over even- and odd-degree vertices: the even-degree part is automatically even, so the odd-degree part (a sum of odd numbers) must also be even – forcing an even *count* of odd terms. ■

**Theorem 11.6** (Two vertices share a degree, CN Thm 56 – proof by cases). *Every graph on  $n \geq 2$  vertices has two vertices of equal degree.*

*Proof.* Possible degrees lie in  $\{0, 1, \dots, n-1\}$ ,  $n$  values.

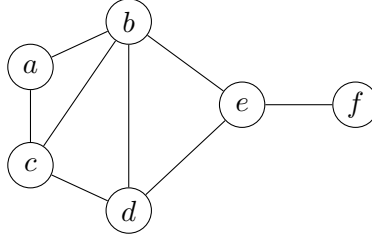
**Case 1:** no vertex has degree  $n-1$ . Then degrees lie in  $\{0, \dots, n-2\}$ , only  $n-1$  values for  $n$  vertices – pigeonhole (§8.2) forces a repeat.

**Case 2:** some vertex  $v$  has degree  $n-1$  (adjacent to all others). Then every other vertex has degree  $\geq 1$ , so no vertex has degree 0: degrees lie in  $\{1, \dots, n-1\}$ , again  $n-1$  values for  $n$  vertices – pigeonhole forces a repeat. ■



## 11.9 Moving around

**Definition 11.16** (Walk, closed walk). A *walk* from  $v$  to  $w$ :  $v_0, v_1, \dots, v_k$  with  $v_0 = v$ ,  $v_k = w$ , each  $v_{i-1} \sim v_i$ . *Length* =  $k$ . *Closed* iff  $v_0 = v_k$ .



$a, b, d, e, f$  is a walk of length 4 (also a path);  $a, b, c, a$  is a closed walk/cycle of length 3

**Definition 11.17** (Trail, path, cycle). A *trail*: walk with no repeated edge. A *path*: trail with no repeated vertex. A *cycle*: closed trail with no repeated vertex except  $v_0 = v_k$ .

*Note* (Strictly increasing restriction).

walk  $\supseteq$  trail  $\supseteq$  path.

**Definition 11.18** (Distance).  $d(v, w)$  = length of the shortest path (equivalently, shortest walk) between  $v, w$ .

**Theorem 11.7** (Odd closed walk  $\iff$  odd cycle, CN Thm 60).  $G$  has an odd-length closed walk  $\iff G$  has an odd cycle.

*Proof.* ( $\Leftarrow$ ) Every cycle is a closed walk.

( $\Rightarrow$ ) Let  $W = v_0, \dots, v_k$  ( $k$  odd) be the *shortest* odd closed walk. If  $W$  repeats no vertex besides  $v_0 = v_k$ ,  $W$  is already an odd cycle.

Otherwise, suppose  $v_i = v_j$  for some  $0 \leq i < j \leq k - 1$ . Split  $W$  into two closed walks:

$$v_i, \dots, v_j \quad \text{and} \quad v_j, \dots, v_{k-1}, v_0, \dots, v_i.$$

Their lengths sum to  $k$  (odd), so one is odd, one even; the odd one is a shorter odd closed walk than  $W$  – contradicting minimality of  $W$ .

So  $W$  has no repeated vertex besides  $v_0 = v_k$ :  $W$  is an odd cycle. ■

*Note* (Fails for “even”).  $K_2$ : closed walk  $v, w, v$  (even, length 2) exists, but no cycle at all exists.

**Definition 11.19** (Hamiltonian cycle). A cycle including every vertex of  $G$ .

*Note* (Travelling Salesman Problem). Finding a minimum-total-weight Hamiltonian cycle – among the most famous NP-hard problems (§6.9).

### 11.10 Vertex and edge connectivity

**Definition 11.20** (Cut vertex, bridge).  $v$  is a *cut vertex* iff  $G - v$  has more components than  $G$ .  $e$  is a *bridge* iff  $G - e$  does.

**Theorem 11.8.**  $e = \{u, v\}$  is a bridge  $\iff e$  lies on no cycle.

**Definition 11.21** (Vertex/edge connectivity, Rosen).  $\kappa(G)$  = minimum number of vertices whose removal disconnects  $G$  (or leaves 1 vertex).  $\lambda(G)$  = minimum number of edges whose removal disconnects  $G$ .

**Theorem 11.9** (Menger's theorem, Rosen – harder). For non-adjacent  $s, t$ : the maximum number of vertex-disjoint  $s$ - $t$  paths equals the minimum number of vertices whose removal disconnects  $s$  from  $t$ .

*Note.* A cornerstone duality result (max-flow-min-cut's combinatorial ancestor); full proof beyond this course.

**Definition 11.22** (Strongly connected, Rosen – directed graphs). Directed  $G$  is *strongly connected* iff, for every  $u, v$ , a directed path exists  $u \rightarrow v$  and  $v \rightarrow u$ .

**Definition 11.23** (Strongly connected component). A maximal strongly-connected subgraph.

*Note.* Unlike undirected components (§11.12), a directed graph's strongly connected components can have edges *between* them (one-directional) – they don't simply "partition into isolated pieces" the way undirected components do.

### 11.11 Shortest paths

**Definition 11.24** (Shortest path problem). For weighted  $G$ ,  $w : E \rightarrow \mathbb{R}^+$ : find, for source  $s$ , the minimum-weight path to each other vertex.

---

**Algorithm 4:** Dijkstra's algorithm

---

**Input:** Weighted graph  $G = (V, E, w)$ , source  $s$   
**Output:**  $d[v]$  = shortest distance from  $s$  to every  $v$

```
1  $d[s] \leftarrow 0$ ;  
2 foreach  $v \in V \setminus \{s\}$  do  
3    $d[v] \leftarrow \infty$   
4 end  
5  $S \leftarrow \emptyset$ ;  
6 while  $S \neq V$  do  
7    $u \leftarrow \arg \min_{v \notin S} d[v]$ ;  
8    $S \leftarrow S \cup \{u\}$ ;  
9   foreach neighbour  $v$  of  $u$  do  
10    if  $d[u] + w(u, v) < d[v]$  then  
11       $d[v] \leftarrow d[u] + w(u, v)$ ;  
12    end  
13  end  
14 end
```

---

**Theorem 11.10** (Correctness). *When  $u$  is added to  $S$ ,  $d[u]$  is  $u$ 's true shortest distance from  $s$ .*

*Proof sketch, induction on  $|S|$ .* Suppose true for all previously-added vertices. Suppose  $d[u]$  were not the true shortest distance – some shorter  $s$ - $u$  path  $P$  exists.  $P$  must leave  $S$  at some point, at edge  $x \rightarrow y$  ( $x \in S, y \notin S$ ). Since edge weights are positive, the portion of  $P$  up to  $y$  already has length  $\geq d[y]$  (by relaxation via  $x$ , whose distance was already correct by the inductive hypothesis), and the rest of  $P$  from  $y$  to  $u$  adds more positive weight – so  $P$ 's total length is  $\geq d[y] \geq d[u]$  (since  $u$  was chosen as the minimum  $d$ -value not yet in  $S$ ), contradicting  $P$  being shorter. ■

*Note* (Complexity). Naive implementation:  $O(n)$  to find the minimum each round,  $n$  rounds:  $O(n^2)$  (§6.9). A priority-queue implementation improves this to  $O((n+m) \log n)$ .

**Example 11.2** (Why Dijkstra fails with negative weights – harder).  $s \rightarrow a$  weight 1,  $s \rightarrow b$  weight 4,  $a \rightarrow b$  weight  $-3$ . Dijkstra finalizes  $a$  first ( $d[a] = 1$ ), then  $s$  or  $a$  next by tentative distance; but it finalizes  $s \rightarrow b$  directly at 4 before ever relaxing via  $a$  (since  $a$ 's  $d$ -value  $1 < 4$ ,  $a$  is chosen *before*  $b$  is finalized – actually here it does relax correctly). A cleaner failing example:  $s \rightarrow a$  weight 2,  $s \rightarrow b$  weight 1,  $b \rightarrow a$  weight  $-2$ : Dijkstra finalizes  $b$  first ( $d = 1$ ), relaxes  $a$  to  $\min(2, 1 - 2) = -1$  correctly here too – but if  $b$  had been finalized *before* discovering a cheaper route via a vertex not yet reached, that finalized value can never be revisited. In general: once a vertex is finalized, Dijkstra never revisits it, so a later-discovered negative-weight shortcut into an already-finalized vertex is missed entirely.

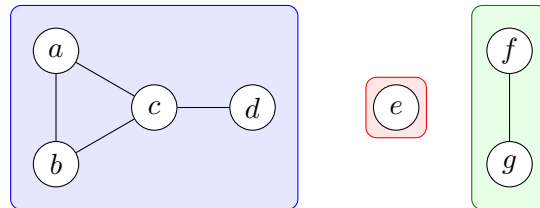
*Note* (Unweighted case reduces to BFS). When all weights equal 1: Dijkstra's "always expand the nearest unfinalized vertex" is exactly breadth-first search (§12.7) – BFS suffices, no priority queue needed.

## 11.12 Connectivity

**Definition 11.25** (Connected graph, component).  $v, w$  are *connected* iff a path (equivalently, walk) joins them.  $G$  is *connected* iff every pair is. A *component* is a maximal connected subgraph.

**Theorem 11.11.** *Connectivity is an equivalence relation on  $V$ ; its equivalence classes are exactly the vertex sets of  $G$ 's components.*

*Proof.* Reflexive: length-0 walk. Symmetric: reverse a walk. Transitive: concatenate walks at the shared vertex. By §2.16's theorem, equivalence classes partition  $V$ . ■

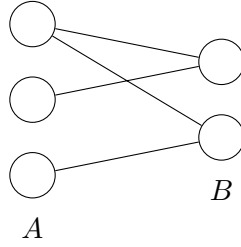


The three components of §11.1's graph – each shaded region is one equivalence class

## 11.13 Bipartite graphs

**Definition 11.26** (Bipartite).  $G$  is *bipartite* iff  $V = A \sqcup B$  with every edge having one endpoint in each part.

**Definition 11.27** (2-colouring). A function  $V \rightarrow \{\text{Black}, \text{White}\}$  with adjacent vertices differently coloured.



A bipartite graph: every edge crosses between  $A$  and  $B$

**Theorem 11.12.**  $G$  bipartite  $\iff G$  has a 2-colouring.

*Proof.* ( $\Rightarrow$ ) Colour  $A$  Black,  $B$  White; every edge crosses parts, so is Black-White.

( $\Leftarrow$ ) Given 2-colouring  $f$ , let  $A = f^{-1}(\text{Black})$ ,  $B = f^{-1}(\text{White})$ : a partition (§1.15), and every edge connects differently-coloured, hence differently-partitioned, vertices. ■

**Theorem 11.13** (Bipartite  $\iff$  no odd closed walk).  $G$  bipartite  $\iff G$  has no odd-length closed walk.

*Proof.* ( $\Rightarrow$ ) In a bipartite graph, every walk alternates  $A, B, A, B, \dots$ ; returning to the start after  $k$  steps requires  $k$  even.

( $\Leftarrow$ ) In each component, fix a ground vertex  $g$ ; colour  $v$  Black if  $d(g, v)$  even, White if odd. If some edge  $\{v, w\}$  had  $v, w$  same colour,  $d(g, v), d(g, w)$  have equal parity, so a shortest- $g$ - $v$ -path, plus edge  $\{v, w\}$ , plus a shortest- $w$ - $g$ -path forms a closed walk of length  $d(g, v) + d(g, w) + 1$  – odd (even + even + 1, or odd + odd + 1) – contradicting the hypothesis. So no such edge exists: this coloring is a valid 2-colouring. ■

**Corollary 11.2.**  $G$  bipartite  $\iff G$  has no odd cycle.

*Proof.* Combine with §11.9's odd-closed-walk-  $\iff$  -odd-cycle theorem. ■

## 11.14 Euler tours

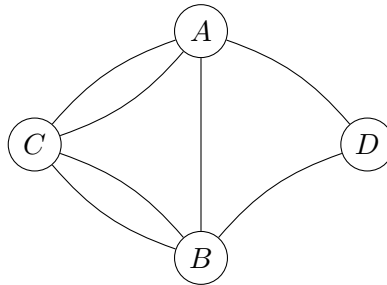
**Definition 11.28** (Euler tour). A closed trail using every edge exactly once.

**Theorem 11.14** (Euler, 1736 – Seven Bridges of Königsberg). A connected graph  $G$  has an Euler tour  $\iff$  every vertex has even degree.

*Proof.* ( $\Rightarrow$ ) Every vertex-visit in a trail uses one incoming and one outgoing edge – 2 edges per visit, so each vertex's incident-edge count is even.

( $\Leftarrow$ ) Algorithm: start a trivial closed trail  $T$  at any vertex. While unused edges remain: find  $v \in T$  with an unused incident edge (exists, since  $G$  connected – any unused edge's component meets  $T$  somewhere, by connectivity); build a new closed trail  $U$  from  $v$  using only unused edges (possible at every step since  $T$ 's even-degree property leaves an even, hence nonzero-if-nonempty, number of unused edges at each vertex encountered); splice  $U$  into  $T$  at  $v$ . Terminates (finite edges), and every edge gets used exactly once. ■

*Note* (Königsberg). Modelled the four landmasses as vertices, seven bridges as edges: each vertex has odd degree (3 or 5), so by the theorem no Euler tour exists – resolving the centuries-old puzzle.



Königsberg bridges graph:  $\deg(A) = \deg(B) = \deg(D) = 3$  (odd),  $\deg(C) = 5$  (odd) – all four vertices odd, so no Euler tour exists

*Note* (Extends to multigraphs/pseudographs). Holds with multiple edges; loops count 2 toward their vertex's degree.

**Theorem 11.15** (Route inspection / Chinese postman, Rosen – harder extension). *If  $G$  connected with  $2k$  odd-degree vertices ( $k \geq 1$ ; even by §11.8's corollary), pairing them off and adding  $k$  new edges (one per pair, parallel edges allowed) makes every degree even, giving an Euler tour of the augmented graph.*

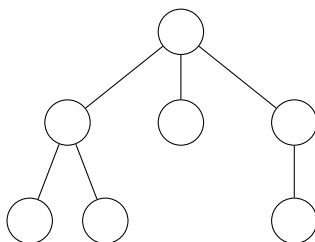
*Note.* Minimizing the total added length (pairing odd vertices via shortest paths to minimize duplicated edges) is the *route inspection problem* – unlike TSP (§11.9), this is solvable in polynomial time via minimum-weight perfect matching.

## 12 Graph Theory II

### 12.1 Trees

**Definition 12.1** (Tree). A *tree* is a connected graph with no cycles.

*Note* (Applications). Communication networks (minimal connectivity), hierarchies (directories, class inheritance under single inheritance, org charts), decision trees, parse trees, rooted trees (a distinguished root, e.g. Unix's /).



A tree

### 12.2 Equivalent characterizations of trees

**Theorem 12.1** (Rosen – TFAE). For a graph  $G$  with  $n$  vertices, the following are equivalent:

- (i)  $G$  is a tree.
- (ii)  $G$  is connected and acyclic.
- (iii)  $G$  is connected with  $n - 1$  edges.
- (iv)  $G$  is acyclic with  $n - 1$  edges.
- (v)  $G$  is connected, and every edge is a bridge.
- (vi) Every pair of vertices is joined by a unique path.
- (vii)  $G$  is acyclic, but adding any one edge creates exactly one cycle.

*Note.* Each pairwise equivalence traces back to results already proved: (i) $\Leftrightarrow$ (ii) is the definition; (ii) $\Leftrightarrow$ (v) is §12.3's minimal-connected theorem; (ii) $\Rightarrow$ (iii) and (ii) $\Rightarrow$ (iv) use §12.3's edge-count theorem; (ii) $\Leftrightarrow$ (vi) was noted after that theorem.

### 12.3 Properties of trees

**Definition 12.2** (Minimal connected graph).  $G = (V, E)$  connected, but deleting *any* edge disconnects it.

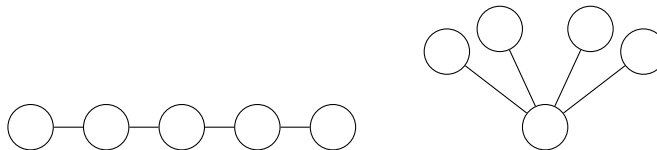
**Theorem 12.2** (CN Thm 66).  $G$  is a tree  $\iff G$  is a minimal connected graph on  $V$ .

*Proof.* ( $\Rightarrow$ ) Let  $G$  be a tree, and suppose (for contradiction) some edge  $e = \{v, w\}$  can be deleted without disconnecting  $G$ . Then a  $v$ - $w$  path avoiding  $e$  exists; adding  $e$  back creates a cycle – contradicting  $G$  acyclic.

( $\Leftarrow$ )  $G$  minimal connected; suppose (for contradiction)  $G$  has a cycle  $C$ , edge  $e \in C$ . For any  $v, w \in V$ : any walk using  $e$  can detour around the rest of  $C$  instead, giving a walk avoiding  $e$ . So  $G - e$  is still connected on the same vertex set – contradicting minimality. ■

*Note* (Unique paths). Consequently, every pair of vertices in a tree is joined by a *unique* path.

*Note* (Extremes among trees,  $n$  vertices). Path  $P_{n-1}$ : longest possible path ( $n - 1$ ), lowest possible max-degree (2). Star  $K_{1,n-1}$ : shortest possible paths (length  $\leq 2$ ), highest possible max-degree ( $n - 1$ ).



$P_4$  (left) and  $K_{1,4}$  (right) – the two extreme trees on 5 vertices

**Theorem 12.3** (CN Thm 67). *Every tree with  $\geq 2$  vertices has a leaf.*

**Theorem 12.4** (CN Thm 68 – edge count). *Every tree on  $n$  vertices has  $n - 1$  edges.*

*Induction on  $n$ .* **Base case** ( $n = 1$ ): the single-vertex tree has  $0 = 1 - 1$  edges.

**Inductive step:** let  $k \geq 1$ ; assume every  $k$ -vertex tree has  $k - 1$  edges. Let  $T$  have  $k + 1$  vertices. By the leaf theorem,  $T$  has a leaf  $\ell$ ; let  $S = T - \ell$  (delete  $\ell$  and its one incident edge).

$S$  is still a tree (removing a leaf disconnects nothing, creates no cycle), with  $k$  vertices – so by the inductive hypothesis,  $S$  has  $k - 1$  edges.  $T$  has exactly one more edge than  $S$  (the deleted one), so  $T$  has  $(k - 1) + 1 = k$  edges.

By induction, every  $n$ -vertex tree has  $n - 1$  edges. ■

*Note* (Applications of trees, Rosen §11.2 – extension). *Binary search trees*: each node has  $\leq 2$  children, left subtree smaller, right larger – search, insert in  $O(\log n)$  for balanced trees. *Huffman coding*: build an optimal prefix-free binary code by repeatedly merging the two least-frequent symbols into a subtree, symbol frequency  $\rightarrow$  leaf depth (frequent symbols get short codes). *Decision trees*: each internal node a test, each leaf an outcome – used to prove lower bounds on comparison-based algorithms (§6.9).

**Definition 12.3** (Ordered rooted tree, binary tree, Rosen). A *rooted tree* has a distinguished root; children of each node are the tree's neighbours farther from the root. An *ordered* rooted tree ranks each node's children (first, second, ...). A *binary tree*: every node has  $\leq 2$  children, distinguished as left/right.

*Note* (Tree traversals, Rosen §11.3 – harder). For a binary tree with root  $r$ , left subtree  $T_L$ , right subtree  $T_R$ :

preorder :  $r, \text{pre}(T_L), \text{pre}(T_R)$ ;      inorder :  $\text{in}(T_L), r, \text{in}(T_R)$ ;      postorder :  $\text{post}(T_L), \text{post}(T_R), r$ .

**Example 12.1** (Expression tree, Rosen – harder).  $(3 + 4) \times (5 - 2)$  as a binary tree: root  $\times$ , left subtree  $+$  with leaves 3, 4, right subtree  $-$  with leaves 5, 2.

preorder:  $\times, +, 3, 4, -, 5, 2$  (prefix/Polish notation)

inorder:  $3, +, 4, \times, 5, -, 2$  (infix, matches ordinary notation up to parentheses)

postorder:  $3, 4, +, 5, 2, -, \times$  (postfix/reverse Polish notation – directly stack-evaluable)

**Definition 12.4** (Prefix code, Rosen). A binary code where no codeword is a prefix of another – guarantees unambiguous decoding of concatenated codewords.

*Note* (Correspondence with binary trees). A prefix code corresponds to a binary tree: each codeword is the path (L/R) from root to a leaf, symbols only at leaves.

**Theorem 12.5** (Kraft's inequality). *A prefix code with codeword lengths  $l_1, \dots, l_k$  exists  $\iff$*

$$\sum_{i=1}^k 2^{-l_i} \leq 1.$$

**Definition 12.5** (Huffman coding, Rosen). Repeatedly merge the two least-frequent symbols/subtrees into a new subtree (frequency = sum), until one tree remains; codeword = root-to-leaf path.

**Theorem 12.6** (Huffman coding is optimal). *Among all prefix codes, Huffman's construction minimizes expected codeword length  $\sum_i f_i l_i$ .*

*Proof sketch, exchange argument.* In an optimal tree, the two least-frequent symbols can always be assumed siblings at maximum depth (swapping any other arrangement to make this so never increases cost). Merging them into one symbol (frequency summed) and solving the smaller  $(k - 1)$ -symbol problem optimally, by induction, then splitting back out, reproduces exactly Huffman's greedy merge – so the greedy construction is optimal at every step. ■

**Theorem 12.7** (Decision-tree lower bound for comparison sorts, Rosen – harder). *Any comparison-based sorting algorithm needs  $\Omega(n \log n)$  comparisons in the worst case.*

*Proof.* Model the algorithm as a binary decision tree: each internal node one comparison, each leaf one possible output permutation. There are  $n!$  possible orderings of  $n$  distinct elements, each needing its own leaf, so the tree has  $\geq n!$  leaves. A binary tree with  $\geq n!$  leaves has depth  $\geq \log_2(n!)$  (a depth- $d$  binary tree has  $\leq 2^d$  leaves). By Stirling's approximation,  $\log_2(n!) = \Theta(n \log n)$ . The worst-case number of comparisons is the tree's depth, so  $\geq \Omega(n \log n)$ . ■

*Note* (Game trees, Rosen – minimax, extension). A two-player game's positions form a tree: root = start, children = legal moves, leaves = terminal positions with a score. *Minimax*: leaves scored from one player's perspective; internal nodes alternately take max (that player's turn) or min (opponent's turn) over children, propagating optimal-play values up to the root.

**Example 12.2** (Tic-tac-toe fragment). At a node where X is about to move with two available squares leading to a forced win and a forced draw: X (maximizing) picks the win – the max-node's value is "win," inherited from whichever child gives the best outcome for X.

**Definition 12.6** (Universal address system, Rosen). Root labelled 0; a node's  $i$ -th child (left to right) appends  $.i$  to the parent's label (1.2: second child of the first child of the root).

*Note.* Gives a canonical way to compare/order any two nodes (lexicographic order on labels) – exactly the ordering that preorder traversal (§12.3) visits nodes in.



## 12.4 Forests

**Definition 12.7** (Forest). A graph with no cycles (not necessarily connected).

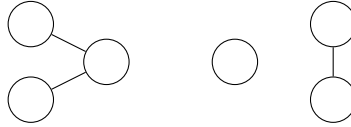
*Note.* Each component of a forest is itself connected and acyclic – hence a tree. A forest is exactly a disjoint union of trees.

**Theorem 12.8** (CN Thm 69). *A forest with  $n$  vertices and  $k$  components has  $n - k$  edges.*

*Proof.* Let  $F_1, \dots, F_k$  be the components,  $n_i, m_i$  their vertex/edge counts. Then  $n = \sum_i n_i$ ,  $m = \sum_i m_i$ , and each  $F_i$  (a tree) has  $m_i = n_i - 1$  (§12.3). So

$$m = \sum_{i=1}^k m_i = \sum_{i=1}^k (n_i - 1) = \sum_{i=1}^k n_i - k = n - k.$$

■



A forest with 3 components (trees): 6 vertices, 3 edges =  $n - k$

## 12.5 Spanning tree algorithms

---

**Algorithm 5:** Depth-first search

---

**Input:** Graph  $G = (V, E)$ , start vertex  $s$

**Output:** Spanning tree  $T$  of  $s$ 's component

```
1 mark  $s$  visited;
2  $T \leftarrow (\{s\}, \emptyset)$ ;
3 Function DFS( $u$ ):
4   foreach neighbour  $v$  of  $u$  do
5     if  $v$  not visited then
6       mark  $v$  visited;
7       add  $v$  to  $T$ 's vertices, edge  $\{u, v\}$  to  $T$ 's edges;
8       DFS( $v$ );
9     end
10  end
11 call DFS( $s$ );
```

---

*Note* (Backtracking algorithms, Rosen – application). DFS underlies *backtracking*: explore a choice, recurse; if a dead end (constraint violated) is reached, undo and try the next choice. This is exactly how a constraint-satisfaction search (e.g. the  $n$ -queens problem, §4.17) can be solved directly, as an alternative to encoding it as SAT and calling a solver.

---

**Algorithm 6:** Breadth-first search

---

**Input:** Graph  $G = (V, E)$ , start vertex  $s$

**Output:** Spanning tree  $T$  of  $s$ 's component

```
1 mark  $s$  visited;
2  $T \leftarrow (\{s\}, \emptyset)$ ;
3  $Q \leftarrow$  queue containing  $s$ ;
4 while  $Q$  not empty do
5    $u \leftarrow Q.dequeue()$ ;
6   foreach neighbour  $v$  of  $u$  do
7     if  $v$  not visited then
8       mark  $v$  visited;
9       add  $v$  to  $T$ 's vertices, edge  $\{u, v\}$  to  $T$ 's edges;
10       $Q.enqueue(v)$ ;
11   end
12 end
13 end
```

---

**Theorem 12.9** (BFS solves unweighted shortest paths). *In BFS from  $s$ , each vertex  $v$  is first discovered at exactly  $d(s, v)$  (graph distance, §11.9).*

*Proof sketch.* Induction on distance  $k$ : BFS discovers exactly the distance- $k$  vertices at level  $k$ , since it exhausts all distance- $(k-1)$  vertices' neighbours before any distance- $(k+1)$  vertex is reached. ■

---

**Algorithm 7:** Prim's algorithm

---

**Input:** Weighted connected graph  $G = (V, E, w)$

**Output:** Minimum spanning tree  $T$

```
1 pick any  $s \in V$ ;
2  $T \leftarrow (\{s\}, \emptyset)$ ;
3 while  $T$ 's vertex set  $\neq V$  do
4    $e \leftarrow$  minimum-weight edge with exactly one endpoint in  $T$ 's vertices;
5   add  $e$ 's edge and its outside endpoint to  $T$ ;
6 end
```

---

**Theorem 12.10** (Prim's algorithm is optimal).

*Proof sketch, cut property.* At each step, the chosen minimum-weight edge crosses the "cut" between  $T$ 's current vertices and the rest – and the minimum-weight edge crossing any cut is always part of *some* MST (if it weren't, swapping it into any MST for whatever edge does cross that cut, at no greater cost, produces another valid MST) – so greedily taking it never rules out optimality. ■

## 12.6 Graph colouring

**Definition 12.8** (Chromatic number).  $\chi(G)$  = minimum  $k$  such that  $G$  has a proper  $k$ -colouring (adjacent vertices differ).

**Theorem 12.11** (Greedy upper bound).

$$\chi(G) \leq \Delta(G) + 1,$$

where  $\Delta(G) = \max_v \deg(v)$ .

*Proof.* Order vertices arbitrarily; colour each in turn with the smallest colour not used by its already-coloured neighbours. Each vertex has  $\leq \Delta(G)$  neighbours, so some colour among  $\{1, \dots, \Delta(G) + 1\}$  is always free. ■

**Theorem 12.12.**

$$\chi(G) = 2 \iff G \text{ bipartite} \iff G \text{ has no odd cycle}$$

(restating §11.13's theorem in colouring language).

**Example 12.3** (Applications, Rosen). *Exam/course scheduling*: courses as vertices, edge iff they share a student – colours = time slots, proper colouring avoids conflicts. *Frequency assignment*: transmitters as vertices, edge iff they'd interfere – colours = frequencies.

*Note.*  $\chi(G) \leq 4$  for every planar  $G$  (§12.8's four colour theorem) – but computing  $\chi(G)$  exactly, for general graphs, is NP-hard.

## 12.7 Spanning trees

**Definition 12.9** (Spanning tree). A subgraph of connected  $G$  that is a tree and includes every vertex of  $G$ .

**Theorem 12.13** (Existence). *Every connected graph has a spanning tree.*

---

**Algorithm 8:** Spanning tree via edge addition

---

**Input:** Connected graph  $G = (V, E)$

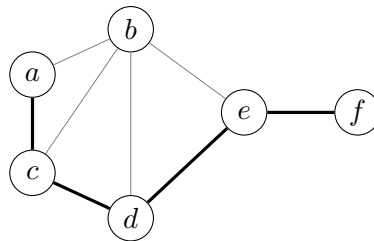
**Output:** Spanning tree  $F = (V, X)$

```

1  $X \leftarrow \emptyset$ ;
2 while  $\exists e \in E \setminus X$  such that  $X \cup \{e\}$  is acyclic do
3   | pick such an  $e$ ;
4   |  $X \leftarrow X \cup \{e\}$ ;
5 end
```

---

*Correctness of the algorithm above.*  $F$  is acyclic by construction (a forest).  $F$  is connected: if not, some component boundary edge of  $G$  could be added to  $X$  without creating a cycle (endpoints in different components), contradicting termination. So  $F$  is a connected forest – a tree – spanning  $V$ . ■



A spanning tree (thick edges) of §11.9's example graph

**Definition 12.10** (Minimum spanning tree). For weighted connected  $G$ : a spanning tree minimizing  $w(X) = \sum_{e \in X} w(e)$ .

---

**Algorithm 9:** Kruskal's algorithm

---

**Input:** Weighted connected graph  $G = (V, E, w)$

**Output:** Minimum spanning tree  $F = (V, X)$

```
1  $X \leftarrow \emptyset$ ;  
2 sort  $E$  by increasing weight;  
3 foreach  $e \in E$ , in sorted order do  
4   | if  $X \cup \{e\}$  is acyclic then  
5   |   |  $X \leftarrow X \cup \{e\}$ ;  
6   | end  
7 end
```

---

**Theorem 12.14** (Kruskal's algorithm is optimal, CN Thm 70). *Kruskal's algorithm produces a minimum spanning tree.*

*Note* (Proof idea – the exchange argument, Rosen – harder). Suppose Kruskal's tree  $T$  is not minimum; let  $T^*$  be a minimum spanning tree agreeing with  $T$  on as many edges as possible, and let  $e$  be the first edge Kruskal chose that is *not* in  $T^*$ . Adding  $e$  to  $T^*$  creates a cycle  $C$ ; since  $T^*$  is a tree,  $C$  has some edge  $f \notin T$  (else  $T$  itself would contain a cycle). Since Kruskal chose  $e$  before considering  $f$  (or never needed  $f$ , having already connected those components via earlier, no-more-expensive edges),  $w(e) \leq w(f)$ . Swapping:  $T^{**} = T^* - f + e$  is also a spanning tree, with  $w(T^{**}) \leq w(T^*)$  (so still minimum), and agrees with  $T$  on strictly more edges – contradicting the choice of  $T^*$ . So no such  $e$  exists:  $T = T^*$ .

*Note* (Prim's algorithm, Rosen – alternate greedy MST algorithm). Grow a single tree from one starting vertex: repeatedly add the minimum-weight edge connecting the current tree to a new vertex (rather than Kruskal's "cheapest edge anywhere, cycle permitting"). Also provably optimal, by a similar exchange argument.

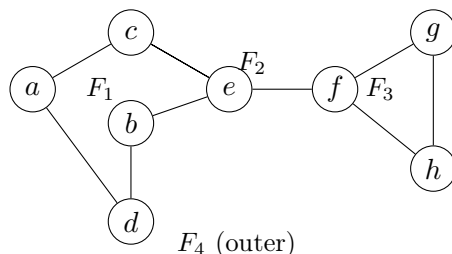
*Note* (Why greedy is exceptional here). Greedy algorithms rarely give optimal solutions in general (e.g. greedy shortest-path search can fail badly, §6.9's complexity themes). MST is a striking exception – the underlying reason (the “matroid” structure of forests) is studied in *matroid theory*, which characterizes exactly when greedy algorithms are guaranteed optimal.

## 12.8 Planarity

**Definition 12.11** (Planar graph).  $G$  is *planar* iff it can be drawn in the plane with vertices as distinct points, edges as non-self-intersecting curves meeting only at shared endpoints – no crossings.

**Definition 12.12** (Face). A maximal region of the plane whose points can be joined by curves avoiding all vertices/edges; the unbounded region is the *outer face*.

*Note.* Each face's boundary, traced around, is a closed walk. Each edge borders exactly two face-sides (possibly the same face twice).



$n = 8$ ,  $m = 10$ ,  $f = 4$ : check  $n - m + f = 8 - 10 + 4 = 2$  (Euler's formula)

**Theorem 12.15** (Euler's formula). *For a connected plane graph with  $n$  vertices,  $m$  edges,  $f$  faces:*

$$n - m + f = 2.$$

*Induction on  $m$ . Base case* ( $m = n - 1$ ):  $G$  connected with  $n - 1$  edges is a tree (§12.3), so has just one face ( $f = 1$ ):

$$n - (n - 1) + 1 = 2.$$

**Inductive step:** let  $m \geq n - 1$ ; assume the formula for every connected plane graph with  $m$  edges. Let  $G$  have  $n$  vertices,  $m + 1$  edges,  $f$  faces.

Since  $m + 1 > n - 1$ ,  $G$  is not a tree, so has a cycle  $C$ ; let  $e \in C$ . The cycle  $C$  separates the plane into inside/outside, so  $e$ 's two sides lie in *different* faces. Deleting  $e$  merges those two faces into one: let  $H = G - e$ , with  $n$  vertices,  $m$  edges,  $f - 1$  faces.

By the inductive hypothesis applied to  $H$ :  $n - m + (f - 1) = 2$ , i.e.  $n - (m + 1) + f = 2$  – exactly the formula for  $G$ .

By induction, the formula holds for all  $m \geq n - 1$ . ■

**Corollary 12.1** (Edge bound). *For a simple, connected planar graph with  $n \geq 3$  vertices,  $m$  edges:*

$$m \leq 3n - 6.$$

*Proof.* Each face's boundary has  $\geq 3$  edges (simple graph,  $n \geq 3$ ), and each edge borders exactly 2 face-sides, so  $3f \leq 2m$ , i.e.  $f \leq \frac{2}{3}m$ . Substituting into Euler's formula:  $2 = n - m + f \leq n - m + \frac{2}{3}m = n - \frac{1}{3}m$ , so  $m \leq 3n - 6$ . ■

**Theorem 12.16** ( $K_5$  is not planar).

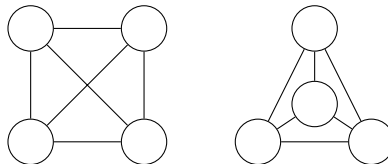
*Proof.*  $K_5$ :  $n = 5$ ,  $m = \binom{5}{2} = 10$ . But  $3n - 6 = 9 < 10$  – violates the corollary. ■

**Theorem 12.17** ( $K_{3,3}$  is not planar).

*Proof.*  $K_{3,3}$ :  $n = 6$ ,  $m = 9$ , bipartite so no triangles – every face boundary has  $\geq 4$  edges, giving  $4f \leq 2m$ ,  $f \leq m/2$ . Euler:  $2 = n - m + f \leq n - m + m/2 = n - m/2$ , so  $m \leq 2n - 4 = 8 < 9$  – contradiction. ■

**Theorem 12.18** (Kuratowski's theorem, Rosen – harder, full characterization).  *$G$  is planar  $\iff G$  contains no subgraph that is a subdivision of  $K_5$  or  $K_{3,3}$  (a copy of  $K_5/K_{3,3}$  with edges possibly replaced by paths).*

*Note.* The two non-planarity “obstructions” above ( $K_5, K_{3,3}$ ) turn out to be the *only* ones, up to subdivision – a remarkably clean complete characterization, though the proof is well beyond this course.



$K_4$ : a crossing drawing (left) vs. a planar drawing (right) – the same graph,  $K_4$  genuinely is planar

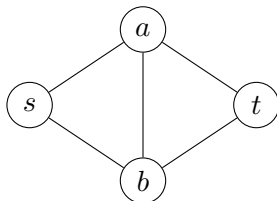
**Theorem 12.19** (Four colour theorem, Rosen – famous, proof omitted). *Every planar graph's vertices can be 4-coloured so that adjacent vertices differ.*

*Note.* Conjectured 1852, proved 1976 (Appel–Haken) – the first major theorem proved with essential computer assistance (checking  $\sim 1900$  unavoidable configurations), which was controversial at the time since no human could verify the proof by hand.

## 12.9 Games on graphs (omega, advanced)

### 12.9.1 Shannon's Switching Game

**Definition 12.13** (Shannon's Switching Game). Connected graph  $G$ , distinguished vertices  $s, t$ . Players *Cut*, *Join* alternate turns. *Cut* crosses out an edge (permanently unusable); *Join* thickens an edge (permanently marked). *Join* wins as soon as thickened edges form an  $s$ - $t$  path; *Cut* wins as soon as  $s, t$  are separated in the uncrossed-out graph.



The diamond graph used in CN's worked example of the Shannon Switching Game

**Theorem 12.20** (The game always has a winner – no draws).

*Proof.* If play continues until every edge is thickened or crossed out, the thickened-edge subgraph either contains an  $s$ - $t$  path (*Join* wins) or doesn't ( $s, t$  separated – *Cut* wins). One of these must hold. ■

**Theorem 12.21** (No graph favours the second player). *For every graph, either some player has a winning strategy regardless of who starts, or the first player (whichever role) has a winning strategy – never the second player.*

*Strategy-stealing argument.* Suppose, for contradiction, the second player had a winning strategy  $\sigma$  on some graph, regardless of who starts. Let the first player make *any* initial move, then follow  $\sigma$  as if they were "the second player" from that point on. Since making a move is never a disadvantage in this game (an extra thickened or crossed-out edge only helps, never hurts, its player – unlike games such as Chess or Draughts), this extra first move cannot hurt them, and  $\sigma$ 's guarantee still applies – so the (actual) first player wins. This contradicts  $\sigma$  being a winning strategy for the second player. ■

*Note* (Same strategy-stealing pattern as Chomp). Identical in structure to the Chomp argument (§3.8): "moving is never a disadvantage"  $\Rightarrow$  the second player's supposed winning strategy could be stolen by the first player, a contradiction.

*Note* (Bridg-It). A commercial version of this game (David Gale, 1960) played on a specific planar graph.

### 12.9.2 Slither

**Definition 12.14** (Slither). Players alternately extend a path by one edge, incident to either current endpoint, to a vertex not yet on the path. First player unable to move loses.

*Note.* Unlike Shannon's Switching Game, Slither's outcome theory is far less settled – CN treats it as an open exploration rather than a solved game, in contrast to the strategy-stealing argument above.

## Notation summary

### Sets (Ch. 1)

|                                                                                                                      |                                               |
|----------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| $\in, \notin$                                                                                                        | membership                                    |
| $\emptyset, \{\}$                                                                                                    | empty set                                     |
| $\{x : P(x)\}$                                                                                                       | set-builder                                   |
| $[a, b], (a, b), [a, b), (a, b]$                                                                                     | intervals                                     |
| $\mathbb{N}, \mathbb{Z}, \mathbb{Z}^+, \mathbb{Z}^-, \mathbb{Q}, \mathbb{R}, \mathbb{R}^+, \mathbb{R}^-, \mathbb{C}$ | standard number sets                          |
| $ A $                                                                                                                | cardinality                                   |
| $\Sigma, \Sigma^n, \Sigma^*$                                                                                         | alphabet, strings of length $n$ , all strings |
| $\subseteq, \subsetneq, \supseteq$                                                                                   | subset, proper subset, superset               |
| $\mathcal{P}(A)$                                                                                                     | power set                                     |
| $\binom{n}{k}$                                                                                                       | binomial coefficient                          |
| $A - B$ (or $A \setminus B$ ), $\overline{A}$                                                                        | set difference, complement                    |
| $\cup, \cap$                                                                                                         | union, intersection                           |
| $\Delta$                                                                                                             | symmetric difference                          |
| $\times, A^n$                                                                                                        | Cartesian product, $n$ -fold power            |

### Functions and relations (Ch. 2)

|                                      |                                                   |
|--------------------------------------|---------------------------------------------------|
| $f : A \rightarrow B, f(x)$          | function declaration, value                       |
| $\text{dom}(f)$                      | domain                                            |
| $i_A$                                | identity function                                 |
| $\chi_A$                             | indicator/characteristic function                 |
| $c_a$                                | constant function                                 |
| $\lfloor x \rfloor, \lceil x \rceil$ | floor, ceiling                                    |
| $f _X$                               | restriction                                       |
| $f^{-1}$                             | inverse function (or preimage, context-dependent) |
| $g \circ f$                          | composition                                       |
| $f^{(n)}$                            | iterated composition                              |
| $e_k, d_k$                           | encryption/decryption (cryptosystem)              |
| $R \subseteq A \times B, xRy$        | binary relation, infix notation                   |
| $R(x), R^{-1}(y)$                    | image set, inverse-image set (relation)           |
| $R \circ S$                          | relation composition                              |
| $R^{(n)}, R^+$                       | iterated relation, transitive closure             |
| $[x]_R$                              | equivalence class                                 |
| $[a_{ij}]$                           | matrix                                            |
| $\wedge, \vee$ (matrices)            | meet, join (zero-one matrices)                    |
| $\odot$                              | Boolean matrix product                            |
| $\preceq$                            | partial order                                     |
| $\vee, \wedge$ (posets)              | join (LUB), meet (GLB)                            |

### Propositional logic (Ch. 4)

|        |              |
|--------|--------------|
| $T, F$ | truth values |
|--------|--------------|



|                    |                            |
|--------------------|----------------------------|
| $\neg$             | negation                   |
| $\wedge, \vee$     | conjunction, disjunction   |
| $\implies, \iff$   | implication, biconditional |
| $\oplus$           | exclusive-or               |
| $ $                | Sheffer stroke (NAND)      |
| $\equiv$ (or $=$ ) | logical equivalence        |

## Predicate logic (Ch. 5)

|                              |                                                         |
|------------------------------|---------------------------------------------------------|
| $P(x)$                       | predicate applied to a term                             |
| $\exists x, \exists x \in A$ | existential quantifier, restricted                      |
| $\forall x, \forall x \in A$ | universal quantifier, restricted                        |
| $\text{dom}(f)$              | (as in Ch. 2 – domain of a variable/predicate argument) |

## Sequences and series (Ch. 6)

|                                                   |                                                    |
|---------------------------------------------------|----------------------------------------------------|
| $(a_n), f_n, (n^2 : n \in \mathbb{N})$            | sequence, $n$ -th term, formula notation           |
| $a, d$                                            | arithmetic sequence: first term, common difference |
| $a, r$                                            | geometric sequence: first term, common ratio       |
| $\varphi, \psi$                                   | golden ratio and its conjugate (Fibonacci/Binet)   |
| $O, \Omega, \Theta$                               | big-O, big-Omega, big-Theta                        |
| $P, NP$                                           | complexity classes                                 |
| $\sum, \prod$                                     | summation, product notation                        |
| $A_n$                                             | partial sum, Abel summation                        |
| $\lim_{n \rightarrow \infty} a_n, \varepsilon, N$ | sequence limit, $\varepsilon$ - $N$ definition     |
| $H_n$                                             | harmonic number                                    |
| $\gamma$                                          | Euler's constant                                   |
| $G(x)$                                            | generating function                                |

## Number theory (Ch. 7)

|                                |                                                                 |
|--------------------------------|-----------------------------------------------------------------|
| $d \mid n, d \nmid n$          | $d$ divides $n$ / does not divide $n$                           |
| $d\mathbb{Z}$                  | set of multiples of $d$                                         |
| $n \bmod d$                    | remainder of $n$ on division by $d$                             |
| $\lfloor n/d \rfloor$          | quotient of $n$ on division by $d$                              |
| $\gcd(a, b), \text{lcm}(a, b)$ | greatest common divisor, least common multiple                  |
| $\text{CD}(x, y)$              | set of common divisors of $x, y$                                |
| $m\mathbb{Z} + n\mathbb{Z}$    | integer linear combinations $\{xm + yn : x, y \in \mathbb{Z}\}$ |
| $(a, x, y), (b, z, w)$         | EEA triples, $a = xm + yn, b = zm + wn$                         |
| $a \equiv b \pmod{n}$          | congruence mod $n$ : $n \mid (a - b)$                           |
| $[a], a + n\mathbb{Z}$         | residue class of $a$ mod $n$                                    |
| $\mathbb{Z}_n$                 | $\{0, \dots, n-1\}$ with arithmetic mod $n$                     |
| $\mathbb{Z}_n^*$               | elements of $\mathbb{Z}_n$ with a multiplicative inverse        |
| $x^{-1} \pmod{n}$              | multiplicative inverse of $x$ in $\mathbb{Z}_n$                 |
| $\varphi(n)$                   | Euler totient function, $ \mathbb{Z}_n^* $                      |
| $\text{ds}(n), \text{ads}(n)$  | digital sum, alternating digital sum                            |

|                      |                                                       |
|----------------------|-------------------------------------------------------|
| $\log_a y \pmod n$   | discrete logarithm: $x$ with $a^x \equiv y \pmod n$   |
| $p, q$               | primes (generic; RSA's two secret primes in examples) |
| $e, d$               | RSA public/private exponents                          |
| $x_A, x_B, y_A, y_B$ | Diffie–Hellman private/public numbers                 |
| $k_{AB}, k_{BA}$     | Diffie–Hellman shared key                             |

## Counting & combinatorics (Ch. 8)

|                                          |                                                       |
|------------------------------------------|-------------------------------------------------------|
| $\lceil x \rceil, \lfloor x \rfloor$     | ceiling, floor (pigeonhole bounds)                    |
| $R(3, 3)$                                | Ramsey number                                         |
| $\bigcup, \bigcap$ (inclusion-exclusion) | union/intersection over index sets, §8.3              |
| $D_n$                                    | derangement count                                     |
| $(n)_r, {}_nP_r, P(n, r)$                | falling factorial, permutations                       |
| $\binom{n}{r}, {}_nC_r$                  | binomial coefficient, combinations                    |
| $\binom{r+n-1}{r}$                       | unordered selection with replacement (stars and bars) |
| $S(n, k)$                                | Stirling number of the second kind                    |

## Proof and general notation (Ch. 3, throughout)

|                             |                                    |
|-----------------------------|------------------------------------|
| ■                           | end of proof                       |
| $ $ (number theory)         | divides ( $a   b$ )                |
| $\Rightarrow$ (proof steps) | logical consequence within a proof |